Chapter 1. Using Access Method Services

Access method services (AMS) is a utility you can use to establish and maintain catalogs and data sets. The Storage Management Subsystem (SMS) and its classes, in conjunction with the automatic class selection (ACS) routines, automate many access method services commands and their specified parameters. However, if your storage administrator has not established these routines, you can construct and execute them manually. You can find these commands and parameters in this book.

IDCAMS is the program name for access method services (AMS).

There are two types of access method services commands: *Functional commands*, which you use to define data sets or list catalogs; and *modal commands*, which you use to specify the conditional execution of functional commands. If you are a time sharing option (TSO/E) user, you are restricted to using only the functional commands. For detailed information about these commands, their parameters and subparameters, refer to the subsequent chapters in this book.

Use this book as a reference only. It is not a complete source of information about access method services. For more information about access method services tasks, refer to *z/OS DFSMS Managing Catalogs* and *z/OS DFSMS Using Data Sets*. To find out more about SMS, its keywords, the ACS routines and requirements, see *z/OS DFSMS Introduction*.

This chapter covers the following topics:

Topic
"Identifying Data Sets and Volumes" on page 1
"Invoking Access Method Services" on page 4
"Access Method Services Tape Library Support" on page 6
"Order of Catalog Use" on page 9
"Understanding the Order of Assigned Data Set Attributes" on page 12

Identifying Data Sets and Volumes

When you use access method services commands, you must identify data sets and volumes. Data sets must be identified when they are accessed. Volumes must be identified when the system accesses the volume table of contents (VTOC), allocates or releases space or accesses a VSAM volume data set (part of the catalog).

VSAM data sets or volumes can be identified through the ALLOCATE command, through job control language (JCL), or by the data set name or volume serial number within the command that requires the data set or volume for its execution. If you do not use JCL or the ALLOCATE command, an attempt is made to dynamically allocate the data set or volume as required.

Under the SMS, you should not explicitly identify volumes. The system identifies the necessary volumes when a storage class is assigned to the data set. You can allocate your data set to a *specific* volume only if your storage administrator has set GUARANTEED SPACE=YES in the storage class assigned to your data set. See *z/OS DFSMSdfp Storage Administration* for further information about SMS volume selection.

Dynamic Allocation

You can dynamically allocate a data set if the data set name exists and is cataloged. The catalog containing the entry must have a name or alias that matches one or more of the qualifiers of the qualified data set name. All referenced catalogs must be connected to the system master catalog.

© Copyright IBM Corp. 1973, 2021

Access method services does not support the S99TIOEX, S99ACUCB, and S99DSABA options of dynamic allocation. These three options are called the XTIOT, UCB nocapture and DSAB-above-the-line options. This restriction applies to the required SYSIN and SYSPRINT data sets and to the DD names that are identified by keywords on the commands.

This restriction does not apply to data sets that are handled by the user routines that are described in "User I/O Routines" on page 435.

Access method services dynamically allocates VSAM and non-VSAM data sets with a disposition of OLD.

To dynamically allocate a volume, the volume must already be mounted as permanently resident or reserved. You should carefully consider the PRIVATE and PUBLIC use attributes when you mount a volume.

Security Authorization

Passwords are no longer honored for protecting a catalog, or data sets contained in a catalog. If they are specified, they will be ignored and no message will be issued. Previously, passwords were ignored only for SMS-managed data sets. You should use z/OS Security Server RACF® or an equivalent security package, to protect your data. Most instances of passwords have been deleted from this publication. Passwords in keywords such as ATTEMPTS, AUTHORIZATION, CODE, and LOCK will be ignored.

For information on RACF® authorization levels, see Appendix A, "Security Authorization Levels," on page 357. RACF applies to both SMS-managed and non-SMS-managed data sets and catalogs.

If a catalog is shared with a downlevel system, data sets will remain password-protected in the downlevel system, but not in a DFSMS/MVS® Version 1 Release 5 system or z/OS V1R1 system.

If you are transferring data from a system with RACF to a system which does not have RACF, data sets in a catalog will not be protected.

Storage Management Subsystem (SMS) Considerations

Do not direct an SMS-managed data set to a specific catalog. Allow the system to determine the catalog through the usual catalog search order. Naming a catalog for an SMS-managed data set requires authority from the RACF STGADMIN.IGG.DIRCAT FACILITY class profile. For information about specifying catalogs with an SMS-managed data set, see *z/OS DFSMS Managing Catalogs*.

JCL DD Statements

When you use a JCL DD statement to identify a data set, include this on the DD statement:

- User data set name
- Catalog name of the BCS
- VVDS name
- Unit and volume serial numbers, if the data set is not cataloged
- Disposition
- AMP='AMORG' is required for VVDSs

JCL DD Statement for a VSAM Data Set

You can allocate VSAM data sets directly with the access method services ALLOCATE command. The following DD statements demonstrate two additional methods of describing and allocating a VSAM data set:

- For allocating and creating a new data set:

```
//ddname DD DSNAME=dsname,DISP=(NEW,CATLG),RECORG=KS,
//          SPACE=(TRK,10,10),STORCLAS=xxxxx
```

- For allocating an existing data set:

```
//ddname DD DSNAME=dsname,DISP=OLD
```

Access method services does not provide protection for data sets in a shared environment. Therefore, you should use DISP=OLD on the DD statement for any data set that can be accessed improperly in a shared environment.

JCL DD Statement for a Volume

To identify and allocate a volume, include:

- Volume serial number
- Disposition
- Unit

This DD statement identifies and allocates volume VSER01:

```
//VOLDD    DD   VOL=SER=VSER01,UNIT=3380,DISP=OLD
```

For information on concatenated DD statements, see the FILE parameter descriptions in Chapter 3, “Functional Command Syntax,” on page 25. Examples using concatenated DD statements follow the description of the REPRO command that begins on Chapter 30, “REPRO,” on page 309. For additional information about the various types of concatenated DD statements, see the section on special DD statements in *z/OS MVS JCL Reference*.

JCL DD Statement for a Non-VSAM Data Set

You can allocate non-VSAM data sets with the access method services ALLOCATE command. See the DD statements in the examples that follow the descriptions of the BLDINDEX, EXPORT, IMPORT, REPRO, and PRINT commands for additional methods of describing and allocating non-VSAM data sets.

JCL DD Statement for a Snap Dump

If access method services encounters a condition that requires it to abnormally end a job, it takes a snap dump of virtual storage. You must write an AMSDUMP DD statement to get the snap dump; that is,

```
//AMSDUMP  DD   SYSOUT=A
```

If you do not supply an AMSDUMP DD statement and access method services encounters a condition requiring the job to abnormally end, it produces only an abbreviated dump.

JCL DD Statement for a Target Data Set

The usual target data set for listing is SYSPRINT. The default parameters of this data set are:

- Record format: variable blocked (VBA)
- Logical record length: 125, that is, (121+4)
- Block size: 0

Print lines are 121 bytes long. The first byte is the ANSI (American National Standards Institute) control character. The minimum LRECL is 121 (U-format records only). If a smaller size is used, it is overridden to 121.

You can alter the defaults by placing other values in the DCB parameter of the SYSPRINT statement. You cannot, however, use a record format of F or fixed block (FB); those are changed to VBA.

JCL DD Statement for an Alternate Target Data Set

In several commands you can use an alternate target data set for listing, but do not use F or FB record formats.

JCL statements, system messages, and job statistics are written to the SYSPRINT output device, not to the alternate target data set.

JCL DD Statement for a Data Set on a read-only volume

To allocate an existing data set on a read-only volume, include this on the DD statement:

```
ROACCESS=(ALLOW,TRKLOCK)
```

For data sets accessed on a read-only PPRC secondary volume, access method services supports allocation using the JCL DD statement. An IEFA181I or IGD17265 error message is returned if dynamic allocation is used. LISTCAT and EXAMINE are exceptions to this rule.

Direct Allocation Using JCL

You can directly allocate VSAM data sets through JCL.

The following example allocates a new data set and, with DATACLAS, uses the allocation attributes predetermined by the storage administrator through the ACS routines.

```
//DD1     DD DSNAME=EXAMPLE1,DATACLAS=DCLAS01,
//        DISP=(NEW,KEEP)
```

See *z/OS MVS JCL User's Guide* and *z/OS MVS JCL Reference* for information about JCL keywords.

Invoking Access Method Services

When you want to use an access method services function, enter a command and specify its parameters. Your request is decoded one command at a time; the appropriate functional routines perform all services required by that command.

You can call the access method services program:

- As a job or jobstep
- From a TSO/E session
- From within your own program

You can run the IDCAMS program (the access method services operating system) and include the command and its parameters as input to the program. You can also call the IDCAMS program from within another program and pass the command and its parameters to the IDCAMS program.

Time sharing option (TSO/E) users can run access method services functional commands from a TSO/E session as though they were TSO/E commands.

See Appendix D, " Invoking Access Method Services from Your Program," on page 429, for more information.

Using a Job or Jobstep to invoke Access Method Services

You can use (JCL) statements to call access method services. PGM=IDCAMS identifies the access method services program.

For example:

```
//YOURJOB  JOB  YOUR INSTALLATION'S JOB=ACCOUNTING DATA
//STEP1    EXEC PGM=IDCAMS
//SYSPRINT DD   SYSOUT=A
//SYSIN    DD   *

 access method services commands and their parameters

/*
```

- //YOURJOB, the JOB statement, is required to describe your job to the system. You might be required to supply user identification, accounting, and authorization information with the JOB statement's parameters.
- //STEP1, the EXEC statement, is required. With PGM=IDCAMS, this statement calls access method services to decode and process the access method services commands and parameters contained in the SYSIN data set. You can use the PARM operand of the EXEC statement to pass parameters to the access method services program. Chapter 2, “Modal Commands,” on page 15, describes the PARM command and explains the options you can use.
- //SYSPRINT, the SYSPRINT DD statement, is required. It identifies the output device to which access method services sends messages and output information.
- //SYSIN, the SYSIN DD statement, is required to identify the source of the input statements. An input statement is a functional or modal command and its parameters. When you code SYSIN DD *, you identify the following statements as input.

The last input statement can be followed by a delimiter statement that has an * in the first two columns.

From a Time Sharing Option Session

You can use the time sharing option (TSO/E) with VSAM and access method services to:

- Run access method services functional commands
- Run a program to call access method services

Each time you enter an access method services command as a TSO/E command, TSO/E builds the appropriate interface information and calls access method services.

You can enter one command at a time. Access method services processes the command completely before TSO/E lets you continue processing. Except for ALLOCATE, all the access method services functional commands are supported in a TSO/E environment.

To use IDCAMS and some of its parameters from TSO/E, your system programmer must update the system by one of these means:

- Update the IKJTSOxx member of SYS1.PARMLIB. This is the method that IBM recommends. Add IDCAMS to the list of authorized programs (AUTHPGM). If you want to use SHCDS, SETCACHE, LISTDATA, DEFINE or IMPORT from TSO/E, add them (and abbreviations) to the authorized command list (AUTHCMD).
- Update the IKJEGSCU CSECT instead of IKJTSOxx, see *z/OS TSO/E Customization* for more information.

The restricted functions performed by access method services that cannot be requested in an unauthorized state are:

- DEFINE—when the RECATALOG parameter is specified
- DEFINE—when the define is for an alias of a UCAT
- DELETE—when the RECOVERY parameter is specified
- EXPORT—when the object to be exported is a BCS
- IMPORT—when the object to be imported is a BCS
- PRINT—when the object to be printed is a catalog
- LISTDATA—all functions
- REPRO—when a BCS is copied or merged
- SETCACHE—all functions
- SHCDS—all functions
- VERIFY—when a BCS is to be verified.

When you use TSO/E with access method services, note that:

- You can use the first characters of a keyword as an abbreviation of the keyword. You must use enough initial characters to make the keyword unique. TRACKS, for example, can be abbreviated TR, TRA, or TRAC, because no other keyword within the same command can be abbreviated in the same way.

  You cannot use some abbreviations (such as REC, RECORD) under TSO/E because the abbreviations do not have enough initial characters to make the keyword unique.
- When a parameter's value is one or more parenthesized parameter sets, the outer parentheses surrounding the list are always required. For example, if `lowkey` and `highkey` form a parameter set that can be repeated several times, then the outer parentheses are required even when just one parameter set is specified; as follows:

  ```
  KEYWORD((lowkey  highkey))
  ```
- In TSO/E, a volume serial number can contain alphanumeric characters, national characters ($, @, or #), and hyphens (-). alphabetic, national, numeric, or hyphen only; if any other characters are used, the volume serial number cannot be used as an entry name.
- A data set name can be placed within quotation marks or not. In TSO/E however, you add a prefix (for example, the user ID) to a name not in quotation marks. The prefix becomes the first qualifier in the name.
- In TSO/E, you add the necessary qualifiers to the specified name. However, you can be prompted to complete a fully qualified name. You are also prompted to supply any required parameters you have omitted.
- The modal commands, IF-THEN-ELSE, DO-END, SET, and PARM, are not allowed in TSO/E.
- In TSO/E, the SYSPRINT data set is not used. The OUTFILE parameter can be used with certain commands to specify a data set to receive access method services output.

Other TSO/E restrictions are noted in Chapter 3, “Functional Command Syntax,” on page 25.

For details about the format of a displayed catalog entry (resulting from a LISTCAT command) for a TSO/E user, see Appendix B, “Interpreting LISTCAT Output Listings,” on page 367.

For details about writing and executing programs with TSO/E, see *z/OS TSO/E User's Guide* and *z/OS TSO/E Command Reference*.

Access Method Services Tape Library Support

Access method services provides support for tape library functions. The access method services ALTER, CREATE, and DELETE commands, however, should be used only to recover from tape volume catalog errors. Because access method services cannot change the library manager inventory in an automated tape library, ISMF should be used for usual tape library ALTER, CREATE, and DELETE functions.

Summary of Tape Library Support

Access method services supports the following tape library functions:

- Creating, altering, deleting, copying, and listing catalog entries for tape library and tape volume entries
- Merging tape volume entries into other volume catalogs
- Providing support for functions that maintain an up-to-date tape library inventory.

The CATALOG parameter is ignored when specified on any command that affects a tape library entry except for the LISTCAT command.

A tape library entry is the record for a tape library. A tape volume entry is the record for a cartridge tape in a tape library.

Access Method Services Commands for Tape Library Support

Use the following access method service commands to interact with a tape library:

ALTER LIBRARYENTRY

Lets you alter all tape library entry fields except the library name.

ALTER VOLUMEENTRY

Lets you alter all tape volume entry fields except for the tape volser.

CREATE LIBRARYENTRY

Lets you create a tape library entry.

CREATE VOLUMEENTRY

Lets you create a tape volume entry.

DEFINE USERCATALOG

Lets you specify the VOLCATALOG parameter to define a volume catalog. A volume catalog is a catalog that contains only tape library and tape volume entries.

DELETE

Use this command to delete tape library and tape volume entries.

- Specify the LIBRARYENTRY parameter to delete a tape library entry.
- Specify the VOLUMEENTRY parameter to delete a tape volume entry.
- The NOSCRATCH/SCRATCH parameter does not apply to tape library or tape volume entries, because the entries have no VVDS or VTOC entries.
- You can use the PURGE parameter to delete tape volume entries, regardless of expiration dates.
- If you use the FORCE parameter in LIBRARYENTRY, the tape library entry will be deleted. Any tape volume entries associated with the deleted tape library entry remain in the volume catalog. If you do not use FORCE, the tape library entry will be deleted only if it has no associated tape volume entries.

DIAGNOSE

Identifies tape library and tape volume record types. DIAGNOSE checks the cell structure of the volume catalog.

EXPORT/IMPORT

Imports and exports volume catalogs.

LISTCAT

Displays fields that are associated with tape library and tape volume entries.

- Use LIBRARYENTRIES to list tape library entries.
- Choose VOLUMEENTRIES to list tape volume entries.
- Use the CATALOG parameter to retrieve tape volume entries from a specified volume catalog.
- To group tape library and tape volume entries, use the ALL parameter. The HISTORY, VOLUME, and ALLOCATION parameters are not valid and will be ignored.

REPRO MERGECAT

Merges entries from one volume catalog to another. REPRO retrieves tape library or tape volume entries and redefines them in a target volume catalog. You cannot use the LEVEL parameter when merging volume catalogs.

If the character prior to the last character in both VOLCATs is 'V', both VOLCATs are specific. You cannot use MERGECAT of two specifics as this would mix VOLSER names.

For a target VOLCAT that is SPECIFIC (not VGENERAL), you must specify the VOLUMEENTRIES parameter. You do not want all entries merged into a specific catalog.

If the case is VGENERAL to SPECIFIC, the specified entry characters must match the first two third qualifier characters of the target catalog.

You may not use the REPRO command to copy the catalogs of two VGENERALs, when MERGECAT is specified.

REPRO NOMERGECAT

Copies volume catalogs. When you copy a volume catalog to another volume catalog, REPRO verifies that the target is a volume catalog.

For additional information on tape volume catalogs, VOLUMEENTRIES, and LIBRARYENTRIES, refer to *z/OS DFSMS OAM Planning, Installation, and Storage Administration Guide for Tape Libraries*. Also see *z/OS DFSMS Managing Catalogs*.

The VOLUMEENTRIES parameter cannot be specified on a NONMERGECAT. It always copies all entries in the volume catalog and cannot be restricted to a subset.

If NOMERGECAT or the default of NOMERGECAT is specified, the first qualifiers of the VOLCATs must not match while their third qualifiers must match. This allows users to make a copy of a VOLCAT, but ensures that the copy is of the same type. You do not want VA entries to be copied into the VB catalog, for example.

You may only use the REPRO NOMERGECAT command to copy the catalogs of two VGENERALs. To use REPRO NOMERGECAT to copy a VGENERAL to another VGENERAL the first qualifier MUST be different and the third qualifier MUST be the same.

Tape Library Naming Conventions

Tape LibraryTape Library Names

The 1-to-8 character names of tape library entries can include only alphanumerics and the national characters $, @, or #. The first character of the name must be non-numeric and must not be the letter 'V'.

Tape Volume Names

Tape volume names have a 'V' concatenated with a 1-to-6 character tape volser. The tape volser can include only uppercase alphabetics A–Z and numerics 0–9.

Tape Library Date Formats

For all tape library parameters that require date entry, the date is in the form *YYYY-MM-DD,* where:

- *YYYY* is 0000–2155
- *MM* is 01–12
- *DD* is 01–28, 29, 30, or 31

  Specify a day of 32 for the ‘never expire’ date of 1999-12-32.

  Exception: For both ALTER VOLUMEENTRY and CREATE VOLUMEENTRY requests, you can specify an expiration date of 1999-00-00.

Enter all dates with leading zeros when appropriate.

Order of Catalog Use

To select the catalog to be searched or chosen for an entry, use the CATALOG parameter. You can use the CATALOG parameter with several commands such as ALTER, BLDINDEX, DEFINE, DELETE, EXPORT, and LISTCAT. If you use an alias name, the catalog associated with the name is searched or selected.

The multilevel alias facility enhances catalog selection that is based on high-level qualifiers of the data set name. It employs a right to left search of the multiple levels of qualifiers of the data set name for a matching alias name or user catalog name. The alias name or user catalog name with the greatest number of matching qualifiers is selected. For additional information on the multilevel alias facility, see *z/OS DFSMS Managing Catalogs*.

Throughout the following "catalog search" and "catalog selection" sections, the catalog name cannot be specified for SMS-managed data sets unless you have authority from the RACF STGADMIN.IGG.DIRCAT FACILITY class. With this authorization, a data set can be directed to a specific catalog. For more information about this FACILITY class, see *z/OS DFSMSdfp Storage Administration*.

Catalog Search Order for ALTER

1. If a catalog is given in the CATALOG parameter, only that catalog is searched. If the entry is not found, a no-entry-found error is returned.
2. If the entry is identified with a qualified entry name, and it is not generic, and its qualifiers are the same as the name or the alias of a catalog, or if the entry is found, no other catalog is searched.
3. The master catalog is searched. If the entry is not found in any of the indicated catalogs, a no-entry-found error is returned.

Catalog Selection Order for BLDINDEX

This section applies only to users of BLDINDEX who code NOSORTCALL.

1. If a catalog is specified with the CATALOG parameter, that catalog is selected to contain work file entries.
2. If the entry (data set name on the DD statement) is identified with a qualified entry name, and:
   - One or more of its qualifiers is the same as the name or the alias of a catalog, or
   - The first qualifier is the same as the name or the alias of a VSAM user catalog

   then the user catalog so identified is selected to contain the work file entries.
3. The master catalog is selected to contain the work file entries.

Catalog Selection Order for DEFINE

1. If a catalog is defined in the CATALOG parameter, that catalog is selected to contain the to-be-defined entry.
2. When a non-VSAM generation data group (GDG) data set is defined, the catalog containing the GDG base is selected to contain the to-be-defined non-VSAM entry.
3. If no user catalog is specified for the current job step or job, the entry's name is a qualified name, and:
   - One or more of its qualifiers is the same as the name or the alias of a catalog, or
   - The first qualifier is the same as the name or the alias of a VSAM user catalog,

   then the catalog so identified is selected to contain the to-be-defined entry.
4. If no catalog has been identified, either explicitly or implicitly, VSAM defines an object in the master catalog.

Catalog Search Order for DELETE

If this is not a generic delete, the order in which catalogs are searched to locate an entry to be deleted is:

1. If a catalog is given in the CATALOG parameter, only that catalog is searched. If the entry is not found, a no-entry-found error is returned.
2. If the entry is identified with a qualified entry name, and:
   - One or more of its qualifiers is the same as the name or the alias of a catalog,

   If the entry is found, no other catalog is searched.
3. If the entry is not found, the master catalog is searched. If the entry is not found in the master catalog, a no-entry-found error is returned.

If this is a generic delete, the order in which catalogs are searched to locate all applicable entries to be deleted is:

1. If a catalog is given in the CATALOG parameter, only that catalog is searched. If an entry that matches the supplied qualifiers is not found, a no-entry-found error is returned.
2. If the entry is identified with a qualified entry name and if one of the following situations occur, then the catalog search continues with step 4:
   - One or more of its qualifiers is the same as the name of a catalog
   - One or more of its qualifiers is the same as the alias of a catalog
   - The first qualifier is the same as the name of a VSAM user catalog
3. The master catalog is searched.
4. If an entry matching the supplied qualifiers is not found in any of the catalogs searched, a no-entry-found error is returned.

If this is a mask delete, which is having the MASK keyword specified in the DELETE command, the order in which catalogs are searched to locate all applicable entries to be deleted is:

1. If a catalog is given in the CATALOG parameter, only that given catalog is searched. If there are no entries matching the supplied mask filter entry in that catalog, a no-entry-found error is returned.
2. If no catalog is given in the CATALOG parameter, only the master catalog is search. No other catalogs are searched. If there are no entries matching the supplied mask filter entry in the master catalog. A no-entry-found error is returned.

Caution: Unwanted deletions can take place if the catalog is not specified with the CATALOG parameter. Other catalogs are searched, according to the order previously described, and any entries matching the supplied qualifiers are deleted.

For information about generic catalog selection for the DELETE command, see "Generic Catalog Selection for DELETE and LISTCAT" on page 11.

Catalog Selection Order for EXPORT DISCONNECT

1. If a catalog is specified with the CATALOG subparameter, that catalog is selected. If the data set is not found in that catalog, the command will be unsuccessful.
2. If the entry is identified with a qualified entry name and one of the following situations occur, then the catalog search will continue with step 3:
   - One or more of its qualifiers is the same as the name or the alias of a catalog
   - The first qualifier is the same as the name or the alias of a VSAM user catalog
   - The user catalog so identified is searched.
3. Then the master catalog is searched. If the entry is not found in the master catalog, a no-entry-found error is returned.

Catalog Search Order for LISTCAT

When you do not use the ENTRIES parameter, or the command is not run through TSO/E and it is not a generic LISTCAT, the order in which catalogs are searched when entries are to be listed using the LISTCAT command is:

1. If a catalog is specified in the CATALOG parameter, only that catalog is listed.
2. If no user catalog is named in the current job step or job, the master catalog is listed.

If the command is not a generic LISTCAT and the ENTRIES or LEVEL parameter is used, or when the command is run through TSO/E, the order in which catalogs are searched when entries are to be listed using the LISTCAT command is:

1. If a catalog is in the CATALOG parameter, only that catalog is searched. If the entry is not found, a no-entry-found error is returned.
2. If the entry is not found, the entry's name is a qualified name, and:
   - One or more of its qualifiers is the same as the name or the alias of a catalog

   that user catalog is searched. If the entry is found, no other catalog is searched.
3. The master catalog is searched. If the entry is not found, a no-entry-found error is returned.

When the ENTRIES parameter is used and this is a generic LISTCAT, the order in which catalogs are searched when entries are to be listed using the LISTCAT command is:

1. If a catalog is shown in the CATALOG parameter, only that catalog is searched. If an entry is not found that matches the supplied qualifiers, a no-entry-found error is returned.
2. If the entry's name is a qualified name, and:
   - One or more of its qualifiers is the same as the name or the alias of a catalog is searched. The catalog search continues with step 4.
3. The master catalog is searched. If an entry has not been found in any of the catalogs searched that matched the supplied qualifiers, a no-entry-found error is returned.

Generic Catalog Selection for DELETE and LISTCAT

The multilevel alias facility enhances generic catalog selection. If you use generic catalog selection with multilevel aliases, you can select several catalogs if the number of qualification levels of the generic name is less than the maximum your system allows. See *z/OS DFSMS Managing Catalog*s for information about setting multilevel alias levels in the catalog address space. If the number of qualification levels in the data set name is less than the maximum your system allows, and aliases exist that match the generic data set name, then every catalog related to those aliases (including the master catalog) is selected.

The multilevel alias facility and the system-generated name format require special attention, such as:

- During the DEFINE of a VSAM data set, if the specified data/index name does not point to the same catalog as the cluster, an error occurs.
- During the DEFINE of a VSAM cluster or a GDG, if the name of the cluster or GDG matches an existing alias or user catalog, the DEFINE request is denied with a duplicate-name error. This is to prevent the data/index component or a GDS from becoming inaccessible.
- When you add an alias to the catalog, make sure that it does not cause existing data sets to become inaccessible.

With the multilevel alias facility, a non-VSAM data set with the same high-level qualifier as an existing alias of a user catalog can be defined. For more details, see *z/OS DFSMS Managing Catalogs*.

The selection order is based upon alias names encountered that match the generic data set name, not upon the catalogs or the data set names selected. For LISTCAT, therefore, entries appear in the data set within alias entry order.

Should two or more aliases relate to the same catalog, only the first catalog reference is used.

If no catalogs are found in the prior searches, the master catalog is searched.

Examples:

Given that,

Alias A
is related to ICFUCAT1,

Alias A.B
is related to ICFUCAT2,

Alias A.C
is related to ICFUCAT3,

Alias A.C.D
is related to ICFUCAT4,

Alias B
is related to SYSCATLG.V338001 and,

ICFMAST is the master catalog for the system,

1. LISTCAT ENTRY(A.*) selects:

   ICFUCAT1
   ICFUCAT2
   ICFUCAT3
   ICFMAST

   Because the master catalog is selected, the alias entries appear in the listing.
2. LISTCAT ENTRY(B.*) selects:

   SYSCATLG.V338001

The master catalog is not searched.

Understanding the Order of Assigned Data Set Attributes

You can select attributes in more than one way with the DEFINE command. Because more than one value for the same attribute can be given, attributes are selected in the following order of precedence:

1. Explicitly specified attributes
2. Modeled attributes
3. DATA CLASS attributes
4. Access method services command defaults

Model processing is done after automatic class selection (ACS) processing. For this reason, modeled attributes are not available to pass to ACS, and default attribute values can be passed to ACS instead. For example, if you state that recordsize be selected from a modeled data set, the AMS default recordsize of 4089 is passed to ACS instead.

The INDEXED|LINEAR|NONINDEXED|NUMBERED|ZFS parameter is an exception to the attribute selection order. If you do not specify this parameter, the command default (INDEXED) overrides the DATA CLASS attribute.

DATA CLASS define/alter panel has an OVERRIDE SPACE flag which specifies whether DATA CLASS attributes override attributes obtained from other sources (like JCL, AMS control cards, attributes obtained from LIKE=). OVERRIDE SPACE flag has two possible values (YES|NO). The default is 'NO'.

If this flag is set to 'YES', DATA CLASS override will occur. If 'No' is specified, DATA CLASS override not occur. All existing allocations will remain unchanged unless users take deliberate action to modify their DATA CLASS and ACS routines.

The JCL Space subparameters to be overridden (including dynamic allocations such as TSO ALLOCATE) are:

- Space type (CYL,TRK, Block length, or record length plus AVREC)
- Primary Quantity
- Secondary Quantity

- Directory blocks

The IDCAMS DEFINE space parameters to be overridden are:

- Cylinders (Primary, Secondary)
- Tracks (Primary, secondary)
- Kilobytes (Primary, Secondary)
- Megabytes (Primary, Secondary)
- Records (Primary, Secondary)
- Controlintervalsize (CISIZE DATA)
- Freespace (CI-percent CA-percent)

AVGREC, AVG VALUE ,PRIMARY, SECONDARY and DIRECTORY must be explicitly specified. ISMF will return a failure reason code if any one of these fields is not explicitly specified. ISMF will prime all these fields with blanks and the user must explicitly set each field to any valid non-blank value.

If RECORG is specified (i.e set to KS, ES, RR or LS), then CISIZE DATA must be explicitly specified. If RECORG is specified and is set to KS then %FREESPACE CI and %FREESPACE CA must also be explicitly specified. ISMF will prime all these fields with blanks and the user must explicitly set each field to any valid non-blank value based on the rules above. '0' is a valid non-blank value for any of these fields.

Note:

1. PRIMARY SPACE, SECONDARY SPACE, allocation units and directory blocks will all be picked up either from the JCL or from the DATA CLASS to keep with existing logic. The SPACE information in the DATA CLASS must be all inclusive, otherwise jobs may fail or produce unexpected results.
2. The Model’s Volume Count is passed to the ACS routines and will override the Data Class Volume Count.

For example: If the model data set has one volume and no candidate volumes, the new data set will have one volume and no candidate volumes.

To make this new data set a multi volume data set, you would have to add the dynamic volume count to the data class that the new data set picks up or the model will need to have specified candidate volumes.

Chapter 2. Modal Commands

With Access Method Services, you can set up jobs to execute a sequence of modal commands with a single invocation of IDCAMS. Modal command execution depends on the success or failure of prior commands.

In this chapter you will find:

- Modal commands
- Condition codes
- Examples of how to use modal commands.

Modal Commands

You cannot use these commands when Access Method Services is running in Time Sharing Option (TSO).

- IF-THEN-ELSE command sequence, which controls command execution on the basis of condition codes
- NULL command, which causes the program to take no action
- DO-END command sequence, which specifies more than one functional access method services command and its parameters
- SET command, which resets condition codes
- CANCEL command, which ends processing of the current job step
- PARM command, which chooses diagnostic aids and options for printed output.

Commonly used single job step command sequences

A sequence of commands commonly used in a single job step includes DELETE-DEFINE-REPRO or DELETE-DEFINE-BLDINDEX.

- You can specify either a data definition (DD) name or a data set name with these commands.
- When you refer to a DD name, allocation occurs at job step initiation. The allocation can result in a job failure, if a command such as REPRO follows a DELETE-DEFINE sequence that changes the location (volser) of the data set. (Such failures can occur with either SMS-managed data sets or non-SMS-managed data sets.)

Avoiding Potential Command Sequence Failures

To avoid potential failures with a modal command sequence in your IDCAMS job, perform either one of the following tasks:

- Specify the data set name instead of the DD name
- Use a separate job step to perform any sequence of commands (for example, REPRO, IMPORT, BLDINDEX, PRINT, or EXAMINE) that follow a DEFINE command.

IF-THEN-ELSE Command Sequence

The syntax of the IF-THEN-ELSE command sequence, which controls command execution, is:

Command	Parameters
IF	{LASTCC|MAXCC} *operator number* THEN[*command*| DO

© Copyright IBM Corp. 1973, 2021

Command	Parameters
	command set END] [ELSE[*command*| DO *command set* END]]

where:

IF
States that one or more functional commands should run based on a test of a condition code. A SET command sets the condition code, or the condition code reflects the completion status of previous functional commands.

Nested IF commands: When an IF command appears within a THEN or ELSE clause, it is called a nested IF command. See "Using Nested IF Commands: Example 1" on page 17.

- The maximum level of nesting allowed is 10, starting with the first IF.
- Within a nest of IF commands:
- The innermost ELSE clause is associated with the innermost THEN clause,
- The next innermost ELSE clause with the next innermost THEN clause, and so on. (Each ELSE is matched with the nearest preceding unmatched THEN.)
- If there is an IF command that does not require an ELSE clause, follow the THEN clause with a null ELSE clause (ELSE) unless the nesting structure does not require one.

LASTCC
LASTCC specifies that the condition code resulting from the preceding function command be compared, as indicated by the *operator*, to the number that follows the *operator* to determine if the THEN action is to be done.

MAXCC
MAXCC specifies that the maximum condition code value established by any previous function command or by a SET command be compared, as indicated by the *operator*, to the number following the *operator* to determine if the THEN action is to be done.

operator
operator requires a comparison to be made between the variable and *number*. There are six possible comparisons:

Equal to, written as = or EQ
Not equal to, written as ¬= or NE
Greater than, written as > or GT
Less than, written as < or LT
Greater than or equal to, written as >= or GE
Less than or equal to, written as <= or LE

number
number is the decimal integer that the program compares to MAXCC or LASTCC. Access method services initializes both LASTCC and MAXCC to zero upon entry. See "Condition Codes" on page 22 for the meaning of condition codes.

THEN
THEN states that a single command or a group of commands (introduced by DO) is to be run if the comparison is true. THEN can be followed by another IF command.

ELSE
ELSE specifies that a single command or a group of commands (introduced by DO) is to be run if the previous comparison is false. ELSE can be followed by another IF command.

Using Nested IF Commands: Example 1

In this example, nested IF commands are used to determine whether or not a REPRO, DELETE, or PRINT command is run.

```
IF LASTCC > 4  -
  THEN IF MAXCC < 12  -
    THEN REPRO...
    ELSE DELETE...
  ELSE IF LASTCC = 4  -
    THEN
    ELSE PRINT...
```

If LASTCC is greater than 4, MAXCC is tested. If MAXCC is less than 12, the REPRO command is run; if the value of MAXCC is 12 or greater, the DELETE command is run instead. If the value of LASTCC is 4 or less, LASTCC is tested for being exactly 4; if it is, the program takes no action. If LASTCC is less than 4, the program runs the PRINT command.

Using Nested IF Commands: Example 2

In this example, nested IF commands are used to determine whether the program should run a REPRO command or a PRINT command.

```
IF LASTCC > 4  -
  THEN IF MAXCC < 12  -
    THEN REPRO ...
    ELSE
  ELSE IF LASTCC = 4  -
    THEN PRINT ...
```

If LASTCC is greater than 4, and MAXCC is 12 or greater, no functional commands are run. Use the null ELSE command to indicate that the next ELSE is to correspond to the first THEN.

Null Command

The null command is a THEN or ELSE command that is not followed by a command continuation character. If THEN or ELSE is not followed by either a continuation character or by a command in the same record, the THEN or ELSE results in no action. The null command supports an ELSE command that balances an IF-THEN-ELSE command sequence, and allows null THEN commands.

If you want to indicate a null ELSE command, say:

```
ELSE
```

If you want to indicate a null THEN command, say:

```
IF ... THEN
ELSE ...
```

Use the null command to indicate that no action is to be taken if the IF clause is satisfied (a null THEN command) or if the IF clause is not satisfied (a null ELSE command).

DO-END Command Sequence

DO
Requires that the group of commands that follow is to be treated as a single unit. That is, the group of commands run as a result of a single IF command. The END command ends the set of commands. A command following a DO must begin on a new line.

END
Specifies the end of a set of commands initiated by the nearest unended DO. END must be on a line by itself.

Restriction: Do not use continuation characters in the DO-END sequence; they are taken as a null command or cause unpredictable results.

Using the LASTCC Parameter

If the last condition code is 0, the program prints lists a catalog and prints a data set. If the last condition code is greater than 0, the catalog is listed before and after a VERIFY command.

```
IF LASTCC=0
  THEN DO
    LISTCAT
    PRINT INFILE (AJK006)
    END
  ELSE DO
    LISTCAT ENTRY (AJK006) ALL
    VERIFY FILE (AJKJCL6)
    LISTCAT ENTRY (AJK006) ALL
    END
```

SET Command

Use the SET command to change or reset a previously defined condition code. You can end all processing by setting MAXCC or LASTCC to 16. The syntax of the SET command is:

Command	Parameters
SET	{MAXCC|LASTCC}=*number*

where:

SET
States that a condition code value is to be set. A SET command that follows a THEN or ELSE that is not run does not alter LASTCC or MAXCC.

MAXCC
Requires that the value to be reset is the maximum condition code set by a previous functional command. Setting MAXCC does not affect LASTCC.

LASTCC
Specifies that the value to be reset is the condition code set by the immediately preceding functional command.

number
Is the value to be assigned to MAXCC or LASTCC. The maximum value is 16; a greater value is reduced to 16. If the value of LASTCC is greater than MAXCC, MAXCC is set equal to the larger value.

Using the SET command and MAXCC Parameter

In this example, if the maximum condition code is 0, the program lists an entry from a catalog and prints a data set. If the maximum condition code is not 0, set the maximum condition code to 8.

```
IF MAXCC=0
  THEN DO
    LISTCAT  CATALOG (AMASTCAT/MST27) ENT (MN01.B005)
    PRINT INFILE (AJK006)
    END
  ELSE ...
    SET MAXCC=8
```

CANCEL Command

You can use the CANCEL command to end processing of the current job step. When you use the CANCEL command, the remainder of the command stream is not processed, including any part of an unprocessed IF-THEN-ELSE statement or DO-END pair. The step ends with a return code in register 15 equal to the highest condition code encountered before the CANCEL command was run. A termination message is printed indicating that the CANCEL command was issued. The syntax of the CANCEL command is:

Command	Parameters
CANCEL	

It has no parameters.

Using the CANCEL Command

In this example, if the maximum condition code is not 0, the maximum condition code is set to 12 and the step ends with CANCEL.

```
IF MAXCC=0
  THEN DO
    LISTCAT  CATALOG (AMASTCAT/MST27) ENT (MN01.B005)
    PRINT INFILE (AJK006)
    END
  ELSE DO
    SET MAXCC=12
    CANCEL
    END
```

PARM Command

The PARM command specifies processing options to be used during execution. These options remain in effect until changed by another PARM command. You can also use these options in the PARM field of an EXEC statement (in the job control language (JCL)). The syntax of the PARM command is:

Command	Parameters
PARM	[TEST({[TRACE] [AREAS(*areaid*[*areaid*...])] [FULL((*dumpid*[*begin*[*count*]]) [(*dumpid*...)...])]| OFF})] [GRAPHICS(CHAIN(*chain*)|TABLE(*mname*))] [MARGINS(*leftmargin rightmargin*)]

where:

TEST(

{[TRACE]
[AREAS(*areaid*[*areaid*...])]
[FULL((*dumpid*[*begin*[*count*]])
[(*dumpid*...)...])]|
OFF})

Specifies the diagnostic aids that the program should use. After the TEST option has been established, it remains in effect until another PARM command resets it. You should use the TRACE, AREAS, and FULL parameters concurrently. See *z/OS DFSMSdfp Diagnosis* for a description of the IDCAMS diagnostic aids and lists of the dump points and area identifiers.

TRACE
Specifies that the program should print trace tables whenever it encounters a dump point.

AREAS(*areaid*[*areaid*...])
Lists the modules that are to have selected areas of storage that is dumped at their dump points. *areaid* is a 2-character area identifier defined within the implementation.

FULL((*dumpid*[*begin*[*count*]])[(*dumpid*...)...])
States that a full region dump, as well as the trace tables and selected areas, is to be provided at the specified dump points. *dumpid* specifies the 4-character identifier of the dump point.

begin
Is a decimal integer that specifies the iteration through the named dump point at which the dump is to be produced. (The default is 1.)

count
Is a decimal integer that specifies the number of times that the program should produce dumps. (The default is 1.)

If you use the FULL keyword, you must also use an AMSDUMP DD statement; for example:

```
//AMSDUMP DD SYSOUT=A
```

OFF
Stops the testing.

GRAPHICS(CHAIN(*chain*)|TABLE(*mname*))
Indicates the print chain–graphic character set or a special graphics table that the program should use to produce the output.

CHAIN(AN|HN|PN|QN|RN|SN|TN)
Is the graphic character set of the print chains you want to use. The processor uses PN unless the program explicitly directs it to use another set of graphics.

AN
Arrangement A, standard EBCDIC character set, 48 characters

HN
Arrangement H, EBCDIC character set for FORTRAN and COBOL, 48 characters

PN
PL/1 alphanumeric character set

QN
PL/1 preferred alphanumeric character set for scientific applications

RN
Preferred character set for commercial applications of FORTRAN and COBOL

SN
This character set contains lower case and is the preferred character set for text printing

TN
Character set for text printing, 120 characters

TABLE(*mname*)
Is the name of a table you supply. This 256-byte table defines the graphics for each of the 256 possible bit patterns. Any character sent to the printeris translated to the bit pattern found in the

specified table at the position corresponding to its numeric value (0-255). If the print chain does not have a graphic for a byte's bit pattern, the table should specify a period as the output graphic. The table must be stored as a module accessible through the LOAD macro.

MARGINS(*leftmargin rightmargin*)
Changes the margins of input records on which command statements are written. The usual left and right margins are 2 and 72, respectively. If you code MARGINS, the program scans all subsequent input records in accordance with the new margins. You can use this function in conjunction with the comment feature: You can use respecification of margins to cause the /* and */ characters to be omitted from the scan. This causes comments to be treated as commands.

leftmargin
Locates the location of the left margin.

rightmargin
Locates the location of the right margin. The right margin must be greater than the left margin value.

Using the PARM Command: Example 1

In this example, the program produces dumps on the third and fourth time through the dump point ZZCA.

```
//LISTC JOB ...
//STEP1  EXEC PGM=IDCAMS
//AMSDUMP  DD SYSOUT=A
//SYSPRINT DD SYSOUT=A
//SYSIN  DD  *
     PARM -
        TEST -
        (FULL -
            (ZZCA 03 02))
     LISTCAT -
           LEVEL(SYS1) -
           ALL
     PARM -
        TEST(OFF)
/*
```

The JCL statement, AMSDUMP DD, describes the dump data set, and is required when FULL is specified.

The PARM command parameters are:

- TEST indicates diagnostic testing is to be done.
- FULL(ZZCA 03 02) requires that a region dump, as well as the trace tables and selected areas, is to be printed the third and fourth time execution passes through dump point ZZCA.

Using the PARM Command: Example 2

In this example, a dump is produced the first time the program goes through dump points ZZCA or ZZCR:

```
//LISTC JOB ...
//STEP1  EXEC PGM=IDCAMS
//AMSDUMP  DD SYSOUT=A
//SYSPRINT DD SYSOUT=A
//SYSIN  DD  *
     PARM -
        TEST -
        (FULL( -
            (ZZCA 01 01) -
            (ZZCR 01 01)))
     LISTCAT -
           LEVEL(SYS1) -
           ALL
     PARM -
        TEST(OFF)
/*
```

The JCL statement AMSDUMP DD describes the dump data set and is required when FULL is specified.

The parameters are:

- TEST requires diagnostic testing.
- FULL((ZZCA 01 01)(ZZCR 01 01)) states that a region dump, as well as the trace tables and selected areas, is printed the first time through dump points ZZCA and ZZCR.

Using the PARM Command: Example 3

In this example, selected areas of storage are displayed for all dump points starting with ZZ or LC. An AMSDUMP DD card is not required in this example.

```
//LISTC JOB ...
//STEP1  EXEC PGM=IDCAMS
//SYSPRINT DD SYSOUT=A
//SYSIN  DD  *
     PARM -
        TEST -
        (AREAS -
              (ZZ LC))
     LISTCAT -
           LEVEL(SYS1) -
           ALL
     PARM -
        TEST(OFF)
/*
```

The PARM command parameters are:

- TEST indicates diagnostic testing is to be done.
- AREAS(ZZ LC) specifies that trace tables and selected areas of storage are printed. This information is used by service personnel for diagnostic purposes.

Condition Codes

The condition codes that are tested in the IF-THEN-ELSE command sequence are:

0
The function ran as directed and expected. Some informational messages can be issued.

4
A problem occurred in executing the complete function, but it continued. The continuation might not provide you with exactly what you wanted, but no permanent harm was done. A warning message appears. An example is:

```
The system was unable to locate an entry in a LISTCAT
command.
```

8
A requested function was completed, but major specifications were unavoidably bypassed. For example, an entry to be deleted or altered could not be found in the catalog, or a duplicate name was found while an entry was being defined and the define action ended.

12
The program could not perform requested function. The program sets this condition code as a result of a logical error. A logical error condition exists when inconsistent parameters are given, when required parameters are missing, or when a value for key length, record size, or buffer space is too small or too large. More information on logical errors that occur during VSAM record processing is in *z/OS DFSMS Macro Instructions for Data Sets*.

16
A severe error occurred that erased the remainder of the command stream. This condition code results from one of the following:

- The program cannot open a system output data set. (For example, a SYSPRINT DD statement was missing.)

- An irrecoverable error occurred in a system data set
- An access method services encountered improper IF-THEN-ELSE command sequences.

Condition codes that are tested in the IF-THEN-ELSE command sequence or set by the SET command cannot be passed from one job step to the next. However, the maximum condition code value established by any previous functional command or SET command is passed to the operating system when the access method services processor returns control to the system.

Common Continuation Errors in Coding Modal Commands

Use continuation rules cautiously when modal commands appear in the input stream. (See "How to continue commands and parameters" on page xxvii.) The following examples show common continuation errors:

```
IF LASTCC = 0 -
  THEN
    LISTCAT
```

A continuation mark (hyphen) is missing after the THEN keyword. A null command is assumed after the THEN keyword, and the LISTCAT command is unconditionally run.

```
IF LASTCC = 0 -
  THEN -
    REPRO ...
    /*ALTERNATE PATH*/
  ELSE -
    PRINT ...
```

Because no continuation mark (hyphen) follows the comment, the program assumes a null command. The ELSE keyword will not match the THEN keyword. Note the correct use of the continuation marks on the other records.

```
IF LASTCC = 0 -
  THEN -
    REPRO ...
  ELSE -

    PRINT ...
```

Because a blank line with no continuation mark (hyphen) follows the ELSE keyword, the ELSE becomes null and the PRINT command is unconditionally run.

```
PARM TEST ( - /*COMMENT*/
TRACE)
```

The program does not continue the PARM command onto the second record, because characters other than blanks appear between the continuation mark (hyphen) and the end of the record.

```
PARM TEST ( TRA+
/*FIELD CONTINUATION*/
CE)
```

The processor finds the end of the PARM command after the second record, because no continuation was indicated. The processor rejects command.

Chapter 3. Functional Command Syntax

This chapter provides an overview of the access method services functional commands for catalogs and for objects that are cataloged in them. The following chapters discuss each command in detail.

Examples of each command appear at the end of each chapter.

See "Notational conventions" on page xxi for an explanation of the symbols used in the command syntax.
See "How to code subparameters" on page xxiv for coding conventions that apply.

Functional Command Syntax Summary

This chapter provides reference information about the following functional commands.

Table 1. Summary of AMS commands

Command	Functions
ALLOCATE	Allocates Virtual Storage Access Method (VSAM) and non-VSAM data sets.
ALTER	Alters attributes of data sets, catalogs, tape library entries, and tape volume entries that have already been defined.
BLDINDEX	Builds alternate indexes for existing data sets.
CREATE	Creates tape library entries and tape volume entries.
DCOLLECT	Collects data set, volume usage, and migration utility information.
DEFINE	Defines the following objects: • **ALIAS** Defines an alternate name for a non-VSAM data set or a user catalog. • **ALTERNATEINDEX** defines an alternate index. • **CLUSTER** Defines a cluster for an entry-sequenced, key-sequenced, linear, or relative record data set. • **GENERATIONDATAGROUP** Defines a catalog entry for a generation data group. • **NONVSAM** Defines a catalog entry for a non-VSAM data set. • **PAGESPACE** Defines an entry for a page space data set. • **PATH** Defines a path directly over a base cluster or over an alternate index and its related base cluster. • **USERCATALOG|MASTERCATALOG** Defines a user catalog.
DELETE	Deletes catalogs, VSAM data sets, and non-VSAM data sets.

© Copyright IBM Corp. 1973, 2021

Table 1. Summary of AMS commands (continued)	
Command	**Functions**
DIAGNOSE	Scans a basic catalog structure (BCS) or a VSAM volume data set (VVDS) to validate the data structures and detect structure errors.
EXAMINE	Analyzes and reports the structural consistency of either an index or data component of a key-sequence data set cluster.
EXPORT	Disconnects user catalogs, and exports VSAM data sets and catalogs.
EXPORT DISCONNECT	Disconnects a user catalog.
IMPORT	Connects user catalogs, and imports VSAM data sets and catalogs.
IMPORT CONNECT	Connects a user catalog or a volume catalog.
LISTCAT	Lists catalog entries.
PRINT	Prints VSAM data sets, non-VSAM data sets, and catalogs.
REPRO	Performs the following functions: • Copies VSAM and non-VSAM data sets, user catalogs, master catalogs, and volume catalogs • Splits catalog entries between two catalogs • Merges catalog entries into another user or master catalog • Merges tape library catalog entries from one volume catalog into another volume catalog.
SHCDS	Lists SMSVSAM recovery associated with subsystems spheres and controls that recovery. This command works both in batch and in the TSO/E foreground. Includes subcommands that allow you to perform the following tasks. • List information kept by the SMSVSAM server and the catalog as related to VSAM RLS or DFSMStvs. • Take action on work that was shunted. • Control a manual forward recovery. • Run critical non-RLS batch window work if necessary. • Perform a subsystem cold start.

Table 1. Summary of AMS commands (continued)	
Command	**Functions**
VERIFY	Performs the following functions: • Causes a catalog to correctly reflect the end of a data set after an error occurred while closing a VSAM data set. The error might have caused the catalog to be incorrect. • Causes VSAM Record Management VERIFY to back out or complete any interrupted CA reclaim in addition to regular IDCAMS VERIFY functions. • When run with a preceding EXAMINE, causes VERIFY to attempt to fix any errors it can from the information passed in by EXAMINE. Only the HURBA will be corrected if there is concurrent access on the data set when VERIFY is being run. When run with a preceding EXAMINE, causes VERIFY to attempt to fix any errors it can from the information passed in by EXAMINE. Only the HURBA will be corrected if there is concurrent access on the data set when VERIFY is being run.

Chapter 14. DEFINE CLUSTER

Using Access Method Services, you can set up jobs to execute a sequence of commands with a single invocation of IDCAMS. Modal command execution is based on the success or failure of prior commands.

Use DEFINE CLUSTER to define attributes for the cluster as a whole and for the components of the cluster. The general syntax of the DEFINE CLUSTER command is:

- **DEFINE CLUSTER** (parameters) -

  [**DATA**(parameters)] -
  [**INDEX**(parameters)] -
  [**CATALOG**(subparameters)]

See "Understanding the Order of Assigned Data Set Attributes" on page 12 for information about the order of precedence of DEFINE command attributes.

Command	Parameters
DEFINE	CLUSTER (NAME(*entryname*) {CYLINDERS(*primary*[*secondary*])| KILOBYTES(*primary*[*secondary*])| MEGABYTES(*primary*[*secondary*])| RECORDS(*primary*[*secondary*])| TRACKS(*primary*[*secondary*])} VOLUMES(*volser*[*volser...*]) [ACCOUNT(*account–info*)] [BUFFERSPACE(*size*)] [BWO(TYPECICS|TYPEIMS|NO)] [CONTROLINTERVALSIZE(*size*)] [DATABASE(BSON|JSON)KEYNAME|KEYNAMEU(explicit-primary-key-value)] [DATACLASS(*class*)] [EATTR(NO|OPT)] [ERASE|NOERASE] [EXCEPTIONEXIT(*entrypoint*)] [FILE(*ddname*)] [FREESPACE(*CI-percent*[*CA-percent*]|0 0)] [FRLOG(ALL|NONE|REDO|UNDO)] [INDEXED|LINEAR|NONINDEXED|NUMBERED|ZFS] [KEYLABEL(*keyvalue*)] [KEYS(*length offset*| 64 0] [LOG(NONE|UNDO|ALL)]

© Copyright IBM Corp. 1973, 2021

Command	Parameters
	[LOGREPLICATE|NOLOGREPLICATE]
	[LOGSTREAMID(*logstream*)]
	[MANAGEMENTCLASS(*class*)]
	[MODEL(*entryname*[*catname*])]
	[OWNER(*ownerid*)]
	[RECATALOG|NORECATALOG]
	[RECORDSIZE(*average maximum*)]
	[REUSE|NOREUSE]
	[|RLSQUIESCE]RLSENABLE
	[SHAREOPTIONS(*crossregion*[*crosssystem*]|1 3)]
	[SPANNED|NONSPANNED]
	[SPEED|RECOVERY]
	[STORAGECLASS(*class*)]
	[TO(*date*)|FOR(*days*)]
	[WRITECHECK|NOWRITECHECK])
	[DATA(
	{CYLINDERS(*primary*[*secondary*])|
	KILOBYTES(*primary*[*secondary*])|
	MEGABYTES(*primary*[*secondary*])|
	RECORDS(*primary*[*secondary*])|
	TRACKS(*primary*[*secondary*])}
	[VOLUMES(*volser*[*volser...*])]
	[BUFFERSPACE(*size*)]
	[CONTROLINTERVALSIZE(*size*)]
	[ERASE|NOERASE]
	[EXCEPTIONEXIT(*entrypoint*)]
	[FILE(*ddname*)]
	[FREESPACE(*CI-percent*[*CA-percent*])]
	[KEYS(*length offset*)]
	[MODEL(*entryname*[*catname*])]
	[NAME(*entryname*)]
	[OWNER(*ownerid*)]
	[RECORDSIZE(*average maximum*)]
	[REUSE|NOREUSE]
	[SHAREOPTIONS(*crossregion*[*crosssystem*])]
	[SPANNED|NONSPANNED]
	[SPEED|RECOVERY]

Command	Parameters
	[WRITECHECK|NOWRITECHECK])]
	[INDEX(
	{CYLINDERS(*primary*[*secondary*])|
	KILOBYTES(*primary*[*secondary*])|
	MEGABYTES(*primary*[*secondary*])|
	RECORDS(*primary*[*secondary*])|
	TRACKS(*primary*[*secondary*])}
	[VOLUMES(*volser*[*volser*...])]
	[CONTROLINTERVALSIZE(*size*)]
	[EXCEPTIONEXIT(*entrypoint*)]
	[FILE(*ddname*)]
	[MODEL(*entryname*[*catname*])]
	[NAME(*entryname*)]
	[OWNER(*ownerid*)]
	[REUSE|NOREUSE]
	[SHAREOPTIONS(*crossregion*[*crosssystem*])]
	[WRITECHECK|NOWRITECHECK])]
	[CATALOG(*catname*)]

DEFINE Abbreviation: DEF

A sequence of commands commonly used in a single job step includes DELETE––DEFINE––REPRO or DELETE––DEFINE––BLDINDEX. You can specify either a DD name or a data set name with these commands. When you refer to a DD name, however, allocation occurs at job step initiation. This could result in a job failure if a command such as REPRO follows a DELETE––DEFINE sequence that changes the location (volser) of the data set. A failure can occur with either SMS-managed data sets.

A VVDS cannot be modeled in an IDCAMS DEFINE command. Also, SMS forces VVDS defines to be non-SMS-managed, so assignments of EATTR values by either a DATACLAS or a MODEL routine are not allowed with the DEFINE of a VVDS. The EATTR assignments are only allowed explicitly in the DEFINE CLUSTER command. There is no other source of assigning the EATTR value for a VVDS DEFINE.



Attention: IBM does not recommend doing a delete and define for the same data set inside a single step, or even in the same job, with DFSMStvs. The delete throws up an exclusive ENQ that is not released until the job terminates. This is not a problem most of the time because the job owns the ENQ, so it has no trouble allocating the data set. If, however, the unit of recovery ended up in backout for any reason, DFSMStvs would be unable to allocate the data set, and the UR would be shunted.

To avoid potential failures with a modal command sequence in your IDCAMS job:

- Specify the data set name instead of the DD name; or
- Use a separate job step to perform any sequence of commands (for example, REPRO, IMPORT, BLDINDEX, PRINT, or EXAMINE) that follow a DEFINE command.

Recommendation: DB2 uses Access Method Services DEFINE CLUSTER for STOGROUP defined data sets. This can result in performance problems for partitioned table spaces if multiple partitions are defined on the same volume. DB2 uses software striping on partitioned table spaces to improve performance of

sequential queries. The throughput is then gated by the data delivery capability of each volume. Since each partition is a separate data set, this problem can be avoided by allocating all the partitions in a single JCL step in an IEFBR14 (not IDCAMS) job. See *z/OS DFSMS Using Data Sets* for details. Allocating all the partitions in this manner works if there are adequate number of volumes available with the requested space quantity in a single SMS storage group to satisfy all the partitions.

Restriction: If you specify IMBED, KEYRANGE, ORDERED, or REPLICATE it will be ignored.

DEFINE CLUSTER Parameters

The DEFINE CLUSTER command uses the following parameters.

Required Parameters

CLUSTER
CLUSTER defines or recatalogs a cluster or cluster entry.

The CLUSTER keyword is followed by the parameters specified for the cluster as a whole. These parameters are enclosed in parentheses and, optionally, are followed by parameters given separately for the DATA and INDEX components.

Abbreviation: CL

NAME(*entryname*)
Defines the cluster's entryname or the name of each of its components. The entryname used for the cluster as a whole is not propagated to the cluster's components.

For SMS and non-SMS-managed clusters, the component names must resolve to the same catalog as the data set's cluster name.

You can define a separate entryname for the cluster, its data component, and its index component. If no name is specified for the data and index component, a name is generated. When the cluster, data component, and index component are individually named, each can be addressed. For information on system generated names, see *z/OS DFSMS Using Data Sets*.

When defining a VSAM volume data set (VVDS), the entryname for the cluster or the data component must be in the form SYS1.VVDS.V*volser,* where volser is the volume serial number specified by the VOLUMES parameter. The default primary and secondary allocation is 10 tracks. VVDSs cannot be defined in cylinder-managed space. For information on defining a VVDS see *z/OS DFSMS Managing Catalogs*.

CYLINDERS(*primary*[*secondary*])

KILOBYTES(*primary*[*secondary*]) |

MEGABYTES(*primary*[*secondary*]) |

RECORDS(*primary*[*secondary*]) |

TRACKS(*primary*[*secondary*]) |
The amount of space in cylinders, kilobytes, megabytes, records, or tracks allocated to the cluster from the volume's available space. A kilobyte or megabyte allocation resolves to either tracks or cylinders; record allocation resolves to tracks.

If the override indicator in Data Class is set ON, the user specified amount of space in DEFINE command will be overridden by the space values specified in data Class. This enforces the installation standard of the system.

Requests for space are directed to DADSM and result in a format-1 or format-8 DSCB for all entries.

If the cluster is not SMS-managed, you must use the amount of space allocated, either through this parameter, or through the DATACLASS, MODEL, or RECATALOG parameters. This parameter is optional

if the cluster is managed by SMS. If it is used, it overrides the DATACLASS space specification. If it is not used, it can be modeled or defaulted by SMS. If it cannot be determined, the DEFINE is unsuccessful.

If you select KILOBYTES or MEGABYTES, the amount of space allocated is the minimum number of tracks or cylinders required to contain the specified number of kilobytes or megabytes. The maximum space can be specified with unit of KILOBYTES or MEGABYTES is 16,777,215. If the amount requested exceeds this value, you should specify a larger allocation unit.

If you select RECORDS, the amount of space allocated is the minimum number of tracks that are required to contain the given number of records. The maximum number of records is 16,777,215. If RECORDS is specified for a linear data set, space is allocated with the number of control intervals equal to the number of records.

The maximum TRACKS or CYLINDERS value that can be specified on the DEFINE CLUSTERS command is X'FFFFFF' or 16777215, because of the 3 byte space parameter fields.

Recommendation: To maintain device independence, do not use the TRACKS or CYLINDERS parameters. If you use them for an SMS-managed data set, space is allocated on the volumes selected by SMS in units equivalent to the device default geometry. If there is an allocation failure due to lack of space, SMS retries allocation with a reduced space quantity. However, any retry, including reduced space quantity, is only attempted if Space Constraint Relief ⇒ Y is specified. SMS also removes other limitations if the data class allows space constraint relief.

Regardless of the allocation type, the calculation of the CA (control area) size is based on the smaller of the two allocation quantities (primary or secondary) in the DEFINE command. A CA is never greater than a single cylinder, it might be less (that is, some number of tracks), depending on the allocation amount and type used. When tracks or records are used, the space allocation unit (the CA size) can be adjusted to one cylinder. This adjustment is made if the calculated CA size contains more tracks than exist in a single cylinder of the device being used. The CA area size assigned by VSAM is the smallest of:

- One cylinder
- The primary space quantity
- The secondary space quantity

If the CA size assigned is not evenly divisible into either the primary or secondary space quantity, VSAM increases that space to a value evenly divisible by the CA size. If you are defining an extended format data set, you should review "Defining an Extended Format Key-Sequenced Data Set" in *z/OS DFSMS Using Data Sets* for information about additional space requirements.

DEFINE RECORDS allocates sufficient space to the specified number of records, but factors unknown at define time (such as key compression or method of loading records) can result in inefficient use of the space allocated. This might prevent every data CA from being completely used, and you might be unable to load the specified number of records without requiring secondary allocation.

When multiple volumes are used for a data set, these rules and conditions apply:

- The first volume is defined as the prime volume. The initial allocation of a data set is on the prime volume. The remaining volumes are defined as candidate volumes.
- A data set's primary space allocation (defined for each data set) is the amount of space initially allocated on both the prime volume and on any candidate volumes the data set extends to. If the data set is striped then the space is allocated for each stripe. To allocate a data set of a particular size divide the space by the number of stripes. Note the size may be rounded up or down, so check using LISTCAT ENT(...) ALLOC.
- A data set's secondary space allocation (if it is defined) is the space allocated when the primary space is filled and the data set needs additional space on the same volume.
- If a data set extends to a candidate volume, the amount of space initially allocated on the candidate volume is the primary space allocation. If the data set extends beyond the primary allocation on the candidate volume, then the amount of space allocated is the secondary space allocation.

- With a DEFINE request, the primary space allocation must be fulfilled in five DASD extents unless the Space Constraint Relief option is specified in the associated SMS data class.

  However, the request is not successful if you do not fulfill each secondary space allocation in five DASD extents.

  A DASD extent is the allocation of one available area of contiguous space on a volume. For example, if a data set's primary space allocation is 100 cylinders, you must allocate a maximum of five DASD extents that add up to 100 cylinders.

Secondary amounts can be allocated on all volumes available to contain parts of the cluster regardless of the key ranges.

You can specify the amount of space as a parameter of CLUSTER, as a parameter of DATA, or as a parameter of both. When a key-sequenced cluster is being defined, and the space is a parameter of:

- CLUSTER, the amount is divided between the data and index components. The division algorithm is a function of control interval size, record size, device type, and other data set attributes.

  If the division results in an allocation for the data component that is not an integral multiple of the required control area size, the data component's allocation is rounded up to the next higher control area multiple. This rounding can result in a larger total allocation for your cluster.
- DATA, the entire amount specified is allocated to the data component. An additional amount of space, depending on control interval size, record size, device type, and other data set attributes, is allocated to the index component.

  Note: If not specified, SMS estimates the size of the index component to be 10% of the data component. Ensure that enough space is present on the volume(s) to account for this estimation.

To determine the exact amount of space allocated to each component, list the cluster's catalog entry, using the LISTCAT command.

The primary and each secondary allocation must be able to be satisfied in five DASD extents; otherwise, your DEFINE or data set extension is unsuccessful. Starting z/OS V1R13, if the primary or secondary space allocation is 16777215 (X'FFFFFF') tracks or cylinders, the value is decreased by 1 CA worth. This is because the AMS LISTCAT command does not recognize all F's as a valid value and prints a zero instead.

primary
Allocates the initial amount of space to the cluster. This amount is also used for extending to additional volumes with variations depending on the type of data set and its attributes. See Primary and Secondary Space Allocation without the Guaranteed Space Attribute in *z/OS DFSMS Using Data Sets* for more information.

secondary
Allocates an amount of space each time the cluster extends, as a secondary extent. You can use this secondary allocation to add space for the data or index components of the cluster. If a secondary quantity of 0 is specified, no secondary extents will be taken; however, the data set may still be eligible for primary extents. See Primary and Secondary Space Allocation without the Guaranteed Space Attribute in *z/OS DFSMS Using Data Sets* for more information.

VOLUMES(*volser*[*volser*...])
Specifies the volumes on which a cluster's components are to have space. If you do not use the MODEL parameter, or if the cluster is not SMS-managed, VOLUMES must be used either as a parameter of CLUSTER, or as a parameter of both DATA and INDEX.

VOLUMES can be specified or modeled for a data set that is to be SMS-managed; know that the volumes specified might not be used and result in an error. See *z/OS DFSMSdfp Storage Administration* for information about SMS volume selection.

Note that extending a data set to candidate volumes is different for non-SMS, SMS and SMS guaranteed space VSAM data sets:

- VSAM data sets allocated to volumes not under the control of SMS, also known as non SMS VSAM data sets, have specific candidate volumes. When a prime volume is full, the next specific candidate

volume in the list in the catalog entry is used. Should space be unavailable on that volume, the next volume in the list is used. The process is continued until either the allocation is satisfied or the candidate list is exhausted. Thus, candidate volumes can be intermingled with prime volumes as the list is not re-sorted after a volume is used. Should sufficient space become available on a candidate volume that was previously full, that candidate volume will be used.

- SMS-managed VSAM data sets that are not guaranteed space have nonspecific candidate volumes in the catalog entry. When a prime volume is full, SMS selects a volume that has sufficient space and causes the VOLSER of that volume to be placed as the next prime volume in the volume list in the catalog entry.
- SMS-managed VSAM data sets that are guaranteed space can have specific candidate volumes with pre-allocated space. These volumes are called candidates with space volumes. The pre-allocated space for the candidate with space volumes is obtained when the VSAM data set is defined. If the space cannot be pre-allocated, the define for the VSAM data set fails. When a prime volume is full, the next candidate with space volume in the list is used. Since the space on the volume is already allocated, only the resetting of RBA values is necessary to make use of the space and only the pre-allocated space is used at that time. Specific or nonspecific candidate volume names (VOLSERs) can be added to SMS-managed guaranteed space VSAM data sets after the data sets is defined. These candidates follow the candidates with space in the volume list in the catalog entry and are only used after the guaranteed space candidates have all been used. These candidates are used in the order of the volume list in the catalog entry regardless of whether they have specific or nonspecific VOLSERs. The specific VOLSERs are honored if there is space on the volume; if there is not enough space, the extend fails. Nonspecific candidates then follow the same process as SMS-managed VSAM data sets without guaranteed space. Since SMS chooses a volume that has space, the extend will always get the space unless the entire storage pool is full. Because nonspecific candidates have a greater likelihood of succeeding for an extend, it is preferable to have nonspecific volumes.

Letting SMS select the volume from the storage group reduces the chances of allocation errors caused by insufficient space. If the data set is SMS-managed with guaranteed space, SMS places the primary quantity on all the volumes with sufficient space for later extensions. If the SMS-managed data set does not have guaranteed space or is a key range data set, primary space is allocated only on the first volume. For SMS-managed VSAM data sets, the primary space might be allocated on a different volume from the one you specified.

You can let SMS choose the volumes for SMS-managed data sets by coding an * for the volser with the VOLUMES parameter. If both user-specified and SMS-specified volumes are requested, the user-specified volser must be input first in the command syntax. The default is one volume.

For SMS-managed and non-SMS-managed data sets, you can specify up to 59 volume serial numbers. If the combined number of volumes for a cluster and its associated alternate indexes exceeds 59, unpredictable results can occur.

If the data and index components are to reside on different device types, you must specify VOLUMES as a parameter of both DATA and INDEX. If more than one volume is listed with a single VOLUMES parameter, the volumes must be of the same device type.

For SMS-managed data sets, if you want the data and index components to be on separate volumes for non-guaranteed space storage class requests, code two different dummy names in the VOLUME parameter for each component. If there are not enough volumes in the storage group to satisfy this requirement, the allocation will fail.

If a guaranteed space storage class is assigned to the data sets (cluster) and volume serial numbers are used, space is allocated on all specified volumes if the following conditions are met:

- All defined volumes are in the same storage group.
- The storage group to which these volumes belong is in the list of storage groups selected by the ACS routines for this allocation.
- The data set is not a key range data set.

The volume serial number is repeated in the list only if the KEYRANGE parameter is used. You can use this to have more than one key range on the same volume. Repetition is valid when duplicate occurrences are used for the primary allocation of some key range.

If a VVDS is being defined, only one volume can be specified and that volume serial number must be reflected in the name indicated in the NAME parameter.

If you define single volume zFS VSAM linear data sets (LDSs) on a DEFINE RECATALOG command, the VOLUMES parameter can have a special form referred to as indirect volume serial. The indirect volser must point to the original volume that the zFS resides on. This results in the system dynamically resolving the volume serial to the system residence (or its logical extension) serial number. It can only be used with the RECATALOG parameter. It can be used for both SMS and non-SMS managed zFS data sets on a DEFINE RECATALOG of the data set. See the VOLUMES parameter in "DEFINE NONVSAM Parameters" on page 184 for information on how to setup an indirect volume serial.

When you clone a zFS, use the COPY command with the PHYSINDYNAM (PIDY) parameter for the following reasons:

- Using PHYSINDYNAM means you do not have to create a catalog entry, but creates a catalog entry in the catalog for you. The catalog entry is required by other methods.
- Using PHYSINDYNAM (PIDY) lets you use the same name for your original and copied zFS.

For more information on how to clone a zFS, see Making a copy of your system software (cloning) in *z/OS Planning for Installation*.

The VOLUMES parameter interacts with other DEFINE CLUSTER parameters. Ensure that the volume you give for the cluster is consistent with the cluster's other attributes:

- CYLINDERS, KILOBYTES, MEGABYTES, RECORDS, TRACKS: The volume must contain enough unallocated space to satisfy the component's primary space requirement.
- FILE: The volume information supplied with the DD statement pointed to by FILE must be consistent with the information specified for the cluster and its components.

Abbreviation: CYL, KB, MB, REC, TRK

Abbreviation: VOL

Optional Parameters

ACCOUNT(*account_info*)
Defines up to 32 bytes of accounting information and user data for the data set. It *must* be between 1 and 32 bytes, otherwise you will receive an error message.

account_info
Is only supported for SMS-managed VSAM and non-VSAM data sets. It is only used for the data set level (not member level) of PDSE/PDS.

Abbreviation: ACCT

BUFFERSPACE(*size*)
Specifies the minimum space for buffers. If BUFFERSPACE is not coded, VSAM attempts to get enough space to contain two data component control intervals and, if the data is key-sequenced, one index component control interval.

If the data set being defined is a KSDS, and the BUFFERSPACE specified is not large enough to contain two data and one index CIs, VSAM increases the specified buffer space and completes the define. VSAM may also increase index CISIZE and, if necessary, increase the buffer space to accommodate the larger index CISIZE.

size
The space for buffers. *Size* can be given in decimal (n), hexadecimal (X'n'), or binary (B'n') form.

The BUFFERSPACE setting is ignored when the data set is opened for VSAM RLS or DFSMStvs mode.

Note: The limitations of the buffer space value on how many buffers will be allocated is based on storage available in your region, and other parameters or attributes of the data set.

Abbreviations: BUFSP or BUFSPC

BWO(TYPECICS|TYPEIMS|NO)
Use this parameter if backup-while-open (BWO) is allowed for the VSAM sphere. BWO applies only to SMS data sets and cannot be used with TYPE(LINEAR).

If BWO is specified in the SMS data class, the value that is defined is used as the data set definition, unless it has been previously defined with an explicitly specified or modeled DEFINE attribute.

TYPECICS
Use TYPECICS to specify BWO in a CICS or DFSMStvs environment. For RLS processing, this activates BWO processing for CICS, or DFSMStvs, or both. For non-RLS processing, CICS determines whether to use this specification or the specification in the CICS FCT.

Exception: If CICS determines that it will use the specification in the CICS FCT, the specification might override the TYPECICS or NO parameters.

Abbreviation: TYPEC

TYPEIMS
Use to enable BWO processing for IMS data sets.

Abbreviation:TYPEI

NO
Use this when BWO does not apply to the cluster.

Exception: If CICS determines that it will use definitions in the CICS FCT, the TYPECICS or NO parameters might be overwritten.

CATALOG(*catname*)
Identifies the catalog in which the cluster is to be defined. See “Catalog Selection Order for DEFINE” on page 9 for the order in which catalogs are selected.

To specify catalog names for SMS-managed data sets, you must have authority to the RACF STGADMIN.IGG.DIRCAT FACILITY class. See “Storage Management Subsystem (SMS) Considerations” on page 2 for more information.

catname/alias
Names the catalog or an alias that can be resolved to a catalog. For example, if alias ABCD relates to catalog SYS1.USERCAT, then specifying either ABCD or SYS1.USERCAT will cause the cluster to be defined in SYS1.USERCAT.

Abbreviation: CAT

If the catalog's volume is physically mounted, it is dynamically allocated. Mount the volume as permanently resident or reserved.

If the catalog's volume is physically mounted, it is dynamically allocated. The volume must be mounted as permanently resident or reserved.

Abbreviation: CAT

CONTROLINTERVALSIZE(*size*)
The size of the control interval for the cluster or component.

For linear data sets, the specified value in bytes is rounded up to a 4K multiple, up to a maximum of 32K. If the size is not specified, the value specified in the data class that is assigned to the data set is used. Otherwise a default value of 4K is used.

If CONTROLINTERVALSIZE is given on the cluster level, it propagates to the component level at which no CONTROLINTERVALSIZE has been specified.

The size of the control interval depends on the maximum size of the data records and the amount of buffer space you provide.

LSR/GSR buffering technique users can ensure buffer pool selection by explicitly defining data and index control interval sizes.

If CONTROLINTERVALSIZE is not coded, VSAM determines the size of control intervals. VSAM selects a control interval size for the data component that will optimize direct access storage usage. It will then select an index control interval size based on the number of data control intervals in the data control area. If the override indicator in Data Class is set ON, the user specified CISIZE in DEFINE command will be overridden by the CISIZE in the DATACLAS.

size
Indicates a cluster's data and index component size..

If SPANNED is not used, the size of a data control interval must be at least 7 bytes larger than the maximum record length.

If the control interval specified is less than maximum record length plus a 7-byte overhead, VSAM increases the data control interval size to contain the maximum record length plus the needed overhead.

If SPANNED is specified, the control interval size can be less than the maximum record length. You can select a size from 512 to 8K in increments of 512, or from 8K to 32K in increments of 2K. When you choose a size that is not a multiple of 512 or 2048, VSAM chooses the next higher multiple. For a linear data set, the size specified is rounded up to 4096 if specified as 4096 or less. It is rounded to the next higher multiple of 4096 if specified as greater than 4096.

The size of the index control interval is the number of data control intervals in a data control area that need indexing at the sequence set level of the index component. The size of each entry depends on an average compression value for a user key. The keys will compress to 1/3 of the length of the actual key value. In some cases, the general compressed key length on which the algorithm is based will be affected by the actual values and ordering of the user key. The result is that each entry can occupy more space in the index record than that provided. This may result in additional control area splits and in all cases, wasted space in the data set. If after loading the data sets, this condition exists; noted by more than anticipated space to store the data set on the direct access device. You should increase the index control interval size. The size can be increased incrementally until it is felt that this condition no longer exists. The guideline formula documented in the past is as follows:

(KEYLEN/2) * DATA CI/CA less than or equal to INDEX CISIZE.

You should be aware that this is only a guideline and does not take into account the actual algorithm for determining the index control interval size requirement. However, the 2:1 compression of key length in the above formula provides some additional overhead over the actual 3:1 formula used during the actual algorithm. Using the above formula can result in an index control interval size that is too large. This may increase I/O transfer time for each index component record, or it may be too small to address the condition described above.

Refer to "Optimizing VSAM Performance" in *z/OS DFSMS Using Data Sets* for a discussion of control interval size and physical block size.

Abbreviations: CISZ or CNVSZ

DATABASE(BSON|JSON)KEYNAME|KEYNAMEU(explicit-primary-key-value)
Identifies the data format of a NoSQL VSAMDB database being defined, as BSON or JSON. If DATABASE is not specified, the default is non-database.

KEYNAME|KEYNAMEU(explicit-primary-key-name) optionally specifies the explicit and unique primary-key name, which is then used by VSAM RLS to obtain the value from the key name:key value pair to be used as the actual primary key value. The primary key name has a maximum 255-bytes, while the key value can be of any length. VSAMDB will hash the key value into a random and unique 128 byte hashed value followed by a 4-byte block number, together making the internal 132-byte “derived” key that identifies the document.

Explicit key names must have unique key values, for example, '_id', 'Acct#', 'Email', etc. KEYNAME and KEYNAMU are valid only when DATABASE is specified; they are ignored if DATABASE is not specified. If KEYS and KEYNAME and KEYNAMEU are specified together, the DEFINE CLUSTER fails.

KEYNAME means the explicit key name is not in UTF-8 character format, and VSAMDB AMS converts it to UTF-8 format before storing it in the catalog for internal processing. KEYNAMEU means the specified key name is already in UTF-8 format.

The primary key name can also consist of a single- or multi-level “multikey” key, where for BSON files the levels are concatenated with a dot notation “.”, or for JSON with a \. Refer to ‘z/OS DFSMS Using Data sets’ section ‘Defining and Accessing a VSAM RLS Database’ for more information on multi-level keys.

explicit-key-name is already in UTF-8 format. The DEFINE CLUSTER fails if KEYNAME or KEYNAMEU is specified without an explicit-key-name or with more than one explicit-key-name.

For BSON and JSON, there are no default KEYNAME and KEYNAMEU key names. If KEYNAME and KEYNAMEU are not specified, the assumption is there is no primary key and VSAMDB generates a unique random implicit key. VSAMDB then treats that implicit key as any base key and converts it to a hashed derived key. The implicit key is returned to the user in the RPL MSGAREA and can be used in a direct GET request to retrieve the associated document.

A VSAM database with no KEYNAME or KEYNAMEU is a “keyless” database. A VSAMDB PUT DIR (against the base cluster) must be direct and generates an implicit key regardless of what is passed in RPLARG. A GET DIR (NUP or UPD) using the implicit key returned in MSGAREA will return the associated document. POINT FRD, GET FRD, GET SEQ, POINT LRD, and GET BWD are still allowed. Documents stored in a “keyless” base may also be accessed via alternate index keys.

The DATABASE parameter is restricted to RLS key-sequenced data sets (KSDS) only. The minimum control interval size is 4096; the allocation size is one cylinder. DATABASE can only be specified together with the INDEXED and LOG parameters.

Set CISIZE to 4K if it is less than 4KB. The valid CI sizes can be from 4K to 32K. The catalog may change the size again based on device type and other factors. When a CISIZE of 14K, 20K, 22K, 28K, 30K, or 32K is specified, to avoid issues caused by hardware for a CI that crosses track boundaries, AMS and Catalog will change it to the next higher CI size which is not in that list. CI sizes of 28K, 30K or 32K will be set to 26K.

If allocation is less than one cylinder, catalog increases it to one cylinder; for example, TRK(2 0) and KILOBYTES(1 0) will be raised to TRK(15 15) and KILOBYTES of size equivalent to one cylinder for primary and secondary. That increase to a cylinder allocation is done only to the data component of a KSDS. Specified sizes larger than one cylinder is assigned 1 cylinder.

The KEYS parameter is overridden when DATABASE is also specified. DATABASE enforces a value of KEYS(132 0).

RECORDSIZE is also ignored. It is always forced to be the size of a control area of size 1 cylinder. The LISTCAT output always shows MAXLRECL=CA size but AVGLRECL=user specified RECORDSIZE 'average' value, meaning it is possible to see MAXLRECL be smaller than AVGLRECL. The data base is always forced to be defined as SPANNED and CYLINDERS. The maximum VSAMDB data record (called a 'document') size is 2.0984 billion bytes (2GB) with the largest cylinder. A database document that is larger than 1 CA – 1 CI and up to 2GB is internally processed as separate spanned records, each of size 1 CA – 1 CI.

Abbreviation: DB

DATACLASS(*class*)

Identifies the name, 1-to-8 characters, of the data class for the data set. It provides the allocation attributes for new data sets. Your storage administrator defines the data class. However, you can override the parameters defined for DATACLASS by explicitly using other attributes. See “Understanding the Order of Assigned Data Set Attributes” on page 12 for the order of precedence (filtering) the system uses to select which attribute to assign.

DATACLASS parameters apply to both SMS- and non-SMS-managed data sets. If DATACLASS is specified and SMS is inactive, DEFINE is unsuccessful.

DATACLASS cannot be used as a subparameter of DATA or INDEX.

Abbreviation: DATACLAS

[EATTR(NO | OPT)]
A data set level attribute specifying whether a data set can have extended attributes (format 8 and 9 DSCBs) and optionally reside in EAS.

NO
No extended attributes. The data set can not have extended attributes (format 8 and 9 DSCBs) and cannot reside in EAS. This is the default behavior for non-VSAM data sets.

OPT
Extended attributes are optional. The data set can have extended attributes (format 8 and 9 DSCBs) and can optionally reside in EAS. This is the default behavior for VSAM data sets.

DFSMS does not provide an actual default value for the EATTR attribute. If an EATTR attribute is not specified on the DEFINE command, in the SMS DATA CLASS, JCL, dynamic allocation parameters or MODEL data set, the EATTR value is recorded as not being specified in the attributes for the data set.

For VSAM files a not specified value is treated by the system as if OPT was specified for EATTR. For non-VSAM files a not specified value is treated by the system as if NO was specified for EATTR. A not specified value for EATTR is the setting for data sets created prior to EAS support. When an EATTR attribute of not specified is encountered during a DEFINE or end of volume (EOV) extend processing, DFSMS internally performs the default action for the data set type when creating the new extent location.

See Appendix E, “DCOLLECT User Exit,” on page 439 and Appendix F, “Interpreting DCOLLECT Output,” on page 443 for collecting EATTR values stored in the volume table of contents (VTOC) and SMS DATA CLASS. The cataloged EATTR value is provided in DCOLLECT and LISTCAT.

ERASE|NOERASE
Specifies whether the cluster's components are to be erased when its entry in the catalog is deleted.

ERASE
Overwrites each component of the cluster with binary zeros when its catalog entry is deleted. If the cluster is protected by a RACF generic or discrete profile and is cataloged, you can use RACF commands to specify an ERASE attribute. If you do this, the data component is automatically erased upon deletion.

Abbreviation: ERAS

NOERASE
Specifies that each component of the cluster is not to be overwritten with binary zeros. NOERASE will not prevent erasure if the cluster is protected by a RACF generic or discrete profile that specifies the ERASE attribute and if the cluster is cataloged. Use RACF commands to alter the ERASE attribute in a profile.

Abbreviation: NERAS

EXCEPTIONEXIT(*entrypoint*)
Specifies the name of a user-written exception-exit routine that receives control when an exceptional I/O error condition occurs during the transfer of data between your program's address space and the cluster's DASD space. An exception is any condition that causes a SYNAD exit to be taken. The component's exception-exit routine is processed first, then the user's SYNAD exit routine receives control. If an exception-exit routine is loaded from an unauthorized library during access method services processing, an abnormal termination occurs. See *z/OS DFSMS Using Data Sets*.

Abbreviation: EEXT

FILE(*ddname*)
Names the DD statement that identifies and allocates the DASD and volumes that must be available for space allocation on the volumes specified by the VOLUMES keyword. If more than one volume is specified, all volumes must be the same device type.

If data and index components are to reside on separate devices, you can specify a separate FILE parameter as a parameter of DATA and INDEX to point to different DD statements.

If the FILE parameter is not specified, an attempt is made to dynamically allocate the required volumes. The volume must be mounted as permanently resident or reserved. When the FILE parameter is used, the specified volumes are directly allocated before access method services gets control.

An example DD statement is:

```
//ddname DD UNIT=(devtype[,unitcount]),
//    VOL=SER=(volser1,volser2,volser,...),DISP=OLD
```

Restriction: When FILE refers to more than one volume of the same device type, the DD statement that describes the volumes cannot be a concatenated DD statement.

FREESPACE(*CI-percent*[*CA-percent*]|0 0)
Specifies the percentage of each control interval and control area to be set aside as free space when the cluster is initially loaded or when a mass insert is done. CI-percent is a percentage of the amount of space to be preserved for adding new records and updating existing records with an increase in the length of the record. Since a CI is split when it becomes full, the CA might also need to be split when it is filled by CIs created by a CI split. The empty space in the control interval and control area is available for data records that are updated and inserted after the cluster is initially loaded. This parameter applies only to key-sequenced clusters, and variable-length relative records with variable-length records. *CI-percent* is the number of bytes that is equal to, or slightly less than, the percentage value of *CI-percent*. *CA-percent* is the number of control intervals equal to, or less than, the percentage of *CA-percent*.

CI-percent and *CA-percent* must be equal to, or less than, 100. When you use FREESPACE(100 100), one data record is placed in each control interval used for data. One control interval in each control area is used for data (that is, one data record is stored in each control area when the data set is loaded). If you do not use FREESPACE, the default reserves no free space when the data set is loaded. If the override indicator in Data Class is set ON, the user specified CI-percent and CA-percent of FREESPACE keyword in the DEFINE command will be overridden by the CI-percent and CA-percent in the DATACLAS.

When you define the cluster using the RECORDS parameter, the amount of free space specified is not considered in the calculations to determine primary allocation.

Abbreviation: FSPC

FRLOG(ALL|NONE|REDO|UNDO)
Specifies if VSAM batch logging can be performed for your VSAM data set. VSAM batch logging is available with CICS VSAM Recovery V3R1.

There is no default value for FRLOG. If FRLOG is left out, the data set cannot be used for VSAM batch logging. See the ALTER command for enabling VSAM batch logging after a data set is created.

ALL
Enables the changes made to your VSAM data set to be both backed out and forward recovered using the VSAM logging. The LOGSTREAMID parameter indicates the changes that are made by applications that are written to the MVS log stream. When specifying FRLOG(ALL), you must also specify LOGSTREAMID.

NONE
Indicates that the data set can be used for VSAM batch logging. However, the function should be disabled. The LOGSTREAMID parameter indicates changes that are made by applications that are written to the MVS log stream. Specifying FRLOG(NONE) implies that you may use the data set for RLS processing; omitting it indicates that RLS processing will not occur.

REDO
Enables the VSAM batch logging function for your VSAM data set. The LOGSTREAMID parameter indicates changes that are made by applications that are written to the MVS log stream. When specifying FRLOG(REDO), you must also specify LOGSTREAMID.

UNDO
Enables the changes made to your VSAM data set to be backed out using the VSAM logging. The LOGSTREAMID parameter indicates changes that are made by applications that are written to the MVS log stream.

Restrictions:

- If you do not want VSAM batch logging for your data set, do not specify the FRLOG parameter. If you specify FRLOG(NONE), the data set must support VSAM batch logging, but logging is not in effect.
- If FRLOG is specified, the data set:
  - Must be SMS-managed
  - Cannot be LINEAR or a temporary data set

INDEXED | LINEAR | NONINDEXED | NUMBERED | ZFS
Shows the type of data organization for the cluster.



Warning: ZFS data sets that are encrypted cannot use EXPORT, IMPORT, REPRO, or PRINT commands.

If you want a data organization other than INDEXED (the default), you must explicitly use it with this parameter.

When a cluster is defined, you indicate whether the data is to be indexed (key-sequenced), nonindexed (entry-sequenced), numbered (relative record), or linear.

Certain parameters apply only to key-sequenced clusters, as noted in the description of each of these parameters.

Linear data set clusters are treated as ESDS clusters that must be processed using control interval access. ZFS data sets are specially indicated linear data sets.

If you do not choose either the data organization or the MODEL parameter, your cluster defaults to key-sequenced (indexed).

To define an entry-sequenced or a relative record cluster, you must specify the NONINDEXED, the NUMBERED, or the MODEL parameter.

The data organization you select must be consistent with other parameters you specify.

INDEXED
Shows that the cluster being defined is for key-sequenced data. If INDEXED is specified, an index component is automatically defined and cataloged. The data records can be accessed by key or by relative-byte address (RBA).

Abbreviation: IXD

LINEAR
Specifies that the cluster being defined is for linear data. Because linear data set clusters are treated as ESDS clusters that must be processed using control interval access, you can use most of the commands and parameters you use to manipulate ESDS clusters. There are two exceptions:

- Parameters that refer to logical records are not allowed (except RECORDS).
- Use partial printing by specifying the RBA syntax.

Space is allocated for a linear data set with the number of control intervals equal to the number of records. Linear data sets cannot be accessed for RLS processing. The LOG, LOGSTREAMID, and BWO parameters do not apply to linear data sets.

Restriction: Linear data sets cannot be accessed for VSAM RLS or DFSMStvs processing. The LOG, LOGSTREAMID, and BWO parameters do not apply to linear data sets.

Abbreviation: LIN

NONINDEXED
Indicates that the cluster being defined is for entry-sequenced data. The data records can be accessed sequentially or by relative-byte address (RBA).

Abbreviation: NIXD

NUMBERED
Specifies that the cluster's data organization is for relative record data. A relative record cluster, which is similar to an entry-sequenced cluster, has fixed-length records or variable-length records that are stored in slots. The RECORDSIZE parameter determines if the records are fixed-length or variable-length. Empty slots hold space for records to be added later. The data records are accessed by relative record number (slot number).

Abbreviation: NUMD

ZFS

Specifies that the cluster being defined is for linear data, and the linear data set is a Unix System Services zFS file system. All restrictions and conditions applicable to linear data sets apply to ZFS CLUSTERs.

When ZFS is specified:

- Non-SMS data sets and SMS EF data sets are defined as extended addressable.
- The value of SHAREOPTIONS is set to (3 3).

Abbreviation: None.

KEYLABEL(*keylabel*)
Identifies the name, up to 64 bytes, of the key label to be used to encrypt the new data set. However, the key label specified with this parameter can be overridden. See Understanding the Order of Assigned Data Set Attributes for the order of precedence (filtering) the system uses to select key label.

Specifies that the base cluster is to be encrypted. Any alternate index associated with the base cluster will also be encrypted and use the same key label as specified for the cluster.

KEYS(*length offset*|64 0)
Provides information about the prime key field of a key-sequence data set's data records.

This parameter overrides any KEYS specification on the DATACLASS parameter.

This parameter applies only to key-sequenced clusters. The default is a key field of 64 bytes, beginning at the first byte (byte 0) of each data record.

The key field of the cluster's index is called the prime key to distinguish it from other keys, called alternate keys. See Chapter 13, “DEFINE ALTERNATEINDEX,” on page 127 for more details on how to choose alternate indexes for a cluster.

When the data record spans control intervals, the record's key field must be within the part of the record that is in the first control interval.

length offset
specifies the length of the key and its displacement (in bytes) from the beginning of the record. The sum of length plus offset cannot exceed the length of the shortest record. The length of the key can be 1 to 255 bytes.

LOG(NONE|UNDO|ALL)
Establishes whether the sphere to be accessed with VSAM record-level sharing (RLS) or DFSMStvs is recoverable or non-recoverable. It also indicates whether or not forward recovery logging should be performed for the data set. LOG applies to all components in the VSAM sphere. VSAM uses LOG in the following way:

Nonrecoverable Sphere
The sphere is considered nonrecoverable if LOG(NONE) is specified. VSAM allows concurrent read and update sharing across multiple resource managers and other applications.

Recoverable Sphere
The sphere is considered recoverable if LOG(UNDO) or LOG(ALL) is specified. For a recoverable sphere, VSAM does not allow applications that do not support commit and backout to open a data set in the sphere for output using RLS access, but applications can open the sphere for output using DFSMStvs access. The applications can, however, open the sphere for RLS access for input processing only.

If LOG is specified in the SMS data class, the value defined is used as the data set definition, unless it has been previously defined with an explicitly specified or modeled DEFINE attribute.

LOG cannot be used with LINEAR.

LOGSTREAMID cannot be used with LINEAR.

NONE
Indicates that neither an external backout nor a forward recovery capability is available for the sphere accessed in VSAM RLS or DFSMStvs mode. If you use LOG(NONE), RLS and DFSMStvs consider the sphere to be nonrecoverable.

UNDO
Specifies that changes to the sphere accessed in VSAM RLS or DFSMStvs mode can be backed out using an external log. RLS and DFSMStvs consider the sphere to be recoverable when you use LOG(UNDO).

ALL
Specifies that changes to the sphere accessed in RLS and DFSMStvs mode can be backed out and forward recovered using external logs. DFSMStvs and RLS consider the sphere recoverable when you use LOG(ALL). When you specify LOG(ALL), you must also specify the LOGSTREAMID parameter.

VSAM RLS and DFSMStvs allow concurrent read or update sharing for nonrecoverable spheres through commit (CICS) and noncommit protocol applications. For a recoverable sphere, a noncommit protocol application must use DFSMStvs to be able to open the sphere for update using RLS access.

LOGREPLICATE|NOLOGREPLICATE
Identifies whether or not the VSAM data set being define is eligible for VSAM replication.

LOGREPLICATE
VSAM data set is eligible for VSAM replication. The update will be captured in the replication log identified by the LOGSTREAMID parameter. When LOGREPLICATE is specified, LOGSTREAMID must also be specified.

LOGREPLICATE does not apply to LINEAR data sets.

Abbreviation: LOGR

NOLOGREPLICATE
VSAM data set is not eligible for VSAM replication.

NOLOGREPLICATE does not apply to LINEAR data sets.

Abbreviation: NOLOGR

If neither LOGREPLICATE nor NOLOGREPLICATE is specified, the value is:

1. The value of the model object, if there is a model specified for the DEFINE
2. The value of the SMS DATACLAS, if there is no model for the defined object
3. NOLOGREPLICATE, if there neither a model for the DEFINE nor SMS DATACLAS has been specified.

LOGSTREAMID(*logstream*)
Gives the name of the forward recovery log stream. It applies to all components in the VSAM sphere.

If LOGSTREAMID is specified in the SMS data class, the value defined is used as the data set definition, unless it has been previously defined with an explicitly specified or modeled DEFINE attribute.

logstream
The name of the forward recovery log stream. This can be a fully qualified name up to 26 characters, including separators. If LOG(ALL) is specified, LOGSTREAMID(name) must be specified.

Abbreviation:LSID

MANAGEMENTCLASS(*class*)
For SMS-managed data sets: Specifies the name, 1-to-8 characters, of the management class for a new data set. Your storage administrator defines the names of the management classes you can use. If MANAGEMENTCLASS is not used, but STORAGECLASS is used or defaulted, MANAGEMENTCLASS is derived from automatic class selection (ACS). If MANAGEMENTCLASS is specified and STORAGECLASS is not specified or derived, the DEFINE is unsuccessful. If SMS is inactive and MANAGEMENTCLASS is specified, the DEFINE will be unsuccessful. MANAGEMENTCLASS cannot be listed as a subparameter of DATA or INDEX.

Abbreviation:MGMTCLAS

MODEL(*entryname*[*catname*])
Specifies an existing entry to be used as a model for the entry being defined. See "Understanding the Order of Assigned Data Set Attributes" on page 12 for information on how the system selects modeled attributes.

A VVDS cannot be modeled.

DATACLASS, MANAGEMENTCLASS, and STORAGECLASS attributes are not modeled.

You can use an existing cluster's entry as a model for the attributes of the cluster being defined. For details about how a model is used, see *z/OS DFSMS Managing Catalogs*.

You can use some attributes of the model and override others by explicitly specifying them in the definition of the cluster or component. If you do not want to add or change any attributes, you need specify only the entry type (cluster, data, or index) of the model to be used and the name of the entry to be defined.

See "Understanding the Order of Assigned Data Set Attributes" on page 12 for more information about the order in which the system selects an attribute.

When you use a cluster entry as a model for the cluster, the data and index entries of the model cluster are used as models for the data and index components of the cluster still to be defined, unless another entry is specified with the MODEL parameter as a subparameter of DATA or INDEX.

entryname
specifies the name of the cluster or component entry to be used as a model.

OWNER(*ownerid*)
Identifies the cluster's owner.

For TSO/E users, if the owner is not identified with the OWNER parameter, the TSO/E user's userid becomes the ownerid.

RECATALOG | NORECATALOG
Indicates whether the catalog entries for the cluster components are to be re-created from information in the VVDS.

RECATALOG
Recreates the catalog entries if valid VVDS entries are found on the primary VVDS volume. If they are not, the command ends.

When recataloging entries (including zFS files) with indirect volsers, the VVDS on the substituted volser must contain a valid NVRs or VVRs that match the entry name.

Catalog entries can be re-created only in the catalog specified in the VVR except for entries that are swap space, page space, or SYS1 data sets. Change this to Catalog entries can be re-created only in the catalog specified in the VVR except for entries that are swap space, page space, SYS1 data sets or single volume zFS VSAM Linear data sets with an indirect volume serial.

The RECORDSIZE parameter is required when doing a DEFINE RECATALOG of a variable-length relative record data set (VRRDS).

Identification of RECATALOG requires that NAME, INDEXED, LINEAR, NONINDEXED, NUMBERED, and VOLUMES be used as they were when the cluster was originally defined. If you specify RECATALOG, you are not required to use CYLINDERS, RECORDS, or TRACKS.

If ATTEMPTS, AUTHORIZATION, CATALOG, CODE, MODEL, or OWNER parameters are used during the original define, they must be respecified with RECATALOG to restore their original values; otherwise, their default values are used.

When you use the TO parameter with RECATALOG, only the cluster's expiration date is updated. The DATA and INDEX components are not updated.

If the RACF user has ADSP specified, a profile is defined to RACF for the data set being re-cataloged.

If the cluster was SMS-managed, the volume serials should be the same as the volumes actually selected by SMS.

The catalog for the entries being re-created must have the same name as the catalog that contained the original entries.

Abbreviation: RCTLG

NORECATALOG
Indicates that the catalog entries are not re-created from VVDS entries. Catalog entries are created for the first time.

Abbreviation: NRCTLG

RECORDSIZE(*average maximum*|*default*)
Specifies the average and maximum lengths, in bytes, of the records in the data component. The minimum record size is 1 byte.

RECORDSIZE can be given as a parameter of either CLUSTER or DATA.

This parameter overrides the LRECL specification on the DATACLASS parameter.

For nonspanned records, the maximum record size + 7 cannot exceed the data component's control interval size (that is, the maximum nonspanned record size, 32 761, + 7 equals the maximum data component control interval size, 32 768).

When you use a record size that is larger than one control interval, you must also specify spanned records (SPANNED). The formula for the maximum record size of spanned records as calculated by VSAM is as follows:

```
MAXLRECL = CI/CA * (CISZ - 10)
```

where:

- MAXLRECL is the maximum spanned record size.
- CI/CA represents the number of control intervals per control area.
- CISZ is the control interval size.

When you select NUMBERED, you identify a data set as a relative record data set. If you use NUMBERED and select the same value for average as for maximum, the relative records must be fixed-length. If you specify NUMBERED and select two different values for the average and maximum record sizes, the relative records can be variable-length. If you know that your relative records are fixed-length, however, be sure to define them as fixed-length. Performance is affected for relative

record data sets defined as variable-length. Each variable-length relative record is increased internally in length by four.

When your records are fixed length, you can use the following formula to find a control interval size that contains a whole number (n) of records:

```
CISZ = (n x RECSZ) + 10
or
n = (CISZ - 10)
       RECSZ
```

If you select SPANNED or NUMBERED for your fixed-length records:

```
CISZ =(n x (RECSZ + 3)) + 4
or
n = (CISZ - 4)
    (RECSZ + 3)
```

where:

- ***n*** is the number of fixed-length records in a control interval.
- CISZ is the control interval size (see also the CONTROLINTERVALSIZE parameter).
- RECSZ is the average record size.

default
When SPANNED is used, the default is RECORDSIZE(4086 32600). Otherwise, the default is RECORDSIZE(4089 4089).

Example:

```
REC(sec) x RECSZ(avg) > RECSZ(max)
```

- where:
  - REC(sec) is the secondary space allocation quantity, in records.
  - RECSZ(avg) is the average record size (default = 4086 or 4089 bytes).
  - RECSZ(max) is the maximum record size (default = 4089 or 32600 bytes).

When the SPANNED record size default prevails (32600 bytes), the secondary allocation quantity should be at least 8 records.

With REPRO and EXPORT, you cannot use data sets with record sizes greater than 32 760.

Abbreviation: RECSZ

REUSE | NOREUSE
Specifies whether or not the cluster can be opened again and again as a reusable cluster.

If REUSE or NOREUSE is specified in the SMS data class, the value defined is used as the data set definition, unless it has been previously defined with an explicitly specified or modeled DEFINE attribute.

REUSE
Specifies that the cluster can be opened again and again as a reusable cluster. When a reusable cluster is opened, its high-used RBA is set to zero if you open it with an access control block that specifies the RESET attribute.

REUSE lets you create an entry-sequenced, key-sequenced, or relative record work file.

When you create a reusable cluster, you cannot build an alternate index to support it. Also, you cannot create a reusable cluster with key ranges (see the KEYRANGE parameter). Reusable data sets can be multivolume and can have up to 123 physical extents.

If you select REUSE and your command also contains the keyword UNIQUE, you must remove the UNIQUE keyword or the DEFINE command will be unsuccessful.

Abbreviation: RUS

NOREUSE
Indicates that the cluster cannot be opened again as a new cluster.

Abbreviation: NRUS

RLSQUIESCE | RLSENABLE
Specifies whether the cluster's components are created in VSAM record-level sharing (RLS) quiesce or enable mode.

RLSQUIESCE
The cluster component is defined in RLS quiesce mode.

RLSENABLE
The cluster component is defined in RLS enable mode, which is the default.

Abbreviation: RLSQ RLSE

SHAREOPTIONS(*crossregion*[*crosssystem*]|1 3)
Shows how a component or cluster can be shared among users. However, SMS-managed volumes, and catalogs containing SMS-managed data sets, must not be shared with non-SMS systems. For a description of data set sharing, see *z/OS DFSMS Using Data Sets*. To ensure integrity, you should be sure that share options specified at the DATA and INDEX levels are the same.

The value of SHAREOPTIONS is assumed to be (3,3) when the data set is accessed in VSAM RLS or DFSMStvs mode.

crossregion
Specifies the amount of sharing allowed among regions within the same system or within multiple systems using global resource serialization (GRS). Independent job steps in an operating system, or multiple systems in a GRS ring, can access a VSAM data set concurrently. For more information about GRS, see *z/OS MVS Planning: Global Resource Serialization*. To share a data set, each user must use DISP=SHR in the data set's DD statement. You can use the following options:

OPT 1
The data set can be shared by any number of users for read processing, or the data set can be accessed by only one user for read and write processing. VSAM ensures complete data integrity for the data set. This setting does not allow any non-RLS access when the data set is already open for VSAM RLS or DFSMStvs processing. A VSAM RLS or DFSMStvs open will fail with this option if the data set is already open for any processing.

OPT 2
The data set can be accessed by any number of users for read processing, and it can also be accessed by one user for write processing. It is the user's responsibility to provide read integrity. VSAM ensures write integrity by obtaining exclusive control for a control interval while it is being updated. A VSAM RLS or DFSMStvs open is not allowed while the data set is open for non-RLS output.

If the data set has already been opened for VSAM RLS or DFSMStvs processing, a non-RLS open for input is allowed; a non-RLS open for output fails. If the data set is opened for input in non-RLS mode, a VSAM RLS or DFSMStvs open is allowed.

OPT 3
The data set can be fully shared by any number of users. Each user is responsible for maintaining both read and write integrity for the data the program accesses. This setting does not allow any non-RLS access when the data set is already open for VSAM RLS or DFSMStvs processing. If the data set is opened for input in non-RLS mode, a VSAM RLS or DFSMStvs open is not allowed.

OPT 4
The data set can be fully shared by any number of users. For each request, VSAM refreshes the buffers used for direct processing. This setting does not allow any non-RLS access

when the data set is already open for VSAM RLS or DFSMStvs processing. If the data set is opened for input in non-RLS mode, a VSAM RLS or DFSMStvs open is not allowed.

As in SHAREOPTIONS 3, each user is responsible for maintaining both read and write integrity for the data the program accesses.

crosssystem
Specifies the amount of sharing allowed among systems. Job steps of two or more operating systems can gain access to the same VSAM data set regardless of the disposition indicated in each step's DD statement for the data set. However, if you are using GRS across systems or JES3, the data set might not be shared depending on the disposition of the system.

To get exclusive control of the data set's volume, a task in one system issues the RESERVE macro. The level of cross-system sharing allowed by VSAM applies only in a multiple operating system environment.

The cross-system sharing options are ignored by RLS or DFSMStvs processing. The values are:

1
Reserved

2
Reserved

3
Specifies that the data set can be fully shared. With this option, each user is responsible for maintaining both read and write integrity for the data that user's program accesses. User programs that ignore write integrity guidelines can cause VSAM program checks, uncorrectable data set errors, and other unpredictable results. This option requires each user to be responsible for maintenance. The RESERVE and DEQ macros are required with this option to maintain data set integrity. (For information on using RESERVE and DEQ, see *z/OS MVS Programming: Authorized Assembler Services Reference ALE-DYN* and *z/OS MVS Programming: Authorized Assembler Services Reference LLA-SDU*.)

4
Indicates that the data set can be fully shared. For each request, VSAM refreshes the buffers used for direct processing. This option requires that you use the RESERVE and DEQ macros to maintain data integrity while sharing the data set. Improper use of the RESERVE macro can cause problems similar to those described under SHAREOPTIONS 3. (For information on using RESERVE and DEQ, see *z/OS MVS Programming: Authorized Assembler Services Reference ALE-DYN* and *z/OS MVS Programming: Authorized Assembler Services Reference LLA-SDU*.) Output processing is limited to update, or add processing, or both that does not change either the high-used RBA or the RBA of the high key data control interval if DISP=SHR is specified.

To ensure data integrity in a shared environment, VSAM provides users of SHAREOPTIONS 4 (cross-region and cross-system) with the following assistance:

- Each PUT request immediately writes the appropriate buffer to the VSAM cluster's DASD space. That is, the buffer in the user's address space that contains the new or updated data record, and the buffers that contain new or updated index records when the user's data is key-sequenced.
- Each GET request refreshes all the user's input buffers. The contents of each data and index buffer being used by the user's program is retrieved from the VSAM cluster's DASD.

Abbreviation: SHR

SPANNED | NONSPANNED
Specifies whether a data record is allowed to cross control interval boundaries.

If SPANNED or NONSPANNED is specified in the SMS data class, the value defined is used as the data set definition, unless it has been previously defined with an explicitly specified or modeled DEFINE attribute.

This parameter cannot be used when defining a linear data set cluster.

SPANNED
Specifies that, if the maximum length of a data record (as specified with RECORDSIZE) is larger than a control interval, the record is contained on more than one control interval. This allows VSAM to select a control interval size that is optimum for the DASD.

When a data record that is larger than a control interval is put into a cluster that allows spanned records, the first part of the record completely fills a control interval. Subsequent control intervals are filled until the record is written into the cluster. Unused space in the record's last control interval is not available to contain other data records.

Using this parameter for a Variable-Length or a Fixed-Length Relative Record Data Set causes an error.

Abbreviation: SPND

NONSPANNED
Indicates that the record must be contained in one control interval. VSAM selects a control interval size that accommodates your largest record.

Abbreviation: NSPND

SPEED | RECOVERY
Specifies whether the data component's control areas are to be preformatted during loading.

This parameter is only considered during the actual loading (creation) of a data set. Creation occurs when the data set is opened and the high-used RBA is equal to zero. After normal CLOSE processing at the completion of the load operation, the physical structure of the data set and the content of the data set extents are exactly the same, regardless of which option is used. Any processing of the data set after the successful load operation is the same, and the specification of this parameter is not considered.

If you use RECOVERY, the initial load takes longer because the control areas are first written with either empty or software end-of-file control intervals. These preformatted control intervals are then updated, using update writes with the data records. When SPEED is used, the initial load is faster.

SPEED
Does not preformat the data component's space.

If the initial load is unsuccessful, you must load the data set again from the beginning because VSAM cannot determine the location of your last correctly written record. VSAM cannot find a valid end-of-file indicator when it searches your data records.

RECOVERY
Does preformat the data component's space prior to writing the data records.

If the initial load is unsuccessful, VSAM can determine the location of the last record written during the load process.

Abbreviation: RCVY

STORAGECLASS(*class*)
For SMS-managed data sets: Gives the name, 1-to-8 characters of the storage class.

Your storage administrator defines the names of the storage classes you can use. A storage class is assigned either when you use STORAGECLASS, or an ACS routine selects a storage class for the new data set. The storage class provides the storage attributes that are specified on the UNIT and VOLUME operand for non-SMS managed data sets. Use the storage class to select the storage service level to be used by SMS for storage of the data set. If SMS is inactive and STORAGECLASS is used, the DEFINE will be unsuccessful.

STORAGECLASS cannot be selected as a subparameter of DATA or INDEX.

Abbreviation: STORCLAS

TO(*date*)|FOR(*days*)
Specifies the retention period for the cluster being defined. If neither TO nor FOR is used, the cluster can be deleted at any time. The MANAGEMENTCLASS maximum retention period, if selected, limits the retention period specified by this parameter.

For non-SMS-managed data sets, the correct retention period is reflected in the catalog entry. The VTOC entry cannot contain the correct retention period. Enter a LISTCAT command for the correct expiration date.

For SMS-managed data sets, the expiration date in the catalog is updated and the expiration date in the format-1 DSCB is changed. If the expiration date in the catalog does not agree with the expiration date in the VTOC, the VTOC entry overrides the catalog entry.

TO(*date*)
Specifies the earliest date that a command without the PURGE parameter can delete an entry. Specify the expiration date in the form *yyyyddd*, where *yyyy* is a four-digit year (to a maximum of 2155) and *ddd* is the three-digit day of the year from 001 through 365 (for non-leap years) or 366 (for leap years).

The following four values are "never-expire" dates: 99365, 99366, 1999365, and 1999366. Specifying a "never-expire" date means that the PURGE parameter will always be required to delete an entry. For related information, see the "EXPDT Parameter" section of *z/OS MVS JCL Reference*.

Specifying the current date as the expiration date will make an entry eligible for deletion.

FOR(*days*)
Specifies the number of days you want to keep the cluster being defined. The maximum number is 93000. If the number is 0 through 92999 (except for 9999), the entry is retained for the number of days indicated. If the number is either 9999 or 93000, the entry is retained indefinitely. There is a hardware imposed expiration date of 2155.

WRITECHECK|NOWRITECHECK

WRITECHECK

This option is obsolete and has no effect.

Abbreviation: WCK

NOWRITECHECK

Abbreviation: NWCK

Data and Index Components of a Cluster

You should use attributes separately for the cluster's data and index components. A list of the DATA and INDEX parameters is provided at the beginning of this section. These parameters are described in detail as parameters of the cluster as a whole. Restrictions are noted with each parameter's description.

DEFINE CLUSTER Examples

The DEFINE CLUSTER command can perform the functions shown in the following examples.

Define an SMS-Managed Key-Sequenced Cluster: Example 1

In this example, an SMS-managed key-sequenced cluster is defined. The DEFINE CLUSTER command builds a catalog entry and allocates space to define the key-sequenced cluster SMS04.KSDS01.

```
//DEFINE   JOB  ...
//STEP1    EXEC PGM=IDCAMS
//SYSPRINT DD   SYSOUT=A
//SYSIN    DD   *
```

```
    DEFINE CLUSTER -
           (NAME (SMS04.KSDS01) -
           STORAGECLASS (FINCE02) -
           MANAGEMENTCLASS (MC1985) -
           DATACLASS (VSAMDB05))
/*
```

The parameters for this command are:

- STORAGECLASS specifies an installation-defined name of a storage class, FINCE02, to be assigned to this cluster.
- MANAGEMENTCLASS specifies an installation-defined name of a management class, MC1985, to be assigned to this cluster. Attributes of MANAGEMENTCLASS control the data set's retention, backup, migration, etc.
- DATACLASS specifies an installation-defined name of a data class, VSAMDB05, to be assigned to this cluster. Record size, key length and offset, space allocation, etc., are derived from the data class and need not be specified.

Define an SMS-Managed Key-Sequenced Cluster Specifying Data and Index Parameters: Example 2

In this example, an SMS-managed key-sequenced cluster is defined. The SMS data class space allocation is overridden by space allocations at the data and index levels. The DEFINE CLUSTER command builds a catalog entry and allocates space to define the key-sequenced cluster SMS04.KSDS02.

```
//DEFINE   JOB  ...
//STEP1    EXEC PGM=IDCAMS
//SYSPRINT DD   SYSOUT=A
//SYSIN    DD   *
    DEFINE CLUSTER -
           (NAME (SMS04.KSDS02) -
           STORAGECLASS (FINCE02) -
           MANAGEMENTCLASS (MC1985) -
           DATACLASS (VSAMDB05)) -
           LOG(ALL) -
           LOGSTREAMID(LogA) -
         DATA -
           (MEGABYTES (10 2)) -
         INDEX -
           (KILOBYTES (25 5))
/*
```

The parameters for this command are as follows:

- STORAGECLASS is an installation defined name of a storage class, FINCE02, to be assigned to the cluster.
- MANAGEMENTCLASS is an installation defined name of a management class, MC1985, to be assigned to the cluster. Attributes associated with a management class control the cluster's retention, backup, migration, etc.
- DATACLASS is an installation defined name of a data class, VSAMDB05, assigned to the cluster. Record size, key length and offset, etc., are derived from the data class and need not be specified. If MAXVOLUMES or the space parameters (MEGABYTES and KILOBYTES) were not specified, the values in the data class would be used.
- LOG(ALL) specifies that changes to the sphere accessed in RLS and DFSMStvs mode can be backed out and forward recovered using external logs.
- LOGSTREAMID gives the name of the forward recovery log stream.

The DATA and INDEX parameters are:

- MEGABYTES, used for DATA, allocates a primary space of 10 megabytes to the data component. A secondary space of 2 megabytes is specified for extending the data component.
- KILOBYTES, used for INDEX, allocates a primary space of 25 kilobytes to the index component. A secondary space of 5 kilobytes is specified for extending the index component.

Define a Key-Sequenced Cluster Specifying Data and Index Parameters: Example 3

In this example, a key-sequenced cluster is defined. The DATA and INDEX parameters are specified and the cluster's data and index components are explicitly named. This example assumes that an alias name VWX is defined for the catalog RSTUCAT1. This naming convention causes VWX.MYDATA to be cataloged in RSTUCAT1.

```
//DEFCLU1  JOB    ...
//STEP1    EXEC   PGM=IDCAMS
//SYSPRINT DD     SYSOUT=A
//SYSIN    DD     *
    DEFINE CLUSTER -
           (NAME(VWX.MYDATA) -
           VOLUMES(VSER02) -
           RECORDS(1000 500)) -
         DATA -
           (NAME(VWX.KSDATA) -
           KEYS(15 0) -
           RECORDSIZE(250 250) -
           FREESPACE(20 10) -
           BUFFERSPACE(25000)  ) -
         INDEX -
           (NAME(VWX.KSINDEX) -
         CATALOG (RSTUCAT1)
/*
```

The DEFINE CLUSTER command builds a cluster entry, a data entry, and an index entry to define the key-sequenced cluster VWX.MYDATA. The parameters for the cluster as a whole are:

- NAME indicates that the cluster's name is VWX.MYDATA.
- VOLUMES is used when the cluster is to reside on volume VSER02.
- RECORDS specifies that the cluster's space allocation is 1000 data records. The cluster is extended in increments of 500 records. After the space is allocated, VSAM calculates the amount required for the index and subtracts it from the total.

In addition to the parameters specified for the cluster as a whole, DATA and INDEX parameters specify values and attributes that apply only to the cluster's data or index component. The parameters specified for the data component of VWX.MYDATA are:

- NAME indicates that the data component's name is VWX.KSDATA.
- KEYS shows that the length of the key field is 15 bytes and that the key field begins in the first byte (byte 0) of each data record.
- RECORDSIZE specifies fixed-length records of 250 bytes.
- BUFFERSPACE verifies that a minimum of 25 000 bytes must be provided for I/O buffers. A large area for I/O buffers can help to improve access time with certain types of processing. For example, with direct processing if the high-level index can be kept in virtual storage, access time is reduced. With sequential processing, if enough I/O buffers are available, VSAM can perform a read-ahead, thereby reducing system overhead and minimizing rotational delay.
- FREESPACE specifies that 20% of each control interval and 10% of each control area are to be left free when records are loaded into the cluster. After the cluster's records are loaded, the free space can be used to contain new records.

The parameters specified for the index component of VWX.MYDATA are:

-
- NAME specifies that the index component's name is VWX.KSINDEX.
- CATALOG specifies the catalog name.

Define a Key-Sequenced Cluster and an Entry-Sequenced Cluster: Example 4

In this example, two VSAM clusters are defined. The first DEFINE command defines a key-sequenced VSAM cluster, VWX.EXAMPLE.KSDS1. The second DEFINE command defines an entry-sequenced VSAM cluster, KLM.EXAMPLE.ESDS1. In both examples, it is assumed that alias names, VWX and KLM, have been defined for user catalogs RSTUCAT1 and RSTUCAT2, respectively.

```
//DEFCLU2  JOB    ...
//STEP1    EXEC   PGM=IDCAMS
//SYSPRINT DD     SYSOUT=A
//SYSIN    DD     *
    DEFINE CLUSTER -
           (NAME(VWX.EXAMPLE.KSDS1) -
           MODEL(VWX.MYDATA) -
           VOLUMES(VSER02))
    DEFINE CLUSTER -
           (NAME(KLM.EXAMPLE.ESDS1) -
           RECORDS(100 500) -
           RECORDSIZE(250 250) -
           VOLUMES(VSER03) -
           NONINDEXED  )
/*
```

The first DEFINE command builds a cluster entry, a data entry, and an index entry to define the key-sequenced cluster VWX.EXAMPLE.KSDS1. Its parameters are:

- NAME specifies the name of the key-sequenced cluster, VWX.EXAMPLE.KSDS1. The cluster is defined in the user catalog for which VWX has been established as an alias.
- MODEL identifies VWX.MYDATA as the cluster to use as a model for VWX.EXAMPLE.KSDS1. The attributes and specifications of VWX.MYDATA that are not otherwise specified with the DEFINE command parameters are used to define the attributes and specifications of VWX.EXAMPLE.KSDS1. VWX.MYDATA is located in the user catalog for which VWX has been established as an alias.
- VOLUMES specifies that the cluster is to reside on volume VSER02.

The second DEFINE command builds a cluster entry and a data entry to define an entry-sequenced cluster, KLM.EXAMPLE.ESDS1. Its parameters are:

- NAME specifies the name of the entry-sequenced cluster, KLM.EXAMPLE.ESDS1. The cluster is defined in the user catalog for which KLM has been established as an alias.
- RECORDS specifies that the cluster space allocation is 100 records. When the cluster is extended, it is extended in increments of 500 records.
- RECORDSIZE specifies that the cluster records are fixed length (the average record size equals the maximum record size) and 250 bytes long.
- VOLUMES specifies that the cluster is to reside on volume VSER03.
- NONINDEXED specifies that the cluster is to be an entry-sequenced cluster.

Define a Relative Record Cluster in a Catalog: Example 5

In this example, a relative record cluster is defined.

```
//DEFCLU4  JOB   ...
//STEP1    EXEC  PGM=IDCAMS
//SYSPRINT DD    SYSOUT=A
//SYSIN    DD    *
     DEFINE CLUSTER -
           (NAME(EXAMPLE.RRDS1) -
           RECORDSIZE(100 100) -
           VOLUMES(VSER01) -
           TRACKS(10 5) -
           NUMBERED) -
         CATALOG(USERCAT)
/*
```

The DEFINE CLUSTER command builds a cluster entry and a data entry to define the relative record cluster EXAMPLE.RRDS1 in the user catalog. The DEFINE CLUSTER command allocates ten tracks for the cluster's use. The command's parameters are:

- NAME specifies that the cluster's name is EXAMPLE.RRDS1.
- RECORDSIZE specifies that the records are fixed-length, 100 byte records. Average and maximum record length must be equal for a fixed-length relative record data set, but not equal for a variable-length RRDS.
- VOLUMES specifies that the cluster is to reside on volume VSER01. This example assumes that the volume is already cataloged in the user catalog, USERCAT.
- TRACKS specifies that 10 tracks are allocated for the cluster. When the cluster is extended, it is to be extended in increments of 5 tracks.
- NUMBERED specifies that the cluster's data organization is to be relative record.
- CATALOG specifies the catalog name.

Define a Reusable Entry-Sequenced Cluster in a Catalog: Example 6

In this example, a reusable entry-sequenced cluster is defined. You can use the cluster as a temporary data set. Each time the cluster is opened, its high-used RBA can be reset to zero.

```
//DEFCLU5  JOB   ...
//STEP1    EXEC  PGM=IDCAMS
//SYSPRINT DD    SYSOUT=A
//SYSIN    DD    *
     DEFINE CLUSTER -
           (NAME(EXAMPLE.ESDS2) -
           RECORDSIZE(2500 3000) -
           SPANNED -
           VOLUMES(VSER03) -
           CYLINDERS(2 1) -
           NONINDEXED -
           REUSE -
           CATALOG(RSTUCAT2)
/*
```

The DEFINE CLUSTER command builds a cluster entry and a data entry to define the entry-sequenced cluster, EXAMPLE.ESDS2. The DEFINE CLUSTER command assigns two tracks for the cluster's use. The command's parameters are:

- NAME specifies that the cluster's name is EXAMPLE.ESDS2.
- RECORDSIZE specifies that the records are variable length, with an average size of 2500 bytes and a maximum size of 3000 bytes.
- SPANNED specifies that data records can cross control interval boundaries.
- VOLUMES specifies that the cluster is to reside on volume VSER03.
- CYLINDERS specifies that two cylinders are to be allocated for the cluster's space. When the cluster is extended, it is to be extended in increments of 1 cylinder.
- NONINDEXED specifies that the cluster's data organization is to be entry-sequenced. This parameter overrides the INDEXED parameter.
- REUSE specifies that the cluster is to be reusable. Each time the cluster is opened, its high-used RBA can be reset to zero and it is effectively an empty cluster.
- CATALOG specifies that the cluster is to be defined in a user catalog, RSTUCAT2.

Define a Key-Sequenced Cluster in a Catalog: Example 7

In this example, a key-sequenced cluster is defined. In other examples, an alternate index is defined over the cluster, and a path is defined that relates the cluster to the alternate index. The cluster, its alternate index, and the path entry are all defined in the same catalog, USERCAT.

```
//DEFCLU6  JOB   ...
//STEP1    EXEC  PGM=IDCAMS
```

```
//SYSPRINT DD    SYSOUT=A
//SYSIN    DD    *
     DEFINE CLUSTER -
           (NAME(EXAMPLE.KSDS2)) -
         DATA -
           (RECORDS(500 100) -
           EXCEPTIONEXIT(DATEXIT) -
           ERASE -
           FREESPACE(20 10) -
           KEYS(6 4) -
           RECORDSIZE(80 100) -
           VOLUMES(VSER01) ) -
         INDEX -
           (RECORDS(300 300) -
           VOLUMES(VSER01) ) -
         CATALOG(USERCAT)
/*
```

The DEFINE CLUSTER command builds a cluster entry, a data entry, and an index entry to define the key-sequenced cluster, EXAMPLE.KSDS2. The DEFINE CLUSTER command allocates space separately for the cluster's data and index components.

The parameter that applies to the cluster is NAME which specifies that the cluster's name is EXAMPLE.KSDS2.

The parameters that apply only to the cluster's data component are enclosed in the parentheses following the DATA keyword:

- RECORDS specifies that an amount of tracks equal to at least 500 records is to be allocated for the data component's space. When the data component is extended, it is to be extended in increments of tracks equal to 100 records.
- EXCEPTIONEXIT specifies the name of the exception exit routine, DATEXIT, that is to be processed if an I/O error occurs while a data record is being processed.
- ERASE specifies that the cluster's data is to be erased (overwritten with binary zeros) when the cluster is deleted.
- FREESPACE specifies the amounts of free space to be left in the data component's control intervals (20%) and the control areas (10% of the control intervals in the control area) when data records are loaded into the cluster.
- KEYS specifies the location and length of the key field in each data record. The key field is 6 bytes long and begins in the fifth byte (byte 4) of each data record.
- RECORDSIZE specifies that the cluster's records are variable length, with an average size of 80 bytes and a maximum size of 100 bytes.
- VOLUMES specifies that the cluster is to reside on volume VSER01.

The parameters that apply only to the cluster's index component are enclosed in the parentheses following the INDEX keyword:

- RECORDS specifies that an amount of tracks equal to at least 300 records is to be allocated for the index component's space. When the index component is extended, it is to be extended in increments of tracks equal to 300 records.
- VOLUMES specifies that the index component is to reside on volume VSER01.

The CATALOG parameter specifies that the cluster is to be defined in a user catalog, USERCAT4.

Define an Entry-Sequenced Cluster Using a Model: Example 8

In this example, two entry-sequenced clusters are defined. The attributes of the second cluster defined are modeled from the first cluster.

```
//DEFCLU7  JOB    ...
//STEP1   EXEC   PGM=IDCAMS
//SYSPRINT DD     SYSOUT=A
//SYSIN    DD     *
    DEFINE CLUSTER -
           (NAME(GENERIC.A.BAKER) -
```

```
           VOLUMES(VSER02) -
           RECORDS(100 100) -
           RECORDSIZE(80 80) -
           NONINDEXED ) -
         CATALOG(USERCAT4)
    DEFINE CLUSTER -
           (NAME(GENERIC.B.BAKER) -
           MODEL(GENERIC.A.BAKER USERCAT4)) -
         CATALOG(USERCAT4)
/*
```

The first DEFINE CLUSTER command defines an entry-sequenced cluster, GENERIC.A.BAKER. Its parameters are:

- NAME specifies the name of the entry-sequenced cluster, GENERIC.A.BAKER.
- VOLUMES specifies that the cluster is to reside on volume VSER02.
- RECORDS specifies that the cluster's space allocation is 100 records. When the cluster is extended, it is extended in increments of 100 records.
- RECORDSIZE specifies that the cluster's records are fixed length (the average record size equals the maximum record size) and 80 bytes long.
- NONINDEXED specifies that the cluster is entry-sequenced.

The second DEFINE CLUSTER command uses the attributes and specifications of the previously defined cluster, GENERIC.A.BAKER, as a model for the cluster still to be defined, GENERIC.B.BAKER. A list of the parameters follows:

- NAME specifies the name of the entry-sequenced cluster, GENERIC.B.BAKER.
- MODEL identifies GENERIC.A.BAKER, cataloged in user catalog USERCAT4, as the cluster to use as a model for GENERIC.B.BAKER. The attributes and specifications of GENERIC.A.BAKER that are not otherwise specified with the DEFINE command's parameters are used to define the attributes and specifications of GENERIC.B.BAKER.
- CATALOG specifies that the cluster is to be defined in the USERCAT4 catalog.

Define a VSAM Volume Data Set: Example 9

In this example, a VVDS is explicitly defined. The cluster is named using the restricted VVDS name format 'SYS1.VVDS.Vvolser'.

```
//DEFCLU8  JOB    ...
//STEP1   EXEC   PGM=IDCAMS
//SYSPRINT DD     SYSOUT=A
//SYSIN    DD     *
    DEFINE CLUSTER -
           (NAME(SYS1.VVDS.VVSER03) -
           VOLUMES(VSER03) -
           NONINDEXED -
           CYLINDERS(1 1)) -
         /*
```

This DEFINE CLUSTER command defines an entry-sequenced cluster that is used as a VVDS. The parameters are:

- NAME specifies the name of a VVDS, 'SYS1.VVDS.Vvolser', SYS1.VVDS.VVSER03.
- VOLUMES specifies that the cluster is to reside on volume VSER03. Only one volume serial can be specified.
- NONINDEXED specifies that the cluster is entry-sequenced.
- CYLINDERS specifies that the cluster's space allocation is 1 cylinder. When the cluster is extended, it is extended in increments of 1 cylinder.

Define a Relative Record Data Set with Expiration Date: Example 10

In this example, an entry-sequenced cluster is defined specifying an expiration date, using the TO parameter.

```
//DEFCLU8  JOB    ...
//STEP1   EXEC   PGM=IDCAMS
//SYSPRINT DD     SYSOUT=A
//SYSIN    DD     *
     DEFINE CLUSTER -
           (NAME(EXAMPLE.RRDS1) -
           RECORDSIZE(100 100) -
           VOLUMES(VSER01) -
           TRACKS(10 5) -
           NUMBERED -
           TO(2015012) ) -
         CATALOG(USERCAT)
/*
```

The DEFINE CLUSTER command builds a cluster entry and a data entry to define the relative record cluster, EXAMPLE.RRDS1, in the user catalog, USERCAT. The DEFINE CLUSTER command allocates ten tracks for the cluster's use. The expiration date is set to January 12, 2015. The parameters are:

- NAME specifies that the cluster's name is EXAMPLE.RRDS1.
- RECORDSIZE specifies that the records are fixed-length, 100-byte records. Average and maximum record length must be equal for a fixed-length relative record data set, but not equal for a variable-length RRDS.
- VOLUMES specifies that the cluster is to reside on volume VSER01.
- TRACKS specifies that ten tracks are allocated for the cluster. When the cluster is extended, it is to be extended in increments of five tracks.
- NUMBERED specifies that the cluster's data organization is to be relative record.
- CATALOG specifies that the cluster is to be defined in a user catalog, USERCAT.

Define a Linear Data Set Cluster in a Catalog: Example 11

In this example, a linear data set cluster is defined in a catalog.

```
//DEFLDS   JOB    ...
//STEP1   EXEC   PGM=IDCAMS
//SYSPRINT DD     SYSOUT=A
//SYSIN    DD     *
    DEFINE CLUSTER -
           (NAME(EXAMPLE.LDS01) -
           VOLUMES(VSER03) -
           TRACKS(20 10) -
           LINEAR -
         CATALOG(USERCAT)
/*
```

The DEFINE CLUSTER command builds a cluster entry and a data entry to define the linear data set cluster EXAMPLE.LDS01. The parameters are:

- NAME specifies that the cluster's name is EXAMPLE.LDS01.
- VOLUMES specifies that the cluster is to reside on volume VSER03.
- TRACKS specifies that 20 tracks are allocated for the cluster's space. When the cluster is extended, it is to be extended in increments of 10 tracks.
- LINEAR specifies that the cluster's data organization is to be linear.
- CATALOG specifies that the cluster is to be defined in a user catalog, USERCAT.

Define a VSAM Database: Example 12

In this example, a DEFINE CLUSTER command defines a VSAM database, using the DATABASE parameter to specify a BSON data format, and the KEYNAME parameter to specify a primary key field of EMPLOYEENAME:

```
DEFINE CLUSTER (NAME(HL1.BSON.KSDS1)
          RECORDSIZE(720676 720676)

          SHAREOPTIONS(2 3)
```

```
           STORCLAS(SXPXXS01)

 DATACLAS(KSX00002)

           SPANNED
           LOG(NONE)
           DATABASE(BSON)
           KEYNAME(EMPLOYEENAME))

           FREESPACE(0,0)
           INDEXED  )

         DATA(NAME(HL1.BSON.KSDS1.DATA)
            CYLINDERS(10,1)
            VOLUME(*)
            CISZ(32768))

         INDEX(NAME(HL1.BSON.KSDS1.INDEX)
            TRACKS(5,1)
            CISZ(4096)
            VOLUME(*))
```

With the DATABASE parameter, the other parameters in the DEFINE CLUSTER command are affected as follows:

- RECORDSIZE values are ignored. With DATABASE, RLS sets RECORDSIZE to the CA size.
- SPANNED/NONSPANNED values are ignored. With DATABASE, the parameter is always set to SPANNED.
- KEYS values are ignored. With DATABASE, the value is always set to KEYS(132 0).

The DATA and INDEX parameters shown are:

- CYLINDERS – with the DATABASE parameter, the minimum allocation is set to one cylinder, and overridden to one cylinder if a lower value was specified. The CI size is likewise set to a minimum of 4K bytes. (The specified value of 32KB is overridden to become 26KB.)
- Those DATABASE restrictions do not apply to parameters in the index component.

Chapter 20. DELETE

The DELETE command deletes catalogs, VSAM data sets, non-VSAM data sets, and objects. The syntax of the DELETE command is:

Command	Parameters
DELETE	(*entryname*[*entryname* ...])
	[ALIAS|
	ALTERNATEINDEX|
	CLUSTER|
	GENERATIONDATAGROUP|
	LIBRARYENTRY|
	NONVSAM|
	NVR|
	PAGESPACE|
	PATH|
	TRUENAME|
	USERCATALOG|
	VOLUMEENTRY|
	VVR]
	[ERASE|NOERASE]
	[FILE(*ddname*)]
	[FORCE|NOFORCE]
	[MASK|NOMASK]
	NODISCONNECT
	[PURGE|NOPURGE]
	[RECOVERY|NORECOVERY]
	[SCRATCH|NOSCRATCH]
	[CATALOG(*catname*)]

For VSAM RLS recoverable data sets, DELETE CLUSTER removes all pending recovery information for the sphere.

Recommendation: Do not delete any entryname for which there is any activity on any system in the sysplex, that is, active units of recovery, commits or back outs. Before deleting any entryname for which there is currently no activity but for which there are shunted log records, use the SHCDS PURGE command to clear the owed sync point (commit or back out). Otherwise, when the sync point is eventually retried, it will fail. In addition, you might receive unpredictable results on a future backout if a data set with the same name is later created.

DELETE can be abbreviated: DEL

© Copyright IBM Corp. 1973, 2021

DELETE Parameters

The DELETE command uses the following parameters.

Required Parameters

(*entryname*[*entryname*...])

names the entries to be deleted. If you want to delete more than one entry, you must enclose the list of entrynames in parentheses. The maximum number of entrynames you can specify is 100. You can code a generic name to delete multiple entries with one entryname. For example, GENERIC.*.BAKER is a generic name where * is any 1-to-8 character simple name. If you specify a generic name on the LEVEL keyword, only one qualifier can replace an asterisk (*). This applies when you either explicitly specify or default to NOMASK.

When you specify the MASK keyword, you can specify only one entryname, which is then considered a filter key. A filter key is an expanded generic name that allows for more versatile generic characters. The system only honors a filter key when the MASK keyword is specified in the DELETE command. (NOMASK is the default.)

If you specify entry names that include the following generics, the system considers them filter keys and so they can only be used when you specify MASK:

- Double asterisks (**), which represent multiple qualifiers within a data set name
- Percent signs (%), which represents one to eight percent characters within a qualifier
- An asterisk within a qualifier

For additional information on using MASK or NOMASK keyword see “Optional Parameters” on page 218.

If you are deleting a member of a non-VSAM partitioned data set, you must specify *entryname* in the format: *pdsname*(*membername*). The *membername* can contain wild card charaters %, and * to DELETE more than one members at a time.

The partitioned data set name directory is read to obtain a list of members, and each member in the list is filtered using the *membername* with the wildcard.

If a member name or alias name in the list matches the filter criteria, the member or alias is deleted. If the data set is a PDSE and the member name matches the criteria, the member and all aliases are deleted; if the data set is a PDS, only the aliases matching the filter criteria are deleted.

If you specify the entryname in the format of a partition data set, *pdsname*(*) or *pdsname*(**) , the command deletes all the members in the partition data set. If the partitioned data set is not a PDSE, the directory is reinitailized when using *pdsname*(*) , but not for *pdsname*(**).

If you are deleting a non-VSAM data set that was defined by coding DEVICETYPES(0000) and VOLUMES(******), then DELETE only uncatalogs the data set. It does not scratch the data set from the SYSRES volume.

In a TSO environment, when specifying the MASK parameter, the *entryname* is considered to be a string for the mask. Therefore, the UserID is not prefixed to the *entryname* as is normally done for other TSO DELETEs.

Using DELETE with the MASK Filter Option

When you use DELETE with the MASK filter option, the command extends across multiple levels of qualifiers as well as across multiple catalogs.

Using DELETE with MASK for Dependent Cluster Objects

Consider the following when deleting dependent objects when using generic MASK filtering:

- If you specify a base CLUSTER mask pattern, the command will delete all associated AIXs and PATHs. Any subsequent AIX or PATH specification via MASK will result in a CC=8 not found because the objects have already been deleted.
- If you specify an AIX mask pattern on your DELETE, it will delete all PATHs associated between this AIX and its base CLUSTER. Any subsequent PATH specified in a generic delete results in a CC=8 because the PATH has been deleted already.
- When deleting objects with dependent objects, consider the sequence of DELETE commands carefully.

Based on these considerations, we recommend the following:

- To DELETE all CLUSTER objects, restrict the MASK pattern to CLUSTER objects, which deletes all associated AIX and PATH objects as well.
- If you want to DELETE just the PATH objects, restrict the MASK pattern to the PATH subtype. Any subsequent AIX or CLUSTER MASK DELETE will then successfully delete AIX and CLUSTER objects.
- To DELETE AIXs after deleting PATH objects, restrict the MASK pattern to the AIX subtype.

Using a MASK pattern for Objects other than CLUSTER, NONVSAM, and GDS

Using DELETE with a MASK pattern will delete only CLUSTER, NONVSAM, and GDS (Generation Data Set) objects. To delete other objects, you must specify a type parameter, such as PATH or AIX. When you use a type to restrict the objects to delete, IBM suggests that you specify a DELETE with dependent objects first, then their base objects - this avoids possible conflicts with objects already deleted. For example,

- To DELETE PATH objects, do so with MASK and PATH type restrict parameter. You can then issue a DELETE with an AIX parameter, and finally issue a DELETE with a CLUSTER parameter. If you issue the first DELETE with CLUSTER, the command deletes all three sets of objects underneath the matching cluster.
- To delete a GDG BASE and its GDSes, first issue a DELETE with MASK for the GDSs only using a NONVSAM parameter and then delete the GDG BASEs with a GENERATIONDATAGROUP parameter.

Using DELETE with MASK in a TSO environment

In a TSO environment, an *entryname* is considered to be a string for the mask. Therefore, the userID is not prefixed to the *entryname* as is normally done for other TSO DELETEs.

Deleting GDS objects

When you delete GDS objects under a GDG base using a MASK pattern, note that the GDS might not be in ascending numeric sequence. See Chapter 2, “Modal Commands,” on page 15 for more information.

Using DELETE with GDSs and non-VSAM data sets with dependent ALIASes

Generation data sets (GDSs) in a generation data group (GDG) base object and non-VSAM objects and may have ALIASes, which are also dependent objects, so you must consider your DELETE mask pattern carefully. For example, a DELETE mask pattern of MASK NVSAM directed against a set of GDSs results in the associated ALIASes for those GDSes being deleted or scratched as well. The safe way to preserve a GDG BASE, is to perform a DELETE NVSAM with a MASK pattern that specifically identifies the GDSs. For example, for a UCAT A1, and a GDG BASE called BASE1, the following command removes the GDSes and all ALIASes associated with those GDSs:

```
DELETE   A1.BASE1.G%%%%V00 MASK NVSAM
```

You should delete GDG BASE objects with GENERATIONDATAGROUP and RECOVERY parameters specified.

User catalogs can be deleted by specifying both USERCATALOG and RECOVERY.

GDG base objects do not recognize SCRATCH, which is the default in a DELETE command specified with the MASK parameter. A MASK pattern which includes these objects without type-restrictions will have an implied SCRATCH option, resulting in an error.

Deleting USERCATALOG objects

- You cannot specify both MASK and FORCE with a USERCATALOG object because FORCE is not supported with a generic DELETE and USERCATALOG. Instead, specify RECOVERY, which is allowed for both MASK and NOMASK DELETE of USERCATALOGs. FORCE is not supported because it would enable anyone to DELETE an unlimited amount of USERCATALOGs in one statement, leaving no way to recover the data other than system back-up.
- User catalog objects do not recognize SCRATCH, which is the default in a DELETE command specified with the MASK parameter. A MASK pattern which includes these objects without type-restrictions will have an implied SCRATCH option, resulting in an error.

Optional Parameters

ALIAS| ALTERNATEINDEX| CLUSTER| GENERATIONDATAGROUP| LIBRARYENTRY| NONVSAM| NVR| PAGESPACE| PATH| TRUENAME| USERCATALOG| VOLUMEENTRY| VVR

specifies the type of object or entry to be deleted. If the object to be deleted is a catalog, truename entry, or VSAM volume record, USERCATALOG, TRUENAME, NVR, or VVR is required.

If you delete a migrated data set without specifying the entry type, DFSMShsm will delete the data set without recalling it.

ALIAS
specifies that the entry to be deleted is an alias entry.

ALTERNATEINDEX
specifies that the object to be deleted is an alternate index and its data and index entries. When a path entry is associated with the alternate index, the path entry is also deleted.

When the alternate index has the to-be-upgraded attribute and it is the only such alternate index associated with the base cluster, the base cluster's upgrade-set entry is also deleted.

Exception: If RLS recovery is associated with the alternate index, all knowledge of the recovery is lost as part of the delete operation.

Abbreviation: AIX

CLUSTER
specifies that the object to be deleted is a cluster, its associated data and index entries, and any related paths and alternate indexes.

When deleting a VVDS, entryname must be the restricted name 'SYS1.VVDS.Vvolser'.

Exception: If RLS recovery is associated with the sphere, all knowledge of the recovery is lost as part of the delete operation.

Abbreviation: CL

GENERATIONDATAGROUP
specifies that the entry to be deleted is a generation data group (GDG) entry. To delete a generation data group that is not empty, you must specify either the FORCE or the RECOVERY parameter. When FORCE is used, all SMS-managed generation data sets pointed to by the GDG base are scratched. Generation data sets are also removed from the catalog when you use FORCE.

For both SMS-managed and non-SMS-managed GDGs, if you use RECOVERY, the GDG entry is deleted from the catalog and generation data sets remain unaffected in the VTOC. To delete a GDG using RECOVERY or FORCE, you must specify both GENERATIONDATAGROUP and RECOVERY or GENERATIONDATAGROUP and FORCE.

The FORCE and RECOVERY generation data set parameters require RACF FACILITY class authorization. For information concerning RACF authorization levels, see Appendix A, “Security Authorization Levels,” on page 357.

Abbreviation: GDG

LIBRARYENTRY
specifies that the entry to be deleted is a tape library entry. You must specify the FORCE parameter to delete a tape library entry that is not empty. A tape library entry is not empty when tape volume entries are still associated with it.

To delete a tape library entry, you must have authorization to RACF FACILITY class profile STGADMIN.IGG.LIBRARY.

Because access method services cannot change the library manager inventory in an automated tape library, ISMF should be used for normal tape library delete functions. The access method services DELETE LIBRARYENTRY command should be used only to recover from volume catalog errors.

Abbreviation: LIBENTRYor LIBENT

NONVSAM
specifies that the entry to be deleted is a cataloged non-VSAM data set entry or object entry.

If the non-VSAM data set has aliases that are defined using RELATE parameter, all of its ALIAS entries are deleted when you use the DELETE command.

If the non-VSAM data set has aliases that are defined using SYMBOLICRELATE parameter, the ALIAS entries are NOT deleted when you use the DELETE command. See *z/OS DFSMS Managing Catalogs*

If the non-VSAM data set is partitioned, you can delete one of its members by specifying pdsname(membername).

If the non-VSAM data set does not have an entry in a catalog, you can delete its format-1 DSCB from the VTOC by using the SCRATCH function of the IEHPROGM utility. See *z/OS DFSMSdfp Utilities*.

Use this parameter to delete generation data sets (GDSs). You can rerun the job step to reclaim a GDS that is in deferred roll-in state, if GDS reclaim processing is enabled (it is enabled by default). For more information about GDS reclaim processing, see *z/OS DFSMSdfp Storage Administration*.

SMS does not support temporary non-VSAM data sets.

Exception: You can use RACF commands to specify an ERASE attribute in a generic or discrete profile for a non-VSAM data set. Use of the attribute renders all allocated DASD tracks unreadable before space on the volume is made available for reallocation. Refer to the appropriate RACF publications for information about how to specify and use this facility.

Abbreviation: NVSAM

NVR
specifies that the object to be deleted is an SMS-managed non-VSAM volume record (NVR) entry. This parameter must be specified to delete an NVR from a VSAM volume data set (VVDS) and its corresponding record from the VTOC. The NVR/VTOC entries are deleted only if the related non-VSAM object catalog entry does not exist.

Similar to DELETE VVR, the FILE parameter must specify the DD statement name that identifies the volume containing the VVDS. If you select a catalog through alias orientation or by use of the catalog parameter, it must match the catalog name in the isolated NVR (unless you have read authority to the RACF FACILITY class STGADMIN.IGG.DLVVRNVR.NOCAT).

Note: If the DELETE NVR command is issued against a non-VSAM data set that is not SMS-managed, the DSCB for the data sets is SCRATCHed (deleted). If cataloged, the data set's entry is not deleted from the catalog.

PAGESPACE
specifies that an inactive page space is to be deleted. A page space is identified as "active" during the operator's IPL procedure.

To delete a page space in an SMS-managed user catalog you must include the CATALOG parameter.

Abbreviation: PGSPC

PATH
specifies that a path entry is to be deleted. No entries associated with the path are deleted.

TRUENAME
specifies that the object to be deleted is the truename entry for a data or index component of a cluster or alternate index, or the name of an alternate index. This parameter must be specified to delete a truename entry. The truename entry is deleted only if the associated base record is missing or is inaccessible.

Abbreviation: TNAME

USERCATALOG
specifies that the object to be deleted is a user catalog.

See the CATALOG(*catname*) parameter for deleting a specific catalog. Specify USERCATALOG only when you want to delete the entire CATALOG.

The catalog connector entry in the master catalog is deleted. If the user catalog has aliases, all the catalog's alias entries in the master catalog are deleted.

To delete a user catalog when it is empty (that is, it contains only its self-describing entries and its volume's VVDS entry), you must specify USERCATALOG. To delete a user catalog that is not empty, you must specify both USERCATALOG and FORCE.

If you are deleting the catalog as part of recovering from a backup copy, you might want to use the RECOVERY option instead. For more information, see the RECOVERY keyword.

You can set up your system so that when you specify RECOVERY with DELETE USERCATALOG, the system issues WTOR message IDC1999I requesting confirmation before deleting the user catalogs. Use the modify command to enable or disable this WTOR feature as follows: F `CATALOG,ENABLE(DELRECOVWNG)` or F `CATALOG,DISABLE(DELRECOVWNG)`. By default, the WTOR is disabled. See the MODIFY command in *z/OS MVS System Commands*.

Abbreviation: UCAT

VOLUMEENTRY
specifies that the entry to be deleted is a tape library volume.

To delete a tape volume entry, you must have authorization to RACF FACILITY class profile STGADMIN.IGG.LIBRARY. Because access method services cannot change the library manager inventory in an automated tape library, ISMF should be used for normal tape library delete functions. The access method services DELETE VOLUMEENTRY command should be used only to recover from volume catalog errors.

Abbreviation: VOLENTRY or VOLENT

VVR
specifies that the objects to be deleted are one or more unrelated VSAM volume record (VVR) entries. To delete a VVR from both the VSAM volume data set (VVDS) and from the VTOC, you must specify this parameter.

The VVR entry is deleted only if the related cluster or alternate-index data and index component catalog entries do not exist. When VVR is specified, the component name of the cluster or alternate-index to which the VVR was related must be specified in the entryname parameter. If you select a catalog through alias orientation or by use of the catalog parameter, it must match the catalog name in the isolated VVR (unless you have read authority to the RACF FACILITY class STGADMIN.IGG.DLVVRNVR.NOCAT).

The FILE parameter must specify the DD statement name that identifies the volume on which the VVDS resides.

Note: If the catalog search order does not orient to the original catalog in the VVR, and the user does not have READ authority to STGADMIN.IGG.DLVVRNVR.NOCAT, in this case the master catalog name must be specified in the CATALOG parameter as follows.

CATALOG(*catname*)
specifies the name of the catalog that contains the entries to be deleted. See "Catalog Search Order for DELETE" on page 9 for the order in which catalogs are searched.

This parameter cannot be used to delete a user catalog, and is ignored when you delete members of a partitioned data set or the tape library entry.

To specify catalog names for SMS-managed data sets, you must have authority from the RACF STGADMIN.IGG.DIRCAT FACILITY class. See "Storage Management Subsystem (SMS) Considerations" on page 2 for more information.

catname
identifies the catalog that contains the entry to be deleted.

Abbreviation: CAT

ERASE|NOERASE
specifies whether the components of a cluster or alternate index to be deleted are to be erased (overwritten with binary zeros). This parameter overrides whatever was coded when the cluster or alternate index was defined or last altered. Specify this parameter only when a cluster or an alternate index entry is to be deleted.

If you use ERASE, one of the following conditions must be true:

- The entry is in the master catalog.
- The qualifiers in the entry's qualified name are the catalog's name or alias.

ERASE
specifies that the components are to be overwritten with binary zeros when the cluster or alternate index is deleted. If ERASE is specified, the volume that contains the data component must be mounted.

If the cluster is protected by a RACF generic or discrete profile and the cluster is cataloged in a catalog, use RACF commands to specify an ERASE attribute as part of this profile so that the data component is automatically erased upon deletion.

Abbreviation: ERAS

NOERASE
specifies that the components are not to be overwritten with binary zeros when the cluster or alternate index is deleted.

NOERASE will not prevent the component from being erased if the cluster is protected by a RACF generic or discrete profile that specifies the ERASE attribute and the cluster is cataloged in a catalog. You can use RACF commands to alter the ERASE attribute in a profile.

Abbreviation: NERAS

FILE(*ddname*)
specifies the name of the DD statement that identifies:

- The volume that contains a data set to be deleted with SCRATCH.
- The data set to be deleted if ERASE is specified.
- The partitioned data set from which a member (or members) is to be deleted.
- The volumes that contain VVDS entries for the objects cataloged.
- The VVDS volume that contains a VVR or NVR to be deleted.

Use of the FILE parameter improves the performance of the DELETE command.

When you delete a data set, the volume referred to in the DD statement must be the same as the volume referred to in the usercatalog.

If you do not specify FILE and VSAM requires access to a volume or volumes during the delete processing, VSAM tries to dynamically allocate the volumes. When the entryname is

pdsname(membername) VSAM dynamically allocate the entire PDS rather then the pdsname(member).

When more than one volume is to be identified (for example, a multivolume data set), FILE identifies the DD statement that specifies all volumes. If in any of the above cases the volumes are of a different device type, use concatenated DD statements. All volumes that contain associations to a cluster being deleted must also be included on the DD statement referenced by the FILE parameter.

When deleting multivolume non-VSAM data sets with the SCRATCH option, DELETE SCRATCH processing requires access to each volume in the entry's catalog record before the scratch can be issued. This requires either all volumes to be mounted, online, and allocatable to the job, or the use of the FILE parameter specifying a DD statement allocating at least one mountable unit (not permanently resident or reserved). Deferred mount must be specified on the DD statement so that allocation will flag the UCB to allow remove/mount requests to be issued for the unit as required during delete processing. If access to the volumes cannot be provided, use DELETE NOSCRATCH to uncatalog the non-VSAM data set and the user will assume the responsibility of scratching the format-1 DSCBs from all the volumes. If RACF is installed, you must have access authority under RACF to specify DELETE NOSCRATCH.

When the FILE parameter points to a DD statement that has DISP=SHR the data set can be deleted when allocated to another user, but is not open.

The use of this DD name in subsequent commands in the same invocation of IDCAMS may not work properly. Specifically, DEFINE, BLDINDEX, REPRO, and IMPORT may fail if these commands refer to the same DD name for output from those commands. This is because those commands will use volume and device-related information that may no longer be applicable. BLDINDEX, REPRO, and IMPORT should use the OUTDATASET keyword instead of OUTFILE to avoid this problem.

FORCE | NOFORCE

specifies whether entries that are not empty should be deleted.

FORCE

lets you delete generation data groups, tape library entries, and user catalogs without first ensuring that these entries are empty.

Attention: The FORCE parameter deletes all clusters in the catalog.

If you delete a generation data group using FORCE:

- Proper access authority to the RACF resource for catalog functions is necessary for DELETE GDG FORCE. The DELETE GDG FORCE function should not be used to redefine the GDG limit value. ALTER LIMIT should be used instead.
- The GDG entry is deleted even though it might point to non-VSAM entries in the catalog.
- Each SMS-managed non-VSAM data set entry pointed to by the GDG base entry is deleted before the GDG base entry is deleted. The non-VSAM data set is scratched.
- Each non-SMS-managed non-VSAM data set entry pointed to by the GDG base entry is deleted before the GDG base entry is deleted. However, the non-VSAM data set's space and contents on the volume are undisturbed.
- No VVDSs are deleted
- An IKJ56220I is possible when perform in DELETE GDG FORCE against a GDG with a large limit and GDSs with many candidate volumes. Similar to limitations imposed on concatenated data sets in a single DD, allocation of an entire GDG is dependent on the size of the TIOT. DELETE GDG FORCE is a unique command that requires allocation of all generation data sets under a GDG in order to perform the forced delete.

If you delete a tape library entry using FORCE:

- The tape library entry is deleted even if tape volume entries are still associated with the specified tape library.
- Any tape volume entries associated with a deleted tape library entry will remain in the catalog for these tape volume entries.

If you delete a user catalog using FORCE:

Attention: The FORCE parameter deletes all clusters in the catalog.

- The user catalog is deleted even if it contains entries for objects that have not been deleted.
- All data sets cataloged in the user catalog as well as the catalog data set itself are deleted. All volumes on which these data sets reside must be included with the FILE parameter.
- All VSAM clusters are automatically deleted, but the contents of each cluster and alternate index are not erased. (If you specify FORCE, the ERASE parameter is ineffective.)
- SMS-managed non-VSAM data set entries in the user catalog are deleted and the data sets are scratched.
- Non-SMS-managed non-VSAM data set entries in the user catalog are deleted, but the data sets are not scratched. A non-SMS-managed non-VSAM data set can be located with its DSCB in the volume's VTOC.
- If you are using the XFACILIT class STGADMIN.IGG.DELAUDIT.*catalogname* to restrict deletion of data sets in the user catalog, those data sets to which the user does not have authority will not be deleted, but the user catalog will be deleted. The entries failing the delete will not be accessible until they are recataloged.

Abbreviation: FRC

NOFORCE
causes the DELETE command to end when you request the deletion of a generation data group, tape library entry, or user catalog that is not empty.

Abbreviation: NFRC

MASK | NOMASK
The MASK parameter specifies that the entryname is a filter key. The entryname or filter key might contain mask characters such as *, ** or % and is used to search for entries to delete that match the filter key.

MASK
The MASK keyword cannot be specified with the following keywords:

- TRUENAME (TRUENAME)
- Non-VSAM Volume record (NVR)
- VSAM Volume Record (VVR)
- PDSE/PDS member data set
- Library Entry (LIBRARYENTRY)
- Tape Volume entry (VOLUMEENTRY)

The DELETE MASK command allows only one entry-name to be specified. If there are multiple entry-names specified, the request will fail with error messages.

When you specify a filter key with the MASK operand, the following rules apply:

- An asterisk (*) represents 1 to 8 characters in a level of a catalog entry.
- Double asterisks (**) represents multiple qualifiers within a catalog entry. The double asterisks (**) must be preceded by a period, and the double asterisks can be followed by either a period or a blank.
- Percent sign (%) allows 1 to 8 percent signs (%) specified in each qualifier.
- The high level qualifier of the filter key must be fully qualified unless the CATALOG parameter is also supplied. In other words, the high level qualifier cannot contain a *, ** or %. ABC %.DATA.SET and **.DATA.SET and A*.DATA.SET are all examples of an invalid mask. If you want to specify names like ABC%.DATA.SET or **.DATA.SET or A*.DATA.SET where the high level qualifier is not fully qualified, you must also specify the CATALOG parameter.

- When using asterisk within a qualifier, this is considered as a mask entry name and a MASK keyword must be specified. Examples are 'A.B*.C', 'AB.CD*', 'AB.*CD.'.

See Catalog Search Interface User's Guide in *z/OS DFSMS Managing Catalogs* for more information regarding data set name filter keys.

NOMASK

NOMASK is the default. With NOMASK the entryname can be fully qualified or a generic name. When using a generic name, the asterisk(*) will only replace one single qualifier.

Here are examples of how DELETE works using MASK, NOMASK or omitting the MASK or NOMASK parameter, given the following data sets.

```
1)  AAA.BBB.AAA.DDD
2)  AAA.BBB.CCC.DDD
3)  AAA.BBB.CCC.DDD.EEE
4)  AAA.BBB.CCC
5)  BBB.DDD.AAC.BBC.EEE
6)  BBB.DDD.ABC.BBC.EEE
7)  BBB.DDD.ADC.BCCD.EEEE
8)  CCC.GDG.BASE1                       This entry is an empty GDG base. It
has no entries
9)  CCC.GDG.BASE2                       This entry is not an empty GDG base.
It has entries
10) CCC.GDG.BASE2.G0001V00              This is an entry that belongs to
CCC.GDG.BASE2
11) CCC.GDG.BASE2.G0002V00              This is an entry that belongs to
CCC.GDG.BASE2
12) CCC.GDG.FLAT.FILE                   This is a non-GDG
```

When NOMASK or parameter is omitted, the following are the results:

```
DELETE AAA.*                                  results in no data sets being
deleted
DELETE AAA.* NOMASK                           results in no data sets being
deleted
DELETE AAA.*  MASK                          results in no data sets being
deleted
DELETE AAA.BBB.*                              results in the deletion of
data set #4
DELETE AAA.BBB.* NOMASK                       results in the deletion of
data set #4
DELETE AAA.BBB.*.DDD                          results in the deletion of
data sets #1
                                              and #2

DELETE AAA.BBB.* MASK                        results in the deletion of data
set #4
DELETE AAA.BBB.*.DDD NOMASK                   results in the deletion of
data sets #1
                                              and #2
DELETE AAA.BBB.*.DDD.EEE                      results in the deletion of
data set #3.
DELETE AAA.BBB.*.DDD.EEE NOMASK               results in the deletion of
data set #3.
DELETE AAA.BBB.AAA.DDD MASK                   results in the deletion of
data sets #1
DELETE CCC.GDG.*                              results in the deletion of
data set #9
                                              and an error message is issued
for data
                                              set #10, #11, and #12 because
the base
                                              is not empty
DELETE CCC.GDG.* NOMASK                       results in the deletion of
data set #9
                                              and an error message is issued
for data
```

```
                                                 set #10, #11, and #12 because
the base
                                                 is not empty
DELETE CCC.GDG.BASE1                             results in the deletion of
data set #9



DELETE CCC.GDG.BASE1 NOMASK                      results in the deletion of
data set #9
DELETE CCC.GDG.BASE2                             results in the deletion of no
data sets
                                                 as the GDG base has entries.
DELETE CCC.GDG.BASE2 NOMASK                      results in the deletion of no
data sets
                                                 as the GDG base has entries.
```

When MASK is specified, the following are the results:

```
DELETE AAA.** MASK                               results in deletion of #1 #2
#3 and #4
DELETE BBB.DDD.** MASK                           results in deletion of #5 #6
#7 and #8
DELETE BBB.DDD.BBC.A%C.BBC.EEE MASK              results in deletion of #5 #6
DELETE BBB.DDD.ADC.B%%%.EEEE MASK                results in deletion of #7 #8
DELETE AAA.*.** MASK                             results in deletion of #1 to #4
DELETE BBB.DDD.A*.BBC.EEE MASK                   results in deletion of #5 #6
DELETE BBB.DDD.A*C.BBC.EEE MASK                  results in deletion of #5 #6

DELETE CCC.GDG.BASE2.* MASK                      results in deletion of #11 and
#12 GDG



DELETE CCC.GDG.BASE2.G%%%%V00 MASK               results in deletion of #11 and
#12
DELETE CCC.GDG.BASE2.G*V00 MASK                  results in deletion of #11 and
#12
DELETE CCC.GDG.BASE2.G%%%%V00 MASK NVSAM         results in deletion of #11 and
#12
DELETE CCC.GDG.BASE2.G*V00 MASK NVSAM            results in deletion of #11 and
#12
```

NODISCONNECT
Use the NODISCONNECT parameter with the USERCATALOG parameter to retain alias information associated with the user catalog you are deleting. The catalog can then be redefined (DEFINE USERCATALOG) with the RECONNECT option to restore the saved alias information.

PURGE | NOPURGE
specifies whether the entry is to be deleted regardless of the retention period specified. If this parameter is used for objects that do not have a date associated with them (for example, VVRs, aliases, and non-SMS-managed non-VSAM data sets), the PURGE|NOPURGE parameter is ignored and the object is deleted. This parameter cannot be used if a truename entry is to be deleted.

PURGE must be specified to delete an OAM non-VSAM entry, because it has a never-expire retention.

PURGE
specifies that the entry is to be deleted even if the retention period, specified in the TO or FOR parameter, has not expired.

When deleting a tape library volume entry, PURGE must be specified if the volume's retention period has not expired.

PURGE works the same way for migrated objects as it does for non-migrated objects. PURGE overrides any DFSMShsm control over the deletion of VSAM base clusters and non-VSAM data sets. It causes the migrated data set to be deleted regardless of the expiration date.

Abbreviation: PRG

NOPURGE
specifies that the entry is not to be deleted if the retention period has not expired.

Abbreviation: NPRG

RECOVERY | NORECOVERY
specifies whether a user catalog, a VSAM volume data set (VVDS), or a generation data group (GDG) is to be deleted in preparation for recovery.

RECOVERY
When RECOVERY is specified and the entry name identifies a user catalog, the user catalog is to be replaced with an imported backup copy. The user catalog, its VSAM volume record (VVR), and its VTOC entries are deleted. The VVR and DSCBs, for each of the objects defined in the user catalog, are not deleted or scratched. If the catalog is RACF-protected, alter authority is required.

VSAM must be able to read the VVDS or be able to process it as an ESDS for the function to complete successfully.

When RECOVERY is specified and entryname identifies a VVDS, the VVDS is unusable or inaccessible and must be rebuilt by deleting, redefining, and loading the appropriate VSAM data sets on the volume. The VVDS entry's DSCB will be scratched from the VTOC. The CATALOG parameter must contain the name of the master catalog when a VVDS is deleted with the RECOVERY parameter. If RACF protected, ALTER authority is required.

When RECOVERY is specified and the entry name identifies a GDG, the SMS-managed or non-SMS-managed GDG entry is deleted from the catalog and generation data sets remain unaffected in the VTOC.

If a VVDS contains a catalog entry or a system data set (SYS1.) entry that is cataloged in a master catalog, the VVDS catalog entry and the DSCB of the associated VVDS will not be removed.

If you delete a generation data group (DELETE GDG RECOVERY) using RECOVERY, proper authority to the RACF resource for catalog function is necessary.

See the USERCATALOG parameter for information about using RECOVERY with

Abbreviation: RCVRY

NORECOVERY
indicates that the entry is to be processed as described by the other parameters specified.

Abbreviation: NRCVRY

SCRATCH | NOSCRATCH
specifies whether a data set is to be removed from the VTOC of the volume on which it resides. This parameter can be specified only for a cluster, an alternate index, a page space, or a non-VSAM data set.

Exception:

- The SCRATCH parameter is not applicable to tape library and tape volume entries because they have no VVDS or VTOC entries. IDCAMS DELETE will determine if the data set to be deleted is a tape data set and issue the NOSCRATCH option on the delete request. For a data set on tape, using the NONVSAM parameter with a fully qualified entryname might cause dynamic allocation of the data set, and therefore a tape mount. To avoid the tape mount in this situation, either specify NOSCRATCH or omit NONVSAM.
- If data set contains indirect or symbolic VOLSER, the scratch parameter will be ignored, if specified.

SCRATCH
specifies that a data set is to be scratched from (removed from the VTOC of) the volume on which it resides. For VSAM data sets and SMS-managed non-VSAM data sets, the VSAM volume data set (VVDS) entry is also removed.

DELETE SCRATCH will ignore any missing data set components (such as VVRs or F1 DSCBs) and will scratch all the data set parts that can be found at the time the request is issued.

When SCRATCH is specified for a VVDS, the VVDS is scratched and the catalog entry for the VVDS is removed. The VVDS must be empty.

If the catalog entry does not exist for a non-VSAM data set, you can use the SCRATCH function of the OS/VS IEHPROGM utility to remove the format-1 DSCB from the VTOC.

If you select SCRATCH, one of the following statements must be true:

- The entry is in the master catalog.
- One or more of the qualifiers in the entry's qualified name is the same as the catalog's name or alias.
- The FILE parameter is specified.

If you specify SCRATCH when deleting a non-VSAM data set defined with an esoteric device type, SYSDA for example, the DELETE will be unsuccessful under the following circumstances:

- Input/output configuration is changed resulting in addition or deletion of one or more esoteric device types.
- The esoteric device type definitions on the creating and using systems do not match when the catalog is shared between the two systems.

Note: If the VVDS indicates that the data set is owned by a catalog other than that catalog identified through the usual catalog search order for DELETE, a DELETE NOSCRATCH is done against the catalog that resulted from the catalog search, and a zero return code is returned to the user. For example, if you specify a DELETE SCRATCH command against a data set in Catalog A, and the VVDS indicates that the data set is owned by Catalog B, a DELETE NOSCRATCH operation is done against Catalog A, and the data set remains intact and accessible from Catalog B. See “Catalog Search Order for DELETE” on page 9 for information on the catalog search order for DELETE.

Abbreviation: SCR

NOSCRATCH
specifies that the catalog entry is to be deleted from the catalog without mounting the volume that contains the object defined by the entry. VVDS and VTOC entries are not deleted.

If RACF is installed, you must have access authority under RACF to specify NOSCRATCH. With proper authority, DELETE NOSCRATCH is allowed on SMS-managed VSAM and non-VSAM data sets, thus deleting the BCS entry in the catalog without accessing the VVDS or VTOC.

Attention:

- DELETE NOSCRATCH can result in uncataloged SMS-managed data sets.
- DELETE NOSCRATCH cannot be issued against a VSAM data set which is currently open.

For more information on cloning zFS, see the VOLUMES parameter in Access Method Services DEFINE CLUSTER command.

NOSCRATCH removes the catalog entry for a VVDS. This entry can be reinstated with DEFINE RECATALOG. If the volume is mounted and usable, the VVDS is checked to ensure that the catalog entry being removed has no data sets in the VVDS. If the catalog entry indicates there are data sets in the VVDS, the VVDS's VSAM volume control record (VVCR) is removed and the catalog entry for the VVDS is removed.

If the volume is mounted and you specify NOSCRATCH for a VSAM volume data set (VVDS), the catalog entry for the VVDS is removed, and the catalog back pointer in the VSAM volume control record (VVCR) is removed.

You should specify NOSCRATCH for the following:

- If the format-1 DSCB of a non-VSAM data set has already been scratched from the VTOC.
- If you are deleting a non-VSAM data set that was defined with a device type named by the user (for example, SYSDA) and the device type is not valid.
- If the object is defined in a catalog and you want to recatalog the object in the same catalog.

- After you convert a volume, the names of catalogs owning data sets on the volume will still be in the VVCR. Only catalogs that reside on the converted volume need to have their names in the VVCR. You can remove unneeded catalog names from the VVCR by using DELETE VVDS NOSCRATCH with the CATALOG parameter referencing the catalog to be deleted from the VVCR. For coexistence, an error indication is still returned if there are VVR or NVRs on the volume for the referenced catalog.
- NOSCRATCH affects the DFSMShsm delete function interaction for VSAM base clusters and non-VSAM data sets. It causes the migrated data set to be recalled because a migrated data set cannot be uncataloged.

Abbreviation: NSCR

DELETE Examples

The DELETE command can perform the following functions.

Delete a Truename Entry in a Catalog: Example 1

In this example, the truename entry for a data component of an alternate index is deleted. The purpose of this example is to remove the truename of an entry when an error has occurred, leaving the associated base record either inaccessible or missing. Removing the name allows a subsequent DEFINE command to reuse the name without an error caused by a duplicate name situation.

```
//DELET12    JOB   ...
//STEP1     EXEC  PGM=IDCAMS
//SYSPRINT  DD    SYSOUT=A
//SYSIN     DD    *
     DELETE -
            K101.AIX.DATA -
            TRUENAME -
            CATALOG(USERCAT4)
/*
```

The DELETE command deletes a truename entry that exists without its associated base record. The parameters are:

- K101.AIX.DATA is the entryname of the alternate index's data component to be deleted.
- TRUENAME specifies the type of entry to be deleted. When a truename entry is to be deleted, the TRUENAME parameter is required.
- CATALOG identifies the catalog that contains the entry to be deleted, USERCAT4.

Delete a User Catalog for Recovery: Example 2

In this example, a user catalog is deleted in preparation for replacing it with an imported backup copy. The VVDS and VTOC entries for objects defined in the catalog are not deleted and the data sets are not scratched.

```
//DELET13    JOB   ...
//STEP1     EXEC  PGM=IDCAMS
//DD1       DD    VOL=SER=VSER01,UNIT=3380,DISP=OLD
//SYSPRINT  DD    SYSOUT=A
//SYSIN     DD    *
     DELETE -
            USERCAT4 -
            FILE(DD1) -
            RECOVERY -
            USERCATALOG
/*
```

The DELETE command deletes a user catalog without deleting the VVDS and VTOC entries of the objects defined in the catalog. Its parameters are:

- USERCAT4 is the name of the catalog.

- FILE specifies the ddname of a DD statement that describes the user catalog's volume and causes it to be mounted.
- RECOVERY specifies that only the catalog data set is deleted without deleting the objects defined in the catalog.
- USERCATALOG specifies that the entryname identifies a user catalog. When a user catalog is to be deleted, the USERCATALOG parameter is required.

Delete VSAM Volume Records: Example 3

In this example, VSAM volume records (VVRs) belonging to a data component of a key-sequenced cluster are deleted from the VVDS. The purpose of this example is to clean up the VVDS when there are residual records as a result of an error.

```
//DELET14    JOB   ...
//STEP1     EXEC  PGM=IDCAMS
//DD1       DD    VOL=SER=VSER01,UNIT=3380,DISP=OLD
//SYSPRINT  DD    SYSOUT=A
//SYSIN     DD    *
     DELETE -
            EXAMPLE.KSDS01.DATA -
            FILE(DD1) -
            VVR
/*
```

The DELETE command deletes the VVRs associated with a VSAM cluster from the VVDS and from the VTOC. Its parameters are:

- EXAMPLE.KSDS01.DATA is the name of the data component of the cluster.
- FILE specifies the ddname of a DD statement that describes the volumes that contain VVDS entries associated with this cluster.
- VVR specifies that only the VVRs for the cluster are to be deleted.

Delete a Non-VSAM Data Set's Entry: Example 4

In this example, a non-VSAM data set's entry is deleted. The SCRATCH parameter is the default. A FILE parameter and its associated DD statement are provided to allocate the data set's volume. In this example, dynamic allocation is not used to provide catalog or volume allocation. This example applies only to non-VSAM data sets that have catalog entries.

```
//DELET4   JOB   ...
//STEP1    EXEC  PGM=IDCAMS
//DD1      DD    VOL=SER=VSER02,UNIT=3380,DISP=OLD,
//   DSNAME=EXAMPLE.NONVSAM
//SYSPRINT DD    SYSOUT=A
//SYSIN    DD    *
     DELETE -
           EXAMPLE.NONVSAM -
           FILE (DD1) -
           PURGE -
           CATALOG(USERCAT4)
/*
```

The DELETE command deletes the non-VSAM data set EXAMPLE.NONVSAM. Its parameters are:

- EXAMPLE.NONVSAM is the entryname of the object to be deleted.
- FILE specifies the ddname of a DD statement that describes the non-VSAM data set's volume and causes it to be mounted. When the data set is deleted, its DSCB entry in the volume's VTOC is removed.
- PURGE specifies that the non-VSAM data set's retention period or date is to be ignored.
- CATALOG identifies the catalog that contains the entries, USERCAT4.

Delete Entries Associated with a Non-VSAM Object from VVDS and VTOC: Example 5

The following example shows how to delete entries associated with a non-VSAM object from the VVDS and VTOC. The purpose of this command is to clean up the VVDS and VTOC when there are residual records as a result of an error.

```
//DELET14  JOB   ...
//STEP1    EXEC  PGM=IDCAMS
//DD1      DD    VOL=SER=VSER01,UNIT=3380,DISP=OLD
//SYSPRINT DD    SYSOUT=A
//SYSIN    DD    *
     DELETE -
           EXAMPLE.NONVSAM -
           FILE(DD1) -
           NVR
/*
```

The above DELETE command deletes the NVR associated with a non-VSAM object from the VVDS and its corresponding entry from the VTOC if they exist. Its parameters are:

- EXAMPLE.NONVSAM, the name of the non-VSAM object. There must not be a BCS entry for this object.
- FILE, specifies the ddname of a DD statement that describes the volume containing the VVDS entry associated with this object.
- NVR, specifies that only the NVR and its corresponding VTOC entry for this object are to be deleted.

Delete a Key-Sequenced VSAM Cluster in a Catalog: Example 6

In this example, a key-sequenced cluster is deleted. Alternate indexes and paths related to the key-sequenced cluster are deleted automatically by access method services. Access method services will dynamically allocate the key-sequenced data set so that the data can be overwritten (as specified by the ERASE option).

```
//DELET1   JOB   ...
//STEP1    EXEC  PGM=IDCAMS
//DD1      DD    VOL=SER=VSER02,UNIT=3380,DISP=OLD,
//   DSNAME=EXAMPLE.KSDS01
//SYSPRINT DD    SYSOUT=A
//SYSIN    DD    *
     DELETE -
           EXAMPLE.KSDS1 -
           FILE(DD1)
           PURGE -
           ERASE -
           CATALOG(GGGUCAT2)
/*
```

The DELETE command deletes the key-sequenced VSAM cluster from the GGGUCAT2 catalog. Its parameters are:

- EXAMPLE.KSDS1, which is a key-sequenced VSAM cluster, is the entryname of the object being deleted.
- FILE is not required but improves performance if specified.
- PURGE specifies that the cluster is to be deleted regardless of its retention period or date.
- ERASE specifies that the cluster's data component be overwritten with binary zeros. If the NOERASE attribute was specified when the cluster was defined or altered, this is ignored.
- CATALOG identifies the catalog, GGGUCAT2, containing the cluster's entries.

Delete Two Key-Sequenced Clusters in a Catalog: Example 7

In this example, two key-sequenced clusters, EXAMPLE.KSDS01 and EXAMPLE.KSDS02, are deleted from a catalog. It shows how more than one cataloged object is deleted with a single DELETE command.

```
//DELET3   JOB   ...
//STEP1    EXEC  PGM=IDCAMS
```

```
//SYSPRINT DD    SYSOUT=A
//SYSIN    DD    *
     DELETE -
           (EXAMPLE.KSDS01 -
           EXAMPLE.KSDS02) -
           PURGE -
           CLUSTER
/*
```

The DELETE command deletes the key-sequenced clusters EXAMPLE.KSD01 and EXAMPLE.KSD02. Both entries are dynamically allocated in order for their respective catalog recovery areas (CRA) to be updated. The parameters are:

- EXAMPLE.KSDS01 and EXAMPLE.KSDS02 identify the objects to be deleted. These are the entrynames of two key-sequenced clusters.
- PURGE specifies that the cluster be deleted regardless of its retention period or date.
- CLUSTER specifies that EXAMPLE.KSDS01 and EXAMPLE.KSDS02 identify cluster catalog records.

Delete a User Catalog: Example 8

In this example, a user catalog is deleted. A user catalog can be deleted when it is empty—that is, when there are no objects cataloged in it other than the catalog's volume. If the catalog is not empty, it cannot be deleted unless the FORCE parameter is specified.



Attention: The FORCE parameter deletes all clusters in the catalog.

```
//DELET6  JOB   ...
//STEP1    EXEC  PGM=IDCAMS
//SYSPRINT DD    SYSOUT=A
//SYSIN    DD    *
     DELETE -
           XXXUCAT1 -
           PURGE -
           USERCATALOG
/*
```

The DELETE command deletes both the catalog and the catalog's user catalog connector entry in the master catalog. The parameters are:

- XXXUCAT1 is the name of the user catalog.
- PURGE indicates the user catalog's retention period or date is to be ignored. If PURGE is not specified and the catalog's retention period has not yet expired, the catalog will not be deleted.
- USERCATALOG identifies XXXUCAT1 as a user catalog.

Delete an Alias Entry in a Catalog: Example 9

In this example, an alias entry, EXAMPLE.NONVSAM1, is removed from catalog USERCAT4.

```
//DELET7    JOB   ...
//STEP1     EXEC  PGM=IDCAMS
//SYSPRINT  DD    SYSOUT=A
//SYSIN     DD    *
     DELETE -
            EXAMPLE.NONVSAM1 -
            ALIAS -
            CATALOG(USERCAT4)
/*
```

The DELETE command removes an alias entry from catalog USERCAT4. Its parameters are:

- EXAMPLE.NONVSAM1 is the entryname of the object to be deleted. EXAMPLE.NONVSAM1 identifies an alias entry.
- ALIAS specifies the type of entry to be deleted. VSAM verifies that EXAMPLE.NONVSAM1 is an alias entry and then deletes it. If EXAMPLE.NONVSAM1 identifies another entry by mistake, VSAM does not delete the entry, but notes the discrepancy with a message to the programmer.

- CATALOG identifies the catalog containing the entry, USERCAT4.

Delete Generically Named Entries in a Catalog: Example 10

In this example, each catalog entry with the name GENERIC.*.BAKER is deleted, where * is any 1-to-8 character simple name. The name GENERIC.*.BAKER is a generic name, and all catalog entries with the same generic name are deleted. Use this example to delete multiple entries. Multiple entries are entries with three levels of qualification where the first is GENERIC and the third is BAKER.

```
//DELET8   JOB  ...
//STEP1    EXEC PGM=IDCAMS
//SYSPRINT DD   SYSOUT=A
//SYSIN    DD   *
     DELETE -
           GENERIC.*.BAKER -
           PURGE -
           CATALOG(USERCAT4)
/*
```

The DELETE command removes all entries (and their associated entries) with the generic name GENERIC.*.BAKER from catalog USERCAT4. Its parameters are:

- GENERIC.*.BAKER, a generic name, identifies all catalog entries with the high-level qualifier GENERIC and the low-level qualifier BAKER.
- PURGE specifies that each entry is to be purged regardless of the retention period or date specified when it was defined.
- CATALOG identifies the catalog, USERCAT4.

List a Generation Data Group's Entries, Then Delete the Group and Its Data Sets in a Catalog: Example 11

In this example, a generation data group, GDG01, and its associated (generation data set) entries are listed, the only generation data set in the group is deleted, and the generation data group base catalog entry is deleted.

```
//DELET9    JOB   ...
//STEP1     EXEC  PGM=IDCAMS
//SYSPRINT  DD    SYSOUT=A
//SYSIN     DD    *
     LISTCAT -
            ENTRIES(GDG01) -
            ALL
     DELETE -
            GDG01.G0001V00 -
       DELETE -
            GDG01 -
            GENERATIONDATAGROUP -
            PURGE
/*
```

The LISTCAT command lists the generation data group, GDG01, and its associated generation data set entries. The parameters are:

- ENTRIES specifies that the entry GDG01 be listed. Because GDG01 is a generation data group entry, its associated generation data set's (non-VSAM) entries are also listed. If one of the generation data sets has aliases, the alias entries associated with the generation data set's entry are listed.
- ALL specifies that all fields are to be listed.

  The first DELETE command removes the non-VSAM data set entry for the only generation data set, GDG01.G0001V00, in the generation data group. Its parameters are:
- GDG01.G0001V00 is the entryname of the object being deleted. GDG01.G0001V00 identifies the only generation data set in generation data group GDG01.
- GDG01 is the entryname of the object being deleted. GDG01 identifies the generation data group base entry.

- GENERATIONDATAGROUP specifies the type of entry being deleted. VSAM verifies that GDG01 is a generation data group entry, then deletes it. If GDG01 incorrectly specifies another type of entry, VSAM does not delete the entry, but notes the discrepancy with a message to the programmer.
- PURGE specifies that the generation data group's retention period or date be ignored.

Delete a Generation Data Group with Recovery: Example 12

In this example, a generation data group base catalog entry, GDG01, is deleted from the catalog. The generation data sets associated with GDG01 remain unaffected in the VTOC.

```
//DELETXX   JOB   ...
//STEP1     EXEC  PGM=IDCAMS
//SYSPRINT  DD    SYSOUT=A
          DELETE -
          GDG01 -
          GENERATIONDATAGROUP -
          RECOVERY
/*
```

The DELETE command removes the GDG base catalog entry from the catalog. Its parameters are:

- GDG01 is the name of the GDG base entry.
- GENERATIONDATAGROUP specifies the type of entry being deleted. VSAM verifies that GDG01 is a GDG entry, then deletes it. If GDG01 is not a GDG entry, VSAM issues a message and does not delete it.
- RECOVERY specifies that only the GDG base entry name in the catalog is deleted. Its associated generation data sets remain intact in the VTOC.

Delete a Member of a Partitioned (Non-VSAM) Data Set in a Catalog: Example 13

In this example, the MEM1 member of partitioned data set EXAMPLE.NONVSAM2 is deleted, then the data set itself is deleted.

```
//DELET10    JOB   ...
//STEP1     EXEC  PGM=IDCAMS
//SYSPRINT  DD    SYSOUT=A
//SYSIN     DD    *
     DELETE -
            EXAMPLE.NONVSAM2(MEM1) -
     DELETE -
            EXAMPLE.NONVSAM2 -
            PURGE -
            CATALOG(USERCAT4)
/*
```

The first DELETE command deletes a member of a partitioned data set, EXAMPLE.NONVSAM2(MEM1), from the user catalog USERCAT4. Its parameters are:

- EXAMPLE.NONVSAM2(MEM1) is the entryname of a member of the partitioned data set, EXAMPLE.NONVSAM2. The entryname identifies the object to be deleted.

  The second DELETE command deletes all remaining members and then the partitioned non-VSAM data set, EXAMPLE.NONVSAM2, itself. Its parameters are:
- EXAMPLE.NONVSAM2 is the entryname of the object being deleted.
- PURGE specifies that the non-VSAM data set's retention period or date be ignored. If PURGE is not specified and the non-VSAM data set's retention period has not expired, VSAM does not delete its entry.
- CATALOG identifies the catalog, USERCAT4.

In the second part of this example, the DSCB entry in the volume's VTOC is removed. Dynamic allocation is used to allocate the data set's volume.

Delete a Member of a Partitioned (Non-VSAM) Data Set in a Catalog: Example 13-A

In this example, the MEM1 member of partitioned data set EXAMPLE.NONVSAM2 is deleted with minimal allocation serialization.

```
//DELET10    JOB   ...
//STEP1     EXEC  PGM=IDCAMS
//MYMEMBER  DD DISP=SHR,DSN=EXAMPLE.NONVSAM2(MEM1)
//SYSPRINT  DD    SYSOUT=A
//SYSIN     DD    *
     DELETE -
            EXAMPLE.NONVSAM2(MEM1)  FILE(MYMEMBER)-
     /*
```

The DELETE command deletes a member of a partitioned data set, EXAMPLE.NONVSAM2(MEM1). Its parameters are:

- EXAMPLE.NONVSAM2(MEM1) is the entryname of a member of the partitioned data set, EXAMPLE.NONVSAM2. The entryname identifies the object to be deleted.
- The FILE parameter points to the DD statement MYMEMBER which allocates the PDS as SHR to the job, hence avoiding dynamic allocation with OLD of the entire PDS.

The second DELETE command deletes all remaining members.

Delete all members of a Partitioned (Non-VSAM) Data Set in a Catalog: Example 13-B

In this example, all members of partitioned data set EXAMPLE.NONVSAM2 are deleted:

```
  //DELET10 JOB ...
  //STEP1 EXEC PGM=IDCAMS
  //SYSPRINT DD SYSOUT=A
  //SYSIN DD *
        DELETE -
             EXAMPLE.NONVSAM2(*)
   /*
```

The DELETE command deletes all members of a partitioned data set, EXAMPLE.NONVSAM2. Its parameters are:

- EXAMPLE.NONVSAM2(*) is the entryname that includes members of the partitioned data set, EXAMPLE.NONVSAM2. The entryname identifies the object to be deleted.

Delete all members of a Partitioned (Non-VSAM) Data Set in a Catalog: Example 13-C

In this example, selected members of partitioned data set EXAMPLE.NONVSAM2 are deleted:

```
  //DELET10 JOB ...
  //STEP1 EXEC PGM=IDCAMS
  //SYSPRINT DD SYSOUT=A
  //SYSIN DD *
  DELETE -
  EXAMPLE.NONVSAM3(A*)
  /*
```

The DELETE command deletes all members in partitioned data set EXAMPLE.NONVSAM3 matching A*. Its parameters are:

- EXAMPLE.NONVSAM3(A*) is the entryname that includes members of the partitioned data set, EXAMPLE.NONVSAM3. The entryname identifies the object to be deleted. Members with names like A1, AB2, ABC3 will be deleted.

Delete all members of a Partitioned (Non-VSAM) Data Set in a Catalog: Example 13-D

In this example, selected members of partitioned data set EXAMPLE.NONVSAM2 are deleted:

```
//DELET10 JOB ...
//STEP1 EXEC PGM=IDCAMS
//SYSPRINT DD SYSOUT=A
//SYSIN DD *
DELETE -
EXAMPLE.NONVSAM4(A%)
/*
```

The DELETE command deletes all members in partitioned data set EXAMPLE.NONVSAM4 matching A%. Its parameters are:

- EXAMPLE.NONVSAM4(A%) is the entryname that includes members of the partitioned data set, EXAMPLE.NONVSAM4. The entryname identifies the object to be deleted. Members with names like A1, A2, A3, and AB will be deleted.

Delete a Page Space: Example 14

In this example, page space SYS1.PAGE2 is deleted from the master catalog. You must ensure other BCS's do not have that data set cataloged by performing a DELETE NOSCRATCH on each, and then performing a DELETE SCRATCH on the BCS that originally 'owned' the system data set.

```
//DELET11   JOB   ...
//STEP1     EXEC  PGM=IDCAMS
//SYSPRINT  DD    SYSOUT=A
//SYSIN     DD    *
     DELETE -
            SYS1.PAGE2 -
            PURGE -
            PAGESPACE
/*
```

The DELETE command removes the page space entry, SYS1.PAGE2, from the master catalog. Its parameters are:

- SYS1.PAGE2 is the entryname of the object being deleted. SYS1.PAGE2 identifies a page space entry.
- PURGE specifies that the page space entry be deleted regardless of the retention period or date specified when it was defined.
- PAGESPACE specifies the type of entry being deleted. VSAM verifies that SYS1.PAGE2 is a page space entry, then deletes it. If SYS1.PAGE2 incorrectly identifies another type of entry, VSAM does not delete it, but sends an error message to the programmer.

Delete a VVDS with Recovery: Example 15

In this example, the VVDS is deleted. The VTOC and catalog entries for the objects reflected by the VSAM volume records (in the VVDS) remain intact.

```
//DELET13    JOB    ...
//STEP1      EXEC   PGM=IDCAMS
//DD1        DD     VOL=SER=338001,UNIT=3380,DISP=OLD
//SYSPRINT   DD     SYSOUT=A
//SYSIN      DD     *
     DELETE  -
            SYS1.VVDS.V338001 -
            FILE(DD1) -
            RECOVERY
/*
```

- SYS1.VVDS.V338001 is the name of the VVDS.
- FILE specifies the name of a DD statement that both describes the VVDS volume and causes it to be mounted.

- RECOVERY specifies that the VVDS entry is being deleted from the VTOC as part of a recovery operation.

Delete an OAM Collection Name Catalog Entry: Example 16

In this example, an OAM non-VSAM collection name entry is deleted from a catalog. A FILE parameter and its associated DD statement are provided to allocate the volume where the catalog containing the collection name entry is located.

```
//DELET15    JOB    ...
//STEP1      EXEC   PGM=IDCAMS
//DD1        DD     VOL=SER=VSER01, UNIT=3380, DISP=OLD
//SYSPRINT   DD     SYSOUT=A
//SYSIN      DD     *
     DELETE -
             OAM.COLLECTION.NONVSAM -
             FILE (DD1) -
             PURGE -
             NOSCRATCH -
             CATALOG(COLNCAT)
/*
```

The DELETE command deletes the non-VSAM collection name entry from the catalog. Its parameters are:

- OAM.COLLECTION.NONVSAM is the name of the collection name entry in the catalog.
- FILE specifies the DD statement within the JCL that locates the volume where the catalog containing the collection name entry that is marked for deletion exists.
- PURGE specifies that the retention period for the non-VSAM collection name entry is ignored.
- NOSCRATCH specifies that only the collection name entry on the catalog is deleted.
- CATALOG identifies the catalog where the collection name entry that is marked for deletion, or COLNCAT, exists. If you are trying to delete SMS-managed entries, you must have RACF ALTER authority.

As of z/OS V2R3, OAM no longer creates or references OAM collection name entries in the catalog. The DELETE command can be used to remove any existing (no longer referenced) OAM collection name entries from the catalog.

Delete a Tape Library Entry: Example 17

This example deletes a tape library entry. Because the FORCE parameter was not used, the tape library entry ATLLIB1 is deleted only if no tape volume entries are associated with it.

```
//DELLIB     JOB     ...
//STEP1      EXEC   PGM=IDCAMS
//SYSPRINT   DD     SYSOUT=A
//SYSIN      DD     *
     DELETE (ATLLIB1) -
             LIBRARYENTRY
/*
```

This command's parameters are:

- ATLLIB1 is the name of the tape library entry that is to be deleted.
- LIBRARYENTRY specifies the type of entry to be deleted.

Delete a Tape Volume Entry: Example 18

This example deletes a tape volume entry.

```
//DELVOL     JOB     ...
//STEP1      EXEC   PGM=IDCAMS
//SYSPRINT   DD     SYSOUT=A
//SYSIN      DD     *
   DELETE (VAL0001) -
             VOLUMEENTRY -
```

```
                PURGE
/*
```

This command's parameters are:

- VAL0001 is the name of the tape volume entry that is to be deleted. This tape volume entry's volser is AL0001.
- VOLUMEENTRY specifies that a tape volume entry is to be deleted.
- PURGE specifies that the volume entry is to be deleted regardless of the expiration date.

Chapter 27. LISTCAT

The LISTCAT command lists catalog entries. The syntax of the LISTCAT command is:

Command	Parameters
LISTCAT	[ALIAS] [ALTERNATEINDEX] [CLUSTER] [DATA] [GENERATIONDATAGROUP] [INDEX] [LIBRARYENTRIES(*libent*)] [NONVSAM] [PAGESPACE] [PATH] [USERCATALOG] [VOLUMEENTRIES(*volent*)] [CREATION(*days*)] [ENTRIES(*entryname* [*entryname...*])| LEVEL(*level*)] [NOCDILVL|CDILVL] [PREFIX|NOPREFIX] [EXPIRATION(*days*)] [FILE(*ddname*)] [LIBRARY*(libname)*] [NAME|HISTORY|VOLUME| ALLOCATION|ALL] [OUTFILE(*ddname*)] [CATALOG(*catname*)]

LISTCAT can be abbreviated: LISTC

LISTCAT Usage

Not all data set attributes are stored within catalog. Some data attributes such as allocated space are maintained in the Volume Table of Contents (VTOC) and take precedence over reported cataloged values. See IDCAMS DCOLLECT or similar service for listing additional data set attributes.

For automated solutions, IBM recommends using the Catalog Search Interface (CSI) which provides compatibility and toleration support across releases. See *z/OS DFSMS Managing Catalogs* for more information on using CSI.

© Copyright IBM Corp. 1973, 2021

LISTCAT Parameters

The LISTCAT command uses the following parameters.

Required Parameters

The LISTCAT command has no required parameters.

When the LISTCAT command is entered as a job step (that is, not through TSO/E) and no parameters are specified, an entire catalog is listed. See “Catalog Search Order for LISTCAT” on page 10 for a description of how the catalog to be listed is selected.

Catalog management does not maintain the statistics of a catalog's own cluster entry. While LISTCAT will record statistics for data sets defined into a catalog, most of the statistics for the catalog's own entries are not accurately recorded.

Volume High Used RBA statistics do not apply for mutli-striped VSAM data sets. For multi-stripe VSAM data sets, the index component is not striped across volumes. The index component primary space is only obtained on the first volume as non-guaranteed space.

For TSO/E users, when LISTCAT is invoked from a TSO/E terminal and no operands are specified, the prefix (the TSO/E userid) becomes the highest level of entryname qualification and only those entries with a matching highest level of qualification are listed. It is as though you specified:

```
LISTCAT LEVEL(TSO/E user prefix)
```

For RACF users, in a non-SMS environment, LISTCAT checks the authorization at the catalog level before checking the authorization at the data set level. For a SMS environment, LISTCAT checks the authorization at the data set level before checking the authorization at the catalog level.

Optional Parameters

[ALIAS][ALTERNATEINDEX][CLUSTER][DATA]
[GENERATIONDATAGROUP][INDEX][LIBRARYENTRIES]
[NONVSAM][PAGESPACE][PATH][USERCATALOG]
[VOLUMEENTRIES]

specifies that certain types of entries are to be listed. Only those entries whose type is specified are listed. For example, if you specify CLUSTER but not DATA or INDEX, the cluster's entry is listed and its associated data and index entries are not listed.

If you use ENTRIES and also specify an entry type, the entryname is not listed unless it is of the specified type. You can specify as many entry types as desired. When you want to completely list a catalog, do not specify any entry type.

ALIAS
specifies that alias entries are to be listed.

ALTERNATEINDEX
specifies that entries for alternate indexes are to be listed. If ALTERNATEINDEX is specified and DATA and INDEX are not also specified, entries for the alternate index's data and index components are not listed.

Abbreviation: AIX

CLUSTER
specifies that cluster entries are to be listed. If CLUSTER is specified and DATA and INDEX are not also specified, entries for the cluster's data and index components are not listed.

Abbreviation: CL

DATA
specifies that entries for data components of clusters and alternate indexes are to be listed.

If a VSAM object's name is specified and DATA is coded, only the object's data component entry is listed. When DATA is the only entry type parameter coded, the catalog's data component is not listed.

GENERATIONDATAGROUP
specifies that entries for generation data groups are to be listed. GDSs in the active state, existing at the time the LISTCAT command is entered, are identified as such when ALL is specified.

Abbreviation: GDG

INDEX
specifies that entries for index components of key-sequenced clusters and alternate indexes are to be listed. If a VSAM object's name is specified and INDEX is coded, only the object's index component entry is listed. When INDEX is the only entry type parameter coded, the catalog's index component is not listed.

Abbreviation: IX

LIBRARYENTRIES(*libent*)
specifies that tape library entries are to be listed.

libent
specifies the name of the tape library entry.

Abbreviation: LIBENTRIES or LIBENT

NONVSAM
specifies that entries for non-VSAM data sets are to be listed. If a generation data group's name and non-VSAM are specified, GDSs in the deferred, active, or rolled-off state, associated with the GDG are listed by specifying the ALL option.

Abbreviation: NVSAM

PAGESPACE
specifies that entries for page spaces are to be listed.

Abbreviation: PGSPC

PATH
specifies that entries for paths are to be listed.

USERCATALOG
specifies that catalog connectors are to be listed. The user catalog connector entries are in the master catalog. (User catalog connector entries can also be in a user catalog, but the operating system does not recognize them when searching for a user catalog.)

Abbreviation: UCAT

VOLUMEENTRIES(*volent*)
specifies that tape library volume entries are to be listed. Prefix the name of the tape volume with the letter 'V'. For example 'Vxxxxx', where xxxxx equals the volume name. You can specify the CATALOG parameter to list tape volume entries from a specific catalog.

volent
specifies the name of the tape volume entry to be listed.

Abbreviation: VOLENTRIES or VOLENT

CATALOG(*catname*)
specifies the name of the catalog that contains the entries that are to be listed. If CATALOG is coded, only entries from that catalog are listed. See “Catalog Search Order for LISTCAT” on page 10 for information about the order in which catalogs are searched.

catname
is the name of the catalog.

If the catalog's volume is physically mounted, it is dynamically allocated. The volume must be mounted as permanently resident or reserved.

Abbreviation: CAT

CREATION(*days*)
specifies that entries of the indicated type (CLUSTER, DATA, and so on,) are to be listed only if they were created the specified number of days ago or earlier.

days
specifies the number of days ago. The maximum number that can be specified is 9999; zero indicates that all entries are to be listed.

Abbreviation: CREAT

ENTRIES(*entryname*[*entryname*...]) |
LEVEL(*level*)

NOCDILVL | CDILVL

ENTRIES specifies the names of entries to be listed.

Unexpired GDSs that have been rolled off and recataloged can be displayed using LISTCAT ENTRIES(*gdg.**), LISTCAT LEVEL(*gdg*) where *gdg* is the original name of the GDG, LISTCAT HISTORY, and LISTCAT ALL. Current and deferred generations are displayed as well as those that have been rolled off, in alphabetical order.

For TSO/E users, TSO/E will prefix the user ID to the specified data set name when the ENTRIES parameter is unqualified. The userid is not prefixed when the LEVEL parameter is specified.

Exception: You can use LISTCAT ENTRY LEVEL command only to list the cluster information of the entry. To list the data component information for the entry, you must use the LISTCAT ENTRY ALL command.

ENTRIES(*entryname* [*entryname*...])
specifies the name or generic name of each entry to be listed. (See the generic examples following the description of the LEVEL parameter.) When you want to list the entries that describe a user catalog, the catalog's volume must be physically mounted. You then specify the catalog's name as the entryname.

Abbreviation: ENT

LEVEL(*level*)
specifies that all entries that match the level of qualification specified by (level) are to be listed irrespective of the number of additional qualifiers.

If you specify a generic level name, only one qualifier replaces the *. The * must not be the last character specified in the LEVEL parameter. LEVEL(A.*) will give you an error message.

When you are using a generic name with the LEVEL parameter you can use a '%' unless under a TSO environment. The'%'acts as a place holder for a single character. For example, you can specify the following:

```
LISTCAT LVL(DMP.S7P2.D%%%%%%.T%%%%%%.YDMJBA00)
```

LEVEL can result in more than one user catalog being searched if the multilevel alias search level is greater than 1. For example if TEST is an alias for UCAT.ONE and TEST.PROD is an alias for UCAT.TWO, and the multilevel alias search level is 2, LEVEL(TEST) results in both catalogs being searched, and data sets satisfying both aliases are listed. If TEST and TEST.PROD are not defined as aliases, and there are catalogs called TEST.UCAT1 and TEST.UCAT2, LEVEL(TEST) with a multilevel alias search level of 2 results in both catalogs, as well as the master catalog, being searched for data sets with a high-level qualifier of TEST. In this situation, where a level is being searched that is not also an alias, the master catalog and all user catalogs with the same high-level qualifier *and a number of qualifiers equal to the multilevel alias search level* are searched for data sets matching the level requested. This situation should not occur if proper aliases are defined for user catalogs.

When multiple catalogs are searched, the listed entries appear in sorted order within the catalogs to which they belong.

Abbreviation: LVL

NOCDILVL | CDILVL
You can only specify NOCDILLVL | CDILVL parameters with the LEVEL parameter.

NOCDILVL
specifies that only the objects whose patterns match the LEVEL pattern be listed.

CDILVL
specifies that DATA and INDEX objects in CLUSTERs and AIXs be listed if at least one of the three objects matches the LEVEL pattern.

NOCDILVL is the default.

Examples of ENTRIES and LEVEL specifications:

Suppose a catalog contains the following names:

1. A.A.B
2. A.B.B
3. A.B.B.C
4. A.B.B.C.C
5. A.C.C
6. A.D
7. A.E
8. A

If ENTRIES(A.*) is specified, entries 6 and 7 are listed.
If ENTRIES(A.*.B) is specified, entries 1 and 2 are listed.
If LEVEL(A.*.B) is specified, entries 1, 2, 3, and 4 are listed.
If LEVEL(A) is specified, entries 1, 2, 3, 4, 5, 6, 7, and 8 are listed.

When using a generic name with the ENTRIES parameter, entries must have one qualifier in addition to those specified in the command.

When using the LEVEL parameter, associated entries (for example, data and index entries associated with a cluster) are not listed unless their names match the level of qualification.

If the specified cluster name is fully qualified and the data set name is the maximum length of 44 characters, more clusters than expected might be displayed. For fully qualified names use the ENTRIES parameter.

Restriction: LISTCAT LEVEL has a restriction on the number of entries that can be displayed due to the amount of available storage below the line (24-bit addressing).

PREFIX|NOPREFIX
specifies if the user ID will be added as the first qualifier of the entry name. These keywords apply when IDCAMS LISCAT is invoked by TSO and LISTCAT keyword ENTRY or LEVEL is specified. PREFIX/NOPREFIX are ignored when LISTCAT is running in batch. PREFIX and NOPREFIX are mutually exclusive.

PREFIX
specifies that the user ID will be added to entry names which are not enclosed in quotes. This is the default.

NOPREFIX
specifies that the user ID will not be added to the entry names. They will be processed as they are specified in the LISTCAT command.

EXPIRATION(*days*)
specifies that entries of the indicated type (CLUSTER, DATA, and so on) are to be listed only if they will expire in the specified number of days or earlier.

days
specifies the number of days. The maximum number that can be specified is 9999 and it indicates that all entries are to be listed. Zero indicates that only entries that have already expired are to be listed.

Abbreviation: EXPIR

FILE(*ddname*)
specifies the name of a DD statement that identifies the devices and volumes that contain information in the VVDS that is to be listed. If FILE is not specified, the volumes are dynamically allocated. The volumes must be mounted as permanently resident or reserved.

LIBRARY*(libname)*
specifies the name of the tape library entry for which tape volume entries are to be listed. Only those tape volumes that are entries in the specified tape library are listed.

libname
specifies a 1-to-8 character tape library name. You can use a partial tape library name followed by an * to list tape volume entries for more than one tape library.

Abbreviation: LIB

NAME|HISTORY|VOLUME|ALLOCATION|ALL
specifies the fields to be included for each entry listed. Appendix B, “Interpreting LISTCAT Output Listings,” on page 367, shows the listed information that results when you specify nothing (which defaults to NAME), HISTORY, VOLUME, ALLOCATION, or ALL. For SMS-managed data sets and catalogs, the SMS class names and last backup date are listed in addition to the other fields specified. The class definitions are not displayed.

Exception: For tape library and tape volume entries, only the ALL parameter is functional. If the HISTORY, VOLUME, and ALLOCATION parameters are specified for tape library and tape volume entries, these parameters are ignored. If ALL is not specified, only the names of the tape library or tape volume entries are listed.

NAME
specifies that the name and entry type of the entries are to be listed. Some entry types are listed with their associated entries. The entry type and name of the associated entry follow the listed entry's name. For details, see “ASN: Associations Group” on page 376 in Appendix B, “Interpreting LISTCAT Output Listings,” on page 367.

For TSO/E users, only the name of each entry associated with the TSO/E user's prefix is listed when no other parameters are coded.

HISTORY
specifies that only the following information is to be listed for each entry: name, entry type, ownerid, creation date, expiration date, and release. For GDG base and non-VSAM entries, status is also listed. For alternate indexes, "SMS-managed (YES/NO)" is also listed. For SMS-managed data sets, storage class, management class, data class, and last backup date are also listed. If the last backup date is unavailable, as in the case of migrated data sets, LISTCAT displays a field of all "X's" instead of an actual date.

HISTORY can be specified for CLUSTER, DATA, INDEX, ALTERNATEINDEX, PATH, GENERATIONDATAGROUP, PAGESPACE, and NONVSAM. See Figure 33 on page 414.

The OWNER-IDENT field in the HISTORY subset has been renamed DATASET-OWNER. This displays the contents of the data set owner field in the BCS. The ACCOUNT information is listed when the HISTORY or ALL parameter is specified.

Abbreviation: HIST

VOLUME
specifies that the information provided by specifying HISTORY, plus the volume serial numbers and device types allocated to the entries, are to be listed. Volume information is only listed for data and index component entries, non-VSAM data set entries, and user catalog connector entries.

For TSO/E users, only the name and volume serial numbers associated with the TSO/E user's prefix are listed when no other parameters are coded.

When ALIAS is specified with VOLUMES, the alias creation date is not listed. The alias creation date appears only for LISTCAT ALL or LISTCAT HISTORY.

Abbreviation: VOL

ALLOCATION
specifies that the information provided by specifying VOLUME plus detailed information about the allocation are to be listed. The information about allocation is listed only for data and index component entries.

When ALIAS is specified with ALLOCATION, the alias creation date is not listed. The alias creation date appears only for LISTCAT ALL or LISTCAT HISTORY.

Abbreviation: ALLOC

ALL
specifies that all fields are to be listed, including fields that are not listed under the NAME, HISTORY, VOLUME, and ALLOCATION options, such as CA-RECLAIM, and fields under ATTRIBUTES, and STATISTICS.

When multiple catalogs are searched, the listed entries appear in sorted order within the catalogs to which they belong.

OUTFILE(*ddname*)
specifies a data set, other than the SYSPRINT data set, to receive the output produced by LISTCAT (that is, the listed catalog entries). Completion messages produced by access method services are sent to the SYSPRINT data set, along with your job's JCL and input statements.

ddname identifies a DD statement that describes the alternate target data set. If OUTFILE is not specified, the entries are listed in the SYSPRINT data set. If an alternate data set is specified, it must meet the requirements in "JCL DD Statement for an Alternate Target Data Set" on page 3.

Abbreviation: OFILE

LISTCAT Examples

The LISTCAT command can perform the functions shown in the following examples.

List an SMS-Managed Data Set: Example 1

In this example, the HISTORY parameter is used to list an SMS-managed data set.

```
//LISTCAT1 JOB  ...
//STEP1    EXEC PGM=IDCAMS
//SYSPRINT DD   SYSOUT=A
//SYSIN    DD   *
      LISTCAT -
           ENTRIES(USER01.DATA1.EXAMPL) -
           CLUSTER -
           HISTORY
/*
```

The CLUSTER parameter specifies that only the cluster component of the entry identified in the ENTRIES parameter are listed. The HISTORY parameter causes the display of the HISTORY information along with the SMS classes and last backup date. The SMS information appears in the following format:

```
SMSDATA
  STORAGECLASS-----------SC4      MANAGEMENTCLASS-MGTCL004
  DATACLASS-----------DCL021      LBACKUP----2003.221.0255
```

If the last backup date had been unavailable, LISTCAT would have displayed:

```
LBACKUP------XXXX.XXX.XXXX
```

An example of the complete output resulting from this LISTCAT command is shown in "LISTCAT HISTORY Output Listing" on page 411.

List a Key-Sequenced Cluster's Entry in a Catalog: Example 2

In this example, a key-sequenced cluster entry is listed.

```
//LISTCAT1 JOB   ...
//STEP1    EXEC  PGM=IDCAMS
//SYSPRINT DD    SYSOUT=A
//SYSIN    DD    *
     LISTCAT -
           ENTRIES(LCT.EXAMPLE.KSDS1) -
           CLUSTER -
           ALL
/*
```

The LISTCAT command lists values stored in cataloged entries. It is assumed that the high level of the qualified cluster name is the same as the alias of the catalog STCUCAT1; this naming convention directs the catalog search to the appropriate catalog. The parameters are:

- ENTRIES identifies the entry to be listed.
- CLUSTER specifies that only the cluster entry is to be listed. If CLUSTER had not been specified, the cluster's data and index entries would also be listed.
- ALL specifies that all fields of the cluster entry are to be listed.

Alter a Catalog Entry, Then List the Modified Entry: Example 3

In this example, the free space attributes for the data component (LCT.KSDATA) of cluster LCT.MYDATA are modified. Next, the cluster entry, data entry, and index entry of LCT.MYDATA are listed to determine the effect, if any, the modification has on the cluster's other attributes and specifications.

```
//LISTCAT2 JOB    ...
//STEP1    EXEC   PGM=IDCAMS
//SYSPRINT DD   SYSOUT=A
//SYSIN    DD   *
     ALTER -
           LCT.KSDATA -
           FREESPACE(10 10)
     IF LASTCC = 0 -
           THEN -
           LISTCAT -
             ENTRIES(LCT.MYDATA) -
             ALL
  /*
```

The ALTER command modifies the free space specifications of the data component of the key-sequenced VSAM cluster LCT.MYDATA. Its parameters are:

- LCT.KSDATA is the *entryname* of the data component being altered. LCT.KSDATA identifies the data component of a key-sequenced VSAM cluster, LCT.MYDATA. To alter a value that applies only to the cluster's data component, such as FREESPACE does, you must specify the data component's entryname.
- FREESPACE specifies the new free space percentages for the data component's control intervals and control areas.

The IF ... THEN command sequence verifies that the ALTER command completed successfully before the LISTCAT command runs. The LISTCAT command lists the cluster's entry and its data and index entries. The parameters are:

- ENTRIES specifies the *entryname* of the object being listed. Because LCT.MYDATA is a key-sequenced cluster, the cluster entry, its data entry, and its index entry are listed.
- ALL specifies that all fields of each entry are to be listed.

List Catalog Entries: Example 4

This example illustrates how all catalog entries with the same generic name are listed.

```
//LISTCAT3   JOB   ...
//STEP1      EXEC  PGM=IDCAMS
//SYSPRINT   DD    SYSOUT=A
//SYSIN      DD    *
     LISTCAT -
            ENTRIES(GENERIC.*.BAKER) -
            ALL
/*
```

The LISTCAT command lists each catalog entry with the generic name GENERIC.*.BAKER, where * is any 1-to-8 character simple name. The parameters are:

- ENTRIES specifies the *entryname* of the object to be listed. Because GENERIC.*.BAKER is a generic name, more than one entry can be listed.
- ALL specifies that all fields of each entry are to be listed.

List Catalog Entries: Example 5

This example shows the LISTCAT command being used with the HISTORY parameter.

```
//LISTCAT4   JOB   ...
//STEP1      EXEC  PGM=IDCAMS
//SYSPRINT   DD    SYSOUT=A
//SYSIN      DD    *
     LISTCAT -
          ENTRIES(USER01.DATA1.EXMPL) -
          DATA -
          HISTORY
/*
```

The LISTCAT command's parameters are:

- ENTRIES specifies the name of the entry to be listed.
- DATA specifies that only the data component of the entry identified in the ENTRIES parameter are listed.
- HISTORY specifies that the HISTORY information is displayed.

List GDG Catalog Entries: Example 6

This example shows the LISTCAT command being used for a GDG that is an SMS-managed PDSE with the ALL parameter.

```
//LISTCAT4   JOB   ...
//STEP1      EXEC  PGM=IDCAMS
//SYSPRINT   DD    SYSOUT=A
//SYSIN      DD    *
     LISTCAT -
          ENTRIES(A.GDG) -
          ALL
          /*
```

If the entry is a GDS that is also an SMS-managed PDSE, LISTCAT displays a DSNTYPE field of LIBRARY, even for a deferred or rolled off PDSE GDS. The output from this example is in the following format:

```
 LISTC ENT(A.GDG) ALL
GDG BASE ------ A.GDG
     IN-CAT --- SYS1.MVSRES9.MASTCAT
     HISTORY
       DATASET-OWNER-----(NULL)     CREATION--------2011.228
       RELEASE----------------2     LAST ALTER------2011.228
     ATTRIBUTES
       LIMIT------------------2     NOSCRATCH        NOEMPTY
     ASSOCIATIONS
       NONVSAM--A.GDG.G0001V00
   NONVSAM ---- A.GDG.G0001V00
     IN-CAT --- SYS1.MVSRES9.MASTCAT
     HISTORY
       DATASET-OWNER-----(NULL)     CREATION--------2011.228
       RELEASE----------------2     EXPIRATION------0000.000
       ACCOUNT-INFO-----------------------------------(NULL)
       DSNTYPE----------LIBRARY
       STATUS-----------ACTIVE
     SMSDATA
      STORAGECLASS ---S1P03S02     MANAGEMENTCLASS---(NULL)
      DATACLASS --------(NULL)     LBACKUP ---0000.000.0000
   VOLUMES
     VOLSER-----------1P0301     DEVTYPE------X'3010200F'
     FSEQN------------------0
  ASSOCIATIONS
    GDG------A.GDG

    THE NUMBER OF ENTRIES PROCESSED WAS:
          AIX -------------------0
          ALIAS -----------------0
          CLUSTER ---------------0
          DATA ------------------0
          GDG -------------------1
          INDEX -----------------0
          NONVSAM ---------------1
          PAGESPACE -------------0
          PATH ------------------0
          SPACE -----------------0
          USERCATALOG -----------0
          TAPELIBRARY -----------0
          TAPEVOLUME ------------0
 TOTAL -----------------2
```

List a Tape Library Entry: Example 7

This example lists the tape library entry named ATLLIB1.

```
//LISTCLIB    JOB     ...
//STEP1       EXEC   PGM=IDCAMS
//SYSPRINT    DD     SYSOUT=A
//SYSIN       DD     *
     LISTCAT  -
          LIBRARYENTRIES(ATLLIB1) -
          ALL
/*
```

This command's parameters are:

- LIBRARYENTRIES identifies ATLLIB1 as the entry to be listed.
- ALL specifies that all information associated with the tape library entry ATLLIB1 is to be listed.

The tape library entry information is listed in the following format:

```
                          LISTING FROM CATALOG -- SYS1.VOLCAT.VGENERAL

LIBRARY-ENTRY-----------ATLLIB1
     DATA-LIBRARY
     LIBRARY-ID----------12345    DEVICE-TYPE---------3592-LE0    MAX-SLOTS-------------0     SCRATCH-VOLUME---------0
       CONSOLE-NAME-----CONSOLE     LOGICAL-TYPE--------AUTOMATED SLOTS-EMPTY-----------0     SCR-VOL-THRESHOLD------0
                        MEDIA1     MEDIA2   MEDIA3   MEDIA4  MEDIA5  MEDIA6   MEDIA7   MEDIA8   MEDIA9  MEDIA10
       SCRATCH-VOLUME---------0         0        0        0       0       0        0        0        0        0
       SCR-VOL-THRESHOLD------0         0        0        0       0       0        0        0        0        0
                        MEDIA11    MEDIA12  MEDIA13
       SCRATCH-VOLUME---------0         0        0
       SCR-VOL-THRESHOLD------0         0        0
       DESCRIPTION---(NULL)
```

List Tape Volume Entries: Example 8

This example lists all the tape volume entries whose names begin with the letters ‘VA’ in the tape library named ATLLIB1.

```
//LISTCLIB    JOB     ...
//STEP1       EXEC    PGM=IDCAMS
//SYSPRINT    DD      SYSOUT=A
//SYSIN       DD      *
       LISTCAT -
            VOLUMEENTRIES(VA*) -
            LIBRARY(ATLLIB1) -
            ALL
/*
```

This command's parameters are:

- VOLUMEENTRIES specifies that information relating to tape volume entries whose names begin with the letters ‘VA’ are be listed.
- LIBRARY specifies that only tape volume entries associated with the tape library named ATLLIB1 are to be listed.
- ALL requires that all information associated with the specified tape volume entries are to be listed.

The tape volume entries information is listed in the following format:

```
                          LISTING FROM CATALOG -- SYS1.VOLCAT.VGENERAL

VOLUME-ENTRY----------VAL0001
    DATA-VOLUME
      LIBRARY---------ATLLIB1     LOCATION----------LIBRARY       CREATION-DATE---2001-01-01   ENT-EJ-DATE---2001-01-01
      RECORDING-------UNKNOWN     MEDIATYPE----------MEDIA2       COMPACTION-----------NO     SPEC-ATTRIBUTE -----NONE
      STORAGE-GROUP---*SCRTCH*    USE-ATTRIBUTE------SCRATCH      EXPIRATION------2010-12-31   LAST-MOUNTED--2001-01-01
      CHECKPOINT------------Y     ERROR-STATUS-------NOERROR      WRITE-PROTECTED---------N   LAST-WRITTEN--2001-01-01
      SHELF-LOCATION------------(NULL)
      OWNER--------------(NULL)
```

Chapter 30. REPRO

The REPRO command performs the following functions:

- Copies VSAM and non-VSAM data sets. If the data set is a version 2 PDSE with generations, only the current generation of each member is copied.
- Copies catalogs
- Copies or merges tape volume catalogs
- Splits integrated catalog facility catalog entries between two catalogs
- Splits entries from an integrated catalog facility master catalog into another integrated catalog facility catalog
- Merges integrated catalog facility catalog entries into another integrated catalog facility user catalog.

Note: When using REPRO on compressed input and output data sets, andeither an INPUT or OUTPUT I/O Exit is used, the compression interface will not be used and performance may be impacted.To ensure that the compression interface is used, avoid using I/O exits.

Restriction: Access Method Services does not use RLS. If an RLS keyword is specified on the DD statement of a file to be opened by AMS, the keyword will be ignored and the file will be opened and accessed in non-RLS mode.

The syntax of the REPRO command is:

Command	Parameters
REPRO	{INFILE(*ddname*)|INDATASET(*entryname*)}
	{OUTFILE(*ddname*)| OUTDATASET(*entryname*)}
	[CIMODE|NOCIMODE]
	[DBCS]
	[ENTRIES(*entryname* [*entryname...*])|
	LEVEL(*level*)]
	[ERRORLIMIT(*value*)]
	[FILE(*ddname*)]
	[FROMKEY(*key*)|FROMADDRESS(*address*)|
	FROMNUMBER(*number*)|SKIP(*number*)]
	[INSERTSHIFT((*offset1 offset2*)[(*offset1 offset2*)...])| INSERTALL]
	[MERGECAT|NOMERGECAT]
	[MESSAGELEVEL(ALL|SHORT)]
	[REPLACE|NOREPLACE]
	[REUSE|NOREUSE]
	[SKIPDBCSCHECK((*offset1 offset2*)[(*offset1 offset2*)...])| NODBCSCHECK]
	[TOKEY(*key*)|TOADDRESS(*address*)|
	TONUMBER(*number*)|COUNT(*number*)]
	[VOLUMEENTRIES(*entryname*)]
	[ENCIPHER

© Copyright IBM Corp. 1973, 2021

Command	Parameters
	({EXTERNALKEYNAME(*keyname*)|
	INTERNALKEYNAME(*keyname*)|PRIVATEKEY}
	[CIPHERUNIT(*number*|1)]
	[DATAKEYFILE(*ddname*)|DATAKEYVALUE(*value*)]
	[SHIPKEYNAMES(*keyname*[*keyname*...])]
	[STOREDATAKEY|NOSTOREDATAKEY]
	[STOREKEYNAME(*keyname*)]
	[USERDATA(*value*)])]
	[DECIPHER
	({DATAKEYFILE(*ddname*)| DATAKEYVALUE(*value*)|SYSTEMKEY}
	[SYSTEMDATAKEY(*value*)]
	[SYSTEMKEYNAME(*keyname*)])]
	[RLSSOURCE(NO|YES|QUIESCE)]
	[RLSTARGET(NO|YES|QUIESCE)]

REPRO Parameters

Before you begin: Be familiar with the following information before using REPRO parameters:

- Partial copying of linear data set clusters is not permitted. The entire linear data space must be copied. A linear data set cluster can be copied to or from a physical sequential data set if the control interval size of the linear data set is equal to the logical record length of the physical sequential data set.
- The blocksize of the data sets you can use with REPRO depends on whether you have the large block interface (LBI) enabled. (You enable LBI by coding any BLKSIZE value, in the DCBE macro or by turning on the DCBEULBI bit before completion of the DCB OPEN exit - see *z/OS DFSMS Using Data Sets*.)
  - If you have LBI enabled, you can use REPRO with data sets with a blocksize larger than 32760 bytes.
  - If you do not have LBI enabled, you can **not** use REPRO with data sets that have a blocksize larger than 32760 bytes
- Using REPRO to copy linear data sets that are managed outside of VSAM and that do not abide to VSAM standards may produce unpredictable results. Such data sets include those managed by system logger. See *z/OS MVS Diagnosis: Reference* for more information.
- Attributes of data sets created by REPRO follow the same rules as those created by the JCL or by other utilities. Specifications of blocksize, logical record length, and blocking factors must be consistent with the type of data set and its physical characteristics. For more information, see the following documentation:
  - *z/OS MVS JCL Reference*
  - *z/OS MVS JCL User's Guide*
  - *z/OS DFSMS Using Data Sets*
- Because REPRO does not copy the control information stored in the directories, program libraries (in either the PDS or PDSE format) should not be copied. While the PDS as a whole cannot be copied, individual PDS members can be copied.
- If you want to copy a KSDS that is larger than 4GB to a system that does not support extended addressability, you must use the FROMKEY and the TOKEY, or COUNT parameters to reduce the size of the data set or to create several smaller data sets.

- If you use the REPRO command to copy compressed files, the record count information is not provided and message IDC005I is not displayed.
- REPRO copies the records of a VSAM recoverable data set. However, the locks used for VSAM record-level sharing (RLS) are not transferred. For the correct procedure to use when copying or moving data sets with pending recovery, see "Using VSAM Record-Level Sharing" in *z/OS DFSMS Using Data Sets*.

The REPRO command uses the following parameters:

Required Parameters

INFILE(*ddname*)|
INDATASET(*entryname*)
Identifies the source data set to be copied. If the logical record length of a non-VSAM source data set is greater than 32760 bytes, your REPRO command will end with an error message. The keys in the source data set must be in ascending order.

INFILE(*ddname*)
specifies the name of the DD statement that identifies the data set to be copied or the user catalog to be merged. You can copy a base cluster in alternate-key sequence by specifying a path name as the data set name in the DD statement.

Abbreviation: IFILE

INDATASET(*entryname*)
specifies the name of the entry to be copied or user catalog to be merged. If INDATASET is specified, the entryname is dynamically allocated with a disposition of OLD. You can copy a base cluster in alternate-key sequence by specifying a path name for the entryname.

If you are copying a member of a non-VSAM partitioned data set, the entryname must be specified in the format: *pdsname(membername)*

Abbreviation: IDS

OUTFILE(*ddname*)| OUTDATASET(*entryname*)
identifies the target data set. If a VSAM data set defined with a record length greater than 32760 bytes is to be copied to a sequential data set, your REPRO command will end with an error message.

Note: To avoid picking up incorrect volume information from the original DD statement on a data set that has previously been deleted and redefined in this invocation of IDCAMS and the FILE parameter was specified on the delete, you must specify the OUTDATASET keyword instead of OUTFILE. Alternatively, you can issue the REPRO command in a different step from the step that invoked the delete and define commands.

When DELETE is specified for abnormal termination disposition processing within an OUTFILE DD statement, the data set may be retained in cases where IDCAMS intercepts an abend. Ensure that a new output data set is specified in OUTFILE or it is deleted and redefined if you decide to start the REPRO from the beginning of the source data set.

OUTFILE(*ddname*)
specifies the name of a DD statement that identifies the target data set. For VSAM data sets, the data set name can name a path. If the DD statement identifies a SYSOUT data set, the attributes must match those specified in "JCL DD Statement for a Target Data Set" on page 3.

Abbreviation: OFILE

OUTDATASET(*entryname*)
specifies the name of the target data set. If OUTDATASET is specified, the entryname is dynamically allocated with a disposition of OLD. For VSAM data sets, *entryname* can be that of a path.

Abbreviation: ODS

Optional Parameters

CIMODE|NOCIMODE
Indicates a VSAM data set will be OPEN for VSAM CI level processing. These keywords apply to VSAM data sets only and are ignored when specified with other type of data set. CIMODE and NOCIMODE are mutually exclusive. CIMODE can only be specified with keyword INFILE, INDATASET, ERRORLIMIT, FROMADDRESS, SKIP, TOADDRESS, COUNT, NOMERGECAT, NOREPLACE and NOREUSE.

CIMODE
Use CI processing for the input data set on the REPRO command.

NOCIMODE
Do not use CI processing on input data set for the REPRO. NOCIMODE is the default.

DBCS
specifies that bytes in the logical record contain DBCS characters. REPRO checks to ensure the DBCS data meets DBCS criteria. For more information about DBCS support, see *z/OS DFSMS Using Data Sets*. This parameter cannot be specified with MERGECAT.

ENTRIES(*entryname [entryname...]*)| LEVEL(*level*)]
specifies the names of the entries in the source catalog to be merged when MERGECAT is specified.

For TSO/E users, TSO/E will prefix the userid to the specified data set name when ENTRIES is specified with an entry name without the userid. The userid is not prefixed when the LEVEL parameter is specified.

ENTRIES(*entryname* [*entryname...*])
specifies the name or generic name of each entry to be merged. (See the examples of generic entries following the description of the LEVEL parameter.) When using a generic name with the ENTRIES parameter, entries must have one qualifier in addition to those specified in the command.

Note: For information concerning RACF authorization levels, see Appendix A, “Security Authorization Levels,” on page 357. RACF applies to both SMS- and non-SMS-managed data sets and catalogs.

Abbreviation: ENT

LEVEL(*level*)
specifies that all entries matching the level of qualification you indicated with the LEVEL parameter are to be merged irrespective of the number of additional qualifiers. If a generic level name is specified, only one qualifier can replace the *. The * must not be the last character specified in the LEVEL parameter. LEVEL(A.*) will give you an error message.

The LEVEL parameter is not allowed when merging tape volume catalogs. For tape volume catalogs, see “Access Method Services Tape Library Support” on page 6 and VOLUMEENTRIES parameter.

Abbreviation: LVL

Examples of ENTRIES and LEVEL:

Suppose a catalog contains the following names:

1. A.A.B
2. A.B.B
3. A.B.B.C
4. A.B.B.C.C
5. A.C.C
6. A.D
7. A.E
8. A

If ENTRIES(A.*) is specified, entries 6 and 7 are merged.
If ENTRIES(A.*.B) is specified, entries 1 and 2 are merged.
If LEVEL(A.*.B) is specified, entries 1, 2, 3, and 4 are merged.
If LEVEL(A) is specified, entries 1, 2, 3, 4, 5, 6, 7 are merged.

ERRORLIMIT(*value*)
lets you select a failure limit. Use this parameter to set a limit to the number of errors REPRO copy will tolerate. The default is four, but any number from 1 to 2,147,483,647 can be used.

Abbreviation: ELIMIT

FILE(*ddname*)
specifies the name of a DD statement that identifies all the volumes that contain the VVDSs to be updated. If you do not specify FILE, VSAM will try to dynamically allocate the required volumes.

FROMKEY(*key*) | FROMADDRESS(*address*) | FROMNUMBER(*number*) | SKIP(*number*)
specifies the location in the source data set where copying is to start. If no value is coded, the copying begins with the first logical record in the data set. You can use only one of the four choices.

Use the SKIP parameter for a SAM data set.

None of these parameters can be specified if you are copying a linear data set. You must copy the entire linear data set. FROMKEY can be specified when copying a catalog, but none of the other parameters may be used for a catalog.

The starting delimiter must be consistent with the ending delimiter. For example, if FROMADDRESS is specified for the starting location, use TOADDRESS to specify the ending location. The same is true for FROMKEY and TOKEY, and FROMNUMBER and TONUMBER.

FROMKEY(*key*)
specifies the key of the first record you want copied. You can specify generic keys (a portion of the key followed by *). If you specify generic keys, copying begins at the first record with a key matching the specified portion of the key. If you specify a key longer than that defined for the data set, the data set is not copied. If the specified key is not found, copying begins at the next higher key.

FROMKEY can be specified only when copying an alternate index, a KSDS, or a catalog.

key
Can contain 1-to-255 EBCDIC characters. A key ending in X'5C' is processed as a generic key.

If RLSSOURCE(YES) and RLSTARGET(YES) are specified, the keys specified in FROMKEY and TOKEY are expected to be a key-name:key-value pair where key name is what was specified in DEFINE CLUSTER KEYNAME. The key value is hashed by RLS to derive internal keys which might make the FROMKEY value greater than the TOKEY value.

Abbreviation: FKEY

FROMADDRESS(*address*)
specifies the relative byte address (RBA) of the first record you want copied. The RBA value must be the beginning of a logical record. If you specify this parameter for key-sequenced data, the records are copied in physical sequential order instead of in logical sequential order. FROMADDRESS:

- Can be coded only for key-sequenced or entry-sequenced data sets or components.
- Cannot be specified when the data set is being accessed through a path.
- Cannot be specified for a key-sequenced data set with spanned records if any of those spanned records are to be accessed.

address
Can be specified in decimal (n) or hexadecimal (X'n'). The specification cannot be longer than one fullword when specified in decimal.

The largest address you can specify in decimal is 4,294,967,295. If you require a higher value, specify it in hexadecimal.

Abbreviation: FADDR

FROMNUMBER(*number*)
specifies the relative record number of the first record you want copied. FROMNUMBER can be specified only when you copy a relative record data set.

number
Can be specified in decimal (n), hexadecimal (X'n'), or binary (B'n'). The specification cannot be longer than one fullword.

The largest address you can specify in decimal is 4,294,967,295. If you require a higher value, specify it in hexadecimal.

Abbreviation: FNUM

SKIP(*number*)
specifies the number of logical records to skip before beginning to copy records. For example, if you want to copy beginning with record number 500, specify SKIP(499).

SKIP should not be specified when you access the data set through a path; the results are unpredictable.

INSERTSHIFT((*offset1 offset2*)[(*offset1 offset2*)...])|INSERTALL
If DBCS is specified without INSERTSHIFT or INSERTALL, the logical record is assumed to already contain SO and SI characters, and REPRO will check during copying to ensure that the DBCS data meets DBCS criteria. INSERTSHIFT or INSERTALL can be specified only if DBCS is also specified, and the data set being copied is **not** a catalog.

INSERTSHIFT[((*offset1 offset2*)[(*offset1 offset2*)...])
indicates that SO and SI characters are to be inserted in the logical record during REPRO command processing. This action has a permanent effect on the target data set.

offset1
Indicates the byte offset in the logical record to be copied before which a SO character is to be inserted.

offset2
Indicates the byte offset in the logical record to be copied after which a SI character is to be inserted. *offset2* must be greater than *offset1* and the difference must be an odd number.

Offset pairs cannot overlap ranges.

The maximum number of offset pairs that can be specified is 255.

Abbreviation: ISHFT

INSERTALL
specifies the entire logical record is assumed to contain only DBCS characters. An SO character is inserted at the beginning of the record and an SI character is inserted at the end of the record.

Abbreviation: ISALL

MESSAGELEVEL(ALL|SHORT)
Specified the format of the REPRO MERGECAT output. These keywords are ignored when MERGECAT keyword is not specified. LIST and NOLIST are mutually exclusive.

ALL
Will not condense the output listing.

ALL is the default.

SHORT
Will condense the output listing.

MERGECAT | NOMERGECAT

specifies whether entries from the source catalog are to be merged with the entries of the target catalog. When merging catalogs ensure that data sets whose entries are merged can still be located after the merge operation. This parameter cannot be specified with the DBCS parameter.

MERGECAT merges deferred generations if specified with the GDG base during a merge of the entire catalog. Deferred generations retain their deferred status in the target catalog. Rolled-off generations are also merged during a merge of all entries.

MERGECAT can also be specified for tape volume catalogs or VOLCATS. For more information on REPRO MERGECAT of VOLCATS, refer to "Access Method Services Tape Library Support" on page 6

MERGECAT

specifies that the source catalog entries are to be merged with the target catalog entries and that the merged entries are to be deleted from the source catalog upon a successful merge operation.

The merge operation can be restarted if an error occurs, because the target catalog does not have to be empty. A LISTCAT and DIAGNOSE should be run before the MERGE is restarted. If the MERGE ended while processing a generation data group, it might be necessary to delete that generation data group from the target catalog because of duplicate data set names in the source and target catalogs.

Candidate volumes are preserved. MERGECAT retains candidate volume information when moving an entry from one catalog to another.

For some duplicate key errors, the merge does not end, and the processing of the next entry continues. However, some alias associations might not be merged because of the duplicate key error.

MERGECAT performs a series of DELETE NOSCRATCH and DEFINE RECATALOG requests to move entries from one catalog to another. For information concerning security authorization levels, see Appendix A, "Security Authorization Levels," on page 357.

During MERGECAT, if the target catalog name is found in the VVDS, the cluster entry for the VVDS is not recreated in the target catalog. You must use DEFINE CLUSTER RECATALOG to create the VVDS cluster entry in the target catalog.

Note that the use of LEVEL or ENTRIES parameter will not move extended aliases that use the SYMBOLICRELATE parameter to the new catalog.

For more information on how aliases are processed when using REPRO MERGECAT, see z/OS DFSMS: Managing Catalogs under the section Splitting Catalogs or Moving Catalog Entries.

For more information on use of extended aliases see z/OS DFSMS: Managing Catalogs under the section Extended Alias Support.

See *z/OS DFSMS Managing Catalogs* for additional information on the integrity of RACF discrete profiles after using MERGECAT.

Abbreviation: MRGC

NOMERGECAT

specifies that the source catalog is to be completely copied into an empty target catalog.

MERGECAT can also be specified for tape volume catalogs or VOLCATS. For more information on REPRO MERGECAT of VOLCATS, refer to "Access Method Services Tape Library Support" on page 6.

The empty target catalog implies that the copy operation cannot be restarted if an error occurs. Before the copy operation can be restarted, the target catalog must be redefined and all volumes that contain objects must be restored. For VOLCAT this is not applicable because they do not use VVDS. If the NOMERGECAT for VOLCAT fail you only need to delete or define the target VOLCAT and restart the NOMERGECAT.

After a REPRO of one catalog to another, the VVRs are changed to point to the target catalog, and all subsequent processing must be done under the target catalog.



Attention: Performing REPRO on a catalog while data sets are open in the source catalog might result in a loss of information if any of those data sets extend, or other catalog updates are made. The changes might not be copied to the target catalog, resulting in a mismatch between the information contained in the VVDS and the new target BCS. This might cause the data sets to be inaccessible or receive errors.

Abbreviation: NOMRGC

REPLACE | NOREPLACE
specifies whether a record in the source cluster (INFILE or INDATASET) is to replace a record in the target cluster (OUTFILE or OUTDATASET) when the source cluster is copied into the target cluster.

When the source cluster is copied, its records might have keys or relative record numbers identical to the keys or relative record numbers of data records in the target cluster. In this case, the source record replaces the target record.

REPLACE|NOREPLACE is not used when copying integrated catalog facility catalogs because these catalogs do not use the catalog unload and reload functions.

REPLACE
When a key-sequenced data set (other than a catalog) is copied, each source record with a key matching a target record's key replaces the target record. Otherwise, the source record is inserted into its appropriate place in the target cluster.

When a relative record data set is copied, each source record with a relative record number that identifies a data record (rather than an empty slot) in the target data set replaces the target data record. Otherwise, the source data record is inserted into the empty slot its relative record number identifies.

REPLACE cannot be used if the target data set is identified as a path through an alternate index, or if the target data set is a base cluster whose upgrade data set includes an alternate index defined with the unique-key attribute.

Abbreviation: REP

NOREPLACE
When a key-sequenced data set (other than a catalog) is copied, target records are not replaced by source records. For each source record whose key matches a target record's key, a "duplicate record" message is issued.

When a relative record data set is copied, target records are not replaced by source records. For each source record whose relative record number identifies a target data record instead of an empty slot, a "duplicate record" message is issued.

When copying something other than a VSAM data set to a sequential data set, the error limit parameter allows more than four mismatches or errors.

Abbreviation: NREP

REUSE | NOREUSE
specifies if the target data set is to be opened as a reusable data set. This parameter is valid only for VSAM data sets.

REUSE
specifies that the target data set, specified with OUTFILE or OUTDATASET, is opened as a reusable data set whether or not it was defined as reusable with the REUSE parameter. (See the DEFINE CLUSTER command description.) If the data set was defined with REUSE, its high-used relative byte address (RBA) is reset to zero (that is, the data set is effectively empty) and the operation proceeds. When you open a reusable data set with the reset option, that data set cannot be shared with other jobs.

If REUSE is specified and the data set was originally defined with the NOREUSE option, the data set must be empty; if not, the REPRO command ends with an error message.

Abbreviation: RUS

NOREUSE
specifies that records are written at the end of an entry-sequenced data set. (OUTFILE or OUTDATASET must identify a nonempty data set.)

Abbreviation: NRUS

SKIPDBCSCHECK((*offset1 offset2*)[(*offset1 offset2*)...])|NODBCSCHECK
SKIPDBCSCHECK and NODBCSCHECK cannot be specified unless DBCS is also specified.

SKIPDBCSCHECK((*offset1 offset2*)[(*offset1 offset2*)...])
indicates that characters between *offset1* and *offset2* are not to be checked for DBCS criteria during REPRO command processing.

offset1
Indicates the byte offset in the logical record to be copied at which checking is to cease until *offset2* is reached.

offset2
Indicates the byte offset in the logical record after which checking is to resume. *offset2* must be greater than *offset1*.

Offset pairs cannot overlap ranges.

The maximum number of offset pairs that can be specified is 255.

Abbreviation: SKDCK

NODBCSCHECK
indicates DBCS verification checking will not be done.

Abbreviation: NODCK

TOKEY(*key*)|TOADDRESS(*address*)|TONUMBER(*number*)|COUNT(*number*)
specifies where copying is to end in the data set being copied. You can specify only one of these parameters for a copy operation. The location where copying is to end must follow the location where it is to begin. If no value is coded, copying ends at the logical end of the data set or component.

COUNT is the only parameter that can be specified for a SAM data set.

None of these parameters can be specified if you are copying a linear data set. You must copy the entire linear data set. TOKEY can be specified when copying a catalog, but none of the other parameters may be used for a catalog.

The ending delimiter must be consistent with the starting delimiter. For example, if FROMADDRESS is specified for the starting location, use TOADDRESS to specify the ending location. The same is true for FROMKEY and TOKEY, and FROMNUMBER and TONUMBER.

TOKEY(*key*)
specifies the key of the last record you want copied. You can specify generic keys (a portion of the key followed by *). If you specify generic keys, copying stops after the last record whose key matches that portion of the key you specified is copied. If you specify a key longer than the one defined for the data set, the data set is not copied. If the specified key is not found, copying ends at the next lower key. TOKEY can be specified only when copying an alternate index, a KSDS, or a catalog.

key
Can contain 1-to-255 EBCDIC characters.

If RLSSOURCE(YES) and RLSTARGET(YES) are specified, the keys specified in FROMKEY and TOKEY are expected to be a key- name:key-value pair where key name is what was specified in DEFINE CLUSTER KEYNAME. The key value is hashed by RLS to derive internal keys which might make the FROMKEY value greater than the TOKEY value.

TOADDRESS(*address*)
specifies the relative byte address (RBA) of the last record you want copied. Unlike FROMADDRESS, the RBA value does not need to be the beginning of a logical record. The entire record containing the specified RBA is copied.

If you specify this parameter for a KSDS, the records are copied in physical sequential order instead of in logical sequential order.

Use TOADDRESS with VSAM key-sequenced or entry-sequenced data sets or components. TOADDRESS cannot be specified when the data set is accessed through a path. TOADDRESS cannot be specified for a key-sequenced data set with spanned records if any of those spanned records are to be accessed.

address
can be specified in decimal (n) or hexadecimal (X'n'). The specification cannot be longer than one fullword.

The largest address you can specify in decimal is 4,294,967,295. If you require a higher value, specify it in hexadecimal.

Abbreviation: TADDR

TONUMBER(*number*)
specifies the relative record number of the last record you want copied. TONUMBER can be specified only when you copy a relative record data set.

number
Can be specified in decimal (n), hexadecimal (X'n'), or binary (B'n'). The specification cannot be longer than one fullword.

The largest address you can specify in decimal is 4,294,967,295. If you require a higher value, specify it in hexadecimal.

Abbreviation: TNUM

COUNT(*number*)
specifies the number of logical records you want copied. COUNT should not be specified when you access the data set through a path; results are unpredictable.

VOLUMEENTRIES(*entryname*)
specifies the tape volume catalogs to be merged or copied. The LEVEL parameter is not allowed when merging tape volume catalogs. When a tape volume catalog is copied, REPRO verifies that the target is a tape volume catalog.

Abbreviation: VOLENTRIES or VOLENT

RLSSOURCE
specifies how the source dataset will be open.

NO
indicates the source data set will be opened using Non-Shared Resources (NSR).

Abbreviation: N

Default: NO

YES
indicates that the source data set will be opened using Record Level Sharing (RLS), and the data set will have consistent read integrity.

Abbreviation: Y

QUIESCE
: indicates that the source data set will be opened using Record Level Sharing (RLS), and the data set will be quiesced before processing any entries.

Abbreviation: Q

Abbreviation: RLSS

RLSTARGET(NO|YES|QUIESCE)
specifies how the target dataset will be open.

NO
: indicates the target data set will be opened using Non-Shared Resources (NSR).

Abbreviation: N

Default: NO

YES
: indicates that the target data set will be opened using Record Level Sharing (RLS), and the data set will have consistent read integrity.

Abbreviation: Y

QUIESCE
indicates that the source data set is quiesced before AMS calls VSAM for RLS OPEN. VSAM switches the data set out of RLS mode and ultimately open it in nonRLS mode.

Abbreviation: Q

Abbreviation: RLSS

Note: If the request is to REPRO a user catalog, and the user catalog is in RLS mode, for example, defined with RLSENABLE, and the following needs to be noted:

- ACBERFLG 181 is expected, if RLSSOURCES(YES) and/or RLSTARGET(YES) options are specified for INDATASET/OUTDATASET.
- ACBERFLG 168 is expected, if RLSTARGET(NO|QUIESCE) is specified for either OUTFILE or OUTDATASET.

Cryptographic Parameters

You can use the REPRO cryptographic parameters with the following facilities:

- IBM Programmed Cryptographic Facility (PCF) (5740-XY5)

  Change your configuration to use the cryptographic parameters with PCF.
- z/OS Integrated Cryptographic Service Facility (ICSF) (5647-A01)

  Change your ICSF configuration to use the cryptographic parameters with ICSF. For a description of the necessary changes, see *z/OS Cryptographic Services ICSF System Programmer's Guide*.

This section lists and describes the REPRO cryptographic parameters.

ENCIPHER
specifies that the source data set is to be enciphered as it is copied to the target data set.

Abbreviation: ENCPHR

EXTERNALKEYNAME(*keyname*) |INTERNALKEYNAME(*keyname*) |PRIVATEKEY
specifies whether you, PCF, or ICSF manages keys privately.

EXTERNALKEYNAME(*keyname*)
specifies that PCF or ICSF manages keys. This parameter also supplies the 1-to-8 character key name of the external file key that is used to encipher the data encrypting key. The key is known only by the deciphering system. The key name and its corresponding enciphered data encrypting key are listed in SYSPRINT only if NOSTOREDATAKEY is specified.

Abbreviation: EKN

INTERNALKEYNAME(*keyname*)
specifies that PCF or ICSF manages keys. This parameter also supplies the 1-to-8 character key name of the internal file key that is used to encipher the data encrypting key. The key is retained by the key-creating system. The key name and its corresponding enciphered data encrypting key will only be listed in SYSPRINT if NOSTOREDATAKEY is specified.

Abbreviation: IKN

PRIVATEKEY
specifies that the key is to be managed by you.

Abbreviation: PRIKEY

CIPHERUNIT(*number* |1)
specifies that multiple logical source records are to be enciphered as a unit. *Number* specifies the number of records that are to be enciphered together. By specifying that multiple records are to be enciphered together, you can improve your security (chaining is done across logical record boundaries) and also improve your performance. However, there is a corresponding increase in virtual storage requirements. The remaining records in the data set, after the last complete group of multiple records, are enciphered as a group. (If *number* is 5 and there are 22 records in that data set, the last 2 records are enciphered as a unit.)

The value for *number* can range from 1 to 255.

Abbreviation: CPHRUN

DATAKEYFILE(*ddname*) | DATAKEYVALUE(*value*)
specifies that you are supplying a plaintext (not enciphered) data encrypting key. If one of these parameters is not specified, REPRO will generate the data encrypting key. These parameters are valid only when EXTERNALKEYNAME or PRIVATEKEY is specified. If INTERNALKEYNAME and DATAKEYVALUE or DATAKEYFILE are specified, REPRO will generate the data encrypting key and DATAKEYVALUE or DATAKEYFILE are ignored by REPRO.

The plaintext data encrypting key will not be listed in SYSPRINT unless PRIVATEKEY is specified and REPRO provides the key.

DATAKEYFILE(*ddname*)
identifies a data set that contains the plaintext data encrypting key. For ddname, substitute the name of the JCL statement that identifies the data encrypting key data set.

Abbreviation: DKFILE

DATAKEYVALUE(*value*)
specifies the 8-byte value to be used as the plaintext data encrypting key to encipher the data.

Value can contain 1-to-8 EBCDIC characters or 1-to-16 hexadecimal characters coded X'n'. *Value* must be enclosed in single quotation marks if it contains commas, semicolons, blanks, parentheses, or slashes. A single quotation mark must be coded as two single quotation marks. With either EBCDIC or hexadecimal representation, *value* is padded on the right with blanks (X'40') if it is fewer than 8 characters.

Abbreviation: DKV

SHIPKEYNAMES(*keyname*[*keyname*...])
supplies the 1-to-8 character key name of one or more external file keys to be used to encipher the data encrypting key. Each key name and its corresponding enciphered data encrypting key is listed in SYSPRINT, but is not stored in the target data set header. The primary use for this parameter is to establish multiple enciphered data encrypting keys to be transmitted to other locations for use in deciphering the target enciphered data set. This parameter is valid only when INTERNALKEYNAME or EXTERNALKEYNAME is specified.

Abbreviation: SHIPKN

STOREDATAKEY | NOSTOREDATAKEY
specifies whether the enciphered data encrypting key is to be stored in the target data set header. The key used to encipher the data encrypting key is identified by INTERNALKEYNAME or EXTERNALKEYNAME. This parameter is valid only when INTERNALKEYNAME or EXTERNALKEYNAME is specified. If the enciphered data encrypting key is stored in the data set header, it does not have to be supplied by the user when the data is deciphered.

Restriction: A data encrypting key enciphered under the keys identified by SHIPKEYNAMES cannot be stored in the header. Therefore, you might want to avoid using STOREDATAKEY and SHIPKEYNAMES together because this could result in storing header information unusable at some locations.

STOREDATAKEY
specifies that the enciphered data encrypting key is to be stored in the target data set header.

Abbreviation: STRDK

NOSTOREDATAKEY
specifies that the enciphered data encrypting key is not to be stored in the target data set header. The keyname and its corresponding enciphered data encrypting key is listed in SYSPRINT.

Abbreviation: NSTRDK

STOREKEYNAME(*keyname*)
specifies whether to store a keyname for the key used to encipher the data encrypting key in the target data set header. The specified keyname must be the name the key is known by on the system where the REPRO DECIPHER is to be performed. This keyname must be the same one specified in INTERNALKEYNAME if REPRO DECIPHER is to be run on the same system. If REPRO DECIPHER is run on a different system, the specified keyname can be different from the one specified in INTERNALKEYNAME or EXTERNALKEYNAME.

This parameter is valid only when INTERNALKEYNAME or EXTERNALKEYNAME is specified. If the keyname is stored in the data set header, it does not have to be supplied by the user when the data is deciphered.

Restriction: Keyname values identified by the SHIPKEYNAMES parameter cannot be stored in the header. Therefore, you might want to avoid using STOREKEYNAME and SHIPKEYNAMES together because this could result in storing header information unusable at some locations.

Abbreviation: STRKN

USERDATA(*value*)
specifies 1-to-32 characters of user data to be placed in the target data set header. For example, this information can be used to identify the security classification of the data.

Value can contain 1-to-32 EBCDIC characters. If *value* contains a special character, enclose the *value* in single quotation marks (for example, USERDATA('*CONFIDENTIAL*')). If the *value* contains a single quotation mark, code the embedded quotation mark as two single quotation marks (for example, USERDATA('COMPANY''S')).

You can code *value* in hexadecimal form, where two hexadecimal characters represent one EBCDIC character. For example, USERDATA(X'C3D6D4D7C1D5E8') is the same as USERDATA(COMPANY). The string can contain up to 64 hexadecimal characters when expressed in this form, resulting in up to 32 bytes of information.

Abbreviation: UDATA

DECIPHER
specifies that the source data set is to be deciphered as it is copied to the target data set. The information from the source data set header is used to verify the plaintext deciphered data encrypting key supplied, or deciphered from the information supplied, as the correct plaintext data encrypting key for the decipher operation.

Abbreviation: DECPHR

DATAKEYFILE(*ddname*) | DATAKEYVALUE(*value*) | SYSTEMKEY
specifies whether you, PCF, or ICSF manages keys privately.

DATAKEYFILE(*ddname*)
specifies that the key is to be managed by you, and identifies a data set that contains the private data encrypting key that was used to encipher the data. For *ddname*, substitute the name of the JCL statement that identifies the data set containing the private data encrypting key.

Abbreviation: DKFILE

DATAKEYVALUE(*value*)
specifies that the key is to be managed by you, and supplies the 1- to 8-byte value that was used as the plaintext private data encrypting key to encipher the data.

Value can contain 1-to-8 EBCDIC characters, and must be enclosed in single quotation marks if it contains commas, semicolons, blanks, parentheses, or slashes. A single quotation mark contained within *value* must be coded as two single quotation marks. You can code *value* in hexadecimal form, (X'n'), *value* can contain 1-to-16 hexadecimal characters, resulting in 1 to 8 bytes of information. With either EBCDIC or hexadecimal representation, *value* is padded on the right with blanks (X'40') if it is less than 8 characters.

Abbreviation: DKV

SYSTEMKEY
specifies that PCF or ICSF manages keys.

Abbreviation: SYSKEY

SYSTEMDATAKEY(*value*)
specifies the 1- to 8-byte value representing the enciphered system data encrypting key used to encipher the data. This parameter is valid only if SYSTEMKEY is specified. If SYSTEMDATAKEY is not specified, REPRO obtains the enciphered system data encrypting key from the source data set header. In this case, STOREDATAKEY must have been specified when the data set was enciphered.

value can contain 1-to-8 EBCDIC characters and must be enclosed in single quotation marks if it contains commas, semicolons, blanks, parentheses, or slashes. A single quotation mark must be coded as two single quotation marks. You can code value in hexadecimal form, (X'n'). *value* can contain 1-to-16 hexadecimal characters, resulting in 1-to-8 bytes of information. With either EBCDIC or hexadecimal representation, *value* is padded on the right with blanks (X'40') if it is fewer than 8 characters.

Abbreviation: SYSDK

SYSTEMKEYNAME(*keyname*)
specifies the 1-to-8 character key name of the internal key that was used to encipher the data encrypting key. This parameter is only valid if SYSTEMKEY is specified. If SYSTEMKEYNAME is not specified, REPRO obtains the key name of the internal key from the source data set header. In this case, STOREKEYNAME must have been specified when the data set was enciphered.

Abbreviation: SYSKN

REPRO Examples

The REPRO command can perform the functions shown in the following examples.

Copy Records into a VSAM Data Set: Example 1

In this two-part example, data records are copied from the non-VSAM data set SEQ.DRGV, a sequential data set, into a key-sequenced VSAM data set, RPR.MYDATA. Next, records are copied from the key-sequenced data set, RPR.MYDATA, into an entry-sequenced data set, ENTRY.

```
//REPRO2   JOB  ...
//STEP1    EXEC PGM=IDCAMS
//INPUT    DD   DSNAME=SEQ.DRGV,DISP=SHR,DCB=(BUFNO=6)
//SYSPRINT DD   SYSOUT=A
//SYSIN    DD   *
     REPRO -
           INFILE(INPUT) -
           OUTDATASET(RPR.MYDATA)-
           ERRORLIMIT(6)
/*
```

```
//STEP2    EXEC PGM=IDCAMS
//INPUT    DD   DSNAME=RPR.MYDATA,DISP=OLD
//OUTPUT   DD   DSNAME=ENTRY,DISP=OLD
//SYSPRINT DD   SYSOUT=A
```

```
//SYSIN    DD   *
     REPRO -
           INFILE(INPUT) -
           OUTFILE(OUTPUT) -
           FROMKEY(DEAN) -
           TOKEY(JOHNSON)
/*
```

STEP1

Access method services copies records from a sequential data set, SEQ.DRGV, into a key-sequenced data set, RPR.MYDATA. STEP1's job control language statement:

- INPUT DD identifies the sequential data set, SEQ.DRGV, that contains the source records. The BUFNO parameter specifies the number of buffers assigned to the sequential data set. This improves performance when the data set's records are accessed.

  STEP1's REPRO command copies all records from the source data set, SEQ.DRGV, to the target data set, RPR.MYDATA. Its parameters are:
- INFILE points to the INPUT DD statement, which identifies the source data set.
- OUTDATASET identifies the key-sequenced data set into which the source records are to be copied. The data set is dynamically allocated by access method services.
- ERRORLIMIT identifies the number of errors REPRO will tolerate.

STEP2

Access method services copies some of the records of the key-sequenced data set RPR.MYDATA into an entry-sequenced data set, ENTRY. STEP2's job control language statements:

- INPUT DD identifies the key-sequenced cluster, RPR.MYDATA, that contains the source records.
- OUTPUT DD identifies the entry-sequenced cluster, ENTRY, that the records are to be copied into.

  STEP2's REPRO command copies records from the source data set, RPR.MYDATA, to the target data set, ENTRY. Only those records with key values from DEAN to, and including, JOHNSON are copied.

  The parameters are:
- INFILE points to the INPUT DD statement, which identifies the source key-sequenced data set.
- OUTFILE points to the OUTPUT DD statement, which identifies the entry-sequenced data set into which the source records are to be copied.
- FROMKEY and TOKEY specify the lower and upper key boundaries.

If ENTRY already contains records, VSAM merges the copied records with ENTRY's records. A subsequent job step could resume copying the records into ENTRY, beginning with the records with key greater than JOHNSON. If you subsequently copied records with key values less than DEAN into ENTRY, VSAM merges them with ENTRY's records.

Merge an Integrated Catalog Facility User Catalog into Another Integrated Catalog Facility User Catalog: Example 2

This example shows how integrated catalog facility user catalog entries are merged into another integrated catalog facility user catalog. This function effectively combines entries from two catalogs into one catalog.

```
//MERGE6   JOB   ...
//STEP1    EXEC  PGM=IDCAMS
//DD1      DD    VOL=SER=VSER01,UNIT=DISK,DISP=OLD
//         DD    VOL=SER=VSER02,UNIT=DISK,DISP=OLD
//         DD    VOL=SER=VSER03,UNIT=DISK,DISP=OLD
//SYSPRINT DD    SYSOUT=A
//SYSIN    DD    *
     REPRO -
           INDATASET(USERCAT4) -
           OUTDATASET(USERCAT5) -
```

```
           MERGECAT -
           FILE(DD1)
/*
```

The REPRO command moves all the entries from the source catalog, USERCAT4, and merges them into the target catalog, USERCAT5. All the entries moved are no longer accessible in the source catalog.

- INDATASET identifies the source catalog, USERCAT4.
- OUTDATASET identifies the target catalog, USERCAT5.
- MERGECAT specifies that entries from the source catalog are to be merged with entries of the target catalog.
- FILE specifies the ddname of a DD statement that describes all the volumes that contain VVDS entries for all the entries that are being merged.

Merge Selected Entries (Split) from a User Catalog into Another User Catalog: Example 3

This example shows how selected entries from an integrated catalog facility user catalog are merged into another integrated catalog facility user catalog that is empty. This function effectively splits a catalog into two catalogs. However, the MERGECAT parameter allows the target catalog to be empty or nonempty.

```
//MERGE76   JOB   ...
//STEP1    EXEC  PGM=IDCAMS
//DD1      DD    VOL=SER=VSER01,UNIT=DISK,DISP=OLD
//         DD    VOL=SER=VSER02,UNIT=DISK,DISP=OLD
//         DD    VOL=SER=VSER03,UNIT=DISK,DISP=OLD
//SYSPRINT DD    SYSOUT=A
//SYSIN    DD    *
     REPRO -
           INDATASET(USERCAT4) -
           OUTDATASET(USERCAT5) -
           ENTRIES(VSAMDATA.*) -
           MERGECAT -
           FILE(DD1)
/*
```

The REPRO command moves selected entries from the source catalog, USERCAT4, and merges them into the empty target catalog, USERCAT5. All the entries moved are no longer accessible in the source catalog.

- INDATASET identifies the source catalog, USERCAT4.
- OUTDATASET identifies the target catalog, USERCAT5
- ENTRIES specifies a generic name, VSAMDATA.*. All the names of the entries cataloged in the source catalog that satisfy the generic name are selected to be merged.
- MERGECAT specifies that entries from the source catalog are to be merged with entries of the target catalog.
- FILE specifies the ddname of a DD statement that describes all the volumes that contain VVDS entries for all the entries that are being merged.

Copy a Catalog: Example 4

In this example, a catalog is copied to illustrate the catalog copying procedure.

```
//COPYCAT  JOB   ...
//STEP1    EXEC  PGM=IDCAMS
//SYSPRINT DD    SYSOUT=A
//SYSIN    DD    *
        DEFINE USERCATALOG -
           (NAME(COPYUCAT) -
           ICFCATALOG -
           FOR(365) -
           CYLINDERS(20 10) -
           VOLUME(338000) )
/*
//STEP2    EXEC  PGM=IDCAMS
//SYSPRINT DD    SYSOUT=A
```

```
//SYSIN    DD    *
     REPRO NOMERGECAT -
           INDATASET(MYCAT) -
           OUTDATASET(COPYUCAT)
     EXPORT -
           MYCAT -
           DISCONNECT
/*
//STEP3    EXEC  PGM=IDCAMS
//SYSPRINT DD    SYSOUT=A
//SYSIN    DD    *
     LISTCAT NAMES CAT(COPYUCAT)
/*
//STEP4    EXEC  PGM=IDCAMS
//SYSPRINT DD    SYSOUT=A
//SYSIN    DD    *
     DEFINE ALIAS -
           (NAME(MYCAT) -
           RELATE(COPYUCAT) )
/*
```

STEP 1

A user catalog, COPYUCAT, is defined on volume 338000 using the DEFINE USERCATALOG command. Its parameters are:

- NAME specifies the name of the new catalog, COPYUCAT.
- ICFCATALOG specifies the catalog format of COPYUCAT.
- FOR specifies that the catalog is to be retained for 365 days.
- CYLINDERS specifies that the catalog itself is initially to occupy 20 cylinders. When the catalog's data component is extended, it is to be extended in increments of 10 cylinders.
- VOLUME specifies that the catalog is to reside on volume 338000.

STEP 2

The REPRO NOMERGECAT command copies the contents of MYCAT into COPYUCAT. Access method services treats each catalog as a key-sequenced data set and copies each record. The first three records of MYCAT, which describe MYCAT as an integrated catalog facility catalog, are not copied into COPYUCAT. Entries from MYCAT are written into COPYUCAT beginning with record 4 (that is, after the three self-describing records of COPYUCAT). The REPRO command's parameters are:

- INDATASET identifies the source data set, MYCAT. MYCAT is cataloged in the master catalog.
- OUTDATASET identifies the receiving data set, COPYUCAT. COPYUCAT is cataloged in the master catalog.

The EXPORT command removes MYCAT's user catalog connector entry from the master catalog. MYCAT's cataloged objects now are not available to the system. (STEP4 builds an alias entry that relates MYCAT to COPYUCAT, making the cataloged objects available to the system again.)

STEP 3

The LISTCAT command lists the name of each entry in the new catalog, COPYUCAT.

LISTCAT cannot run in a job step where the catalog is empty when it is opened. To ensure that the LISTCAT correctly reflects the contents of the catalog, the LISTCAT was run as a separate job step.

STEP 4

Access method services builds an alias entry that relates MYCAT entries to COPYUCAT.

Copy a DBCS Data Set: Example 5

In this example, the REPRO command is used with the DBCS and INSERTSHIFT parameters. The REPRO command copies the input data set to the output data set inserting SO and SI characters into each logical

record of the output data set. It is assumed that the input data set's logical records contain DBCS characters and have an LRECL, for this example, of 100 bytes and the record format is fixed length records.

```
//REPRO    JOB  ...
//STEP1    EXEC PGM=IDCAMS
//SYSPRINT DD   SYSOUT=A
//OUTDS    DD   DSN=MY.DATA,DISP=(NEW,CATLG),VOL=SER=VSER01,
//           UNIT=3380,DCB=(LRECL=104,RECFM=F),SPACE=(TRK,(20,10))
//SYSIN    DD   *
         REPRO -
           INDATASET(USER.REPRO.EXAMPLE) -
           OUTFILE(OUTDS) -
           DBCS -
           INSERTSHIFT((11 30)(51 60))
/*
```

The parameters are:

- INDATASET specifies the name of USER.REPRO.EXAMPLE the data set to be copied. This data set might not contain SO and SI characters.
- OUTFILE specifies the name of the output data set, MY.DATA. This data set will have SO and SI characters inserted. Because four shift characters are being inserted, the LRECL must be 4 bytes larger than the input data set's LRECL.
- DBCS specifies that the data contains DBCS characters and should be criteria checked.
- INSERTSHIFT specifies that a SO character is inserted before offsets 11 and 51 of the logical record and a SI character is inserted after offsets 30 and 60 of the logical record.

Encipher Using System Keys: Example 6

In this example, an enciphered copy of part of a VSAM relative record data set is produced using a tape as output. The enciphered data set is deciphered at a remote installation. The keys are managed by the Programmed Cryptographic Facility, the Cryptographic Unit Support, or the z/OS Integrated Cryptographic Service Facility.

```
//ENSYS    JOB    ...
//STEP1    EXEC PGM=IDCAMS
//CLEAR    DD   DSN=RRDS1,DISP=SHR
//CRYPT    DD   DSN=RRDSEN,LABEL=(1,SL),DISP=NEW,
//              UNIT=3480,VOL=SER=TAPE01,
//              DCB=(DEN=3,RECFM=FB,LRECL=516,BLKSIZE=5160)
//SYSPRINT DD   SYSOUT=A
//SYSIN    DD   *
     REPRO -
           INFILE(CLEAR) -
           OUTFILE(CRYPT) -
           COUNT(50) -
           ENCIPHER -
             (EXTERNALKEYNAME(AKEY27) -
              STOREDATAKEY -
              CIPHERUNIT(4) -
              USERDATA(CONF))
/*
```

Job control language statements:

- CLEAR DD describes the relative record data set.
- CRYPT DD describes and allocates a magnetic tape file. LRECL is the relative record data set record size plus 4.

  The REPRO command copies 50 records enciphered from a generated data encrypting key, from the source data set, RRDS1, to the output tape. The source records are enciphered in units of 4 records, except for the last 2 records, which are enciphered together. The enciphered data encrypting key is stored in the header of the target data set; therefore, REPRO will not list the key name or enciphered data encrypting key in SYSPRINT. The parameters of the command are:
- INFILE points to the CLEAR DD statement identifying the source data set to be enciphered, RRDS1.

- OUTFILE points to the CRYPT DD statement, identifying the target data set on tape.
- COUNT indicates that 50 records are to be copied.
- ENCIPHER indicates that the target data set is to contain an enciphered copy of the source data set.
- EXTERNALKEYNAME supplies the name, AKEY27, of the external file key to be used to encipher the data encrypting key.
- STOREDATAKEY indicates that the data encrypting key enciphered under the secondary file key is to be stored in the header of the target data set.
- CIPHERUNIT indicates that 4 source records at a time are to be enciphered as a unit.
- USERDATA specifies a character string, CONF, to be stored in the header of the target data set as user data.

Decipher Using System Keys: Example 7

In this example, the enciphered data set produced by the job in "Encipher Using System Keys: Example 6" on page 326 is deciphered, using a VSAM relative record data set as the target for the plaintext (deciphered) data. The empty slots in the original data set are reestablished. Keys are managed by the Programmed Cryptographic Facility or the Cryptographic Support Unit.

```
//DESYS    JOB    ...
//STEP2    EXEC PGM=IDCAMS
//CRYPT    DD   DSN=RRDSEN,LABEL=(1,SL),DISP=OLD,
//              UNIT=3480,VOL=SER=TAPE01,
//              DCB=DEN=3
//CLEAR    DD   DSN=RRDS2,DISP=SHR
//SYSPRINT DD   SYSOUT=A
//SYSIN    DD   *
     REPRO -
           INFILE(CRYPT) -
           OUTFILE(CLEAR) -
           DECIPHER -
             (SYSTEMKEY -
             SYSTEMKEYNAME(BKEY27))
/*
```

Job control language statements:

- CRYPT DD describes and allocates the magnetic tape containing the enciphered data.
- CLEAR DD describes the relative record data set.

  The REPRO command copies and deciphers the enciphered data set from the source tape to the target data set RRDS2. The enciphered data encrypting key is obtained from the header of the source data set. Use the internal file key (BKEY27) to decipher the enciphered data encrypting key that is then used to decipher the data. The parameters of the REPRO command are:
- INFILE points to the CRYPT DD statement, identifying the tape containing the enciphered source data.
- OUTFILE points to the CLEAR DD statement, identifying the data set to contain the deciphered data, RRDS2. The defined record size must be the same as that of the original relative record data set.
- DECIPHER indicates that the source data set is to be deciphered as it is copied to the target data set.
- SYSTEMKEY indicates that keys are managed by the Program Cryptographic Facility, the Cryptographic Unit Support, or the z/OS Integrated Cryptographic Service Facility.
- SYSTEMKEYNAME supplies the key name, BKEY27, of the internal file key that was used to encipher the system data encrypting key. The file key must be an internal file key in this system.

Encipher Using Private Keys: Example 8

In this example, an enciphered copy of a SAM data set is produced by using an entry-sequenced data set as the target data set. The enciphered data set resides on a volume that is to be stored offline at the local

installation. Each record in the target data set is enciphered separately, using a data encrypting key supplied by the user with a data encrypting key data set. Keys are managed privately by the user.

```
//ENPRI    JOB    ...
//STEP1    EXEC PGM=IDCAMS
//CLEAR    DD   DSN=SAMDS1,DISP=OLD,
//              VOL=SER=VOL005,UNIT=DISK
//CRYPT    DD   DSN=ESDS1,DISP=OLD
//KEYIN    DD   *
     X'53467568503A7C29'
/*
//SYSPRINT DD   SYSOUT=A
//SYSIN    DD   *
     REPRO -
           INFILE(CLEAR) -
           OUTFILE(CRYPT) -
           REUSE -
           ENCIPHER -
             (PRIVATEKEY -
             DATAKEYFILE(KEYIN))
  /*
```

Job control language statements:

- CLEAR DD describes the SAM data set.
- CRYPT DD describes the entry-sequenced data set.
- KEYIN DD describes the data encrypting key data set consisting of a single record containing the data encrypting key.

The REPRO command copies all records enciphered under the supplied data encrypting key, from the source data set, SAMDS1, to the target data set, ESDS1. The plaintext private data encrypting keys is not listed on SYSPRINT, because the user manages the key. The parameters of the REPRO command are:

- INFILE points to the CLEAR DD statement, identifying the source data set to be enciphered, SAMDS1.
- OUTFILE points to the CRYPT DD statement, identifying the target data set, ESDS1. The defined maximum record size of the entry-sequenced data set must be large enough to accommodate the largest SAM record.
- REUSE indicates that the target data set is to be opened as a reusable data set. If the data set was defined as REUSE, it is reset to empty; otherwise, the REPRO command will end.
- ENCIPHER indicates that the target data set is to contain an enciphered copy of the source data set.
- PRIVATEKEY indicates that the key is to be managed by the user.
- DATAKEYFILE points to the KEYIN DD statement that supplies the plaintext data encrypting key, X'53467568503A7C29', to be used to encipher the data.

Decipher Using Private Keys: Example 9

In this example, the enciphered data set produced by the job in “Encipher Using Private Keys: Example 8” on page 327 is deciphered at the same location, using an entry-sequenced data set as the target for the plaintext (deciphered) data. Keys are managed privately by the user.

```
//DEPRI    JOB    ...
//STEP1    EXEC PGM=IDCAMS
//CRYPT    DD   DSN=ESDS1,DISP=OLD
//CLEAR    DD   DSN=ESDS3,DISP=OLD
//SYSPRINT DD   SYSOUT=A
//SYSIN    DD   *
     REPRO -
           INFILE(CRYPT) -
           OUTFILE(CLEAR) -
           DECIPHER -
             (DATAKEYVALUE(X'53467568503A7C29'))
/*
```

Job control language statements:

- CRYPT DD describes the enciphered source entry-sequenced data set.

- CLEAR DD describes the target entry-sequenced data set.

The REPRO command copies and deciphers the enciphered data set from the source data set, ESDS1, to the target data set, ESDS3. The supplied plaintext data encrypting key is used to decipher the data. The parameters of the REPRO command are:

- INFILE points to the CRYPT DD statement identifying the source data set, ESDS1.
- OUTFILE points to the CLEAR DD statement, identifying the target data set, ESDS3, which must be empty. The defined maximum record size of the target entry-sequenced data set must be large enough to accommodate the largest source entry-sequenced data set record.
- DECIPHER indicates that the source data set is to be deciphered as it is copied to the target data set.
- DATAKEYVALUE indicates that keys are to be managed by the user, and supplies the plaintext private data encrypting key, X'53467568503A7C29', used to encipher the data.

Appendix B. Interpreting LISTCAT Output Listings

The various LISTCAT command options allow you to select the LISTCAT output that gives you the information you want. This appendix provides information on the structure of LISTCAT output if you use certain options. Fields that can be printed for each type of catalog entry are listed and described.

Each catalog entry is identified by its type (for example: cluster, non-VSAM, data) and by its entryname. Entries are listed in alphabetic order of the entrynames, unless the ENTRIES parameter is used. The entries are then listed in the order they are specified in the ENTRIES parameter.

An entry that has associated entries is immediately followed by the listing of each associated entry. That is, a cluster's data component (and, if the cluster is key-sequenced, its index component) is listed immediately following the cluster. The associated entry is excluded if type options (CLUSTER, DATA, SPACE, and so on) or a generic entryname list are specified.

This appendix has three parts:

- "LISTCAT Output Keywords" on page 367 lists all field names that can be listed for each type of entry.
- "Description of Keyword Fields" on page 375 describes each field name within a group of related field names.
- "Examples of LISTCAT Output Listings" on page 386 describes and illustrates the LISTCAT output that results when various LISTCAT options are specified.

LISTCAT Output Keywords

This section lists the field names associated with each type of catalog entry. Each field name is followed by an abbreviation that points to a group of related field descriptions in the next section. Keywords are listed in alphabetic order, not in the order of appearance in the LISTCAT output.

The group names and abbreviations are:

Abbreviations
Group Names

ALC
allocation group

ASN
associations group

ATT
attributes group

GDG
generation data group base entry, special fields

HIS
history group

NVS
non-VSAM entry, special field

PRT
protection group

STA
statistics group

VLS
volumes group

© Copyright IBM Corp. 1973, 2021

Alias Entry Keywords

ASSOCIATIONS (ASN)
entryname (HIS)
HISTORY (HIS)
RELEASE (HIS)

Alternate-Index Entry Keywords

ASSOCIATIONS (ASN)
ATTEMPTS (PRT)
ATTRIBUTES (ATT)
CLUSTER (ASN)
CODE (PRT)
CONTROLPW (PRT)
DATA (ASN)
entryname (HIS)
HISTORY (HIS)
  CREATION (HIS)
  DATASET-OWNER(HIS)
  EXPIRATION (HIS)
  RELEASE (HIS)
  SMS-MANAGED
INDEX (ASN)
MASTERPW (PRT)
NOUPGRADE (ATT)
PATH (ASN)
PROTECTION (PRT)
RACF (PRT)
READPW (PRT)
UPDATEPW (PRT)
UPGRADE (ATT)
USAR (PRT)
USVR (PRT)

Cluster Entry Keywords

AIX (ASN)
ASSOCIATIONS (ASN)
ATTEMPTS (PRT)
CODE (PRT)
CONTROLPW (PRT)
DATA (ASN)
entryname (HIS)
HISTORY (HIS)
  CREATION (HIS)
  DATASET-OWNER (HIS)
  EXPIRATION (HIS)
  RELEASE (HIS)
INDEX (ASN)
MASTERPW (PRT)
PATH (ASN)
PROTECTION (PRT)
RACF (PRT)
READPW (PRT)
RLSDATA
  FRLOG

LOG
LOGSTREAMID
LOGREPLICATE
RECOVERY REQUIRED
RECOVERY TIMESTAMP GMT
RECOVERY TIMESTAMP LOCAL
RLS IN USE
VSAM QUIESCED
SMSDATA
BWO
BWO TIMESTAMP
BWO STATUS
CA RECLAIM
DATACLASS
LBACKUP
MANAGEMENTCLASS
STORAGECLASS
UPDATEPW (PRT)
USAR (PRT)
USVR (PRT)

Data Entry Keywords

ACT-DICT-TOKEN (ATT)
ACCOUNT-INFO
AIX (ASN)
ALLOCATION (ALC)
ASSOCIATIONS (ASN)
ATTEMPTS (PRT)
ATTRIBUTES (ATT)
AVGLRECL (ATT)
AXRKP (ATT)
BINARY (ATT)
BUFND (ATT)
BUFSPACE (ATT)
BYTES/TRACK (VLS)
CCSID (ATT)
CI/CA (ATT)
CISIZE (ATT)
CLUSTER (ASN)
CODE (PRT)
COMP-FORMT (ATT)
COMP-USER-DATA-SIZE (STA)
CONTROLPW (PRT)
EXTENDED (ATT)
DDMEXIST (ATT)
DEVTYPE (VLS)
DSTGWAIT (ATT)
entryname (HIS)
ERASE (ATT)
EXCPEXIT (ATT)
EXCPS (STA)
EXT-ADDR (ATT)
EXTENT-NUMBER (VLS)
EXTENT-TYPE (VLS)
EXTENTS (STA)
EXTENTS (VLS)

LOW-CCHH (VLS)
LOW-RBA (VLS)
TRACKS (VLS)
HIGH-CCHH (VLS)
HIGH-RBA (VLS)
FREESPACE-%CI (STA)
FREESPACE-%CA (STA)
FREESPC (STA)
HI-KEY-RBA (VLS)
HI-A-RBA (ALC)
HI-U-RBA (ALC)
HI-A-RBA (VLS)
HI-U-RBA (VLS)
HIGH-KEY (VLS)
HISTORY (HIS)
CREATION (HIS)
DATASET-OWNER (HIS)
EXPIRATION (HIS)
RELEASE (HIS)
ICFCATALOG (ATT)
INDEX (ASN)
INDEXED (ATT)
INH-UPDATE (ATT)
KEYLEN (ATT)
LINEAR (ATT)
LOW-KEY (VLS)
MASTERPW (PRT)
MAXLRECL (ATT)
MAXRECS (ATT)
NOERASE (ATT)
NONINDEXED (ATT)
NONSPANNED (ATT)
NONUNIQKEY (ATT)
NOREUSE (ATT)
NOSWAP (ATT)
NOTRKOVFL (ATT)
NOTUSABLE (ATT)
NOWRITECHK (ATT)
NUMBERED (ATT)
PGSPC (ASN)
PHYRECS/TRK (VLS)
PHYREC/SIZE (VLS)
PROTECTION (PRT)
RACF (PRT)
READPW (PRT)
RECOVERY (ATT)
REC-DELETED (STA)
REC-INSERTED (STA)
REC-RETRIEVED (STA)
REC-TOTAL (STA)
REC-UPDATED (STA)
RECORDS/CI (ATT)
RKP (ATT)
REUSE (ATT)
RECVABLE (ATT)
SHROPTNS (ATT)
SPACE-PRI (ALC)
SPACE-SEC (ALC)

SPACE-TYPE (ALC)
SPEED (ATT)
SPLITS-CA (STA)
SPLITS-CI (STA)
SPANNED (ATT)
STATISTICS (STA)
STRIPE-COUNT (ATT)
STRNO (ATT)
SWAP (ATT)
SYSTEM-TIMESTAMP (STA)
TEMP-EXP (ATT)
TEXT (ATT)
TRACKS/CA (VLS)
TRKOVFL (ATT)
UNORDERED (ATT)
UNIQUE (ATT)
UNIQUEKEY (ATT)
UPDATEPW (PRT)
USAR (PRT)
USER-DATA-SIZE (STA)
USVR (PRT)
VOLFLAG (VLS)
VOLSER (VLS)
VOLUMES (VLS)
WRITECHECK (ATT)

Index Entry Keywords

AIX (ASN)
ALLOCATION (ALC)
ASSOCIATIONS (ASN)
ATTEMPTS (PRT)
ATTRIBUTES (ATT)
AVGLRECL (ATT)
BUFNI (ATT)
BUFSPACE (ATT)
CI/CA (ATT)
CISIZE (ATT)
CLUSTER (ASN)
CODE (PRT)
CONTROLPW (PRT)
DEVTYPE (VLS)
DSTGWAIT (ATT)
entryname (HIS)
ERASE (ATT)
EXCPEXIT (ATT)
EXCPS (STA)
EXTENTS (STA)
EXTENT-NUMBER (VLS)
EXTENT-TYPE (VLS)
EXTENTS (VLS)
  HIGH-CCHH (VLS)
  HIGH-RBA (VLS)
  LOW-CCHH (VLS)
  LOW-RBA (VLS)
  TRACKS (VLS)
FREESPACE-%CI (STA)

FREESPACE-%CA (STA)
FREESPC (STA)
HI-A-RBA (ALC)
HI-U-RBA (ALC)
HI-A-RBA (VLS)
HI-U-RBA (VLS)
HIGH-KEY (VLS)
HISTORY (HIS)
  CREATION (HIS)
  DATASET-OWNER (HIS)
  EXPIRATION (HIS)
  RELEASE (HIS)
INDEX (STA)
  ENTRIES/SECT (STA)
  HI-LEVEL-RBA (STA)
  LEVELS (STA)
  SEQ-SET-RBA (STA)
INH-UPDATE (ATT)
KEYLEN (ATT)
LOW-KEY (VLS)
MASTERPW (PRT)
MAXLRECL (ATT)
NOERASE (ATT)
NOREUSE (ATT)
NOTUSABLE (ATT)
NOWRITECHK (ATT)
PHYRECS/TRK (VLS)
PHYREC-SIZE (VLS)
PROTECTION (PRT)
RACF (PRT)
READPW (PRT)
RECOVERY (ATT)
REC-DELETED (STA)
REC-INSERTED (STA)
REC-RETRIEVED (STA)
REC-TOTAL (STA)
REC-UPDATED (STA)
RKP (ATT)
REUSE (ATT)
SHROPTNS (ATT)
SPACE-PRI (ALC)
SPACE-SEC (ALC)
SPACE-TYPE (ALC)
SPEED (ATT)
SPLITS-CA (STA)
SPLITS-CI (STA)
STATISTICS (STA)
SYSTEM-TIMESTAMP (STA)
TEMP-EXP (ATT)
TRACKS/CA (VLS)
UNIQUE (ATT)
UPDATEPW (PRT)
USAR (PRT)
USVR (PRT)
VOLFLAG (VLS)
VOLSER (VLS)
VOLUME (VLS)
WRITECHECK (ATT)

Generation Data Group Base Entry Keywords

ASSOCIATIONS (ASN)
ATTRIBUTES (GDG)
  EMPTY (GDG)
  EXTENDED (GDG)
  FIFO (GDG)
  LIFO (GDG)
  LIMIT (GDG)
  NOEMPTY (GDG)
  NOEXTENDED (GDG)
  NOPURGE (GDG))
  NOSCRATCH (GDG)
  PURGE (GDG)
  SCRATCH (GDG)
entryname (HIS)
HISTORY (HIS)
  CREATION (HIS)
  DATASET-OWNER (HIS)
  EXPIRATION (HIS)
  RELEASE (HIS)
NONVSAM (ASN)

Non-VSAM Entry Keywords

ACT-DICT-TOKEN (ATT)
ALIAS (ASN)
ASSOCIATIONS (ASN)
BINARY (ATT)
CCSID (ATT)
COMP-FORMT (ATT)
COMP-USER-DATA-SIZE (STA)
EXTENDED (ATT)
DDMEXIST (ATT)
DEVTYPE(VLS)
entryname (HIS)
FSEQN (NVS)
HISTORY (HIS)
  CREATION (HIS)
  DATASET-OWNER (HIS)
  EXPIRATION (HIS)
  RELEASE (HIS)
  STATUS
OAMDATA
  DIRECTORYTOKEN
SIZES-VALID (STA)
SMSDATA
  DATACLASS
  MANAGEMENTCLASS
  STORAGECLASS
  LBACKUP
STRIPE-COUNT (ATT)
TEXT (ATT)
USER-DATA-SIZE (STA)
VERSION-NUMBER (ATT)
VOLSER(VLS)

Page Space Entry Keywords

ASSOCIATIONS (ASN)
ATTEMPTS (PRT)
CODE (PRT)
CONTROLPW (PRT)
DATA (ASN)
entryname (HIS)
HISTORY (HIS)
  CREATION (HIS)
  DATASET-OWNER (HIS)
  EXPIRATION (HIS)
  RELEASE (HIS)
INDEX (ASN)
MASTERPW (PRT)
PROTECTION (PRT)
RACF (PRT)
READPW (PRT)
UPDATEPW (PRT)
USAR (PRT)
USVR (PRT)

Path Entry Keywords

AIX (ASN)
ASSOCIATIONS (ASN)
ATTEMPTS (PRT)
ATTRIBUTES (ATT)
CLUSTER (ASN)
CODE (PRT)
CONTROLPW (PRT)
DATA (ASN)
entryname (HIS)
HISTORY (HIS)
  CREATION (HIS)
  DATASET-OWNER (HIS)
  EXPIRATION (HIS)
  RELEASE (HIS)
INDEX (ASN)
MASTERPW (PRT)
NOUPDATE (ATT)
PROTECTION (PRT)
RACF (PRT)
READPW (PRT)
UPDATE (ATT)
UPDATEPW (PRT)
USAR (PRT)
USVR (PRT)

User Catalog Entry Keywords

ALIAS (ASN)
ASSOCIATIONS (ASN)
DEVTYPE(VLS)
entryname (HIS)
HISTORY (HIS)
  RELEASE (HIS)

SMSDATA
  DATACLASS
  MANAGEMENTCLASS
  STORAGECLASS
  LBACKUP
VOLFLAG (VLS)
VOLSER (VLS)

Description of Keyword Fields

This section contains a description of each field name. The field names are in the following groups of related information:

Abbreviations
  Group Names

ALC
  allocation group

ASN
  associations group

ATT
  attributes group

GDG
  generation data group base entry, special fields

HIS
  history group

NVS
  non-VSAM entry, special field

PRT
  protection group

STA
  statistics group

VLS
  volumes group.

Groups are in alphabetic order. Field names within each group are in alphabetic order, not the order of appearance in the listed entry.

ALC: Allocation Group

The fields in this group describe the space allocated to the data or index component defined by the entry.

HI-A-RBA—The highest RBA (plus 1) available within allocated space to store data.

HI-U-RBA—The highest RBA (plus 1) within allocated space that actually contains data. (The RBA of the next completely unused control interval.)

SPACE-PRI—Gives the number of units (indicated under TYPE) of space allocated to the data or index component when the cluster was defined. This amount of space is to be allocated whenever a data component, a key range within the data component, or the data component's associated sequence set (if IMBED is an attribute of the cluster) is extended onto a candidate volume.

SPACE-SEC—Gives the number of units (indicated under TYPE) of space to be allocated whenever a data set (or key range within it) is extended on the same volume.

SPACE-TYPE—Indicates the unit of space allocation:

  CYLINDER—Cylinders
  KILOBYTE—Kilobytes

MEGABYTE—Megabytes
TRACK—Tracks

For nonstriped VSAM data sets, you can specify in the SMS data class parameter whether to use primary or secondary allocation amounts when extending to a new volume. VSAM striped data sets extend by stripe.

The actual amount of space allocated may differ from the amount requested as a result of SMS policies, record formats, system exits, device geometries and other software or hardware features. Additional information on space allocation is provided in *z/OS DFSMS Using Data Sets*.

ASN: Associations Group

This group lists the type (cluster or data, for example) and entry names of the objects associated with the present entry. A cluster or alternate index entry will indicate its associated path entries and data and index (if a key-sequenced data set) entries. Similarly, an index or data entry will indicate its associated cluster or the alternate index of which it is a component.

- An alias entry points to:
  - Its associated non-VSAM data set entry. If the associated entry is a symbolic association (for example, defined with the SYMBOLICRELATE keyword), both the unresolved and resolved values will be listed. The resolved value will be the value resulting from using the symbols defined on the system on which the LISTCAT is run.
  - A user catalog entry. (All alias entries for a non-VSAM data set entry are chained together, as are alias entries for a user catalog entry.)
- An alternate index entry points to:
  - Its associated data and index entries.
  - Its base cluster's cluster entry.
  - Each associated path entry.
- An alternate index's data entry points to:
  - Its associated alternate index entry.
- An alternate index's index entry points to:
  - Its associated alternate index entry.
- A cluster entry points to:
  - Its associated data entry.
  - Each associated path entry.
  - For a key-sequenced cluster, its associated index entry.
  - For a cluster with alternate indexes, each associated alternate index entry.
- A cluster's data entry points to:
  - Its associated cluster entry.
- A cluster's index entry points to:
  - Its associated cluster entry.
- A generation data group base entry points to:
  - Its associated non-VSAM data set entries.
- A non-VSAM data set entry points to:
  - Its associated alias entry.
  - Its associated generation data group (for a G0000V00 non-VSAM).
- A page space entry points to:

  - Its associated data entry. The page space is cataloged as an entry-sequenced cluster with a cluster entry and an associated data entry.
- A path entry that establishes the connection between a base cluster and an alternate index points to:
  - Its associated alternate index entry, and the alternate index's associated data and index entries.
  - The data entry of its associated base cluster.
  - For a key-sequenced base cluster, the index entry of its associated base cluster.
- Path entry that is an alias for a cluster entry points to:
  - Its associated base cluster entry.
  - The data entry of its associated base cluster.
  - For a key-sequenced cluster, the index entry of its associated base cluster.
- A user catalog entry points to:
  - Its associated alias entry.

Entries are identified as shown in the list that follows.

- **AIX**—Identifies an alternate index entry.
- **ALIAS**—Identifies an alias entry.
- **CLUSTER**—Identifies a cluster entry.
- **DATA**—Identifies a data entry.
- **GDG**—Identifies a generation data group (GDG) base entry.
- **INDEX**—Identifies an index entry.
- **NONVSAM**—Identifies a non-VSAM data set entry.
- **PGSPC**—Identifies a page space entry.
- **PATH**—Identifies a path entry.
- **UCAT**—Identifies a user catalog entry.

ATT: Attributes Group

The fields in this group describe the miscellaneous attributes of the entry. See the DEFINE command for further discussion of most of these attributes.

ACT-DIC-TOKEN—The active dictionary token or NULL. This attribute is only valid for compressed data sets.

Note: The following information is not an intended programming interface. It is provided for diagnostic purposes only.

The first byte of the dictionary token indicates the type of compression used for the data set.

X'100. ....'
indicates compression has been rejected for the data set. No data is compressed.

X'010. ....'
indicates generic DBB compression is used.

X'011. ....'
indicates tailored compression is used.

AVGLRECL—The average length of data records, in bytes. AVGLRECL equals MAXLRECL when the records are fixed length. Do not, however, set AVGLRECL equal to MAXLRECL for variable-length relative records.

Note: For variable-length RRDSs, the AVLGLRECL shown in the LISTCAT output is 4 greater than the user-specified length, reflecting the system-increased record size.

AXRKP—Indicates, for an alternate index, the offset, from the beginning of the base cluster's data record, at which the alternate-key field begins.

BUFND—The number of buffers provided for catalog data records. The default for BUFND is taken at catalog open and is not reflected in the output from LISTCAT.

BUFNI—The number of buffers provided for catalog index records. The default for BUFNI is taken at catalog open and is not reflected in the output from LISTCAT.

BUFSPACE—The minimum buffer space, in bytes, in virtual storage to be provided by a processing program.

CCSID—The Coded Character Set Identifier attribute that identifies a specific set of encoding scheme identifier, character set identifiers, code page identifiers, or the additional coding required to uniquely identify the coded graphic used.

CI/CA—The number of control intervals per control area. If you wish to learn the number of tracks in each CA, see "VLS: Volumes Group" on page 384 to view the TRACKS/CA value for the component being listed.

CISIZE—The size of a control interval, in bytes.

COMP-FORMT—The data is written to the data set in a format that allows data compression.

EATTR— Indicates whether the VSAM data set can reside in the cylinder-managed space of an EAV.

NULL indicates the attribute was not specified or that the data set type is non VSAM.
NO indicates the VSAM data set can only reside in track managed space.
YES indicates the VSAM data set can reside in track and cylinder managed space.

ECSHARING—Sharing with the coupling facility for this catalog is allowed.

ERASE—The system will erase the medium when the data set is deleted or when the system releases unused space at the end of the data set. After that, data is not recoverable by any System z® software. See the *Erasing DASD Data* section in *z/OS DFSMS Using Data Sets*.

EXCPEXIT—The name of the object's exception exit routine.

EXT-ADDR—Extended addressability indicator.

EXTENDED—The extended format indicator.

ICFCATALOG—The object is part of a cluster for the catalog data set.

INDEXED—The data component has an index; it is key-sequenced.

INH-UPDATE—The data component cannot be updated. Either the data component was exported with INHIBITSOURCE specified, or its entry was modified by way of ALTER, with INHIBIT specified.

KEYLEN—The length of the key field in a data record, in bytes.

LINEAR—The cluster is a linear data set.

MAXLRECL—The maximum length of data or index records, in bytes. MAXLRECL equals AVGLRECL when the records are fixed length. Do not, however, set MAXLRECL equal to AVGLRECL for variable-length relative records.

Note: For variable-length RRDSs, the MAXLRECL shown in the LISTCAT output is 4 greater than the user-specified length, reflecting the system-increased record size.

MAXRECS—Identifies the highest possible valid relative record number, for a relative record data set. This value is calculated as follows: 2 to the 32nd power/CISIZE x number of records slots per control interval

NOECSHARE—Sharing with the coupling facility for this catalog is not allowed.

NOERASE—Records are not to be erased (set to binary 0's) when deleted.

NONINDEXED—The data component has no index; it is entry-sequenced.

NONSPANNED—Data records cannot span control intervals.

NONUNIQKEY—Indicates, for an alternate index, that more than one data record in the base cluster can contain the same alternate-key value.

NOREUSE—The data set cannot be reused.

NOSWAP—The page space is a conventional page space and cannot be used as a high speed swap data set.

NOTRKOVFL—The physical blocks of a page space data set cannot span a track boundary.

NOUPDATE—When the path is opened for processing, its associated base cluster is opened but the base cluster's upgrade set is not opened.

NOUPGRADE—The alternate index is not upgraded unless it is opened and being used to access the base cluster's data records.

NOTUSABLE—The entry is not usable because (1) the catalog could not be correctly recovered by RESETCAT, or (2) a DELETE SPACE FORCE was issued for a volume in the entry's volume list.

NOWRITECHK—Write operations are not checked for correctness.

NUMBERED—The cluster is a relative record data set.

RECORDS/CI—Specifies the number of records, or slots, in each control interval of a relative record data set.

RECOVERY—A temporary CLOSE is issued as each control area of the data set is loaded, so the whole data set will not have to be reloaded if a serious error occurs during loading.

REUSE—The data set can be reused (that is, its contents are temporary and its high-used RBA can be reset to 0 when it is opened).

RKP—The relative key position:the displacement from the beginning of a data record to its key field.

SHROPTNS—(n,m) The numbers n and m identify the types of sharing permitted. See SHAREOPTIONS in the DEFINE CLUSTER section for more details.

SIZES-VALID—Indicates if user data sizes are valid (YES) or are not valid (NO).

SPANNED—Data records can be longer than control interval length, and can cross, or span, control interval boundaries.

SPEED—CLOSE is not issued until the data set has been loaded.

STRIPE-COUNT—The number of stripes for the data set. This number will always be 1 for extended format VSAM KSDS.

STRNO—The number of concurrent RPLs the catalog is prepared to accommodate. The default for STRNO is taken at catalog open and is not reflected in the output from LISTCAT.

Note: LISTCAT ALL indicates a value of 0 (the default) if no other value is specified, rather than the expected value of 2.

SWAP—The page space is a high speed swap data set used by Auxiliary Storage Management during a swap operation to store and retrieve the set of LSQA pages owned by an address space.

TEMP-EXP—The data component was temporarily exported.

TRKOVFL—The physical blocks of a page space data set can span a track boundary.

UNIQUEKEY—Indicates, for an alternate index, that the alternate-key value identifies one, and only one, data record in the base cluster.

UPDATE—When the path is opened, the upgrade set's alternate indexes (associated with the path's base cluster) are also opened and are updated when the base cluster's contents change.

UPGRADE—When the alternate index's base cluster is opened, the alternate index is also opened and is updated to reflect any changes to the base cluster's contents.

VERSION-NUMBER — Version of the format of the extended format sequential data set.

WRITECHECK—Write operations are checked for correctness.

GDG: Generation Data Group Base Entry, Special Fields

The special fields for a generation data group base entry describe attributes of the generation data group.

ATTRIBUTES
This field includes the following fields:

EMPTY
All generation data sets in the generation data group are uncataloged when the maximum number (given under LIMIT) is reached and one more data set is to added to the group.

EXTENDED
The maximum number of generation data sets can be 999.

LIMIT
The maximum number of generation data sets allowed in the generation data group.

NOEMPTY
Only the oldest generation data set in the generation data group is uncataloged when the maximum number (given under LIMIT) is reached and one more data set is to be added to the group.

NOEXTENDED
The maximum number of generation data sets can be 255.

NOPURGE
Does not override expiration dates when scratching the GDS (see PURGE below).

NOSCRATCH
Generation data sets are not to be scratched (see SCRATCH below) when uncataloged.

PURGE
Overrides expiration date when scratching a GDS.

SCRATCH
Generation data sets are to be scratched (that is, the DSCB describing each one is removed from the VTOC of the volume where it resides) when uncataloged.

NVS: Non-VSAM Entry, Special Field

The special field for a non-VSAM data set describes a non-VSAM data set stored on magnetic tape.

FSEQN—The sequence number (for the tape volume indicated under the "VOLUMES group" keyword VOLSER) of the file in which the non-VSAM data set is stored.

HIS: History Group

The fields in this group identify the object's owner and give the object's creation and expiration dates.

entryname—The name of the cataloged object. The entryname can be specified with the ENTRIES parameter of LISTCAT to identify a catalog entry.

HISTORY—This field includes the following fields:

- **CREATION**—The Julian date (YYYY.DDD) on which the entry was created.

  Note: For a currently migrated VSAM data set, CREATION will be the date the data set was migrated (because a new data set was created to encapsulate the original version of the data set, plus related control information). When the data set is later restored, CREATION will be set to the creation date of the original data set.

  For a non-VSAM data set, CREATION will always be set to the creation date of the original data set, even if the data set is currently migrated.
- **DATASET-OWNER**—Contents of the data set owner field in the BCS. Was formerly called **OWNER-IDENT** field.
- **LAST ALTER DATE**—The julian date (YYYY.DDD) on which the GDG base was last altered, by adding or removing a GDS.

- **RELEASE**—The release of VSAM under which the entry was created:

  1 = OS/VS2 Release 3 and releases preceding Release 3
  2 = OS/VS2 Release 3.6 and any later releases

- **STATUS**—The possible values this field can contain are active, deferred, library, or rolled-off.

  For generation data set entries, the status is indicated by active, deferred, or rolled-off.

  For non-VSAM entries, a status of library indicates a partitioned data set extended (PDSE).

- **OAMDATA**—For OAM entries, this field contains the following:
  - **DIRECTORYTOKEN**—The OAM directory token (1-to-8 characters).
- **RLSDATA**—For RLS/Recovery entries, this field has the following:
  - **FRLOG**—Gives the value of the FRLOG parameter specified on the DEFINE CLUSTER.
  - **LOG**—Gives the value of the LOG parameter specified on DEFINE CLUSTER.
  - **LOGSTREAMID**—This gives you the value of the LOGSTREAMID parameter specified on DEFINE CLUSTER.
  - **LOGREPLICATE**—This gives you the value of the LOGREPLICATE parameter specified on DEFINE CLUSTER.
  - **RECOVERY REQUIRED**—Indicates whether the sphere is currently in the process of being forward recovered.
  - **RECOVERY TIMESTAMP**—This gives the time the most recent backup was taken when the data set was accessed by CICS using VSAM RLS.
  - **RLS IN USE**—Indicates whether the sphere is using RLS. A sphere uses RLS if:
    - It was last opened for RLS processing.
    - It is not opened for RLS processing but is recoverable and either has retained locks protecting updates or is in a lost locks state.
  - **VSAM QUIESCED**—Indicates that the sphere has been quiesced for RLS. You cannot open the sphere using RLS.
- **SMSDATA**—For SMS-managed data sets, this field has:
  - **BWO**—Data set is enabled for backup-while-open.
  - **BWO STATUS**—Indicates the status of the data set. Status can be:
    - Data set is enabled for backup-while-open
    - Control interval or control area split in progress
    - Data set has been restored and is down level. It might need to be updated with forward recovery logs.
  - **BWO TIMESTAMP**—A CICS timestamp that indicates the time from which forward recovery logs have to be applied to a restored copy of the data set.
  - **CA RECLAIM**—The cataloged value for the CA reclaim attribute, as specified in the data class when the data set was defined or when modified with the ALTER command. CA reclaim specifies whether empty CA space will be reclaimed. This value does not reflect settings in PARMLIB member IGDSMS*xx* or SETSMS commands, which may disable CA reclaim. This line is not displayed for VSAM data set types that are ineligible for CA Reclaim: ESDS, RRDS, VRRDS, LDS and KSDS with the IMBED option.
  - **DATACLASS**—The name of the data class assigned to the cluster.
  - **LAST TRANSITION**—The date of 'Last Successful Class Transition Date' (LSCTD).
  - **LBACKUP**—The last date that the cluster was backed up. If this date is unavailable, this field will contain an entry of all "Xs" rather than an actual date.
  - **MANAGEMENTCLASS**—The name of the management class assigned to the cluster.
  - **STORAGECLASS**—The name of the storage class assigned to the cluster.

PRT: Protection Group

The fields in this group describe how the alternate index, cluster, data component, index component, or path defined by the entry is password-protected or RACF protected. NULL or SUPPRESSED might be listed under password protection and YES or NO might be listed under RACF protection.

NULL indicates that the object defined by the entry has no passwords.

SUPP indicates that the master password of neither the catalog nor the entry was specified, so authority to list protection information is not granted.

RACF—Indicates if the entry is protected by the Resource Access Control Facility.

YES—Entry is RACF protected.
NO—Entry is not RACF protected.

ATTEMPTS—Gives the number of times the console operator is allowed to try to enter a correct password.

CODE—Gives the code used to tell the console operator which alternate index, catalog, cluster, path, data component, or index component requires a password to be entered. NULL is listed under CODE if a code is not used—the object requiring the password is identified with its full name.

CONTROLPW—The control interval password (that is, the password for control interval access). NULL indicates no control interval password.

MASTERPW—The master password.

READPW—The read-only password. NULL indicates no read-only password.

UPDATEPW—The update password. NULL indicates no update password.

USAR—The contents (1-to-255 bytes, in character format) of the USAR (user-security-authorization record). This is the information specified in the string subparameter of the AUTH subparameter of the DEFINE command.

USVR—The name of the USVR (user-security-verification routine) that is to be invoked to verify authorization of any access to the entry.

STA: Statistics Group

The fields in this group give numbers and percentages that tell how much activity has taken place in the processing of a data or index component. The statistics in the catalog are updated when the data set is closed. Therefore, if an error occurs during CLOSE, the statistics might not be valid.

If the data set has not been properly closed, the statistics are not updated and will therefore be incorrect. Once the data set is properly closed after a previous close failure, a LISTCAT of the data set will show these statistics as invalid. VERIFY cannot correct these statistics. To correct these statistics, you can either use the EXPORT and IMPORT commands or you can REPRO the data set to a new data set. When using compressed VSAM data sets, REPRO should specify the REPLACE option to ensure that the correct statistics are calculated.

The statistics may also not be correct if the data set was open for update in more than one address space at the same time. Depending on the processing occurring (especially with inserts and deletes) and if the data set extends you may not see all the statistics properly reflected. In some cases the statistics will not include all occurrences and depending on the sequence of processing the number of logical records in the data set may be incorrect. In some cases this may show as a very large value for the number of logical records which usually indicates that at some point the counter for the number of logical records has gone negative.

COMP-USER-DATA-SIZE—The total length of data after compression. If the data lengths are too large to be presented in decimal, they are presented in hexadecimal format.

FREESPACE-%CI—Percentage of space to be left free in a control interval for subsequent processing.

FREESPACE-%CA—Percentage of control intervals to be left free in a control area for subsequent processing.

FREESPC—Actual number of bytes of free space in the total amount of space allocated to the data or index component. Free space in partially used control intervals is not included in this statistic. Some of this space may not be accessible due to the current amount of key compression that can be performed in the index.

INDEX—This field appears only in an index entry. The fields under it describe activity in the index component.

ENTRIES/SECT—The number of entries in each section of entries in an index record.

HI-LEVEL-RBA—The RBA (relative byte address) of the highest-level index record.

LEVELS—The number of levels of records in the index. The number is 0 if no records have been loaded into the key-sequenced data set to which the index belongs.

SEQ-SET-RBA—The RBA (relative byte address), in decimal, of the first sequence-set record. The sequence set can be separated from the index set by some amount of RBA space.

The remaining fields in the statistics group (except for the system timestamp field), are updated only when the data set is closed.

EXCPS—EXCP (run channel program—SVC 0) macro instructions issued by VSAM against the data or index component.

EXTENTS—Extents in the data or index component.

REC-DELETED—The number of records that have been deleted from the data or index component. Statistics for records deleted are not maintained when the data set is processed in control interval mode. For a data set that is eligible for CA_RECLAIM on a system with CA_RECLAIM enabled, this value for the index component will be the number of times CA_RECLAIM has occurred for this data set.

REC-INSERTED—For a key-sequenced data set, the number of records that have been inserted into the data component before the last record; records originally loaded and records added to the end are not included in this statistic. For relative record data sets, it is the number of records inserted into available slots; the number of records originally loaded are included in this statistic. Statistics for records inserted are not maintained when the data set is processed in control interval mode. For a data set that is eligible for CA_RECLAIM on a system with CA_RECLAIM enabled, this value for the index component will be the number of CA's that have been reused for this data set.

REC-RETRIEVED—The number of records that have been retrieved from the data or index component, whether for update or not for update. Statistics for records retrieved are not maintained when the data set is processed in control interval mode.

REC-TOTAL—The total number of records actually in the data or index component. This statistic is not maintained when the data set is processed in control interval mode. For a variable-length RRDS, this is the count of slots in the data set.

REC-UPDATED—The number of records that have been retrieved for update and rewritten. This value does not reflect those records that were deleted, but a record that is updated and then deleted is counted in the update statistics. Statistics for records updated are not maintained when the data set is processed in control interval mode.

SPLITS-CA—Control area splits. Half the data records in a control area were written into a new control area and then were deleted from the old control area. For an index component, the value reported is the number of times a split occurred when inserting a record in the next higher level of the index above the sequence set records. For a data set that is eligible for CA_RECLAIM on a system with CA_RECLAIM enabled, the value reported here will also include the number of times that CA_RECLAIM has occurred for this data set. For a data set that is eligible for CA_RECLAIM to determine the number of CA splits that were not related to CA_RECLAIM processing take the SPLITS-CA value for the data component and subtract the value of REC-DELETED from the index component.

SPLITS-CI—Control interval splits. Half the data records in a control interval were written into a new control interval and then were deleted from the old control interval. For an index component the value reported is the number of times a split occurred when inserting a record into the sequence set.

SYSTEM-TIMESTAMP—The time (system time-of-day clock value) the data or index component was last closed (after being opened for operations that might have changed its contents).

USER-DATA-SIZE—Displays the total length of data before compression. If the data lengths are too large to be presented in decimal, they are presented in hexadecimal format.

VLS: Volumes Group

The fields in this group identify the volume on which a data component, index component, user catalog, or non-VSAM data set is stored. It also identifies candidate volumes for a data or index component. The fields describe the type of volume and give, for a data or index component, information about the space the object uses on the volume.

- If an entry-sequenced or relative record cluster's data component has more than one VOLUMES group, each group describes the extents that contain data records for the cluster on a specific volume.
- If a key-sequenced cluster's data component has more than one VOLUMES group, each group describes the extents that contain data records for the cluster, or one of its key ranges, on a specific volume.
- If a key-sequenced cluster's index component has more than one VOLUMES group, each group describes the extents that contain index records for the cluster, or one of its key ranges, on a specific volume. The first VOLUMES group describes the extent that contains the high-level index records (that is, index records in levels above the sequence set level). Each of the next groups describes the extents that contain sequence-set index records for the cluster, or one of its key ranges, on a specific volume. The index component of a key-sequenced data set with the IMBED attribute will have a minimum of two volume groups, one for the embedded sequence set, and one for the high-level index. The extents for the embedded sequence set are the same as those for the data component.

BYTES/TRACK—The number of bytes that VSAM can write on a track (listed for page spaces only).

DEVTYPE—The type of device to which the volume belongs.

EXTENT-NUMBER—The number of extents allocated for the data or index component on the volume.

EXTENT-TYPE—The type of extents:

00—The extents are contiguous.
40—The extents are not preformatted.
80—A sequence set occupies a track adjacent to a control area.
FF—A candidate volume.

EXTENTS—Gives the physical and relative byte addresses of each extent.

LOW-CCHH—The track address of the beginning of the extent where CCHH is a 28-bit cylinder address. A 28-bit cylinder address is described by the X'CCCCcccH' format, where 'ccc' is the high order 12-bits and 'CCCC' is the low order 16-bits of the cylinder address. The 'H' is the 4-bit track address. The high order 12-bits 'ccc' will be non-zero for a cylinder address greater than 65535.
LOW-RBA—The RBA (relative byte address), in decimal, of the beginning of the extent.
TRACKS—The number of tracks in the extent, from low to high device addresses.
HIGH-CCHH—The track address of the end of the extent where CCHH is a 28-bit cylinder address. A 28-bit cylinder address is described by the X'CCCCcccH' format, where 'ccc' is the high order 12-bits and 'CCCC' is the low order 16-bits of the cylinder address. The 'H' is the 4-bit track address. The high order 12-bits ('ccc') will be non-zero for a cylinder address greater than 65535.
HIGH-RBA—The RBA (relative byte address), in decimal, of the end of the extent.

HIGH-KEY [1] —For a key-sequenced data set with the KEYRANGE attribute, the highest hexadecimal value allowed on the volume in the key field of a record in the key range. A maximum of 64 bytes can appear in HIGH-KEY.

HI-KEY-RBA [1]—For a key-sequenced data set, the RBA (relative byte address), in decimal, of the control interval on the volume that contains the highest keyed record in the data set or key range.

[1] Multiple key ranges can reside on a single volume; the volumes group is repeated for each such key range field.

LOW-KEY [1]—For a key-sequenced data set with the KEYRANGE attribute, the lowest hexadecimal value allowed on the volume in the key field of a record in the key range. A maximum of 64 bytes can appear in LOW-KEY.

PHYRECS/TRK—The number of physical records (of the size indicated under PHYRECS-SIZE) that VSAM can write on a track on the volume.

PHYREC-SIZE—The number of bytes that VSAM uses for a physical record in the data or index component.

HI-A-RBA—The highest RBA (plus 1) available within allocated space to store data component, its key range, the index component, or the sequence set records of a key range.

HI-U-RBA—The highest RBA (plus 1) within allocated space that actually contains data component, its key range, the index component, or the sequence set records of a key range. (The RBA of the next completely unused control interval.)

TRACKS/CA—The number of tracks in a control area for the component being listed. (This value is computed when the entry is defined. This value reflects the optimum size of the control area for the given device and the nature of the entry, whether indexed, nonindexed, or numbered.) For a key-sequenced data set with the imbedded attribute, this value includes the sequence set track.

VOLFLAG—Indicates if the volume is a candidate volume and if the volume is a prime or overflow volume on which data in a given key range is stored.

CANDIDATE—The volume is a candidate for storing the data or index component.

CAND-SPACE—The volume is a candidate for storing the data or index component, and it has a primary extent preallocated (the data set was defined with a guaranteed-space storage class).

OVERFLOW—The volume is an overflow volume on which data records in a key range are stored. The KEYRANGE begins on another (PRIME) volume.

PRIME—The volume is the first volume on which data records in a key range are stored.

VOLSER—The serial number of the volume.

Device Type Translate Table

The following table lists the LISTCAT codes for supported device types.

Table 14. Device Type Translate Table

Generic Name	LISTCAT Code	Device Type
3380	3010 200E	3380, all models
3390	3010 200F	3390, all models
9345	3010 2004	9345, all models
3400-2	30C0 8003	3420 Models 3, 5, and 7
3400-5	3200 8003	3420 Models 4, 6, and 8 (9 track, 6250 BPI)
3400-6	3210 8003	3420 Models 4, 6, and 8 (9 track, 1600/6250 BPI)
3400-9	3300 8003	3420C (3480 coexistence mode)
3400-3	3400 8003	3430, 9 track, 1600/6250 BPI tape
3480	7800 8080	3480 Magnetic Tape Unit and 3490 Magnetic Tape Subsystem Models A01, A02, B02, B04, D31, and D32
3480X	7804 8080	3480 Magnetic Tape Unit with IDRC enabled and 3490 Magnetic Tape Subsystem Models A01, A02, B02, B04, D31, and D32 with IDRC enabled

Table 14. Device Type Translate Table (continued)

Generic Name	LISTCAT Code	Device Type
3490	7804 8081	3490 Magnetic Tape Subsystem Enhanced Capability Models A10, A20, B20, B40, D41, and D42
3590-1	7804 8083	IBM 3590 High Performance Tape Subsystem Models A00, 3591 A01, A14, B11, B1A, C12

Examples of LISTCAT Output Listings

This section illustrates the kind of output you can get when you specify LISTCAT parameters. It also describes the job control language you can specify and the output messages you get when the LISTCAT procedure runs successfully.

Job Control Language (JCL) for LISTCAT Jobs

The job control language (JCL) statements that can be used to list a catalog's entries are:

```
//LISTCAT  JOB   ...
//STEP1    EXEC  PGM=IDCAMS
//OUTDD    DD    DSN=LISTCAT.OUTPUT,UNIT=3480,
//         VOL=SER=TAPE10,LABEL=(1,NL),DISP=(NEW,KEEP),
//         DCB=(RECFM=VBA,LRECL=125,BLKSIZE=629)
//SYSPRINT DD    SYSOUT=A
//SYSIN    DD    *
     LISTCAT -
          CATALOG(YOURCAT) -
          OUTFILE(OUTDD) -
          ...
/*
```

Note: Additional keywords can be included.

The JOB statement contains user and accounting information required for your installation.

The EXEC statement identifies the program to be run, IDCAMS (that is, the access method services program).

- OUTDD, which specifies an alternate output file, so that the LISTCAT output can be written onto an auxiliary storage device. The LISTCAT command's OUTFILE parameter points to the OUTDD DD statement. Only the LISTCAT output is written to the alternate output device. JCL statements, system messages, and job statistics are written to the SYSPRINT output device.
  - DSN=LISTCAT.OUTPUT specifies the name for the magnetic tape file.
  - UNIT=3480 and VOL=SER=TAPE10 specifies that the file is to be contained on magnetic tape volume TAPE10.
  - LABEL=(1,NL) specifies that this is the first file on a nonlabeled tape. You can also use a standard labeled tape by specifying LABEL=(1,SL). If subsequent job steps produce additional files of LISTCAT output on the same tape volume, you should increase the file number in each job step's LABEL subparameter (that is, LABEL=(2,NL) for the second job step, LABEL=(3,NL) for the third job step, etc.)
  - DISP=(NEW,KEEP) specifies that this is a new tape file and is to be rewound when the job finishes. If a subsequent job step prints the tape, DISP=(NEW,PASS) should be specified. If your job step contains more than one LISTCAT command, use DISP=(MOD,KEEP) or DISP=(MOD,PASS) to concatenate all of the LISTCAT output in one sequential file.
  - DCB=(RECFM=VBA,LRECL=125,BLKSIZE=629) specifies that the LISTCAT output records are variable-length, blocked 5-to-1, and are preceded by an ANSI print control character.
- SYSPRINT DD, which is required for each access method services job step. It identifies the output queue, SYSOUT=A, on which all LISTCAT output and system output messages are printed (unless the OUTFILE parameter and its associated DD statement is specified—see OUTDD above).

Note: If you want *all* output to be written to an auxiliary storage device, replace X'OUTDD' with X'SYSPRINT' in the OUTDD DD statement and omit the SYSPRINT DD SYSOUT=A statement.

- SYSIN DD, which specifies, with an asterisk (*), that the statements that follow are the input data statements. A '/*' ends the input data statements.

The LISTCAT command parameters shown in the preceding example are common to the LISTCAT examples that follow. Other LISTCAT parameters are coded with each example and the output that results is illustrated. These two parameters are optional:

- CATALOG, which identifies YOURCAT as the catalog whose entries are to be listed.
- OUTFILE, which points to the OUTDD DD statement. The OUTDD DD statement allocates an alternate output file for the LISTCAT output.

If you want to print the LISTCAT output that is contained on an alternate output file, you can use the IEBGENER program. The following shows the JCL required to print the alternate output file, LISTCAT.OUTPUT, that was allocated previously:

```
//PRINTOUT  JOB   ...
//STEP1     EXEC  PGM=IEBGENER
//SYSUT1    DD    DSN=LISTCAT.OUTPUT,UNIT=2400-3,
//          VOL=SER=TAPE10,LABEL=(1,NL),DISP=(OLD,KEEP),
//          DCB=(RECFM=VBA,LRECL=125,BLKSIZE=629)
//SYSUT2    DD    SYSOUT=A
//SYSPRINT  DD    SYSOUT=A
//SYSIN     DD    DUMMY
/*
```

Note: If you have the DFSORT product installed, consider using ICEGENER as an alternative to IEBGENER when making an unedited copy of a data set or member. It is usually faster than IEBGENER. It might already be installed on your system using the name IEBGENER.

LISTCAT and Access Method Services Output Messages

When the LISTCAT job completes, access method services provides messages and diagnostic information. If an error occurred, an analysis of the error message can be found in *z/OS TSO/E User's Guide*. When your LISTCAT job completes successfully, access method services provides messages that follow the entry listing (see Figure 7 on page 387):

```
                                          LISTING FROM CATALOG -- ICFUCAT1

        THE NUMBER OF ENTRIES PROCESSED WAS:
                  AIX ------------------1
                  ALIAS ----------------1
                  CLUSTER --------------4
                  DATA -----------------5
                  GDG ------------------1
                  INDEX ----------------4
                  NONVSAM --------------9
                  PAGESPACE ------------0
                  PATH -----------------2
                  SPACE ----------------0
                  USERCATALOG ----------0
                  TOTAL ----------------27

        THE NUMBER OF PROTECTED ENTRIES SUPPRESSED WAS 0

IDC0001I FUNCTION COMPLETED, HIGHEST CONDITION CODE WAS 0

IDC0002I IDCAMS PROCESSING COMPLETE, MAXIMUM CONDITION CODE WAS 0
```

Figure 7. Messages That Follow the Entry Listing

The first line identifies the catalog that contains the listed entries. The next group of lines specify the number of each entry type, and the total number of entries, that were listed. This statistical information can help you determine the approximate size, in records, of your catalog. The next line specifies the number of entries that could not be listed because the appropriate password was not specified. The last two messages indicate that the LISTCAT command (FUNCTION) and the job step (IDCAMS) completed successfully. When LISTCAT is invoked from a TSO terminal, IDC0001I is not printed.

LISTCAT Output Listing

If you use LISTCAT with no parameters, the entryname and type of each entry are listed (see Figure 8 on page 388). You can use this type of listing to list the name of each cataloged object and to determine the number of entries in the catalog. The total number of entries is an approximate size, in records, of your catalog.

```
  LISTCAT -
         CATALOG(ICFUCAT1)          /* IN CATALOG ICFUCAT1       */

                              LISTING FROM CATALOG -- ICFUCAT1
CLUSTER ------- 00000000000000000000000000000000000000000000000000
   DATA ------- ICFUCAT1
   INDEX ------ ICFUCAT1.CATINDEX
CLUSTER ------- SYS1.VVDS.V1P0101
   DATA ------- SYS1.VVDS.V1P0101
CLUSTER ------- SYS1.VVDS.V1P0201
   DATA ------- SYS1.VVDS.V1P0201
CLUSTER ------- SYS1.VVDS.V1P0202
   DATA ------- SYS1.VVDS.V1P0202
CLUSTER ------- SYS1.VVDS.V1P0301
   DATA ------- SYS1.VVDS.V1P0301
CLUSTER ------- SYS1.VVDS.V1P0302
   DATA ------- SYS1.VVDS.V1P0302
CLUSTER ------- SYS1.VVDS.V338001
   DATA ------- SYS1.VVDS.V338001
ALIAS --------- USER.ALIAS
CLUSTER ------- USER.DUMMY
   DATA ------- USER.DUMMY.CLDATA
   INDEX ------ USER.DUMMY.CLINDEX
GDG BASE ------ USER.GDGBASE
   NONVSAM ---- USER.GDGBASE.G0003V00
   NONVSAM ---- USER.GDGBASE.G0004V00
   NONVSAM ---- USER.GDGBASE.G0005V00
   NONVSAM ---- USER.GDGBASE.G0006V00
NONVSAM ------- USER.GDGBASE.G0001V00
NONVSAM ------- USER.GDGBASE.G0002V00
AIX ----------- USER.KSDS1.AIX1CLUS
   DATA ------- USER.KSDS1.AIX1DATA
   INDEX ------ USER.KSDS1.AIX1INDX
   PATH ------- USER.KSDS1.PATHAIX1
CLUSTER ------- USER.KSDS1.CLUSTER
   DATA ------- USER.KSDS1.CLDATA
   INDEX ------ USER.KSDS1.CLINDEX
   PATH ------- USER.KSDS1.PATHCL
CLUSTER ------- USER.LINEAR
   DATA ------- USER.LINEAR.DATA
NONVSAM ------- USER.MODEL
NONVSAM ------- USER.NONVSAM.DATA.SET
NONVSAM ------- USER.PDSE
CLUSTER ------- USER.SPANNED.CLUSTER
   DATA ------- USER.SPANNED.DATA
   INDEX ------ USER.SPANNED.INDEX

       THE NUMBER OF ENTRIES PROCESSED WAS:
                  AIX -------------------1
                  ALIAS -----------------1
                  CLUSTER --------------11
                  DATA ------------------12
                  GDG -------------------1
                  INDEX -----------------5
                  NONVSAM ---------------9
                  PAGESPACE -------------0
                  PATH ------------------2
                  SPACE -----------------0
                  USERCATALOG -----------0
                  TOTAL ----------------42
         THE NUMBER OF PROTECTED ENTRIES SUPPRESSED WAS 0
IDC0001I FUNCTION COMPLETED, HIGHEST CONDITION CODE WAS 0
IDC0002I IDCAMS PROCESSING COMPLETE. MAXIMUM CONDITION CODE WAS 0
```

Figure 8. Example of LISTCAT Output When No Parameters Are Specified

LISTCAT NAMES Output Listing

When you specify LISTCAT with the NAMES parameter, only the entryname and type of each entry are listed (see Figure 9 on page 389). In this example, LEVEL(USER) limits the data sets that are listed by name to those with a high-level qualifier of USER.

```
  LISTCAT -
         LEVEL(USER)                /* LIST ALL 'USER' ENTRIES   */ -
         NAME                       /* NAME INFORMATION ONLY     */ -
         CATALOG(ICFUCAT1)          /* IN CATALOG ICFUCAT1       */

                            LISTING FROM CATALOG -- ICFUCAT1
ALIAS --------- USER.ALIAS
CLUSTER ------- USER.DUMMY
DATA ---------- USER.DUMMY.CLDATA
INDEX --------- USER.DUMMY.CLINDEX
GDG BASE ------ USER.GDGBASE
NONVSAM ------- USER.GDGBASE.G0003V00
NONVSAM ------- USER.GDGBASE.G0004V00
NONVSAM ------- USER.GDGBASE.G0005V00
NONVSAM ------- USER.GDGBASE.G0006V00
NONVSAM ------- USER.GDGBASE.G0001V00
NONVSAM ------- USER.GDGBASE.G0002V00
AIX ----------- USER.KSDS1.AIX1CLUS
DATA ---------- USER.KSDS1.AIX1DATA
INDEX --------- USER.KSDS1.AIX1INDX
DATA ---------- USER.KSDS1.CLDATA
INDEX --------- USER.KSDS1.CLINDEX
CLUSTER ------- USER.KSDS1.CLUSTER
PATH ---------- USER.KSDS1.PATHAIX1
PATH ---------- USER.KSDS1.PATHCL
CLUSTER ------- USER.LINEAR
DATA ---------- USER.LINEAR.DATA
NONVSAM ------- USER.MODEL
NONVSAM ------- USER.NONVSAM.DATA.SET
NONVSAM ------- USER.PDSE
CLUSTER ------- USER.SPANNED.CLUSTER
DATA ---------- USER.SPANNED.DATA
INDEX --------- USER.SPANNED.INDEX
         THE NUMBER OF ENTRIES PROCESSED WAS:
                   AIX ------------------1
                   ALIAS ----------------1
                   CLUSTER --------------4
                   DATA -----------------5
                   GDG ------------------1
                   INDEX ----------------4
                   NONVSAM --------------9
                   PAGESPACE ------------0
                   PATH -----------------2
                   SPACE ----------------0
                   USERCATALOG ----------0
                   TOTAL ----------------27
         THE NUMBER OF PROTECTED ENTRIES SUPPRESSED WAS 0
IDC0001I FUNCTION COMPLETED, HIGHEST CONDITION CODE WAS 0
IDC0002I IDCAMS PROCESSING COMPLETE. MAXIMUM CONDITION CODE WAS 0
```

Figure 9. Example of LISTCAT NAME Output

LISTCAT VOLUME Output Listing

When the LISTCAT command is specified with the VOLUME parameter, the volume serial number and device type of each volume that contains part or all of the cataloged object are listed (see Figure 10 on page 390).

```
  LISTCAT -
         LEVEL(USER)                /* LIST ALL 'USER' ENTRIES   */ -
         VOLUME                     /* VOLUME INFORMATION        */ -
         CLUSTER                    /* INCLUDE CLUSTERS          */ -
         DATA                       /*   AND DATA COMPONENTS     */ -
         INDEX                      /*   AND INDEX COMPONENTS    */ -
         ALTERNATEINDEX             /*   AND ALTERNATEINDEXES    */ -
         PATH                       /*   AND PATHS               */ -
         GENERATIONDATAGROUP        /*   AND GDG BASES           */ -
         NONVSAM                    /*   AND NONVSAM DATA SETS   */ -
         CATALOG(ICFUCAT1)          /* IN CATALOG ICFUCAT1       */

                             LISTING FROM CATALOG -- ICFUCAT1
CLUSTER ------- USER.DUMMY
     HISTORY
       DATASET-OWNER-----(NULL)     CREATION--------2004.345
       RELEASE----------------2     EXPIRATION------0000.000
     SMSDATA
       STORAGECLASS ---S1P02S02     MANAGEMENTCLASS---(NULL)
       DATACLASS --------(ABCD)     LBACKUP ---0000.000.0000
       CA RECLAIM --------(YES)
            BWO STATUS------11100000     BWO TIMESTAMP---0000 00:00:00.0
       BWO-------------  (NULL)
     RLSDATA
       LOG --------------(NULL)     RECOVERY REQUIRED --(NO)
       VSAM QUIESCED      (NO)      RLS IN USE ---------(NO)
       LOGSTREAMID -----------------------------------(NULL)
       RECOVERY TIMESTAMP LOCAL-----X'0000000000000000'
       RECOVERY TIMESTAMP GMT-------X'0000000000000000'
DATA ---------- USER.DUMMY.CLDATA
     HISTORY
       DATASET-OWNER-----(NULL)     CREATION--------2004.215
       RELEASE----------------2     EXPIRATION------0000.000
       ACCOUNT-INFO----------------------ALTER ACCOUNT INFO
     VOLUMES
       VOLSER------------1P0201     DEVTYPE------X'3010200E'
INDEX --------- USER.DUMMY.CLINDEX
     HISTORY
       DATASET-OWNER-----(NULL)     CREATION--------2004.221
       RELEASE----------------2     EXPIRATION------0000.000
     VOLUMES
       VOLSER------------1P0201     DEVTYPE------X'3010200E'
GDG BASE ------ USER.GDGBASE
     HISTORY
       DATASET-OWNER-----(NULL)     CREATION--------2003.323
       RELEASE----------------2     EXPIRATION------2005.001
       ACCOUNT-INFO-----------------------------------(NULL)
NONVSAM ------- USER.GDGBASE.G0003V00
     HISTORY
       DATASET-OWNER-----(NULL)     CREATION--------2003.323
       RELEASE----------------2     EXPIRATION------2005.001
       ACCOUNT-INFO-----------------------------------(NULL)
     SMSDATA
       STORAGECLASS ---S1P02S01     MANAGEMENTCLASS-S1P01M02
       DATACLASS ------PS000000     LBACKUP ---0000.000.0000
     VOLUMES
       VOLSER------------1P0202     DEVTYPE------X'3030200E'
```

Figure 10. Example of LISTCAT VOLUME Output (1 of 5)

```
NONVSAM ------- USER.GDGBASE.G0004V00
     HISTORY
       DATASET-OWNER-----(NULL)     CREATION--------2003.323
       RELEASE----------------2     EXPIRATION------2005.001
       ACCOUNT-INFO-----------------------------------(NULL)
     SMSDATA
       STORAGECLASS ---S1P02S01     MANAGEMENTCLASS-S1P01M02
       DATACLASS ------PS000000     LBACKUP ---0000.000.0000
     VOLUMES
       VOLSER------------1P0202     DEVTYPE------X'3030200E'
NONVSAM ------- USER.GDGBASE.G0005V00
     HISTORY
       DATASET-OWNER-----(NULL)     CREATION--------2003.323
       RELEASE----------------2     EXPIRATION------2005.001
       ACCOUNT-INFO-----------------------------------(NULL)
     SMSDATA
       STORAGECLASS ---S1P02S01     MANAGEMENTCLASS-S1P01M02
       DATACLASS ------PS000000     LBACKUP ---0000.000.0000
     VOLUMES
       VOLSER------------1P0202     DEVTYPE------X'3030200E'
NONVSAM ------- USER.GDGBASE.G0006V00
     HISTORY
       DATASET-OWNER-----(NULL)     CREATION--------2003.323
       RELEASE----------------2     EXPIRATION------2005.001
       ACCOUNT-INFO-----------------------------------(NULL)
     SMSDATA
       STORAGECLASS ---S1P02S01     MANAGEMENTCLASS-S1P01M02
       DATACLASS ------PS000000     LBACKUP ---0000.000.0000
     VOLUMES
       VOLSER------------1P0202     DEVTYPE------X'3030200E'
NONVSAM ------- USER.GDGBASE.G0001V00
     HISTORY
       DATASET-OWNER-----(NULL)     CREATION--------2003.323
       RELEASE----------------2     EXPIRATION------2005.001
       ACCOUNT-INFO-----------------------------------(NULL)
     SMSDATA
       STORAGECLASS ---S1P02S01     MANAGEMENTCLASS-S1P01M02
       DATACLASS ------PS000000     LBACKUP ---0000.000.0000
     VOLUMES
       VOLSER------------1P0202     DEVTYPE------X'3030200E'
NONVSAM ------- USER.GDGBASE.G0002V00
     HISTORY
       DATASET-OWNER-----(NULL)     CREATION--------2003.323
       RELEASE----------------2     EXPIRATION------2005.001
       ACCOUNT-INFO-----------------------------------(NULL)
     SMSDATA
       STORAGECLASS ---S1P02S01     MANAGEMENTCLASS-S1P01M02
       DATACLASS ------PS000000     LBACKUP ---0000.000.0000
     VOLUMES
       VOLSER------------1P0202     DEVTYPE------X'3030200E'
AIX ----------- USER.KSDS1.AIX1CLUS
     HISTORY
       DATASET-OWNER-----(NULL)     CREATION--------2003.323
       RELEASE----------------2     EXPIRATION------2005.254
       SMS MANAGED--------(YES)
```

Figure 11. Example of LISTCAT VOLUME Output (2 of 5)

```
DATA ---------- USER.KSDS1.AIX1DATA
     HISTORY
       DATASET-OWNER-----(NULL)     CREATION--------2003.323
       RELEASE----------------2     EXPIRATION------2005.254
     VOLUMES
       VOLSER------------1P0201     DEVTYPE------X'3010200E'
       VOLSER------------1P0202     DEVTYPE------X'3010200E'
INDEX --------- USER.KSDS1.AIX1INDX
     HISTORY
       DATASET-OWNER-----(NULL)     CREATION--------2003.323
       RELEASE----------------2     EXPIRATION------2005.254
     VOLUMES
       VOLSER------------1P0201     DEVTYPE------X'3010200E'
       VOLSER------------1P0202     DEVTYPE------X'3010200E'
DATA ---------- USER.KSDS1.CLDATA
     HISTORY
       DATASET-OWNER-----(NULL)     CREATION--------2003.323
       RELEASE----------------2     EXPIRATION------2005.254
     VOLUMES
       VOLSER------------1P0201     DEVTYPE------X'3010200E'
       VOLSER------------1P0202     DEVTYPE------X'3010200E'
INDEX --------- USER.KSDS1.CLINDEX
     HISTORY
       DATASET-OWNER-----(NULL)     CREATION--------2003.323
       RELEASE----------------2     EXPIRATION------2005.254
     VOLUMES
       VOLSER------------1P0201     DEVTYPE------X'3010200E'
       VOLSER------------1P0202     DEVTYPE------X'3010200E'
CLUSTER ------- USER.KSDS1.CLUSTER
     HISTORY
       DATASET-OWNER-----(NULL)     CREATION--------2003.323
       RELEASE----------------2     EXPIRATION------2005.254
     SMSDATA
       STORAGECLASS ---S1P02S02     MANAGEMENTCLASS---(NULL)
       DATACLASS --------(ABCD)     LBACKUP ---0000.000.0000
       CA RECLAIM --------(YES)
       BWO STATUS------11100000     BWO TIMESTAMP---0000 00:00:00.0
       BWO---------------(NULL)
     RLSDATA
       LOG --------------(NULL)     RECOVERY REQUIRED --(NO)
       VSAM QUIESCED      (NO)      RLS IN USE ---------(NO)
       LOGSTREAMID -----------------------------------(NULL)
       RECOVERY TIMESTAMP LOCAL-----X'0000000000000000'
       RECOVERY TIMESTAMP GMT-------X'0000000000000000'
PATH ---------- USER.KSDS1.PATHAIX1
     HISTORY
       DATASET-OWNER-----(NULL)     CREATION--------2003.323
       RELEASE----------------2     EXPIRATION------2005.254
PATH ---------- USER.KSDS1.PATHCL
     HISTORY
       DATASET-OWNER-----(NULL)     CREATION--------2003.323
       RELEASE----------------2     EXPIRATION------2005.254
```

Figure 12. Example of LISTCAT VOLUME Output (3 of 5)

```
CLUSTER ------- USER.LINEAR
       HISTORY
         DATASET-OWNER-----(NULL)     CREATION--------2003.323
         RELEASE----------------2     EXPIRATION------2005.254
       SMSDATA
         STORAGECLASS ---S1P02S02     MANAGEMENTCLASS---(NULL)
         DATACLASS --------(NULL)     LBACKUP ---0000.000.0000
         BWO STATUS------11100000     BWO TIMESTAMP---0000 00:00:00.0
         BWO---------------(NULL)
       RLSDATA
         LOG --------------(NULL)     RECOVERY REQUIRED --(NO)
         VSAM QUIESCED      (NO)      RLS IN USE ---------(NO)
         LOGSTREAMID -----------------------------------(NULL)
         RECOVERY TIMESTAMP LOCAL-----X'0000000000000000'
         RECOVERY TIMESTAMP GMT-------X'0000000000000000'
DATA ---------- USER.LINEAR.DATA
       HISTORY
         DATASET-OWNER-----(NULL)     CREATION--------2003.323
         RELEASE----------------2     EXPIRATION------9999.999
       VOLUMES
         VOLSER------------338001     DEVTYPE------X'3010200E'
NONVSAM ------- USER.MODEL
       HISTORY
         DATASET-OWNER-----(NULL)     CREATION--------2003.323
         RELEASE----------------2     EXPIRATION------0000.000
       VOLUMES
         VOLSER------------338001     DEVTYPE------X'3010200E'
NONVSAM ------- USER.NONVSAM.DATA.SET
       HISTORY
         DATASET-OWNER-----(NULL)     CREATION--------2003.323
         RELEASE----------------2     EXPIRATION------0000.000
       SMSDATA
         STORAGECLASS ---S1P01S02     MANAGEMENTCLASS-S1P01M02
         DATACLASS ------PS000000     LBACKUP ---0000.000.0000
       VOLUMES
         VOLSER------------1P0101     DEVTYPE------X'3030200E'
NONVSAM ------- USER.PDSE
       HISTORY
         DATASET-OWNER-----(NULL)     CREATION--------2003.323
         RELEASE----------------2     EXPIRATION------2005.244
       SMSDATA
         STORAGECLASS ---S1P03S01     MANAGEMENTCLASS---(NULL)
         DATACLASS --------(NULL)     LBACKUP ---0000.000.0000
       VOLUMES
         VOLSER------------1P0302     DEVTYPE------X'3030200E'
CLUSTER ------- USER.SPANNED.CLUSTER
       HISTORY
         DATASET-OWNER-----(NULL)     CREATION--------2004.345
         RELEASE----------------2     EXPIRATION------0000.000
       SMSDATA
         STORAGECLASS ---S1P02S02     MANAGEMENTCLASS---(NULL)
         DATACLASS --------(NULL)     LBACKUP ---0000.000.0000
         CA RECLAIM --------(YES)
         BWO STATUS------11100000     BWO TIMESTAMP---0000 00:00:00.0
         BWO-------------  (NULL)
       RLSDATA
         LOG --------------(NULL)     RECOVERY REQUIRED --(NO)
         VSAM QUIESCED      (NO)      RLS IN USE ---------(NO)
         LOGSTREAMID -----------------------------------(NULL)
         RECOVERY TIMESTAMP LOCAL-----X'0000000000000000'
         RECOVERY TIMESTAMP GMT-------X'0000000000000000'
```

Figure 13. Example of LISTCAT VOLUME Output (4 of 5)

```
DATA ---------- USER.SPANNED.DATA
     HISTORY
       DATASET-OWNER-----(NULL)     CREATION--------2003.323
       RELEASE----------------2     EXPIRATION------2007.365
     VOLUMES
       VOLSER------------1P0301     DEVTYPE------X'3010200E'
       VOLSER------------1P0302     DEVTYPE------X'3010200E'
INDEX --------- USER.SPANNED.INDEX
     HISTORY
       DATASET-OWNER-----(NULL)     CREATION--------2003.323
       RELEASE----------------2     EXPIRATION------2007.365
     VOLUMES
       VOLSER------------1P0301     DEVTYPE------X'3010200E'
       VOLSER-----------------*     DEVTYPE------X'3010200E'
       VOLSER------------1P0301     DEVTYPE------X'3010200E'
       VOLSER------------1P0302     DEVTYPE------X'3010200E'
         THE NUMBER OF ENTRIES PROCESSED WAS:
                   AIX -------------------1
                   ALIAS -----------------0
                   CLUSTER ---------------4
                   DATA ------------------5
                   GDG -------------------1
                   INDEX -----------------4
                   NONVSAM ---------------9
                   PAGESPACE -------------0
                   PATH ------------------2
                   SPACE -----------------0
                   USERCATALOG -----------0
                   TOTAL ----------------26
         THE NUMBER OF PROTECTED ENTRIES SUPPRESSED WAS 0
IDC0001I FUNCTION COMPLETED, HIGHEST CONDITION CODE WAS 0
IDC0002I IDCAMS PROCESSING COMPLETE. MAXIMUM CONDITION CODE WAS 0
```

Figure 14. Example of LISTCAT VOLUME Output (5 of 5)

LISTCAT ALL Output Listing

When you specify the LISTCAT command and include the ALL parameter, all the information for each catalog entry is listed (see Figure 15 on page 395). This example illustrates the LISTCAT output for each type of catalog entry. You can use this type of listing to obtain all cataloged information (except password and security information) about each entry that is listed.

Note: When ENTRIES is specified, you specify only those entry names that identify catalog entries which are not volume entries. If a volume serial number is specified with the ENTRIES parameter, then entry names of other entry types cannot also be specified. However, if the ENTRIES parameter is not specified and if entry types are not specified (that is, CLUSTER, SPACE, DATA, etc.), all entries in the catalog, including volume entries, are listed.

Note: When Encryption value is NO, there is no line 'DATA SET KEY LABEL ---' displayed. for example,

```
       DATA SET ENCRYPTION-----YES
       DATA SET KEY LABEL---key-label
```

and

```
       DATA SET ENCRYPTION-----NO
```

```
  LISTCAT -
          LEVEL(USER)                /* LIST ALL 'USER' ENTRIES   */ -
          ALL                        /* SHOW ALL INFORMATION      */ -
          CATALOG(ICFUCAT1)          /* IN CATALOG ICFUCAT1       */

                              LISTING FROM CATALOG -- ICFUCAT1
ALIAS --------- USER.ALIAS
     HISTORY
       RELEASE----------------2
     ASSOCIATIONS
       NONVSAM--USER.NONVSAM.DATA.SET
CLUSTER ------- USER.DUMMY
     HISTORY
       DATASET-OWNER-----(NULL)     CREATION--------2003.323
       RELEASE----------------2     EXPIRATION------0000.000
     SMSDATA
       STORAGECLASS ---S1P02S02     MANAGEMENTCLASS---(NULL)
       DATACLASS --------(NULL)     LBACKUP ---0000.000.0000
       CA RECLAIM---------(YES)
       BWO STATUS------00000000     BWO TIMESTAMP---00000 00:00:00:0
       BWO                (NULL)
       BWO-------------TYPECICS
     RLSDATA
       LOG -------------(NULL)     RECOVERY REQUIRED --(NO)
       VSAM QUIESCED      (NO)     RLS IN USE ---------(NO)
       LOGSTREAMID -----------------------------------(NULL)
       RECOVERY TIMESTAMP LOCAL-----X'00000000000000000000'
       RECOVERY TIMESTAMP GMT-------X'00000000000000000000'
     ENCRYPTIONDATA
       DATA SET ENCRYPTION-----(YES/NO)
       DATA SET KEY LABEL---(NULL/key-label)
     PROTECTION-PSWD-----(NULL)     RACF----------------(NO)
     ASSOCIATIONS
       DATA-----USER.DUMMY.CLDATA
       INDEX----USER.DUMMY.CLINDEX
DATA ---------- USER.DUMMY.CLDATA
     HISTORY
       DATASET-OWNER-----(NULL)     CREATION--------2003.323
       RELEASE----------------2     EXPIRATION------0000.000
       ACCOUNT-INFO-----------------------------------(NULL)
     PROTECTION-PSWD-----(NULL)     RACF----------------(NO)
     ASSOCIATIONS
       CLUSTER--USER.DUMMY
     ATTRIBUTES
       KEYLEN-----------------4     AVGLRECL------------2000     BUFSPACE------------6656     CISIZE--------------2048
       RKP--------------------0     MAXLRECL------------2000     EXCPEXIT----------(NULL)     CI/CA---------------270
       AXRKP------------------0
       VERSION-NUMBER---------1
       STRIPE-COUNT------(NULL)
       ACT-DIC-TOKEN----X'FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF'
       SHROPTNS(1,3)   RECOVERY     UNIQUE           NOERASE     INDEXED       NOWRITECHK
       UNORDERED        NOREUSE     NONSPANNED    NONUNIQKEY     TEXT
       CCSID-----------------37     CECP EBCDIC
     STATISTICS
       REC-TOTAL--------------0     SPLITS-CI--------------0     EXCPS------------------0
       REC-DELETED------------0     SPLITS-CA--------------0     EXTENTS----------------1
       REC-INSERTED-----------0     FREESPACE-%CI----------0     SYSTEM-TIMESTAMP:
       REC-UPDATED------------0     FREESPACE-%CA----------0          X'0000000000000000'
       REC-RETRIEVED----------0     FREESPC----------9400320
```

Figure 15. Example of LISTCAT ALL Output (1 of 12)

```
     ALLOCATION
       SPACE-TYPE------CYLINDER     HI-A-RBA---------9400320
       SPACE-PRI-------------17     HI-U-RBA--------------0
       SPACE-SEC--------------0
     VOLUME
       VOLSER------------1P0201     PHYREC-SIZE---------2048     HI-A-RBA---------400320      EXTENT-NUMBER----------1
       DEVTYPE------X'3010200E'     PHYRECS/TRK-----------18     HI-U-RBA-------------0       EXTENT-TYPE--------X'40'
       VOLFLAG------------PRIME     TRACKS/CA-------------15
       EXTENTS:
       LOW-CCHH-----X'00020000'     LOW-RBA---------------0     TRACKS---------------255
       HIGH-CCHH----X'0012000E'     HIGH-RBA---------9400319
INDEX --------- USER.DUMMY.CLINDEX
     HISTORY
       DATASET-OWNER-----(NULL)     CREATION--------2003.323
       RELEASE----------------2     EXPIRATION------0000.000
     PROTECTION-PSWD-----(NULL)     RACF----------------(NO)
     ASSOCIATIONS
       CLUSTER--USER.DUMMY
     ATTRIBUTES
       KEYLEN-----------------4     AVGLRECL---------------0     BUFSPACE---------------0     CISIZE--------------2560
       RKP--------------------0     MAXLRECL------------2553     EXCPEXIT----------(NULL)     CI/CA-----------------15
       SHROPTNS(1,3)   RECOVERY     UNIQUE           NOERASE     NOWRITECHK       UNORDERED
       NOREUSE
     STATISTICS
       REC-TOTAL--------------0     SPLITS-CI--------------0     EXCPS------------------0     INDEX:
       REC-DELETED------------0     SPLITS-CA--------------0     EXTENTS----------------1     LEVELS----------------0
       REC-INSERTED-----------0     FREESPACE-%CI----------0     SYSTEM-TIMESTAMP:            ENTRIES/SECT----------16
       REC-UPDATED------------0     FREESPACE-%CA----------0          X'0000000000000000'     SEQ-SET-RBA-----------0
       REC-RETRIEVED----------0     FREESPC-----------76800                                  HI-LEVEL-RBA----------0
     ALLOCATION
       SPACE-TYPE---------TRACK     HI-A-RBA----------76800
       SPACE-PRI--------------2     HI-U-RBA--------------0
       SPACE-SEC--------------0
     VOLUME
       VOLSER------------1P0201     PHYREC-SIZE---------2560     HI-A-RBA----------76800     EXTENT-NUMBER----------1
       DEVTYPE------X'3010200E'     PHYRECS/TRK-----------15     HI-U-RBA--------------0     EXTENT-TYPE--------X'40'
       VOLFLAG------------PRIME     TRACKS/CA-------------1
       EXTENTS:
       LOW-CCHH-----X'0000000B'     LOW-RBA---------------0     TRACKS-----------------2
       HIGH-CCHH----X'0000000C'     HIGH-RBA----------76799
GDG BASE ------ USER.GDGBASE
     IN-CAT --- SYS1.MVSRES9.MASTCAT
     HISTORY
       DATASET-OWNER-----(NULL)     CREATION--------2014.071
       RELEASE----------------2     LAST ALTER------0000.000
     ATTRIBUTES
       LIMIT------------------4     NOSCRATCH  NOEMPTY    LIFO       NOPURGE    NOEXTENDED
     ASSOCIATIONS
       NONVSAM--USER.GDGBASE.G0003V00
       NONVSAM--USER.GDGBASE.G0004V00
       NONVSAM--USER.GDGBASE.G0005V00
       NONVSAM--USER.GDGBASE.G0006V00
NONVSAM ------- USER.GDGBASE.G0003V00
     HISTORY
       DATASET-OWNER-----(NULL)     CREATION--------2003.323
       RELEASE----------------2     EXPIRATION------2005.001
       STATUS------------ACTIVE
     SMSDATA
       STORAGECLASS ---S1P02S01     MANAGEMENTCLASS-S1P01M02
       DATACLASS ------PS000000     LBACKUP ---0000.000.0000
     VOLUMES
       VOLSER------------1P0202     DEVTYPE------X'3030200E'     FSEQN-----------------0
     ASSOCIATIONS
       GDG------USER.GDGBASE
```

Figure 16. Example of LISTCAT ALL Output (2 of 12)

```
NONVSAM ------- USER.GDGBASE.G0004V00
     HISTORY
       DATASET-OWNER-----(NULL)     CREATION--------2003.323
       RELEASE----------------2     EXPIRATION------2005.001
       STATUS------------ACTIVE
     SMSDATA
       STORAGECLASS ---S1P02S01     MANAGEMENTCLASS-S1P01M02
       DATACLASS ------PS000000     LBACKUP ---0000.000.0000
     VOLUMES
       VOLSER------------1P0202     DEVTYPE------X'3030200E'     FSEQN-----------------0
     ASSOCIATIONS
       GDG------USER.GDGBASE
NONVSAM ------- USER.GDGBASE.G0005V00
     HISTORY
       DATASET-OWNER-----(NULL)     CREATION--------2003.323
       RELEASE----------------2     EXPIRATION------2005.001
       STATUS------------ACTIVE
     SMSDATA
       STORAGECLASS ---S1P02S01     MANAGEMENTCLASS-S1P01M02
       DATACLASS ------PS000000     LBACKUP ---0000.000.0000
     VOLUMES
       VOLSER------------1P0202     DEVTYPE------X'3030200E'     FSEQN-----------------0
     ASSOCIATIONS
       GDG------USER.GDGBASE
NONVSAM ------- USER.GDGBASE.G0006V00
     HISTORY
       DATASET-OWNER-----(NULL)     CREATION--------2003.323
       RELEASE----------------2     EXPIRATION------2005.001
       STATUS------------ACTIVE
     SMSDATA
       STORAGECLASS ---S1P02S01     MANAGEMENTCLASS-S1P01M02
       DATACLASS ------PS000000     LBACKUP ---0000.000.0000
     VOLUMES
       VOLSER------------1P0202     DEVTYPE------X'3030200E'     FSEQN-----------------0
     ASSOCIATIONS
       GDG------USER.GDGBASE
NONVSAM ------- USER.GDGBASE.G0001V00
     HISTORY
       DATASET-OWNER-----(NULL)     CREATION--------2003.323
       RELEASE----------------2     EXPIRATION------2005.001
       STATUS--------ROLLED-OFF
     SMSDATA
       STORAGECLASS ---S1P02S01     MANAGEMENTCLASS-S1P01M02
       DATACLASS ------PS000000     LBACKUP ---0000.000.0000
     VOLUMES
       VOLSER------------1P0202     DEVTYPE------X'3030200E'     FSEQN-----------------0
     ASSOCIATIONS--------(NULL)
NONVSAM ------- USER.GDGBASE.G0002V00
     HISTORY
       DATASET-OWNER-----(NULL)     CREATION--------2003.323
       RELEASE----------------2     EXPIRATION------2005.001
       STATUS----------DEFERRED
     SMSDATA
       STORAGECLASS ---S1P02S01     MANAGEMENTCLASS-S1P01M02
       DATACLASS ------PS000000     LBACKUP ---0000.000.0000
     VOLUMES
       VOLSER------------1P0202     DEVTYPE------X'3030200E'     FSEQN-----------------0
     ASSOCIATIONS--------(NULL)
```

Figure 17. Example of LISTCAT ALL Output (3 of 12)

```
AIX ----------- USER.KSDS1.AIX1CLUS
     HISTORY
       DATASET-OWNER-----(NULL)     CREATION--------2003.323
       RELEASE----------------2     EXPIRATION------2005.254
       SMS MANAGED--------(YES)
     PROTECTION-PSWD-----(NULL)     RACF----------------(NO)
     ASSOCIATIONS
       DATA-----USER.KSDS1.AIX1DATA
       INDEX----USER.KSDS1.AIX1INDX
       CLUSTER--USER.KSDS1.CLUSTER
       PATH-----USER.KSDS1.PATHAIX1
     ATTRIBUTES
       UPGRADE
DATA ---------- USER.KSDS1.AIX1DATA
     HISTORY
       DATASET-OWNER-----(NULL)     CREATION--------2003.323
       RELEASE----------------2     EXPIRATION------2005.254
     PROTECTION-PSWD-----(NULL)     RACF----------------(NO)
     ASSOCIATIONS
       AIX------USER.KSDS1.AIX1CLUS
     ATTRIBUTES
       KEYLEN-----------------4     AVGLRECL------------4086     BUFSPACE-----------29184     CISIZE-------------14336
       RKP--------------------5     MAXLRECL-----------32600     EXCPEXIT----------(NULL)     CI/CA------------------3
       VERSION-NUMBER---------1
       STRIPE-COUNT-----------1
       SHROPTNS(1,3)   RECOVERY     UNIQUE           NOERASE     INDEXED       NOWRITECHK
       UNORDERED        NOREUSE     NONSPANNED      EXTENDED
       CCSID-------------65535
     STATISTICS
       REC-TOTAL------------180     SPLITS-CI--------------0     EXCPS----------------373
       REC-DELETED------------0     SPLITS-CA--------------0     EXTENTS----------------1
       REC-INSERTED----------59     FREESPACE-%CI---------20     SYSTEM-TIMESTAMP:
       REC-UPDATED------------0     FREESPACE-%CA---------20          X'A0D8F717D7463101'
       REC-RETRIEVED--------479     FREESPC------------28672
     ALLOCATION
       SPACE-TYPE---------TRACK     HI-A-RBA-----------86016
       SPACE-PRI--------------1     HI-U-RBA-----------43008
       SPACE-SEC--------------1
     VOLUME
       VOLSER------------1P0201     PHYREC-SIZE--------14336     HI-A-RBA-----------43008     EXTENT-NUMBER----------1
       DEVTYPE------X'3010200E'     PHYRECS/TRK------------3     HI-U-RBA--------------0     EXTENT-TYPE--------X'40'
       VOLFLAG------------PRIME     TRACKS/CA--------------1
       EXTENTS:
       LOW-CCHH-----X'0000000E'     LOW-RBA----------------0     TRACKS-----------------1
       HIGH-CCHH----X'0000000E'     HIGH-RBA-----------43007
     VOLUME
       VOLSER------------1P0202     PHYREC-SIZE--------14336     HI-A-RBA-----------86016     EXTENT-NUMBER----------1
       DEVTYPE------X'3010200E'     PHYRECS/TRK------------3     HI-U-RBA-----------43008     EXTENT-TYPE--------X'00'
       VOLFLAG------------PRIME     TRACKS/CA--------------1
       EXTENTS:
       LOW-CCHH-----X'00020007'     LOW-RBA------------43008     TRACKS-----------------1
       HIGH-CCHH----X'00020007'     HIGH-RBA-----------86015
```

Figure 18. Example of LISTCAT ALL Output (4 of 12)

```
INDEX --------- USER.KSDS1.AIX1INDX
     HISTORY
       DATASET-OWNER-----(NULL)     CREATION--------2003.323
       RELEASE----------------2     EXPIRATION------2005.254
     PROTECTION-PSWD-----(NULL)     RACF----------------(NO)
     ASSOCIATIONS
       AIX------USER.KSDS1.AIX1CLUS
     ATTRIBUTES
       KEYLEN-----------------4     AVGLRECL---------------0     BUFSPACE---------------0     CISIZE----------------512
       RKP--------------------5     MAXLRECL-------------505     EXCPEXIT----------(NULL)     CI/CA-----------------46
       SHROPTNS(2,3)   RECOVERY     UNIQUE           NOERASE     INDEXED       NOWRITECHK     UNORDERED
       UNORDERED        NOREUSE     NONSPANNED
     STATISTICS
       REC-TOTAL--------------1     SPLITS-CI--------------0     EXCPS-----------------15     INDEX:
       REC-DELETED------------0     SPLITS-CA--------------0     EXTENTS----------------1     LEVELS----------------1
       REC-INSERTED-----------0     FREESPACE-%CI----------0     SYSTEM-TIMESTAMP:            ENTRIES/SECT----------1
       REC-UPDATED------------0     FREESPACE-%CA----------0          X'A0D8F717D7463101'     SEQ-SET-RBA--------23552
       REC-RETRIEVED----------0     FREESPC------------23040                                 HI-LEVEL-RBA-------23552
     ALLOCATION
       SPACE-TYPE---------TRACK     HI-A-RBA-----------47104
       SPACE-PRI--------------1     HI-U-RBA-----------24064
       SPACE-SEC--------------1
     VOLUME
       VOLSER------------1P0201     PHYREC-SIZE----------512     HI-A-RBA-----------23552     EXTENT-NUMBER----------1
       DEVTYPE------X'3010200E'     PHYRECS/TRK-----------46     HI-U-RBA---------------0     EXTENT-TYPE--------X'40'
       VOLFLAG------------PRIME     TRACKS/CA--------------1
       EXTENTS:
       LOW-CCHH-----X'00010000'     LOW-RBA----------------0     TRACKS-----------------1
       HIGH-CCHH----X'00010000'     HIGH-RBA-----------23551
     VOLUME
       VOLSER------------1P0202     PHYREC-SIZE----------512     HI-A-RBA-----------47104     EXTENT-NUMBER----------1
       DEVTYPE------X'3010200E'     PHYRECS/TRK-----------46     HI-U-RBA-----------24064     EXTENT-TYPE--------X'00'
       VOLFLAG------------PRIME     TRACKS/CA--------------1
       EXTENTS:
       LOW-CCHH-----X'00020008'     LOW-RBA------------23552     TRACKS-----------------1
       HIGH-CCHH----X'00020008'     HIGH-RBA-----------47103
DATA ---------- USER.KSDS1.CLDATA
     HISTORY
       DATASET-OWNER-----(NULL)     CREATION--------2003.323
       RELEASE----------------2     EXPIRATION------2005.254
     PROTECTION-PSWD-----(NULL)     RACF----------------(NO)
     ASSOCIATIONS
       CLUSTER--USER.KSDS1.CLUSTER
     ATTRIBUTES
       KEYLEN-----------------4     AVGLRECL------------2000     BUFSPACE------------6144     CISIZE--------------2048
       RKP--------------------0     MAXLRECL------------2000     EXCPEXIT----------(NULL)     CI/CA-----------------90
       SHROPTNS(1,3)   RECOVERY     UNIQUE           NOERASE     INDEXED       NOWRITECHK
       UNORDERED        NOREUSE     NONSPANNED
```

Figure 19. Example of LISTCAT ALL Output (5 of 12)

```
     STATISTICS
       REC-TOTAL------------180     SPLITS-CI--------------0     EXCPS---------------1524
       REC-DELETED------------0     SPLITS-CA--------------1     EXTENTS---------------2
       REC-INSERTED----------59     FREESPACE-%CI---------20     SYSTEM-TIMESTAMP:
       REC-UPDATED----------179     FREESPACE-%CA---------20          X'A0D8F6E3D70D9401'
       REC-RETRIEVED-------1051     FREESPC----------------0
     ALLOCATION
       SPACE-TYPE---------TRACK     HI-A-RBA----------552960
       SPACE-PRI--------------5     HI-U-RBA----------552960
       SPACE-SEC--------------5
     VOLUME
       VOLSER------------1P0201     PHYREC-SIZE---------2048     HI-A-RBA----------184320     EXTENT-NUMBER----------1
       DEVTYPE------X'3010200E'     PHYRECS/TRK-----------18     HI-U-RBA--------------0     EXTENT-TYPE--------X'40'
       VOLFLAG------------PRIME     TRACKS/CA--------------5
       EXTENTS:
       LOW-CCHH-----X'0013000A'     LOW-RBA----------------0     TRACKS-----------------5
       HIGH-CCHH----X'0013000E'     HIGH-RBA----------184319
     VOLUME
       VOLSER------------1P0202     PHYREC-SIZE---------2048     HI-A-RBA----------552960     EXTENT-NUMBER----------2
       DEVTYPE------X'3010200E'     PHYRECS/TRK-----------18     HI-U-RBA----------552960     EXTENT-TYPE--------X'00'
       VOLFLAG------------PRIME     TRACKS/CA--------------5
       EXTENTS:
       LOW-CCHH-----X'0000000B'     LOW-RBA-----------184320     TRACKS-----------------5
       HIGH-CCHH----X'00010000'     HIGH-RBA----------368639
       LOW-CCHH-----X'0002000B'     LOW-RBA-----------368640     TRACKS-----------------5
       HIGH-CCHH----X'00030000'     HIGH-RBA----------552959
INDEX --------- USER.KSDS1.CLINDEX
     HISTORY
       DATASET-OWNER-----(NULL)     CREATION--------2003.323
       RELEASE----------------2     EXPIRATION------2005.254
     PROTECTION-PSWD-----(NULL)     RACF----------------(NO)
     ASSOCIATIONS
       CLUSTER--USER.KSDS1.CLUSTER
     ATTRIBUTES
       KEYLEN-----------------4     AVGLRECL---------------0     BUFSPACE---------------0     CISIZE--------------2048
       RKP--------------------0     MAXLRECL------------2041     EXCPEXIT----------(NULL)     CI/CA-----------------18
       SHROPTNS(1,3)   RECOVERY     UNIQUE           NOERASE     NOWRITECHK       UNORDERED
       NOREUSE
     STATISTICS
       REC-TOTAL--------------4     SPLITS-CI--------------1     EXCPS---------------1377     INDEX:
       REC-DELETED------------0     SPLITS-CA--------------0     EXTENTS---------------1     LEVELS-----------------2
       REC-INSERTED-----------0     FREESPACE-%CI----------0     SYSTEM-TIMESTAMP:           ENTRIES/SECT-----------9
       REC-UPDATED----------183     FREESPACE-%CA----------0          X'A0D8F6E3D70D9401'     SEQ-SET-RBA--------36864
       REC-RETRIEVED----------0     FREESPC------------28672                                 HI-LEVEL-RBA-------40960
     ALLOCATION
       SPACE-TYPE---------TRACK     HI-A-RBA-----------73728
       SPACE-PRI--------------1     HI-U-RBA-----------45056
       SPACE-SEC--------------1
     VOLUME
       VOLSER------------1P0201     PHYREC-SIZE---------2048     HI-A-RBA-----------36864     EXTENT-NUMBER----------1
       DEVTYPE------X'3010200E'     PHYRECS/TRK-----------18     HI-U-RBA--------------0     EXTENT-TYPE--------X'40'
```

Figure 20. Example of LISTCAT ALL Output (6 of 12)

```
       VOLFLAG------------PRIME     TRACKS/CA--------------1
       EXTENTS:
       LOW-CCHH-----X'0000000D'     LOW-RBA----------------0     TRACKS-----------------1
       HIGH-CCHH----X'0000000D'     HIGH-RBA-----------36863
     VOLUME
       VOLSER-------------1P0202    PHYREC-SIZE---------2048     HI-A-RBA-----------73728     EXTENT-NUMBER----------1
       DEVTYPE------X'3010200E'     PHYRECS/TRK-----------18     HI-U-RBA-----------45056     EXTENT-TYPE--------X'00'
       VOLFLAG------------PRIME     TRACKS/CA--------------1
       EXTENTS:
       LOW-CCHH-----X'00010001'     LOW-RBA------------36864     TRACKS-----------------1
       HIGH-CCHH----X'00010001'     HIGH-RBA-----------73727
CLUSTER ------- USER.KSDS1.CLUSTER
     HISTORY
       DATASET-OWNER-----(NULL)     CREATION--------2004.345
       RELEASE----------------2     EXPIRATION------0000.000
     SMSDATA
       STORAGECLASS ---S1P02S02     MANAGEMENTCLASS---(NULL)
       DATACLASS --------(NULL)     LBACKUP ---0000.000.0000
       CA RECLAIM---------(YES)
       BWO STATUS------11100000     BWO TIMESTAMP---0000 00:00:00.0
       BWO-------------  (NULL)
     RLSDATA
       LOG --------------(NULL)     RECOVERY REQUIRED --(NO)
       VSAM QUIESCED      (NO)      RLS IN USE ---------(NO)
       LOGSTREAMID ------------------------------------(NULL)
       RECOVERY TIMESTAMP LOCAL-----X'0000000000000000'
       RECOVERY TIMESTAMP GMT-------X'0000000000000000'
     ENCRYPTIONDATA
       DATA SET ENCRYPTION-----(YES/NO)
       DATA SET KEY LABEL---(NULL/key-label)
     PROTECTION-PSWD-----(NULL)     RACF----------------(NO)
     ASSOCIATIONS
       DATA-----USER.KSDS1.CLDATA
       INDEX----USER.KSDS1.CLINDEX
       AIX------USER.KSDS1.AIX1CLUS
       PATH-----USER.KSDS1.PATHCL
PATH ---------- USER.KSDS1.PATHAIX1
     HISTORY
       DATASET-OWNER-----(NULL)     CREATION--------2003.323
       RELEASE----------------2     EXPIRATION------2005.254
     PROTECTION-PSWD-----(NULL)     RACF----------------(NO)
     ASSOCIATIONS
       AIX------USER.KSDS1.AIX1CLUS
       DATA-----USER.KSDS1.AIX1DATA
       INDEX----USER.KSDS1.AIX1INDX
       DATA-----USER.KSDS1.CLDATA
       INDEX----USER.KSDS1.CLINDEX
     ATTRIBUTES
       UPDATE
PATH ---------- USER.KSDS1.PATHCL
     HISTORY
       DATASET-OWNER-----(NULL)     CREATION--------2003.323
       RELEASE----------------2     EXPIRATION------2005.254
     PROTECTION-PSWD-----(NULL)     RACF----------------(NO)
     ASSOCIATIONS
       CLUSTER--USER.KSDS1.CLUSTER
       DATA-----USER.KSDS1.CLDATA
       INDEX----USER.KSDS1.CLINDEX
     ATTRIBUTES
       UPDATE
```

Figure 21. Example of LISTCAT ALL Output (7 of 12)

```
CLUSTER ------- USER.LINEAR
     HISTORY
       DATASET-OWNER---DEPTUSER     CREATION--------2003.323
       RELEASE----------------2     EXPIRATION------9999.999
       BWO STATUS------11100000     BWO TIMESTAMP---0000 00:00:00.0
     PROTECTION-PSWD-----(NULL)     RACF----------------(NO)
     ASSOCIATIONS
       DATA----USER.LINEAR.DATA
DATA ---------- USER.LINEAR.DATA
     HISTORY
       DATASET-OWNER-----(NULL)     CREATION--------2003.323
       RELEASE----------------2     EXPIRATION------9999.999
     PROTECTION-PSWD-----(NULL)     RACF----------------(NO)
     ASSOCIATIONS
       CLUSTER------USER.LINEAR
     ATTRIBUTES
       KEYLEN-----------------0     AVGLRECL---------------0     BUFSPACE------------8192     CISIZE--------------4096
       RKP--------------------0     MAXLRECL---------------0     EXCPEXIT----------(NULL)     CI/CA----------------20
       SHROPTNS(1,3)   RECOVERY     UNIQUE           NOERASE     LINEAR        NOWRITECHK
       UNORDERED        NOREUSE     NONSPANNED
     STATISTICS
       REC-TOTAL--------------0     SPLITS-CI--------------0     EXCPS------------------0
       REC-DELETED------------0     SPLITS-CA--------------0     EXTENTS----------------1
       REC-INSERTED-----------0     FREESPACE-%CI----------0     SYSTEM-TIMESTAMP:
       REC-UPDATED------------0     FREESPACE-%CA----------0          X'0000000000000000'
       REC-RETRIEVED----------0     FREESPC-----------163840
     ALLOCATION
       SPACE-TYPE---------TRACK     HI-A-RBA----------163840
       SPACE-PRI--------------4     HI-U-RBA----------------0
       SPACE-SEC--------------2
     VOLUME
       VOLSER------------338001     PHYREC-SIZE---------4096     HI-A-RBA----------163840     EXTENT-NUMBER----------1
       DEVTYPE------X'3010200E'     PHYRECS/TRK----------10     HI-U-RBA----------------0    EXTENT-TYPE--------X'40'
       VOLFLAG------------PRIME     TRACKS/CA--------------2
       EXTENTS:
       LOW-CCHH-----X'0000000B'     LOW-RBA----------------0     TRACKS-----------------4
       HIGH-CCHH----X'0000000E'     HIGH-RBA----------163839
NONVSAM ------- USER.MODEL
     HISTORY
       DATASET-OWNER-----(NULL)     CREATION--------2004.213
       RELEASE----------------2     EXPIRATION------0000.000
     VOLUMES
       VOLSER------------338001     DEVTYPE------X'3010200E'     FSEQN------------------0
     ASSOCIATIONS--------(NULL)
```

Figure 22. Example of LISTCAT ALL Output (8 of 12)

```
NONVSAM ------- USER.NONVSAM.DATA.SET
     HISTORY
       DATASET-OWNER-----(NULL)     CREATION--------2005.294
       RELEASE----------------2     EXPIRATION------0000.000
     SMSDATA
       STORAGECLASS ---S1P01S02     MANAGEMENTCLASS---(NULL)
       DATACLASS ------SRX00001     LBACKUP ---0000.000.0000
     VOLUMES
       VOLSER------------1P0302     DEVTYPE------X'3010200E'     FSEQN------------------0
     ASSOCIATIONS--------(NULL)
       ALIAS---------USER.ALIAS
     ATTRIBUTES
       VERSION-NUMBER---------1
       STRIPE-COUNT-----------1
       ACT-DICT-TOKEN---X'4000000C01C401E301F001F101F201F301F401F501F601F701F801F907030D0108FE0DFE'
       COMP-FORMT
       CCSID--------------65535
     STATISTICS
       USER-DATA-SIZE--------------------------------3389920     COMP-USER-DATA-SIZE----------------------------910988
       SIZES-VALID--------(YES)
NONVSAM ------- USER.PDSE
     HISTORY
       DATASET-OWNER-----(NULL)     CREATION--------2003.244
       RELEASE----------------2     EXPIRATION------2004.244
       DSNTYPE----------LIBRARY
     SMSDATA
       STORAGECLASS ---S1P03S01     MANAGEMENTCLASS---(NULL)
       DATACLASS ------SRX00001     LBACKUP ---0000.000.0000
     VOLUMES
       VOLSER------------1P0302     DEVTYPE------X'3030200E'     FSEQN------------------0
     ASSOCIATIONS--------(NULL)
```

Figure 23. Example of LISTCAT ALL Output (9 of 12)

```
CLUSTER ------- USER.SPANNED.CLUSTER
     HISTORY
       DATASET-OWNER-----(NULL)     CREATION--------2004.345
       RELEASE----------------2     EXPIRATION------0000.000
     SMSDATA
       STORAGECLASS ---S1P02S02     MANAGEMENTCLASS---(NULL)
       DATACLASS --------(NULL)     LBACKUP ---0000.000.0000
       CA RECLAIM---------(YES)
       BWO STATUS------11100000     BWO TIMESTAMP---0000 00:00:00.0
       BWO-------------  (NULL)
     RLSDATA
       LOG --------------(NULL)     RECOVERY REQUIRED --(NO)
       VSAM QUIESCED      (NO)      RLS IN USE ---------(NO)
       LOGSTREAMID -----------------------------------(NULL)
       RECOVERY TIMESTAMP LOCAL-----X'0000000000000000'
       RECOVERY TIMESTAMP GMT-------X'0000000000000000'
     ENCRYPTIONDATA
       DATA SET ENCRYPTION-----(YES/NO)
       DATA SET KEY LABEL---(NULL/key-label)
     PROTECTION
       MASTERPW--------CLUSMPW1     UPDATEPW----------(NULL)     CODE--------------(NULL)     RACF----------------(NO)
       CONTROLPW---------(NULL)     READPW------------(NULL)     ATTEMPTS--------------2     USVR--------------(NULL)
       USAR--------------(NONE)
     ASSOCIATIONS
       DATA-----USER.SPANNED.DATA
       INDEX----USER.SPANNED.INDEX
DATA ---------- USER.SPANNED.DATA
     HISTORY
       DATASET-OWNER-----(NULL)     CREATION--------2003.323
       RELEASE----------------2     EXPIRATION------2007.365
     PROTECTION-PSWD-----(NULL)     RACF----------------(NO)
     ASSOCIATIONS
       CLUSTER--USER.SPANNED.CLUSTER
     ATTRIBUTES
       KEYLEN-----------------4     AVGLRECL------------6000     BUFSPACE------------6144     CISIZE--------------2048
       RKP--------------------10     MAXLRECL------------6000     EXCPEXIT----------(NULL)     CI/CA-----------------90
       SHROPTNS(1,3)   RECOVERY     UNIQUE           NOERASE     INDEXED       NOWRITECHK     IMBED
       UNORDERED        NOREUSE     SPANNED
     STATISTICS
       REC-TOTAL------------100     SPLITS-CI--------------0     EXCPS----------------726
       REC-DELETED------------0     SPLITS-CA--------------2     EXTENTS----------------4
       REC-INSERTED----------40     FREESPACE-%CI----------0     SYSTEM-TIMESTAMP:
       REC-UPDATED------------0     FREESPACE-%CA----------0          X'A0D8F75A5E7E1700'
       REC-RETRIEVED--------100     FREESPC-----------491520
```

Figure 24. Example of LISTCAT ALL Output (10 of 12)

```
     ALLOCATION
       SPACE-TYPE---------TRACK     HI-A-RBA----------737280
       SPACE-PRI----------------6     HI-U-RBA----------737280
       SPACE-SEC----------------6
     VOLUME
       VOLSER------------1P0301     PHYREC-SIZE---------2048     HI-A-RBA----------552960     EXTENT-NUMBER----------2
       DEVTYPE------X'3010200E'     PHYRECS/TRK-----------18     HI-U-RBA----------552960     EXTENT-TYPE--------X'00'
       VOLFLAG------------PRIME     TRACKS/CA--------------6
       LOW-KEY---------00000010
       HIGH-KEY--------00000320
       HI-KEY-RBA--------491520
       EXTENTS:
       LOW-CCHH-----X'0000000B'     LOW-RBA----------------0     TRACKS-----------------6
       HIGH-CCHH----X'00010001'     HIGH-RBA----------184319
       LOW-CCHH-----X'00020008'     LOW-RBA-----------368640     TRACKS-----------------6
       HIGH-CCHH----X'0002000D'     HIGH-RBA----------552959
     VOLUME
       VOLSER------------1P0302     PHYREC-SIZE---------2048     HI-A-RBA----------737280     EXTENT-NUMBER----------2
       DEVTYPE------X'3010200E'     PHYRECS/TRK-----------18     HI-U-RBA----------737280     EXTENT-TYPE--------X'00'
       VOLFLAG------------PRIME     TRACKS/CA--------------6
       LOW-KEY---------00000330
       HIGH-KEY--------00000640
       HI-KEY-RBA--------675840
       EXTENTS:
       LOW-CCHH-----X'00020007'     LOW-RBA----------184320     TRACKS-----------------6
       HIGH-CCHH----X'0002000C'     HIGH-RBA----------368639
       LOW-CCHH-----X'0002000D'     LOW-RBA-----------552960     TRACKS-----------------6
       HIGH-CCHH----X'00030003'     HIGH-RBA----------737279
INDEX --------- USER.SPANNED.INDEX
     HISTORY
       DATASET-OWNER-----(NULL)     CREATION--------2003.323
       RELEASE----------------2     EXPIRATION------2007.365
     PROTECTION-PSWD-----(NULL)     RACF----------------(NO)
     ASSOCIATIONS
       CLUSTER--USER.SPANNED.CLUSTER
     ATTRIBUTES
       KEYLEN-----------------4     AVGLRECL----------------0     BUFSPACE--------------0     CISIZE--------------2048
       RKP--------------------10     MAXLRECL------------2041     EXCPEXIT----------(NULL)     CI/CA----------------18
       SHROPTNS(1,3)   RECOVERY     UNIQUE           NOERASE     NOWRITECHK         IMBED     REPLICATE      UNORDERED
       NOREUSE
     STATISTICS
       REC-TOTAL--------------5     SPLITS-CI--------------2     EXCPS----------------401     INDEX:
       REC-DELETED------------0     SPLITS-CA--------------0     EXTENTS----------------5     LEVELS-----------------2
       REC-INSERTED-----------0     FREESPACE-%CI----------0     SYSTEM-TIMESTAMP:            ENTRIES/SECT-----------9
       REC-UPDATED----------183     FREESPACE-%CA----------0          X'A0D8F75A5E7E1700'     SEQ-SET-RBA---------2048
       REC-RETRIEVED----------0     FREESPC----------------0                                 HI-LEVEL-RBA-----------0
     ALLOCATION
       SPACE-TYPE---------TRACK     HI-A-RBA-----------10240
       SPACE-PRI----------------1     HI-U-RBA-----------10240
       SPACE-SEC----------------1
     VOLUME
       VOLSER------------1P0301     PHYREC-SIZE---------2048     HI-A-RBA------------2048     EXTENT-NUMBER----------1
       DEVTYPE------X'3010200E'     PHYRECS/TRK-----------18     HI-U-RBA------------2048     EXTENT-TYPE--------X'00'
       VOLFLAG------------PRIME     TRACKS/CA--------------1
       EXTENTS:
       LOW-CCHH-----X'0001000C'     LOW-RBA----------------0     TRACKS-----------------1
       HIGH-CCHH----X'0001000C'     HIGH-RBA------------2047
```

Figure 25. Example of LISTCAT ALL Output (11 of 12)

```
     VOLUME
       VOLSER----------------*     PHYREC-SIZE-----------0     HI-A-RBA--------------0     EXTENT-NUMBER----------0
       DEVTYPE------X'3010200E'    PHYRECS/TRK-----------0     HI-U-RBA--------------0     EXTENT-TYPE--------X'FF'
       VOLFLAG--------CANDIDATE    TRACKS/CA-------------0
     VOLUME
       VOLSER------------1P0301    PHYREC-SIZE---------2048    HI-A-RBA-----------8192     EXTENT-NUMBER----------2
       DEVTYPE------X'3010200E'    PHYRECS/TRK-----------18    HI-U-RBA-----------8192     EXTENT-TYPE--------X'80'
       VOLFLAG------------PRIME    TRACKS/CA-------------6
       LOW-KEY---------00000010
       HIGH-KEY--------00000320
       EXTENTS:
       LOW-CCHH-----X'0000000B'    LOW-RBA-------------2048    TRACKS-----------------6
       HIGH-CCHH----X'00010001'    HIGH-RBA------------4095
       LOW-CCHH-----X'00020008'    LOW-RBA-------------6144    TRACKS-----------------6
       HIGH-CCHH----X'0002000D'    HIGH-RBA------------8191
     VOLUME
       VOLSER------------1P0302    PHYREC-SIZE---------2048    HI-A-RBA-----------10240    EXTENT-NUMBER----------2
       DEVTYPE------X'3010200E'    PHYRECS/TRK-----------18    HI-U-RBA-----------10240    EXTENT-TYPE--------X'80'
       VOLFLAG------------PRIME    TRACKS/CA-------------6
       LOW-KEY---------00000330
       HIGH-KEY--------00000640
       EXTENTS:
       LOW-CCHH-----X'00020007'    LOW-RBA-------------4096    TRACKS-----------------6
       HIGH-CCHH----X'0002000C'    HIGH-RBA------------6143
       LOW-CCHH-----X'0002000D'    LOW-RBA-------------8192    TRACKS-----------------6
       HIGH-CCHH----X'00030003'    HIGH-RBA-----------10239
         THE NUMBER OF ENTRIES PROCESSED WAS:
                   AIX -------------------1
                   ALIAS -----------------1
                   CLUSTER ---------------4
                   DATA ------------------5
                   GDG -------------------1
                   INDEX -----------------4
                   NONVSAM ---------------9
                   PAGESPACE -------------0
                   PATH ------------------2
                   SPACE -----------------0
                   USERCATALOG -----------0
                   TOTAL ----------------27
         THE NUMBER OF PROTECTED ENTRIES SUPPRESSED WAS 0
IDC0001I FUNCTION COMPLETED, HIGHEST CONDITION CODE WAS 0
IDC0002I IDCAMS PROCESSING COMPLETE. MAXIMUM CONDITION CODE WAS 0
```

Figure 26. Example of LISTCAT ALL Output (12 of 12)

LISTCAT ALL Output Listing for a Non-VSAM Tailored Compressed Data Set

The following example illustrates the output produced for a non-VSAM tailored compressed data set. Tailored compression (used only with non-VSAM compressed data sets) creates dictionaries tailored specifically to the initial data written to the data set. Once derived, the dictionary is stored within the data set. This technique improves compression ratios over generic DBB compression.

Note: The following information is not an intended programming interface. It is provided for diagnostic purposes only.

The first byte of the dictionary token indicates the type of compression used for the data set.

X'100. ....'
indicates compression has been rejected for the data set. No data is compressed.

X'010. ....'
indicates generic DBB compression is used.

X'011. ....'
indicates tailored compression is used.

```
    LISTCAT LEVEL(ZTAMS002) ALL
NONVSAM ------- ZTAMS002.COMPRS.DATA01
     IN-CAT --- SYS1.MVSRES.MASTCAT
     HISTORY
       DATASET-OWNER-----(NULL)     CREATION--------2004.297
       RELEASE----------------2     EXPIRATION------0000.000
       ACCOUNT-INFO-----------------------------------(NULL)
       EATTR--------------(OPT)
     SMSDATA
       STORAGECLASS ---S1P03S12     MANAGEMENTCLASS---(NULL)
       DATACLASS ------SRX00001     LBACKUP ---0000.000.0000
     VOLUMES
       VOLSER------------1P0301     DEVTYPE------X'3010200F'
FSEQN-----------------0
       VOLSER------------1P0302     DEVTYPE------X'3010200F'
FSEQN-----------------0
     ASSOCIATIONS--------(NULL)
     ATTRIBUTES
       VERSION-NUMBER---------1
       STRIPE-COUNT-----------2

ACT-DIC-TOKEN----X'6200000B0000000000000000000000000000000000000000000000000000000
0000000000000'
       COMP-FORMT      EXTENDED
     STATISTICS
       USER-DATA-SIZE-------------------------------4509920
COMP-USER-DATA-SIZE---------------------------351227
       SIZES-VALID--------(YES)
```

Figure 27. Example of LISTCAT ALL Output for a Non-VSAM Tailored Compressed Dataset

LISTCAT ALLOCATION Output Listing

When you specify the LISTCAT command and include the ALLOCATION parameter, each cataloged object with space allocated to it is listed (see Figure 28 on page 407). All information about the object's space is listed, but none of the object's other cataloged information is listed. The entry types that can be specified when the ALLOCATION parameter is specified are limited to DATA and INDEX.

Example : LISTCAT ALLOCATION Output

```
  LISTCAT -
          LEVEL(USER)                /* LIST ALL 'USER' ENTRIES   */ -
          ALLOCATION                 /* ALLOCATION INFORMATION    */ -
          CLUSTER                    /* INCLUDE CLUSTERS          */ -
          DATA                       /* AND DATA COMPONENTS       */ -
          INDEX                      /* AND INDEX COMPONENTS      */ -
          ALTERNATEINDEX             /* AND ALTERNATEINDEXES      */ -
          PATH                       /* AND PATHS                 */ -
          CATALOG(ICFUCAT1)          /* IN CATALOG ICFUCAT1       */

                            LISTING FROM CATALOG -- ICFUCAT1
CLUSTER ------- USER.DUMMY
     HISTORY
       DATASET-OWNER-----(NULL)     CREATION--------2003.323
       RELEASE----------------2     EXPIRATION------0000.000
       BWO STATUS------11100000     BWO TIMESTAMP---0000 00:00:00.0
     SMSDATA
       STORAGECLASS ---S1P02S02     MANAGEMENTCLASS---(NULL)
       DATACLASS --------(NULL)     LBACKUP ---0000.000.0000
       CA RECLAIM --------(YES)
DATA ---------- USER.DUMMY.CLDATA
     HISTORY
       DATASET-OWNER-----(NULL)     CREATION--------2003.323
       RELEASE----------------2     EXPIRATION------0000.000
     ALLOCATION
       SPACE-TYPE------CYLINDER     HI-A-RBA---------9400320
       SPACE-PRI-------------17     HI-U-RBA---------------0
       SPACE-SEC--------------0
     VOLUME
       VOLSER------------1P0201     PHYREC-SIZE---------2048     HI-A-RBA---------9400320     EXTENT-NUMBER----------1
       DEVTYPE------X'3010200E'     PHYRECS/TRK-----------18     HI-U-RBA---------------0     EXTENT-TYPE--------X'40'
       VOLFLAG------------PRIME     TRACKS/CA-------------15
       EXTENTS:
       LOW-CCHH-----X'00020000'     LOW-RBA----------------0     TRACKS----------------255
       HIGH-CCHH----X'0012000E'     HIGH-RBA---------9400319
INDEX --------- USER.DUMMY.CLINDEX
     HISTORY
       DATASET-OWNER-----(NULL)     CREATION--------2003.323
       RELEASE----------------2     EXPIRATION------0000.000
     ALLOCATION
       SPACE-TYPE---------TRACK     HI-A-RBA-----------76800
       SPACE-PRI--------------2     HI-U-RBA---------------0
       SPACE-SEC--------------0
     VOLUME
       VOLSER------------1P0201     PHYREC-SIZE---------2560     HI-A-RBA-----------76800     EXTENT-NUMBER----------1
       DEVTYPE------X'3010200E'     PHYRECS/TRK-----------15     HI-U-RBA---------------0     EXTENT-TYPE--------X'40'
       VOLFLAG------------PRIME     TRACKS/CA--------------1
       EXTENTS:
       LOW-CCHH-----X'0000000B'     LOW-RBA----------------0     TRACKS-----------------2
       HIGH-CCHH----X'0000000C'     HIGH-RBA-----------76799
AIX ----------- USER.KSDS1.AIX1CLUS
     HISTORY
       DATASET-OWNER-----(NULL)     CREATION--------2003.323
       RELEASE----------------2     EXPIRATION------2005.254
       SMS MANAGED--------(YES)
DATA ---------- USER.KSDS1.AIX1DATA
     HISTORY
       DATASET-OWNER-----(NULL)     CREATION--------2003.323
       RELEASE----------------2     EXPIRATION------2005.254
     ALLOCATION
       SPACE-TYPE---------TRACK     HI-A-RBA-----------86016
       SPACE-PRI--------------1     HI-U-RBA-----------43008
       SPACE-SEC--------------1
     VOLUME
       VOLSER------------1P0201     PHYREC-SIZE--------14336     HI-A-RBA-----------43008     EXTENT-NUMBER----------1
       DEVTYPE------X'3010200E'     PHYRECS/TRK------------3     HI-U-RBA---------------0     EXTENT-TYPE--------X'40'
       VOLFLAG------------PRIME     TRACKS/CA--------------1
       EXTENTS:
       LOW-CCHH-----X'0000000E'     LOW-RBA----------------0     TRACKS-----------------1
       HIGH-CCHH----X'0000000E'     HIGH-RBA-----------43007
```

Figure 28. Example of LISTCAT ALLOCATION Output (Part 1 of 5)

```
     VOLUME
       VOLSER------------1P0202     PHYREC-SIZE--------14336     HI-A-RBA-----------86016     EXTENT-NUMBER----------1
       DEVTYPE------X'3010200E'     PHYRECS/TRK------------3     HI-U-RBA-----------43008     EXTENT-TYPE--------X'00'
       VOLFLAG------------PRIME     TRACKS/CA--------------1
       EXTENTS:
       LOW-CCHH-----X'00020007'     LOW-RBA------------43008     TRACKS-----------------1
       HIGH-CCHH----X'00020007'     HIGH-RBA-----------86015
INDEX --------- USER.KSDS1.AIX1INDX
     HISTORY
       DATASET-OWNER-----(NULL)     CREATION--------2003.323
       RELEASE----------------2     EXPIRATION------2005.254
     ALLOCATION
       SPACE-TYPE---------TRACK     HI-A-RBA-------47104
       SPACE-PRI--------------1     HI-U-RBA--------24064
       SPACE-SEC--------------1
     VOLUME
       VOLSER------------1P0201     PHYREC-SIZE----------512     HI-A-RBA-----------23552     EXTENT-NUMBER----------1
       DEVTYPE------X'3010200E'     PHYRECS/TRK----------46     HI-U-RBA--------------0     EXTENT-TYPE--------X'40'
       VOLFLAG------------PRIME     TRACKS/CA--------------1
       EXTENTS:
       LOW-CCHH-----X'00010000'     LOW-RBA----------------0     TRACKS-----------------1
       HIGH-CCHH----X'00010000'     HIGH-RBA-----------23551
     VOLUME
       VOLSER------------1P0202     PHYREC-SIZE----------512     HI-A-RBA-----------47104     EXTENT-NUMBER----------1
       DEVTYPE------X'3010200E'     PHYRECS/TRK----------46     HI-U-RBA-----------24064     EXTENT-TYPE--------X'00'
       VOLFLAG------------PRIME     TRACKS/CA--------------1
       EXTENTS:
       LOW-CCHH-----X'00020008'     LOW-RBA------------23552     TRACKS-----------------1
       HIGH-CCHH----X'00020008'     HIGH-RBA-----------47103
DATA ---------- USER.KSDS1.CLDATA
     HISTORY
       DATASET-OWNER-----(NULL)     CREATION--------2003.323
       RELEASE----------------2     EXPIRATION------2005.254
     ALLOCATION
       SPACE-TYPE---------TRACK     HI-A-RBA------552960
       SPACE-PRI--------------5     HI-U-RBA-------552960
       SPACE-SEC--------------5
     VOLUME
       VOLSER------------1P0201     PHYREC-SIZE---------2048     HI-A-RBA------184320     EXTENT-NUMBER----------1
       DEVTYPE------X'3010200E'     PHYRECS/TRK----------18     HI-U-RBA------------0     EXTENT-TYPE--------X'40'
       VOLFLAG------------PRIME     TRACKS/CA--------------5
       EXTENTS:
       LOW-CCHH-----X'0013000A'     LOW-RBA----------------0     TRACKS-----------------5
       HIGH-CCHH----X'0013000E'     HIGH-RBA----------184319
     VOLUME
       VOLSER------------1P0202     PHYREC-SIZE---------2048     HI-A-RBA----------552960     EXTENT-NUMBER----------2
       DEVTYPE------X'3010200E'     PHYRECS/TRK----------18     HI-U-RBA----------552960     EXTENT-TYPE--------X'00'
       VOLFLAG------------PRIME     TRACKS/CA--------------5
       EXTENTS:
       LOW-CCHH-----X'0000000B'     LOW-RBA-----------184320     TRACKS-----------------5
       HIGH-CCHH----X'00010000'     HIGH-RBA----------368639
       LOW-CCHH-----X'0002000B'     LOW-RBA-----------368640     TRACKS-----------------5
       HIGH-CCHH----X'00030000'     HIGH-RBA----------552959
```

Figure 29. Example of LISTCAT ALLOCATION Output (Part 2 of 5)

```
INDEX --------- USER.KSDS1.CLINDEX
     HISTORY
       DATASET-OWNER-----(NULL)     CREATION--------2003.323
       RELEASE----------------2     EXPIRATION------2005.254
     ALLOCATION
       SPACE-TYPE---------TRACK     HI-A-RBA-----------73728
       SPACE-PRI----------------1   HI-U-RBA-----------45056
       SPACE-SEC----------------1
     VOLUME
       VOLSER------------1P0201     PHYREC-SIZE---------2048     HI-A-RBA-----------36864     EXTENT-NUMBER----------1
       DEVTYPE------X'3010200E'     PHYRECS/TRK-----------18     HI-U-RBA---------------0     EXTENT-TYPE--------X'40'
       VOLFLAG------------PRIME     TRACKS/CA--------------1
       EXTENTS:
       LOW-CCHH-----X'0000000D'     LOW-RBA----------------0     TRACKS-----------------1
       HIGH-CCHH----X'0000000D'     HIGH-RBA-----------36863
     VOLUME
       VOLSER------------1P0202     PHYREC-SIZE---------2048     HI-A-RBA-------73728     EXTENT-NUMBER----------1
       DEVTYPE------X'3010200E'     PHYRECS/TRK-----------18     HI-U-RBA--------45056     EXTENT-TYPE--------X'00'
       VOLFLAG------------PRIME     TRACKS/CA--------------1
       EXTENTS:
       LOW-CCHH-----X'00010001'     LOW-RBA------------36864     TRACKS-----------------1
       HIGH-CCHH----X'00010001'     HIGH-RBA-----------73727
CLUSTER ------- USER.KSDS1.CLUSTER
     HISTORY
       DATASET-OWNER-----(NULL)     CREATION--------2003.323
       RELEASE----------------2     EXPIRATION------2005.254
       BWO STATUS------11100000     BWO TIMESTAMP---0000 00:00:00.0
     SMSDATA
       STORAGECLASS ---S1P02S02     MANAGEMENTCLASS---(NULL)
       DATACLASS --------(NULL)     LBACKUP ---0000.000.0000
       CA RECLAIM --------(YES)
PATH ---------- USER.KSDS1.PATHAIX1
     HISTORY
       DATASET-OWNER-----(NULL)     CREATION--------2003.323
       RELEASE----------------2     EXPIRATION------2005.254
PATH ---------- USER.KSDS1.PATHCL
     HISTORY
       DATASET-OWNER-----(NULL)     CREATION--------2003.323
       RELEASE----------------2     EXPIRATION------2005.254
CLUSTER ------- USER.LINEAR
     HISTORY
       DATASET-OWNER---DEPTUSER     CREATION--------2003.323
       RELEASE----------------2     EXPIRATION------9999.999
       BWO STATUS------11100000     BWO TIMESTAMP---0000 00:00:00.0
DATA ---------- USER.LINEAR.DATA
     HISTORY
       DATASET-OWNER-----(NULL)     CREATION--------2003.323
       RELEASE----------------2     EXPIRATION------9999.999
     ALLOCATION
       SPACE-TYPE---------TRACK     HI-A-RBA----------163840
       SPACE-PRI----------------4   HI-U-RBA----------------0
       SPACE-SEC----------------2
     VOLUME
       VOLSER------------338001     PHYREC-SIZE---------4096     HI-A-RBA----------163840     EXTENT-NUMBER----------1
       DEVTYPE------X'3010200E'     PHYRECS/TRK-----------10     HI-U-RBA----------------0     EXTENT-TYPE--------X'40'
       VOLFLAG------------PRIME     TRACKS/CA--------------2
       EXTENTS:
       LOW-CCHH-----X'0000000B'     LOW-RBA----------------0     TRACKS-----------------4
       HIGH-CCHH----X'0000000E'     HIGH-RBA----------163839
CLUSTER ------- USER.SPANNED.CLUSTER
     HISTORY
       DATASET-OWNER-----(NULL)     CREATION--------2003.323
       RELEASE----------------2     EXPIRATION------2007.365
       BWO STATUS------11100000     BWO TIMESTAMP---0000 00:00:00.0
     SMSDATA
       STORAGECLASS ---S1P03S01     MANAGEMENTCLASS---(NULL)
       DATACLASS --------(NULL)     LBACKUP ---0000.000.0000
       CA RECLAIM --------(YES)
```

Figure 30. Example of LISTCAT ALLOCATION Output (Part 3 of 5)

```
DATA ---------- USER.SPANNED.DATA
     HISTORY
       DATASET-OWNER-----(NULL)     CREATION--------2003.323
       RELEASE----------------2     EXPIRATION------2007.365
     ALLOCATION
       SPACE-TYPE---------TRACK     HI-A-RBA----------737280
       SPACE-PRI----------------6   HI-U-RBA----------737280
       SPACE-SEC----------------6
     VOLUME
       VOLSER------------1P0301     PHYREC-SIZE---------2048     HI-A-RBA----------552960     EXTENT-NUMBER----------2
       DEVTYPE------X'3010200E'     PHYRECS/TRK-----------18     HI-U-RBA----------552960     EXTENT-TYPE--------X'00'
       VOLFLAG------------PRIME     TRACKS/CA--------------6
       LOW-KEY---------00000010
       HIGH-KEY--------00000320
       HI-KEY-RBA--------491520
       EXTENTS:
       LOW-CCHH-----X'0000000B'     LOW-RBA----------------0     TRACKS-----------------6
       HIGH-CCHH----X'00010001'     HIGH-RBA----------184319
       LOW-CCHH-----X'00020008'     LOW-RBA-----------368640     TRACKS-----------------6
       HIGH-CCHH----X'0002000D'     HIGH-RBA----------552959
     VOLUME
       VOLSER------------1P0302     PHYREC-SIZE---------2048     HI-A-RBA----------737280     EXTENT-NUMBER----------2
       DEVTYPE------X'3010200E'     PHYRECS/TRK-----------18     HI-U-RBA----------737280     EXTENT-TYPE--------X'00'
       VOLFLAG------------PRIME     TRACKS/CA--------------6
       LOW-KEY---------00000330
       HIGH-KEY--------00000640
       HI-KEY-RBA--------675840
       EXTENTS:
       LOW-CCHH-----X'00020007'     LOW-RBA-----------184320     TRACKS-----------------6
       HIGH-CCHH----X'0002000C'     HIGH-RBA----------368639
       LOW-CCHH-----X'0002000D'     LOW-RBA-----------552960     TRACKS-----------------6
       HIGH-CCHH----X'00030003'     HIGH-RBA----------737279
INDEX --------- USER.SPANNED.INDEX
     HISTORY
       DATASET-OWNER-----(NULL)     CREATION--------2003.323
       RELEASE----------------2     EXPIRATION------2007.365
     ALLOCATION
       SPACE-TYPE---------TRACK     HI-A-RBA-----------10240
       SPACE-PRI----------------1   HI-U-RBA-----------10240
       SPACE-SEC----------------1
     VOLUME
       VOLSER------------1P0301     PHYREC-SIZE---------2048     HI-A-RBA------------2048     EXTENT-NUMBER----------1
       DEVTYPE------X'3010200E'     PHYRECS/TRK-----------18     HI-U-RBA------------2048     EXTENT-TYPE--------X'00'
       VOLFLAG------------PRIME     TRACKS/CA--------------1
       EXTENTS:
       LOW-CCHH-----X'0001000C'     LOW-RBA----------------0     TRACKS-----------------1
       HIGH-CCHH----X'0001000C'     HIGH-RBA------------2047
     VOLUME
       VOLSER-----------------*     PHYREC-SIZE------------0     HI-A-RBA----------------0     EXTENT-NUMBER----------0
       DEVTYPE------X'3010200E'     PHYRECS/TRK------------0     HI-U-RBA----------------0     EXTENT-TYPE--------X'FF'
       VOLFLAG--------CANDIDATE     TRACKS/CA--------------0
```

Figure 31. Example of LISTCAT ALLOCATION Output (Part 4 of 5)

```
     VOLUME
       VOLSER------------1P0301     PHYREC-SIZE---------2048     HI-A-RBA------------8192     EXTENT-NUMBER----------2
       DEVTYPE------X'3010200E'     PHYRECS/TRK-----------18     HI-U-RBA------------8192     EXTENT-TYPE--------X'80'
       VOLFLAG------------PRIME     TRACKS/CA--------------6
       LOW-KEY---------00000010
       HIGH-KEY--------00000320
       EXTENTS:
       LOW-CCHH-----X'0000000B'     LOW-RBA-------------2048     TRACKS-----------------6
       HIGH-CCHH----X'00010001'     HIGH-RBA------------4095
       LOW-CCHH-----X'00020008'     LOW-RBA-------------6144     TRACKS-----------------6
       HIGH-CCHH----X'0002000D'     HIGH-RBA------------8191
     VOLUME
       VOLSER------------1P0302     PHYREC-SIZE---------2048     HI-A-RBA-----------10240     EXTENT-NUMBER----------2
       DEVTYPE------X'3010200E'     PHYRECS/TRK-----------18     HI-U-RBA-----------10240     EXTENT-TYPE--------X'80'
       VOLFLAG------------PRIME     TRACKS/CA--------------6
       LOW-KEY---------00000330
       HIGH-KEY--------00000640
       EXTENTS:
       LOW-CCHH-----X'00020007'     LOW-RBA-------------4096     TRACKS-----------------6
       HIGH-CCHH----X'0002000C'     HIGH-RBA------------6143
       LOW-CCHH-----X'0002000D'     LOW-RBA-------------8192     TRACKS-----------------6
       HIGH-CCHH----X'00030003'     HIGH-RBA-----------10239
         THE NUMBER OF ENTRIES PROCESSED WAS:
                   AIX -------------------1
                   ALIAS -----------------0
                   CLUSTER ---------------4
                   DATA ------------------5
                   GDG -------------------0
                   INDEX -----------------4
                   NONVSAM ---------------0
                   PAGESPACE -------------0
                   PATH ------------------2
                   SPACE -----------------0
                   USERCATALOG -----------0
                   TOTAL ----------------16
         THE NUMBER OF PROTECTED ENTRIES SUPPRESSED WAS 0
IDC0001I FUNCTION COMPLETED, HIGHEST CONDITION CODE WAS 0
IDC0002I IDCAMS PROCESSING COMPLETE. MAXIMUM CONDITION CODE WAS 0
```

Figure 32. Example of LISTCAT ALLOCATION Output (Part 5 of 5)

LISTCAT HISTORY Output Listing

When you specify the LISTCAT command and include the HISTORY or ALL parameter, only the name, *ownerid*, creation date, account information, and expiration date are listed for each entry that is selected, as shown in the following example. Only these types of entries have HISTORY information: ALTERNATEINDEX, CLUSTER, DATA, GDG, INDEX, NONVSAM, PAGESPACE and PATH.

```
  LISTCAT -
         LEVEL(USER)                /* LIST ALL 'USER' ENTRIES   */ -
         HISTORY                    /* SHOW HISTORY INFORMATION  */ -
         CATALOG(ICFUCAT1)          /* IN CATALOG ICFUCAT1       */

                            LISTING FROM CATALOG -- ICFUCAT1
ALIAS --------- USER.ALIAS
     HISTORY
       RELEASE----------------2
CLUSTER ------- USER.DUMMY
     HISTORY
       DATASET-OWNER-----(NULL)     CREATION--------2003.323
       RELEASE----------------2     EXPIRATION------0000.000
       BWO STATUS------11100000     BWO TIMESTAMP---0000 00:00:00.0
     SMSDATA
       STORAGECLASS ---S1P02S02     MANAGEMENTCLASS---(NULL)
       DATACLASS --------(NULL)     LBACKUP ---0000.000.0000
DATA ---------- USER.DUMMY.CLDATA
     HISTORY
       DATASET-OWNER-----(NULL)     CREATION--------2003.323
       RELEASE----------------2     EXPIRATION------0000.000
INDEX --------- USER.DUMMY.CLINDEX
     HISTORY
       DATASET-OWNER-----(NULL)     CREATION--------2003.323
       RELEASE----------------2     EXPIRATION------0000.000
GDG BASE ------ USER.GDGBASE
     HISTORY
       DATASET-OWNER-----(NULL)     CREATION--------2003.323
       RELEASE----------------2     LAST ALTER DATE-2005.001
NONVSAM ------- USER.GDGBASE.G0003V00
     HISTORY
       DATASET-OWNER-----(NULL)     CREATION--------2003.323
       RELEASE----------------2     EXPIRATION------2005.001
       STATUS------------ACTIVE
     SMSDATA
       STORAGECLASS ---S1P02S01     MANAGEMENTCLASS-S1P01M02
       DATACLASS ------PS000000     LBACKUP ---0000.000.0000
NONVSAM ------- USER.GDGBASE.G0004V00
     HISTORY
       DATASET-OWNER-----(NULL)     CREATION--------2003.323
       RELEASE----------------2     EXPIRATION------2005.001
       STATUS------------ACTIVE
     SMSDATA
       STORAGECLASS ---S1P02S01     MANAGEMENTCLASS-S1P01M02
       DATACLASS ------PS000000     LBACKUP ---0000.000.0000
NONVSAM ------- USER.GDGBASE.G0005V00
     HISTORY
       DATASET-OWNER-----(NULL)     CREATION--------2003.323
       RELEASE----------------2     EXPIRATION------2005.001
       STATUS------------ACTIVE
     SMSDATA
       STORAGECLASS ---S1P02S01     MANAGEMENTCLASS-S1P01M02
       DATACLASS ------PS000000     LBACKUP ---0000.000.0000
NONVSAM ------- USER.GDGBASE.G0006V00
     HISTORY
       DATASET-OWNER-----(NULL)     CREATION--------2003.323
       RELEASE----------------2     EXPIRATION------2005.001
       STATUS------------ACTIVE
     SMSDATA
       STORAGECLASS ---S1P02S01     MANAGEMENTCLASS-S1P01M02
       DATACLASS ------PS000000     LBACKUP ---0000.000.0000
NONVSAM ------- USER.GDGBASE.G0001V00
     HISTORY
       DATASET-OWNER-----(NULL)     CREATION--------2003.323
```

```
       RELEASE----------------2     EXPIRATION------2005.001
       STATUS--------ROLLED-OFF
     SMSDATA
       STORAGECLASS ---S1P02S01     MANAGEMENTCLASS-S1P01M02
       DATACLASS ------PS000000     LBACKUP ---0000.000.0000
NONVSAM ------- USER.GDGBASE.G0002V00
     HISTORY
       DATASET-OWNER-----(NULL)     CREATION--------2003.323
       RELEASE----------------2     EXPIRATION------2005.001
       STATUS----------DEFERRED
     SMSDATA
       STORAGECLASS ---S1P02S01     MANAGEMENTCLASS-S1P01M02
       DATACLASS ------PS000000     LBACKUP ---0000.000.0000
AIX ----------- USER.KSDS1.AIX1CLUS
     HISTORY
       DATASET-OWNER-----(NULL)     CREATION--------2003.323
       RELEASE----------------2     EXPIRATION------2005.254
       SMS MANAGED--------(YES)
DATA ---------- USER.KSDS1.AIX1DATA
     HISTORY
       DATASET-OWNER-----(NULL)     CREATION--------2003.323
       RELEASE----------------2     EXPIRATION------2005.254
INDEX --------- USER.KSDS1.AIX1INDX
     HISTORY
       DATASET-OWNER-----(NULL)     CREATION--------2003.323
       RELEASE----------------2     EXPIRATION------2005.254
DATA ---------- USER.KSDS1.CLDATA
     HISTORY
       DATASET-OWNER-----(NULL)     CREATION--------2003.323
       RELEASE----------------2     EXPIRATION------2005.254
INDEX --------- USER.KSDS1.CLINDEX
     HISTORY
       DATASET-OWNER-----(NULL)     CREATION--------2003.323
       RELEASE----------------2     EXPIRATION------2005.254
CLUSTER ------- USER.KSDS1.CLUSTER
     HISTORY
       DATASET-OWNER-----(NULL)     CREATION--------2003.323
       RELEASE----------------2     EXPIRATION------2005.254
       BWO STATUS------11100000     BWO TIMESTAMP---0000 00:00:00.0
     SMSDATA
       STORAGECLASS ---S1P02S02     MANAGEMENTCLASS---(NULL)
       DATACLASS --------(NULL)     LBACKUP ---0000.000.0000
PATH ---------- USER.KSDS1.PATHAIX1
     HISTORY
       DATASET-OWNER-----(NULL)     CREATION--------2003.323
       RELEASE----------------2     EXPIRATION------2005.254
PATH ---------- USER.KSDS1.PATHCL
     HISTORY
       DATASET-OWNER-----(NULL)     CREATION--------2003.323
       RELEASE----------------2     EXPIRATION------2005.254
CLUSTER ------- USER.LINEAR
     HISTORY
       DATASET-OWNER---DEPTUSER     CREATION--------2003.323
       RELEASE----------------2     EXPIRATION------9999.999
       BWO STATUS------11100000     BWO TIMESTAMP---0000 00:00:00.0
DATA ---------- USER.LINEAR.DATA
     HISTORY
       DATASET-OWNER-----(NULL)     CREATION--------2003.323
       RELEASE----------------2     EXPIRATION------9999.999
NONVSAM ------- USER.MODEL
     HISTORY
       DATASET-OWNER-----(NULL)     CREATION--------2003.323
       RELEASE----------------2     EXPIRATION------0000.000
NONVSAM ------- USER.NONVSAM.DATA.SET
     HISTORY
       DATASET-OWNER-----(NULL)     CREATION--------2003.323
       RELEASE----------------2     EXPIRATION------0000.000
     SMSDATA
       STORAGECLASS ---S1P01S02     MANAGEMENTCLASS-S1P01M02
       DATACLASS ------PS000000     LBACKUP ---0000.000.0000

NONVSAM ------- USER.PDSE
```

```
     HISTORY
       DATASET-OWNER-----(NULL)     CREATION--------2003.323
       RELEASE----------------2     EXPIRATION------2005.244
       DSNTYPE----------LIBRARY
     SMSDATA
       STORAGECLASS ---S1P03S01     MANAGEMENTCLASS---(NULL)
       DATACLASS --------(NULL)     LBACKUP ---0000.000.0000
CLUSTER ------- USER.SPANNED.CLUSTER
     HISTORY
       DATASET-OWNER-----(NULL)     CREATION--------2003.323
       RELEASE----------------2     EXPIRATION------2007.365
       BWO STATUS------11100000     BWO TIMESTAMP---0000 00:00:00.0
     SMSDATA
       STORAGECLASS ---S1P03S01     MANAGEMENTCLASS---(NULL)
       DATACLASS --------(NULL)     LBACKUP ---0000.000.0000
       CA RECLAIM --------(YES)
DATA ---------- USER.SPANNED.DATA
     HISTORY
       DATASET-OWNER-----(NULL)     CREATION--------2003.323
       RELEASE----------------2     EXPIRATION------2007.365
INDEX --------- USER.SPANNED.INDEX
     HISTORY
       DATASET-OWNER-----(NULL)     CREATION--------2003.323
       RELEASE----------------2     EXPIRATION------2007.365
         THE NUMBER OF ENTRIES PROCESSED WAS:
                   AIX -------------------1
                   ALIAS -----------------1
                   CLUSTER ---------------4
                   DATA ------------------5
                   GDG -------------------1
                   INDEX -----------------4
                   NONVSAM ---------------9
                   PAGESPACE -------------0
                   PATH ------------------2
                   SPACE -----------------0
                   USERCATALOG -----------0
                   TOTAL ----------------27
         THE NUMBER OF PROTECTED ENTRIES SUPPRESSED WAS 0
IDC0001I FUNCTION COMPLETED, HIGHEST CONDITION CODE WAS 0
IDC0002I IDCAMS PROCESSING COMPLETE. MAXIMUM CONDITION CODE WAS 0
DATA ---------- USER.LINEAR.DATA
     HISTORY
       DATASET-OWNER-----(NULL)     CREATION--------2003.323
       RELEASE----------------2     EXPIRATION------9999.999
NONVSAM ------- USER.MODEL
     HISTORY
       DATASET-OWNER-----(NULL)     CREATION--------2003.323
       RELEASE----------------2     EXPIRATION------0000.000
NONVSAM ------- USER.NONVSAM.DATA.SET
     HISTORY
       DATASET-OWNER-----(NULL)     CREATION--------2003.323
       RELEASE----------------2     EXPIRATION------0000.000
     SMSDATA
       STORAGECLASS ---S1P01S02     MANAGEMENTCLASS-S1P01M02
       DATACLASS ------PS000000     LBACKUP ---0000.000.0000
NONVSAM ------- USER.PDSE
     HISTORY
       DATASET-OWNER-----(NULL)     CREATION--------2003.323
       RELEASE----------------2     EXPIRATION------2005.244
       DSNTYPE----------LIBRARY
     SMSDATA
       STORAGECLASS ---S1P03S01     MANAGEMENTCLASS---(NULL)
       DATACLASS --------(NULL)     LBACKUP ---0000.000.0000
CLUSTER ------- USER.SPANNED.CLUSTER
     HISTORY
       DATASET-OWNER-----(NULL)     CREATION--------2003.323
       RELEASE----------------2     EXPIRATION------2007.365
       BWO STATUS------11100000     BWO TIMESTAMP---0000 00:00:00.0
     SMSDATA
       STORAGECLASS ---S1P03S01     MANAGEMENTCLASS---(NULL)
       DATACLASS --------(NULL)     LBACKUP ---0000.000.0000
       CA RECLAIM --------(YES)
```

```
DATA ---------- USER.SPANNED.DATA
     HISTORY
       DATASET-OWNER-----(NULL)     CREATION--------2003.323
       RELEASE----------------2     EXPIRATION------2007.365
INDEX --------- USER.SPANNED.INDEX
     HISTORY
       DATASET-OWNER-----(NULL)     CREATION--------2003.323
       RELEASE----------------2     EXPIRATION------2007.365
         THE NUMBER OF ENTRIES PROCESSED WAS:
                   AIX -------------------1
                   ALIAS -----------------1
                   CLUSTER ---------------4
                   DATA ------------------5
                   GDG -------------------1
                   INDEX -----------------4
                   NONVSAM ---------------9
                   PAGESPACE -------------0
                   PATH ------------------2
                   SPACE -----------------0
                   USERCATALOG -----------0
                   TOTAL -----------------27
          THE NUMBER OF PROTECTED ENTRIES SUPPRESSED WAS 0
IDC0001I FUNCTION COMPLETED, HIGHEST CONDITION CODE WAS 0
IDC0002I IDCAMS PROCESSING COMPLETE. MAXIMUM CONDITION CODE WAS 0
```

LISTCAT LEVEL Output Listing

LISTCAT LEVEL(USER) is specified in various LISTCAT examples throughout this appendix. Refer to the LISTCAT ALL or LISTCAT ALLOCATION examples for an example of a LISTCAT LEVEL output listing.

LISTCAT ENTRIES Output Listing

When you specify the LISTCAT command and include the ENTRIES parameter, entries specified by the entryname are listed.

```
  LISTCAT -
          ENTRIES(USER.GDGBASE.*)     /* LIST ALL 'USER.GDGBASE'   */ -
          NAME                        /* NAMES ONLY                */ -
          CATALOG(ICFUCAT1)           /* IN CATALOG ICFUCAT1       */

                              LISTING FROM CATALOG -- ICFUCAT1
NONVSAM ------- USER.GDGBASE.G0001V00
NONVSAM ------- USER.GDGBASE.G0002V00
NONVSAM ------- USER.GDGBASE.G0003V00
NONVSAM ------- USER.GDGBASE.G0004V00
NONVSAM ------- USER.GDGBASE.G0005V00
NONVSAM ------- USER.GDGBASE.G0006V00
          THE NUMBER OF ENTRIES PROCESSED WAS:
                    AIX -------------------0
                    ALIAS -----------------0
                    CLUSTER ---------------0
                    DATA ------------------0
                    GDG -------------------0
                    INDEX -----------------0
                    NONVSAM ---------------6
                    PAGESPACE -------------0
                    PATH ------------------0
                    SPACE -----------------0
                    USERCATALOG -----------0
                    TOTAL -----------------6
          THE NUMBER OF PROTECTED ENTRIES SUPPRESSED WAS 0
IDC0001I FUNCTION COMPLETED, HIGHEST CONDITION CODE WAS 0
IDC0002I IDCAMS PROCESSING COMPLETE. MAXIMUM CONDITION CODE WAS 0
```

Figure 33. Example of LISTCAT ENTRIES Output

LISTCAT CREATION/EXPIRATION Output Listing

When you specify the LISTCAT command and include the CREATION or EXPIRATION parameter (or both), entries that have a creation or expiration date are selected according to the number of days you specify in the subparameter.

For example, in Figure 34 on page 415 only the entry USER.ALIAS is listed as a result of the LISTCAT CREATION(5) job because it was created more than 5 days ago and all the objects were created on the same day as the LISTCAT. When that job is run on an older catalog, each entry that was created the specified number of days ago or earlier is listed (that is, the CREATION number of days specifies that all objects in the catalog at least 5 days old are to be listed). The creation date of the data and index objects of a cluster or alternate index is always the same as the creation date of its associated cluster or alternate index object.

When you list all entries of a catalog, and you specify the CREATION parameter, each user catalog connector entry is also listed regardless of its creation date.

The LISTCAT CREATION keywords on the LISTCAT command are ignored if the LIBRARYENTRIES or VOLUMEENTRIES keywords are also supplied. Date filtering cannot be done for these types of entries being listed.

When the LISTCAT EXPIRATION(365) job is run, each entry whose expiration date occurs within 365 days of today's date is listed, as in Figure 35 on page 416.

When you list all entries of a catalog and you specify the EXPIRATION parameter, each volume entry is listed because volume entries have no expiration date.

The following can have a creation or expiration date: ALTERNATEINDEX, CLUSTER, DATA, GDG, INDEX, NONVSAM, PAGESPACE and PATH.

The LISTCAT EXPIRATION keywords on the LISTCAT command are ignored if the LIBRARYENTRIES or VOLUMEENTRIES keywords are also supplied. Date filtering cannot be done for these types of entries being listed.

```
*************************************************************************************************************/
/* LIST EACH CATALOG ENTRY WHOSE CREATION DATE IS 5 DAYS AGO OR EARLIER    */
/* (THAT IS, THE OBJECT IS AT LEAST 5 DAYS OLD)                                                          */
/*************************************************************************************************************/
LISTING FROM CATALOG -- ICFUCAT1
LISTCAT -
   CREATION(5) -
   CATALOG(ICFUCAT1/ )

ALIAS --------- USER.ALIAS
THE NUMBER OF ENTRIES PROCESSED WAS:
AIX ------------------------0
ALIAS ----------------------1
CLUSTER --------------0
DATA -----------------------0
GDG ------------------------0
INDEX ----------------------0
NONVSAM -------------0
PAGESPACE ---------0
PATH -----------------------0
SPACE ----------------------0
USERCATALOG ----0
TOTAL ----------------------1
THE NUMBER OF PROTECTED ENTRIES SUPPRESSED WAS 0
IDC0001I FUNCTION COMPLETED, HIGHEST CONDITION CODE WAS 0
```

Figure 34. Example of LISTCAT CREATION(5) Output

```
 /*****************************************************************/
 /* LISTCAT 'USER' ENTRIES THAT HAVE EXPIRATION DATES WITHIN THE   */
 /* NEXT 365 DAYS                                                 */
 /*****************************************************************/
  LISTCAT -
         EXPIRATION(365)            /* LIST EXPIRATION OF 365    */ -
         NAME                       /* NAMES ONLY                */ -
         CATALOG(ICFUCAT1)          /* IN CATALOG ICFUCAT1       */

                           LISTING FROM CATALOG -- ICFUCAT1
CLUSTER ------- 00000000000000000000000000000000000000000000000000
   DATA ------- ICFUCAT1
   INDEX ------ ICFUCAT1.CATINDEX
CLUSTER ------- SYS1.VVDS.V1P0101
   DATA ------- SYS1.VVDS.V1P0101
CLUSTER ------- SYS1.VVDS.V1P0201
   DATA ------- SYS1.VVDS.V1P0201
CLUSTER ------- SYS1.VVDS.V1P0202
   DATA ------- SYS1.VVDS.V1P0202
CLUSTER ------- SYS1.VVDS.V1P0301
   DATA ------- SYS1.VVDS.V1P0301
CLUSTER ------- SYS1.VVDS.V1P0302
   DATA ------- SYS1.VVDS.V1P0302
CLUSTER ------- SYS1.VVDS.V338001
   DATA ------- SYS1.VVDS.V338001
ALIAS --------- USER.ALIAS
CLUSTER ------- USER.DUMMY
   DATA ------- USER.DUMMY.CLDATA
   INDEX ------ USER.DUMMY.CLINDEX
GDG BASE ------ USER.GDGBASE
   NONVSAM ---- USER.GDGBASE.G0003V00
   NONVSAM ---- USER.GDGBASE.G0004V00
   NONVSAM ---- USER.GDGBASE.G0005V00
   NONVSAM ---- USER.GDGBASE.G0006V00
NONVSAM ------- USER.GDGBASE.G0001V00
NONVSAM ------- USER.GDGBASE.G0002V00
AIX ------------ USER.KSDS1.AIX1CLUS
   DATA ------- USER.KSDS1.AIX1DATA
   INDEX ------ USER.KSDS1.AIX1INDX
   PATH ------- USER.KSDS1.PATHAIX1
CLUSTER ------- USER.KSDS1.CLUSTER
   DATA ------- USER.KSDS1.CLDATA
   INDEX ------ USER.KSDS1.CLINDEX
   PATH ------- USER.KSDS1.PATHCL
NONVSAM ------- USER.MODEL
NONVSAM ------- USER.NONVSAM.DATA.SET
NONVSAM ------- USER.PDSE
         THE NUMBER OF ENTRIES PROCESSED WAS:
                   AIX -------------------1
                   ALIAS -----------------1
                   CLUSTER ---------------9
                   DATA ------------------10
                   GDG -------------------1
                   INDEX -----------------4
                   NONVSAM ---------------9
                   PAGESPACE -------------0
                   PATH ------------------2
                   SPACE -----------------0
                   USERCATALOG -----------0
                   TOTAL -----------------37
         THE NUMBER OF PROTECTED ENTRIES SUPPRESSED WAS 0
IDC0001I FUNCTION COMPLETED, HIGHEST CONDITION CODE WAS 0
IDC0002I IDCAMS PROCESSING COMPLETE. MAXIMUM CONDITION CODE WAS 0
```

Figure 35. Example of LISTCAT EXPIRATION(365) Output

Examples of LISTCAT in a TSO Environment

The following examples illustrate the output produced at a TSO terminal for a LISTCAT NAMES (default) and LISTCAT VOLUME. A TSO logon ID of IBMUSER is assumed.

For LISTCAT NAMES, the catalog name is printed followed by the names of all entries that have a high-level qualifier equal to the USER logon ID.

For LISTCAT VOLUME, all entrynames that have a high-level qualifier equal to the USER logon ID are printed, followed by the volume serial numbers for those entries that contain volume information.

Note: Because volume serial numbers for a cluster or an alternate index are contained in the data and index components, the data and index must have been named on the initial DEFINE in order to list the volume serial numbers.

LOGON IBMUSER

READY